Grid vs. Graph Representations for Self-Play Learning in Hive

A pre-registered comparison under limited compute

Mohamed Bechir Kefi

Independent research report (not peer-reviewed)

Version v2.1-draft, October 2026

Abstract

Hive is a boardless hexagonal strategy game whose moves are (piece, destination) pairs over an ever-changing set of cells, which makes it, in principle, a natural candidate for graph neural encodings in place of the convolutional grid encodings standard in AlphaZero-style systems. We test that intuition under a pre-registered protocol frozen before any comparison run: one grid CNN and one capacity-matched (+1.5%) relational message-passing network share a rules-validated engine, one action decoder, identical training settings, a frozen three-opponent population and 250 frozen openings, evaluated under two budget readings (equal training examples; equal wall-clock at a pre-registered cutoff) with five independent seeds per arm and truncation reported as its own outcome. The hypothesis is rejected: the graph arm scores lower against two of three opponents under both readings, at twice the wall-clock cost, with its deficit concentrated in converting won positions. Ablations show that the graph arm's typed edge relations are necessary for optimization stability, while its global pooling is dispensable. A second study, pre-registered separately (protocol frozen on 13 October 2026) and run on five new seeds per arm under a warm-started loop with a three-generation replay window, tested the main objection to that verdict: no graph advantage was detected under either reading, every graph–grid interval includes zero (largest upper bounds +0.090 and +0.055), the first study's deficit is no longer visible at equal examples and reduced at equal wall-clock, and the conversion pathology persists in three graph seeds of five at 2.34 times the cost. Both results are pre-registered and reproducible from the released records.

Keywords. Hive; AlphaZero; graph neural networks; representation learning; pre-registration; negative result.

Résumé

Hive est un jeu de stratégie hexagonal sans plateau dont les coups sont des paires (pièce, destination) sur un ensemble de cellules en perpétuel changement, ce qui en fait, en principe, un candidat naturel pour des encodages par réseaux de neurones en graphe plutôt que pour les encodages convolutifs en grille standards des systèmes de type AlphaZero. Nous testons cette intuition sous un protocole pré-enregistré, gelé avant toute exécution comparative : un CNN en grille et un réseau relationnel à passage de messages de capacité appariée (+1.5%) partagent un moteur validé côté règles, un décodeur d'actions unique, des réglages d'entraînement identiques, une population gelée de trois adversaires et 250 ouvertures gelées, évalués sous deux lectures budgétaires (à exemples d'entraînement égaux ; à temps mural égal à un seuil pré-enregistré), avec cinq graines indépendantes par bras et la troncature rapportée comme un résultat à part entière. L'hypothèse est rejetée : le bras graphe obtient un score inférieur contre deux des trois adversaires sous les deux lectures, à un coût en temps mural double, son déficit se concentrant dans la conversion des positions gagnées. Les ablations montrent que les relations d'arêtes typées du bras graphe sont nécessaires à la stabilité de l'optimisation, tandis que son agrégation globale (global pooling) est dispensable. Une seconde étude, pré-enregistrée séparément (protocole gelé le 13 octobre 2026) et menée sur cinq nouvelles graines par bras sous une boucle à démarrage à chaud avec une fenêtre de rejeu de trois générations, a testé la principale objection à ce verdict : aucun avantage du graphe n'a été détecté sous l'une ou l'autre lecture, chaque intervalle graphe−grille inclut zéro (plus grandes bornes supérieures +0.090 et +0.055), le déficit de la première étude n'est plus visible à exemples égaux et est réduit à temps mural égal, et la pathologie de conversion persiste dans trois graines graphe sur cinq, à 2.34 fois le coût. Les deux résultats sont pré-enregistrés et reproductibles à partir des enregistrements publiés.

Mots-clés. Hive ; AlphaZero ; réseaux de neurones en graphe ; apprentissage de représentations ; pré-enregistrement ; résultat négatif.

Status, code and declaration of assistance

Status. Independent research report prepared in view of a doctoral application. It is not peer-reviewed and is not a publication of any institution. Version 2.1-draft, October 2026; the experimental content is final and no number differs from the frozen raw results of 9 October 2026 (first study) and 27 October 2026 (second study).

Code and artifacts. The engine, the training pipeline, both encoders, the frozen configurations, the raw per-game evaluation records and the scripts that regenerate every table and figure of this report form one public repository, https://github.com/mohamedbechirkefi/hive-graph-selfplay, released at tag v2.0-report on 9 October 2026 for the first study (the records archive is attached to that release); the second study's records and this revision accompany the release tagged v2.1-report. A fresh-environment reproduction scenario is described in Appendix G; the provenance of every result is tabulated in Appendix E.

Declaration of assistance by an artificial-intelligence system. This study was executed under a human-as-principal-investigator methodology in which an AI research assistant (Claude, Anthropic) implemented code, ran the computational campaigns, kept the experiment journals and drafted text, under a system of six decision gates reserving every scientific commitment to the human author: the freezing of protocol, opponents and openings, every expenditure of compute, anything made public and the deletion of any data. The human author chose the question, approved each gated decision in writing, verified the conclusions and owns every claim. Section 10 documents this working methodology, its rationale and its record.

Conventions. Scores are means over decided games (win 1, draw 0.5, loss 0) against a fixed opponent population; games stopped at the experimental move cap are *truncated* and are reported as a separate outcome, never as draws. Intervals in brackets are 95% percentile bootstrap intervals with the independent training run (seed) as the resampling unit. The arm trained on the grid encoding is the *grid arm*; the arm trained on the graph encoding is the *graph arm*. A glossary with the French equivalents of all technical terms is given in Appendix F.

Contents

Abstract . . . 1
Résumé . . . 2
Status, code and declaration of assistance . . . 3
1 Introduction . . . 11
1.1 Context: learning to play from self-play, and the cost of representation . . . 11
1.2 The problem: an intuition that cuts both ways . . . 11
1.3 Research questions and hypothesis . . . 12
1.4 What was done . . . 12
1.5 What was found . . . 12
1.6 Contributions . . . 13
1.7 How this report is organised . . . 13
1.8 How to read the numbers . . . 14
2 Background: Hive, self-play learning and state representations . . . 15
2.1 The game of Hive . . . 15
2.2 Formalisation used in the study . . . 16
2.3 Self-play learning in the AlphaZero family . . . 18
2.4 State representations . . . 19
2.5 Compute budgets and the two readings . . . 21
2.6 Evaluating agents at small scale . . . 22
3 Related work and positioning . . . 24
3.1 Programs that play Hive . . . 24
3.2 Network-guided search and the cost of self-play . . . 25
3.3 Representing states and variable action sets . . . 25
3.4 Direct grid-versus-graph comparisons . . . 26
3.5 Evidence standards for few-run comparisons . . . 27
3.6 Summary of the closest works . . . 27
3.7 Positioning . . . 29
4 How and when the system and the study were built . . . 32
4.1 The engine project (July 2026) . . . 32
4.2 The prior self-play loop (July–August 2026) . . . 32
4.3 The research programme (from 9 September 2026) . . . 33
4.4 Timeline . . . 34
5 The Hive engine: design, implementation and validation . . . 37
5.1 Design goals . . . 37
5.2 Architecture . . . 37
5.3 Board representation and hashing . . . 38
5.4 Move generation and rule edge cases . . . 39
5.5 Validation campaign (September 2026) . . . 40
5.6 Throughput profile . . . 43
5.7 Two engine-side fixes recorded before training . . . 43
6 The self-play learning pipeline . . . 45
6.1 The generation loop . . . 45
6.2 Self-play workers and search settings . . . 46
6.3 The training record . . . 47

6.4 The training procedure 47
6.5 Export and inference 48
6.6 The evaluation arena 49
6.7 Seven automated pre-training checks 49
6.8 The pilot of 10 September 2026 50
6.9 A harness defect caught during the pilot 51
7 The two state representations and their networks 53
7.1 What varies between the arms, and what provably does not 53
7.2 The grid encoding 54
7.3 The graph encoding 55
7.4 Capacity matching and measured cost asymmetries 59
7.5 Pinning the encoders: golden and property tests 60
7.6 The two ablation variants 60
7.7 Symmetry augmentation: excluded by decision 61
7.8 Limits of what this chapter establishes 61
8 The frozen opponent population 62
8.1 Why a fixed population rather than a rating against moving targets 62
8.2 The three opponents 62
8.3 Tactical verification before use 63
8.4 Characterisation 63
8.5 The freeze of 9 September 2026 64
9 Experimental methodology: a pre-registered, frozen protocol 67
9.1 Why pre-register, and why freeze, at small compute 67
9.2 Research questions and the hypothesis 68
9.3 Design: two arms, five seeds, ten generations 68
9.4 The two budget readings and the equal-time cutoff 69
9.5 Evaluation: frozen population, frozen openings, pinned search 70
9.6 Statistics and the truncation policy 71
9.7 Exclusion criteria and the record 71
9.8 Ablation design 71
9.9 Resources actually consumed 72
9.10 What was fixed before the first comparison run, and what came after 72
9.11 Question → experiment → result matrix 73
10 Working methodology: gated human–AI research 74
10.1 Why a gated method 74
10.2 Division of labour 74
10.3 The six gates 75
10.4 Scientific principles enforced in every phase 77
10.5 What the discipline caught 77
10.6 What the assistant did not do, and the author's responsibility 80
11 Results: the main comparison (RQ-H1) 82
11.1 The verdict in one paragraph 82
11.2 Reading 1: equal number of training examples 82
11.3 Reading 2: equal wall-clock on the same machine 83
11.4 The arm contrast 84
11.5 How the five-seed analysis relates to the three-seed analysis 84
11.6 Can the move cap rescue the hypothesis? 85
11.7 Where the deficit lives 85

12 Results: cost and efficiency (RQ-H2) 87
12.1 What each arm costs on the study machine 87
12.2 The two readings side by side 88
12.3 Score as a function of time 88
12.4 Resources consumed by the study 88
13 Results: component ablations (RQ-H3) 90
13.1 Design of the three cells 90
13.2 A1: removing the typed relations removes trainability 90
13.3 A2: removing global pooling changes nothing measurable 91
13.4 A1′: the stabilised naive-adjacency variant (two-component supplement) 92
13.5 Summary table H-T3 92
14 Results: failure positions and training behaviour 93
14.1 Three failure positions, selected mechanically 93
14.2 The truncation channel, run by run 94
14.3 What the training metrics show 95
14.4 Per-opponent trajectories 95
15 A second study: the comparison under a warm-started loop with a replay window 97
15.1 Why a second study 97
15.2 What changed, and what did not 97
15.3 Implementation and verification before the freeze 98
15.4 Execution 100
15.5 Results 100
15.6 The two studies side by side 105
16 Discussion and threats to validity 109
16.1 Reading the result 109
16.2 Which explanations are demonstrated, and which are plausible 109
16.3 Relation to prior work 110
16.4 Internal validity: bugs and comparability 110
16.5 Measurement validity: opponents and truncation 111
16.6 Statistical validity: seeds and dependencies 111
16.7 External validity: variant, hardware and budget 111
16.8 A property of the pipeline that bounds the absolute strength reached 112
16.9 What we would do differently 112
17 Conclusion and perspectives 114
17.1 The answer to the research question 114
17.2 Contributions, as demonstrated 114
17.3 Two follow-ups motivated by the data 115
17.4 A method that stands regardless of the sign 115
Bibliography 116
Webography 118
A Annotated position corpora 120
A.1 Critical rules corpus (30 cases) 120
A.2 Tactical verification set (5 cases) 128
B Architectures, decoder and data formats in detail 131
B.1 HiveNet, the grid network 131
B.2 HiveGraphNet, the graph network 132
B.3 The shared action decoder in detail 133

B.4 Record format 134
B.5 Search settings 135
C Frozen constants, hyperparameters, seeds and configurations 136
C.1 Frozen constants 136
C.2 Training hyperparameters 137
C.3 Seed derivation 137
C.4 Configuration of each arm and variant 138
C.5 Measured machine profile 138
C.6 The second study: frozen constants, regime and seeds 139
D Statistical procedures and raw result tables 141
D.1 Score, truncation rate and sensitivity score 141
D.2 The seed-level bootstrap 141
D.3 Raw per-seed tables: same-examples reading 142
D.4 Raw per-seed tables: same-wall-clock reading 142
D.5 Seed-level means, intervals and arm contrasts 143
D.6 The equal-time cutoff and the checkpoint map 144
D.7 Cap-sensitivity bounds 145
D.8 The second study: sensitivity scores, cutoff map and wall-clock 146
D.9 Ablation raw tables 147
D.10 Per-generation evaluation data 149
E Provenance of every result 150
E.1 The evidence chain 150
E.2 From each result to its artifact 150
E.3 Decision index 154
E.4 Claims–evidence register 155
E.5 Raw data, identifiers and frozen hashes 161
F Glossary 164
G Reproduction guide 168
G.1 What is released 168
G.2 The minimal fresh-environment scenario 168
G.3 Full reproduction commands 169
G.4 Expected durations and disk 170
G.5 Identifiers a reproduction must match 171

List of figures

Figure 1 A five-tile Hive position under the two encodings compared in this study. Left, the grid arm's view: the position embedded in a fixed 32×32 frame of hexagonal cells (drawn for orientation), each frame cell carrying 77 feature values, so that tiles appear as activations at frame coordinates. Right, the graph arm's view: the cell graph over the candidate set, in which occupied cells (filled, labelled by colour and tile) and every empty cell adjacent to the hive (open circles), together the set of all legal destinations, are joined by edges typed with the six hexagonal directions. Both views derive from the same engine state without loss; only the structure offered to the network differs. 20

Figure 2 Project chronology, July–October 2026: engine project (12 July), prior self-play loop classified as a demonstration (13 July – 14 August), and research programme (9 September – 9 October) with freezes (population 9 September; protocol and openings 10 September), main campaign (10–17 September), equal-time cutoff (16 September), three-seed analysis (19 September), ablations (20 September – 6 October), seed extension (27 September – 6 October) and final five-seed analysis (9 October); second study (13–27 October): protocol v2 frozen and campaign launched (13 October), ten warm-started runs (13–26 October), equal-time cutoff from the grid clocks (25 October), analysis (27 October). 36

Figure 3 The generation loop shared by both arms. Self-play with Monte-Carlo tree search produces training records; the records train a network; the network is exported to ONNX and drives the next generation's self-play; at fixed generations the exported network is evaluated against the frozen opponent population (B-RND, B-HEU, B-MCTS) on frozen openings. The grid arm and the graph arm differ only in the state encoder and the network body inside the boxes marked "network". 45

Figure 4 Block diagrams of the two networks. Left, HiveNet (grid arm): 3 × 3 convolutional stem, 8 residual blocks of 96 channels with a global-pooling bias in blocks 2 and 5, a flat 28,673-way policy head and a 3-way value head, 1.44 M parameters in total. Right, HiveGraphNet (graph arm): input linear layer to 152 channels, 8 relational message-passing layers with six direction-typed matrices and a global-pooling bias every third layer, a per-candidate policy scorer over destination, source and slot embeddings, and a masked mean‖max value head, 1.47 M parameters in total. Both networks feed the shared action decoder; only the encoder and the body differ. 58

Figure 5 Mean score against the frozen population (average of the three opponents, truncations excluded) as a function of cumulative training wall-clock, for all ten main runs; evaluations at generations 5, 8 and 10 (20, 20 and 100 games per opponent). Blue: grid arm; red: graph arm; one line per seed. The dashed vertical line marks the equal-time cutoff of 18.77 hours. 85

Figure 6 Three commented failure positions, reproduced and verified against their recorded result rows. F1 (left): graph arm, seed 1, against legal-random, opening 2, white; the game reaches the 300-ply cap with the graph arm

materially ahead but unable to complete the surround. F2 (centre): grid arm, seed 2, against the heuristic, opening 2, black; a decided loss in 19 plies to the heuristic's queen-targeting tactics. F3 (right): graph arm, seed 3, against the search opponent, opening 19, white; a 79-ply draw in which the graph arm never generates a winning threat. 93
Figure 7 Final-evaluation truncation rate against the legal-random opponent for each of the ten main runs (final checkpoints; 100 games per run at 400 simulations on the 250 frozen openings; 300-ply cap). Blue: grid arm seeds 1–5; red: graph arm seeds 1–5. Truncation is reported separately from draws throughout the study. 94
Figure 8 Training metrics by generation for the ten main-campaign runs (epoch-1 line of each generation): left, policy top-1 agreement with the MCTS visit distribution on the self-play validation split; right, value accuracy on non-truncated samples. Blue: grid arm; red: graph arm; one line per seed. Ablation runs are excluded. 95
Figure 9 Score against each frozen opponent at the evaluated generations (5, 8 and 10; 20, 20 and 100 games per opponent respectively) for every main run. Blue: grid arm; red: graph arm; one line per seed. Scores exclude truncated games. 95
Figure 10 Second study: share of the 100 final-evaluation games against the legal-random opponent (B-RND) stopped at the 300-ply cap, per run; blue, the five grid seeds (11 to 15, left to right); red, the five graph seeds. Final checkpoints (generation 10) of ten warm-started runs, 100 paired colour-swapped games each on the 250 frozen openings at 400 simulations. 103
Figure 11 Second study: mean score against the frozen population (average of the three opponents, truncations excluded) as a function of cumulative training wall-clock, for all ten warm-started runs; evaluations at generations 5, 8 and 10 (20, 20 and 100 games per opponent). Blue: grid arm; red: graph arm; one line per seed, labelled at its end. The dashed vertical line marks the second study's equal-time cutoff of 17.63 hours. 104
Figure 12 Second study: score against each frozen opponent at generations 5, 8 and 10 (20, 20 and 100 paired games per opponent), one line per run, five warm-started seeds per arm; blue, grid arm; red, graph arm; truncations excluded. Intermediate points carry 20 games and serve the trajectory only. 104
Figure 13 Second study: validation policy top-1 accuracy (left) and value accuracy on non-truncated samples (right) after the first epoch of each generation, as printed by the trainer on the held-out split of the generation's windowed training set; one line per run, five warm-started seeds per arm; blue, grid arm; red, graph arm. Training diagnostics, not strength measurements. . . 106

Part I. Problem and context

1 Introduction

1.1 Context: learning to play from self-play, and the cost of representation

Since AlphaZero, the dominant recipe for learning to play a board game without human examples has been stable: a neural network estimates, for a position, a probability distribution over moves (the *policy*) and an expected outcome (the *value*); a Monte-Carlo tree search guided by the network produces stronger decisions than the network alone; the games the search plays against itself become the training data of the next network (Silver et al., 2018). The recipe is general, but one of its ingredients is not: the way a position is presented to the network. For chess, shogi and Go the answer was natural: the board is a fixed rectangular array, and a position becomes a stack of image-like planes read by a convolutional network. The success of this *grid* encoding rests on a property these games share: every square exists at every moment, at a fixed place.

Hive does not have that property. It is a two-player game of perfect information played with hexagonal tiles and no board: the tiles themselves form the playing surface (the *hive*), the surface changes shape every turn, tiles can climb on top of one another, and a move is best described as "this piece goes to that cell" over a set of cells that exists only relative to the current hive. The natural mathematical object for such a position is a graph rather than an image, with cells as nodes, adjacency as edges and stacks as node attributes; the natural network is then a graph neural network, which reads a variable-size graph directly and needs no frame. Grid encodings can still be applied to Hive by unfolding the hive into a large fixed frame, as every learning system for the game has done so far; but doing so introduces anchoring choices, empty space and a very large discrete move space whose structure the network must learn from scratch.

1.2 The problem: an intuition that cuts both ways

It is tempting to conclude that a graph encoding must learn Hive better. This study was designed because the intuition is genuinely uncertain. On one side, a graph network matches the game's native structure and carries no anchoring artifacts. On the other side, Hive's outcomes hinge on short-range *surround* tactics (a queen bee loses when its six neighbouring cells are occupied), and short-range pattern recognition is precisely the regime in which convolutional locality is strongest. Moreover, a graph over the *occupied* cells alone cannot express the legal destinations of a move, which are properties of empty space; the graph must therefore carry empty candidate cells, and it is not known whether a simple message-passing network, given such a graph, learns the sliding, climbing and connectivity constraints of the rules better than a convolutional network given the unfolded frame.

The prior evidence does not settle the question. The only published AlphaZero-style study of Hive compared five board encodings and found that the choice of encoding measurably changes learning, but all five were grid encodings and the resulting engines remained weaker than plain search (de Goede et al., 2022). The two direct grid-versus-

graph comparisons in other games point in opposite directions: in Hex, under a value-based learner rather than self-play, a graph network dominated on long-range dependencies while the convolutional network stayed sharper at local patterns (Keller et al., 2023); in chess, under self-play, a graph-attention network out-learned convolutional baselines, although from a single training run per model and without seed replication (Rigaux & Kashima, 2024). None of these studies ran both arms under one self-play pipeline, at matched capacity, under a matched budget, with several independent training runs per arm, on Hive. Section 3 develops this positioning.

1.3 Research questions and hypothesis

The study asks one main question and two secondary ones.

- **RQ-H1 (representation).** At comparable training budget, does a graph architecture learn a better policy than a grid architecture for base-game Hive, within an AlphaZero-style self-play pipeline?
- **RQ-H2 (efficiency).** How do the two representations compare when the budget is read as the number of training examples, and when it is read as wall-clock time on the same hardware?
- **RQ-H3 (components).** Which components of the graph architecture carry its behaviour, in particular its direction-typed relations and its global pooling?

The hypothesis under test, **H1**, states that a simple message-passing graph network, receiving the hive as a graph, reaches a higher mean score against a fixed opponent population than a grid convolutional network receiving a 32×32 unfolded frame, at equal training budget. H1 was not presumed true: the protocol (Section 9) states explicitly that a negative or null result is a publishable outcome, and it fixed, before any comparison run, the opponents, the openings, the budgets, the evaluation settings, the statistical unit and the rule by which H1 would be rejected.

1.4 What was done

Both encodings were built behind one shared action decoder on one rules-validated engine; the two networks were matched in capacity to within +1.5% (1.44 M against 1.47 M parameters); five independent seeds per arm were trained under identical self-play settings for ten generations of 500 games each; the comparison was read in two ways, at equal training examples and at equal wall-clock at a pre-registered cutoff, against a frozen population of three opponents (legal-random, a documented heuristic, and a search agent without network at 6,400 simulations) on 250 frozen openings, with truncation treated as a first-class outcome throughout. Two ablations removed one component each of the graph arm at full parity, and a third, explicitly two-component run supplemented the first ablation after it proved untrainable.

1.5 What was found

The hypothesis is rejected. The grid arm scores higher against two of the three opponents under both budget readings, at roughly half the wall-clock cost; the largest upper bound of any interval on a graph advantage is +0.028. The graph arm's deficit concentrates in local tactical conversion: it wins material against the random opponent and

then fails to close, truncating 20–57% of those games where the grid arm truncates almost none. The ablations localise the graph arm's machinery: its direction-typed relations are necessary for optimization itself (removing them made training diverge in every seed), while its global pooling is dispensable.

A second study, pre-registered separately and run after the first had been written up, tested the first objection to that verdict: that re-initialising the network at every generation had kept both arms in a regime too early for a graph prior to pay. Under a warm-started loop with a three-generation replay window, on five new seeds per arm, no graph advantage was detected under either budget reading; every graph–grid interval includes zero, with the largest upper bounds at +0.090 and +0.055; the first study's deficit is no longer visible at equal examples and reduced at equal time; the graph arm's conversion pathology persists in three seeds of five, and its cost ratio rises to 2.34 (Section 15).

1.6 Contributions

Four contributions are claimed, each verifiable from the released repository.

1. **The first controlled, budget-matched, multi-seed grid-versus-graph comparison for Hive**, with both arms under the same AlphaZero-style pipeline, read under two budget equalisations, and a pre-registered negative answer (Section 11, Section 12; raw records released).
2. **A reproducible comparison harness for frameless, stacking games**: a rules engine validated independently of learning; two encoders pinned byte-exactly across two implementation languages; a shared decoder over a variable action set; truncation-aware evaluation; and a frozen-artifact discipline (Section 5 to Section 9).
3. **A component attribution for the graph arm**, in which the typed relations determine trainability and the pooling has a null effect, with an honestly labelled two-component supplement (Section 13).
4. **A second pre-registered comparison under the standard warm-started regime**, with its protocol, equal-time cutoff and seeds fixed before any run, whose null result bounds the first study's negative one and is reported beside it without pooling (Section 15).

The construction of software, however substantial, is not counted as a scientific contribution in itself; it is reported because the result cannot be evaluated without it.

1.7 How this report is organised

Part I states the problem: Section 2 gives the rules of Hive, the formalisation used, and the elements of self-play learning, state representation and small-sample evaluation a reader needs; Section 3 positions the study against prior work. Part II describes how the system was built and when (Section 4): the engine and its validation (Section 5), the learning pipeline (Section 6), the two representations and networks (Section 7), and the frozen opponent population (Section 8). Part III explains the methodology and why it was chosen: the pre-registered experimental protocol (Section 9) and the gated human-AI working method (Section 10). Part IV reports the results by question: the main comparison (Section 11), cost and efficiency (Section 12), ablations (Section 13) and a qualitative analysis of failure positions and training behaviour (Section 14), then the second study under the warm-started regime (Section 15). Part V discusses mechanisms and threats to

validity (Section 16) and concludes (Section 17). The bibliography and webography follow the conclusion. The appendices contain the annotated position corpora, the architectures and data formats in full, every hyperparameter and seed, the statistical procedures with the raw per-seed tables, the provenance of every result, a glossary with French equivalents, and the reproduction guide.

1.8 How to read the numbers

Every score in this report is a mean over *decided* games against one frozen opponent, with win = 1, draw = 0.5 and loss = 0; games stopped at the 300-ply experimental cap are counted separately as *truncations* and are never scored as draws. Every interval is a 95% percentile bootstrap interval whose resampling unit is the independent training run (the *seed*), not the game: a thousand games played by one network are one observation of that network, not a thousand observations of the method. Differences between arms are written as graph minus grid, so a negative number favours the grid arm. All numbers trace to raw per-game records through the chain documented in Appendix E.

2 Background: Hive, self-play learning and state representations

This chapter sets out what the rest of the report takes for granted: the rules of Hive and what makes it awkward for learning machines; the formal objects the study manipulates, including the truncation convention; AlphaZero-style self-play as the system implements it, including a loss that departs from the usual form in two deliberate respects; the two state representations compared; the budget conventions under which they are compared; and the statistics for evaluating agents from a handful of independent training runs.

2.1 The game of Hive

Hive, designed by John Yianni and published by Gen42 Games, is a two-player, perfect-information, zero-sum game of alternating moves. It is played without a board: hexagonal tiles are placed on any flat surface, and the tiles in play define the playing surface. In the base game each player owns eleven tiles: one queen bee, two spiders, two beetles, three grasshoppers and three soldier ants. Three further tiles (mosquito, ladybug, pillbug) exist as expansions; the engine used here implements them, but no result in this report involves an expansion.

2.1.1 Placement

Play begins with one player placing a tile and the other joining a tile of their own to it edge to edge. Afterwards, a tile entering play may not touch a tile of the opponent's colour. The queen bee must enter by each player's fourth turn, and no tile may move before its owner's queen is in play. The study adopts the tournament opening rule (no queen on either player's first turn), which is the convention of the Universal Hive Protocol and of both reference engines against which the rules kernel was validated; the publisher's 2010 rulesheet alone would permit a first-turn queen.

2.1.2 Movement

The queen bee slides one cell. The beetle also moves one cell but may climb on top of the hive; a tile covered by a beetle cannot move, and for placement purposes a stack takes the colour of the beetle on top. The grasshopper does not slide at all: it jumps in a straight line over one or more adjacent tiles to the first empty cell beyond them. The spider slides exactly three cells along the outside of the hive, without backtracking. The soldier ant slides to any reachable cell around the hive. Beetle climbs produce stacks, and a stack's height matters: a tile moving above ground level must also be able to slide, and a *beetle gate* formed by two taller neighbouring stacks blocks it.

Two global constraints cut across these movements. The **One-Hive rule** requires the tiles in play to remain connected at all times: a tile that is the only link between two parts of the hive may not move, whatever its own movement rule would allow. The **freedom-to-move rule** requires a sliding tile to pass physically between its neighbours: a cell whose two flanking cells are both occupied cannot be slid into or out of. Together they

mean that the legal moves of a piece depend on the whole configuration rather than on the piece's neighbourhood alone.

2.1.3 End of the game

The game ends as soon as a queen bee is surrounded on all six sides by tiles of any colour; its owner loses, even when the owner's own move completed the surround. If one move completes the surround of both queens, the game is drawn; the engine additionally declares a draw on threefold repetition. A player who can neither place nor move must pass, and the opponent moves again.

2.1.4 Why Hive is hard for machines

Four properties make Hive an unusual object for the methods of Section 2.3, all of which were developed on games with a fixed board.

Frameless geometry. There is no board tensor to fill in. A position is a finite connected set of occupied cells of arbitrary shape and extent, plus the empty cells around it; its absolute location and orientation carry no meaning. Any fixed-size array encoding must therefore choose an anchor and a frame, a modelling decision with consequences (Section 2.4).

Stacking. Beetles climb, so a cell may carry several tiles, and buried tiles remain relevant: they count toward surrounds and reappear when the beetle leaves.

Branching and length. Kampert et al. (2021) measure an average branching factor that stabilises around 60, roughly twice that of chess, and find that full-width search to depth four already costs tens of seconds on a highly optimised engine, so the bottleneck is evaluation quality rather than search speed. Games are long: the longest search-guided game observed while profiling the engine ran 202 plies, and self-play from a randomly initialised network reached the 300-ply cap of Section 2.2 in 56.7% of games.

Legality is non-local. Under the One-Hive and freedom-to-move rules, whether a move exists depends on the global configuration (articulation points of the hive, gates formed by distant stacks) and on empty space. The same authors found no cheap reliable hand-crafted evaluation: the intuitive count of tiles around the enemy queen is actively misleading when weighted heavily. Hence the case for learning the evaluation, and for asking which representation lets a network learn it most efficiently.

2.2 Formalisation used in the study

Table 1 collects the symbols used from here on.

A **state** s comprises the tiles in play with their cells and stack levels, the tiles still in hand for each player, the side to move, the ply count, and a marker for the tile moved on the previous ply (relevant only under the pillbug expansion; carried for kernel uniformity and part of the hashed position). The engine exposes s over the Universal Hive Protocol, and its rules correctness was established independently of any learning component (Section 5).

An **action** is a pair $a = (p, d)$ of a piece p and a destination cell d, together with one distinguished action, *pass*, which is legal exactly when no move exists. The pair identifies a Hive move uniquely, since the rare cases in which a tile could reach the same cell by two mechanisms produce identical successor states. Pieces are addressed relative to the side to move, so that an action index means the same thing for either colour. The set

of legal actions $A(s)$ is produced by the engine's move generator; both learned architectures receive it and normalise their policies over exactly $A(s)$ through a shared decoder (Section 2.4). The **transition** $s' = T(s, a)$ is the engine's move application.

Outcomes are expressed from the perspective of the player to move in the state being scored: a win is $+1$, a loss -1, a draw 0. Because a player can lose by surrounding their own queen, the implementation computes the sign from the result and the side to move rather than assuming that the previous mover won.

Experimental truncation. Self-play and evaluation games are stopped after 300 plies. A game stopped this way is *not* a draw. It forms a fourth outcome class, *truncated*, carried as such through the data format, the training loss (truncated records contribute no value target, Section 2.3), the match runner, every result table (a separate truncation-rate column) and the statistical analysis. The cap was set from measurement, above the longest observed search-guided game, and was frozen with the rest of the protocol on 10 September 2026, before any comparison run. Treating a capped game as a draw would silently teach the value head that long, unresolved games are balanced; keeping truncation separate makes its frequency a reported quantity and lets the sensitivity of every conclusion to the cap be tested (Section 2.6).

Table 1: Notation used in this report.

Symbol	Meaning
s, s'	a game state and its successor
$A(s)$	the set of legal actions in s (moves, plus pass when no move exists)
$a = (p, d)$	an action: piece p to destination cell d
$T(s, a)$	the transition (the engine's move application)
$z \in \{+1, 0, -1\}$	terminal outcome from the mover's perspective: win, draw, loss
truncated	fourth outcome class: game stopped at the 300-ply cap; never a draw
$p_\theta(a \mid s)$, $v_\theta(s)$	network policy over $A(s)$ and network value in $[-1, 1]$
$P(s, a)$, $N(s, a)$, $Q(s, a)$	prior, visit count and mean value of a search edge
$N(s)$	visit count of the node s
c	exploration constant of the selection rule (1.4)
ε, α	root-noise mixing weight (0.25 in self-play, 0 in evaluation) and concentration (0.15)
$\pi(a \mid s)$	recorded root visit distribution; the policy target
λ_v	weight of the value loss (0.6)
h_i, $h_{i'}$	embedding of graph node i before and after a message-passing layer
$n_i(d)$	the neighbour of node i in hexagonal direction $d \in \{1, \ldots, 6\}$
W_{self}, W_d, b	self-weight, direction-typed weights and bias of a message-passing layer
score	mean of win $= 1$, draw $= 0.5$, loss $= 0$ over non-truncated evaluation games
seed	one independent training run; the unit of analysis

2.3 Self-play learning in the AlphaZero family

2.3.1 Policy-value network and search

AlphaZero (Silver et al., 2018) learns to play from the rules alone. A single network with parameters θ maps a state to a policy $p_\theta(\cdot \mid s)$ over actions and a value $v_\theta(s)$ estimating the outcome for the player to move. The network does not play directly: it guides a Monte-Carlo tree search, and the search, which is stronger than the raw network, generates the games on which the network is then trained. Silver et al. ran 800 simulations per move during self-play and reached superhuman strength in chess, shogi and Go with thousands of specialised accelerators and a single training run per game; the study described here keeps the algorithmic core and changes the scale, the game and the statistics.

Search proceeds by repeated descents from the root. At a node s the implementation selects the edge maximising

$$Q(s,a) + c\,P(s,a)\,\frac{\sqrt{N(s)}}{1+N(s,a)} \tag{1}$$

with $c = 1.4$, which is the PUCT rule of Silver et al. Here $P(s,a)$ is the network prior, normalised over $A(s)$; $N(s,a)$ the number of visits through the edge; $N(s)$ the parent's visit count, taken as at least one; and $Q(s,a)$ the mean of the values backed up through the edge from the parent's point of view (the child's values are negated, its side to move being the opponent). Two implementation details matter for reproducibility. Leaf evaluations are batched: a descent that reaches an unevaluated leaf waits for the batch to fill, and meanwhile a *virtual loss* (one extra visit counted as a loss in both Q and N) is placed on every edge of its path, so that concurrent descents spread out. And an edge whose child has never been visited takes $Q(s,a) = Q(s) - 0.2$, where $Q(s)$ is the parent's own mean value: a *first-play urgency* reduction that discourages opening every child before deepening the promising ones. A new leaf is scored by the network; its value is backed up along the path with alternating sign, and terminal nodes back up their exact outcome. When the simulation budget is spent, the search returns the most-visited root move and the vector of root visit counts.

2.3.2 Exploration in self-play

Two devices make self-play games diverse. At the root, the priors are mixed with noise, $P(s,a) \leftarrow (1-\varepsilon)\,P(s,a) + \varepsilon\,\eta_a$, with $\varepsilon = 0.25$ and η a random vector drawn once per search from an approximate Dirichlet distribution of concentration $\alpha = 0.15$ (approximated in the implementation by normalised transformed uniform draws rather than exact Gamma variates). Silver et al. scaled α to each game's typical number of legal moves (0.3 for chess, 0.15 for shogi, 0.03 for Go); the value used here is their shogi setting. For the first 12 plies of a game the move actually played is sampled in proportion to the root visit counts (temperature one); thereafter the most-visited move is played. Evaluation games apply none of this: $\varepsilon = 0$, no temperature, the most-visited move always played, at 400 simulations per decision, enforced by test and pinned by configuration.

2.3.3 Training targets and the loss as implemented

Every recorded decision yields a training example: the state, the root visit distribution $\pi(\cdot \mid s)$ (the fifteen most-visited actions, renormalised), the legal set $A(s)$, and the game's

eventual outcome z from that state's mover's perspective, or else the marker *truncated*. The policy target is the search distribution rather than the move played, because the search improves on the network's prior and the network is trained to predict that improvement.

Both arms minimise the same loss. For a mini-batch of B records let $K \subseteq \{1, \ldots, B\}$ be the records whose outcome is a win, draw or loss, and let $q_\theta(\cdot \mid s)$ be the three-way softmax of the value head over those three classes. Then

$$\mathcal{L}(\theta) = -\frac{1}{B}\sum_{b=1}^{B}\sum_{a \in A(s_b)} \pi(a \mid s_b)\log p_\theta(a \mid s_b) - \lambda_v \frac{1}{|K|}\sum_{b \in K}\log q_\theta(z_b \mid s_b), \qquad \lambda_v = 0.6. \quad (2)$$

The first term is the cross-entropy between the recorded visit distribution and the network policy, where $p_\theta(\cdot \mid s)$ is a softmax over *exactly* the legal set (the grid arm sets every illegal logit to a large negative constant before the softmax; the graph arm produces logits for legal candidates only), so that illegal actions carry identically zero mass in both arms and training uses the normalisation that search uses. The second term is the cross-entropy of the three-class outcome, averaged over the $|K|$ non-truncated records only (a batch without any contributes nothing) and down-weighted by $\lambda_v = 0.6$. The two departures from a plain joint policy-value loss are the legal masking, which removes a confound between the arms (the grid arm's 28,673-way head would otherwise spend capacity suppressing indices that are never legal), and the exclusion of truncated games from the value target, the training-side half of the truncation convention.

2.3.4 Economies for small budgets

The cost of self-play is dominated by search. Wu (2020) showed in the KataGo project that an AlphaZero-style pipeline can be made far more sample-efficient by a handful of representation-agnostic changes. The one adopted here, identically for both arms, is **playout-cap randomization**: the value head needs many games while the policy head needs deep searches, so most decisions are searched cheaply and not recorded, and a minority receive a full search and become training examples. Wu used full searches of 600–1,000 visits on about a quarter of turns and fast searches of 100–200 visits otherwise; the study uses 128 and 32 simulations and records one decision in four, values that were measured on the study machine, as the protocol chapter explains. Self-play also ends a game by **resignation** when the root value falls below -0.92, recording a loss for the resigning side; in 10% of games, chosen before the first move, resignation is disabled so that the rate at which a "lost" position would in fact have been saved can be audited, a standard safeguard of self-play pipelines.

2.4 State representations

Everything above is indifferent to how a state is handed to the network, which is the variable of this study. Figure 1 shows one position under the two encodings compared.

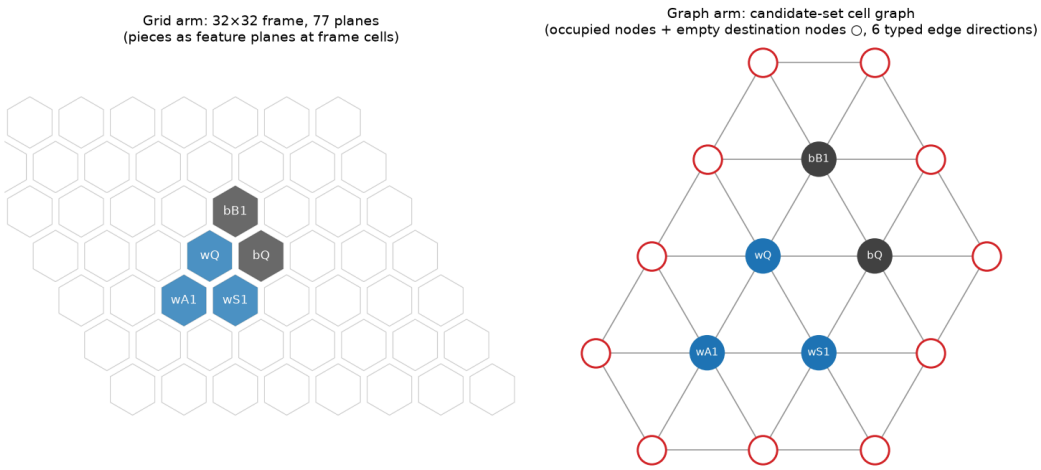


Figure 1: A five-tile Hive position under the two encodings compared in this study. Left, the grid arm's view: the position embedded in a fixed 32×32 frame of hexagonal cells (drawn for orientation), each frame cell carrying 77 feature values, so that tiles appear as activations at frame coordinates. Right, the graph arm's view: the cell graph over the candidate set, in which occupied cells (filled, labelled by colour and tile) and every empty cell adjacent to the hive (open circles), together the set of all legal destinations, are joined by edges typed with the six hexagonal directions. Both views derive from the same engine state without loss; only the structure offered to the network differs.

2.4.1 Stacked planes for convolutional networks

The canonical encoding of the AlphaZero family is a stack of binary or scalar planes over the board array, read by a residual convolutional network, which convolution makes translation-equivariant, with a receptive field that widens with depth. Silver et al. used more than a hundred planes over the 8×8 chess board and 17 planes for Go, and expressed the policy itself as spatial planes. The encoding presupposes a fixed array. For Hive one must manufacture it: the grid arm unwraps the engine's board, stored on a wrapping torus, by breadth-first traversal from any occupied cell, translates the position so that the centre of its bounding box lands on the centre of a 32×32 frame, and writes 77 planes: piece planes indexed by owner, bug type and stack level; planes for pinned tiles, the last-moved tile and both sides' legal placement regions; and constant planes for the side to move, queen liberties, ply and reserves. On axial coordinates the six hexagonal neighbours of a cell are a subset of its 3×3 square neighbourhood, so ordinary 3×3 convolutions cover hexagonal adjacency, two corner weights never meeting a neighbour. The frame is sized so that any base-game position and its ring of candidate cells fit by construction, and the encoder asserts this on every call. No rotation or reflection canonicalisation is applied: the frame fixes translation but not orientation.

2.4.2 Graphs and message passing

The alternative is to hand the network the adjacency structure itself. A graph neural network computes each node's embedding from the node's own features and its neighbours' embeddings, with weights shared across all nodes and all graphs. In the neighbourhood-aggregation scheme of Hamilton et al. (2017), layer k computes $h_v^k = \sigma\big(W \cdot \big[\, h_v^{k-1} \,;\, \mathrm{AGG}\big(\{h_u^{k-1} : u \in \mathcal{N}(v)\}\big)\,\big]\big)$ for every node v, where AGG is a permutation-invariant function of the neighbour set. Because the weights belong to the layer and

not to the node, one trained network embeds a graph of any size or shape. This is the inductive property that a frameless, variable-size Hive position calls for, and that the grid arm's fixed frame only approximates. Plain neighbourhood aggregation ignores edge semantics: a neighbour is a neighbour. In Hive the *direction* of a neighbour matters (a grasshopper jumps along a line; a gate is formed by the two cells flanking one direction), so the graph arm uses **direction-typed relations**, one weight matrix per hexagonal direction. Its message-passing layer updates the embedding of cell i as

$$h_{i'} = \mathrm{ReLU}\left(h_i + W_{\mathrm{self}}\, h_i + b + \sum_{d=1}^{6} W_d\, h_{n_i(d)} \right), \tag{3}$$

where $n_i(d)$ is the neighbour of i in direction d (the term vanishes when no such neighbour exists within the candidate set), W_{self} and b are the self-transformation and its bias, the six W_d are the direction-typed matrices, and the leading h_i is a residual connection. Nodes are the cells of the candidate set, that is, every occupied cell and every empty cell adjacent to the hive, so that each legal destination, including the empty ones, is a first-class object the policy can score; a graph over pieces alone would have nothing to attach a destination to. Tiles are not separate nodes: a cell's features describe its whole stack level by level, so stacking enters as node content rather than graph structure. Global information (side to move, ply, queen liberties, reserves by bug type) enters as a vector broadcast to every node at input and concatenated to the pooled representation at the heads. Because message passing propagates one hop per layer while queen safety depends on the whole hive, every third layer also adds a **global-pooling bias**, after the global-pooling device of Wu (2020): the mean and maximum of the node embeddings, passed through a linear map and added to every node. The value head reads a masked mean-and-max pooling of the final embeddings; the policy head scores each legal (piece, destination) pair from the destination node's embedding, the moving piece's current node (or a learned vector for placements from hand) and an embedding of the piece slot.

2.4.3 What a graph network does and does not provide

A graph encoding grants no invariance for free. The encoding is coordinate-free: no absolute coordinate appears anywhere, so questions of anchoring do not arise. Message passing with direction-typed edges is nevertheless *not* rotation-invariant (rotating the hive permutes the relation types), receptive fields are limited by depth, and no rule of Hive is known to the network. Legality is supplied per position by the engine through the shared decoder's legal mask, identically for both arms, and neither network ever computes it. Any invariance claim about the learned function must be measured, not assumed. Conversely, the grid arm is not without global context: two of its residual blocks also carry a global-pooling bias, so the two arms differ in how adjacency is presented rather than in whether they can see the whole position.

2.5 Compute budgets and the two readings

"At comparable budget" is the clause that gives the research question its meaning, and it is ambiguous. The large self-play papers state budgets in hardware terms. Silver et al. report accelerator counts and wall-clock for a single run per game, with generation and training hardware accounted separately, which permits no cost-normalised comparison between architectures. Wu (2020) instead reports hardware type and count, wall-clock,

self-play games and training samples, and plots strength against cumulative cost rather than iterations. Jones (2021), training AlphaZero-style agents on Hex at a deliberately small-laboratory scale, measures budgets in FLOP-seconds, GPU-hours and samples, draws *compute frontiers* (the best strength attainable per unit of compute), and finds that training and test-time compute trade against each other: about ten times more training compute replaced about fifteen times more search at constant strength. Hence an evaluation must fix the search budget identically across arms, or it confounds representation quality with search.

The study therefore accounts every run in four denominations, namely wall-clock, hardware (machine, cores, accelerator), number of training states consumed, and simulations per decision, and reads the comparison under two equalisations. Under the **same-examples reading** both arms train on the same number of self-play states, generated with the same simulation budget per decision, and the question is which representation extracts more from each example. Under the **same-wall-clock reading** both arms receive the same number of hours on the same machine, and the question is which representation delivers more strength per hour. The two readings can disagree: a graph network that is better per example but slower per example (in self-play inference, in training, or both) can win the first and lose the second. Neither is privileged; both are reported side by side, the cost asymmetry between the arms measured on the study machine and charged rather than equalised away, and both use the same evaluation search budget.

2.6 Evaluating agents at small scale

Measuring the strength of a learned Hive agent raises three questions: against whom, with what score, and with what notion of uncertainty.

Against whom. Hive has no public ladder of reference networks of the kind KataGo was measured against, and no perfect player to anchor a rating scale as MoHex anchors Jones's Hex experiments; win rate against a random player saturates and cannot separate decent agents (Kampert et al., 2021). The study therefore evaluates against a **fixed opponent population** of three agents, frozen on 9 September 2026 before any training run and never retuned, re-versioned or extended afterwards: B-RND, uniform among legal moves; B-HEU, a greedy agent over a documented hand-crafted evaluation with hash-pinned weights; and B-MCTS, the same tree search as the learned agents but with uniform priors and the hand-crafted evaluation as leaf value, at 6,400 simulations per decision. No checkpoint of either arm belongs to the population, so no arm is scored against itself. Every match plays paired games from the same pre-drawn openings with colours swapped, so that all arms, seeds and opponents face identical opening and colour schedules. Elo-style ratings, where printed, are descriptive, relative to this population, and never a headline number.

With what score. The primary metric is the mean **score** against the population, with a win counting 1, a draw 0.5 and a loss 0, computed over non-truncated games only. Truncated games are excluded from the mean and reported as a separate **truncation rate** in every table; a sensitivity column additionally scores truncations as 0.5, and bracketing treatments (truncations as losses, truncations as wins) bound what any alternative cap could change. If the direction of a conclusion changes with the cap, that fragility is itself a reported finding.

With what uncertainty. Agarwal et al. (2021) showed that deep reinforcement learning results from a handful of training runs are routinely over-read: conclusions from point estimates reverse under interval analysis, and uncertainty is badly underestimated below about ten runs. Their remedy, interval estimates from a stratified bootstrap over runs rather than bare means, is adopted with the adaptation they note for a single task: stratification collapses to a bootstrap over seeds. The **seed**, one independent training run, is the unit of analysis. Evaluation games are first aggregated to one score per (seed, opponent) cell; an arm's mean and interval then come from a percentile bootstrap over the seed-level scores, with 10,000 resamples and the 0.025 and 0.975 quantiles as the 95% interval, and the graph-minus-grid contrast resamples both arms' seed sets independently. Games are never pooled as independent observations: thousands of games from one model measure that model precisely but say nothing about the variability between training runs, which is what the comparison is about.

These conventions (frozen opponents, a score that keeps truncation visible, the seed as the unit, intervals over seeds) were fixed in the protocol before any comparison run existed. Section 9 states them in full, with the number of seeds, the pre-registered rejection rule and the equal-time cutoff.

3 Related work and positioning

This chapter is organised by problem rather than by paper: Hive-playing programs; the cost of network-guided self-play; representations of states and variable action sets; direct grid-versus-graph comparisons; and evidence standards for few-run studies. Only works read in full during the study are cited. Table 2 compares the closest of them with the present design, and Section 3.7 states what is reproduced, modified, held identical, and claimed.

3.1 Programs that play Hive

Scholarly work on Hive is thin and almost entirely classical. Kampert et al. (2021) built heuristic minimax and MCTS agents on the BeeKeeper engine and documented the game's size: the average branching factor stabilises around 60, roughly twice that of chess (≈30), and even a highly optimised engine generates only ≈400k nodes per second late in the game. Their most robust finding is negative: the intuitive count of tiles around the enemy queen carries little evaluation signal once its weight is tuned, whereas a distance-to-queen term helps; and the win rate against a random player saturates and cannot separate decent agents. Their agents, like the thesis-level attempts they survey, remained below strong human play; their closing suggestion, to insert a lightweight neural network, is what the next work took up.

AZ-Hive (de Goede et al., 2022) is the closest prior work and the direct motivation of this study: an AlphaZero-style loop on base Hive through the same engine, the hex lattice mapped onto a skewed-axis 26×26 array read by a 4-layer CNN with 12-fold symmetry augmentation, and five board encodings crossed with two action encodings (absolute-coordinate versus tile-relative), each configuration trained for 4 h with 5 repeats. Two results matter here. The choice of encoding has a visible effect: with tile-relative actions the hybrid board encoding clearly wins while the original and simple encodings show no win-rate improvement within 4 h, and with absolute-coordinate actions the ordering changes. And the learned engines stayed weak: after 24 h of training the self-play engine rated 1063 in a BayesElo round robin (400 matches per engine, first-move advantage zeroed) against 1181 for plain MCTS and 1355 for the engine's minimax (untrained 704, random 778). Every encoding in their design space is a dense array over a fixed frame; no graph representation appears anywhere in the paper (confirmed against the full text). The authors estimate a full exploration of that space at tens of training-years, the limited-compute framing this study inherits.

Outside the literature, one public hobby project (hiveGo; Pfeifer, 2018–2026) attaches a feed-forward evaluator to an alpha-beta searcher and a "tiny GNN" to an AlphaZero-style loop for Hive in one code base, with a hexagonal-convolution model flagged as broken. It reports no controlled evaluation (no fixed opponents, game counts, seeds or intervals) and never compares its representation families under matched conditions. It is cited as an existence proof that a graph encoder for Hive has been built, and as a small instance of the gap that runs through the field.

3.2 Network-guided search and the cost of self-play

AlphaZero (Silver et al., 2017/2018) fixed the algorithmic template this study inherits: a policy-value network guides PUCT search in self-play (800 simulations per move), the state is presented as stacked spatial planes (8×8×119 for chess) and the policy itself is expressed as spatial planes, the canonical grid parameterisation from which the grid arm descends. Its evidence style is what a small-compute study must depart from: one training run per game, a budget denominated in hardware (5,000 first-generation TPUs for self-play, 64 second-generation TPUs for training), and evaluation against a single reference engine per game. AlphaZero never varies the encoding at fixed budget.

KataGo (Wu, 2020) showed how compressible that cost is: fewer than 30 V100 GPUs for 19 days (≈1.4 GPU-years, 4.2M self-play games, ≈241M training samples) reached the strength ELF OpenGo had reached with ≈74 GPU-years, a claimed gain of ≈50×, and it set the budget-reporting standard followed here: hardware, wall-clock, games and samples, strength against cumulative cost, and one ablation run per technique. Of its economies this study adopts exactly one, playout-cap randomization, because it is representation-agnostic and applies identically to both arms; the Go-specific input features and auxiliary targets, which would smuggle domain knowledge into one encoding, are not copied. KataGo's global pooling gives a convolutional body global context and partly compensates the locality of a grid encoder, which is why the grid arm keeps a global-pooling bias and the graph arm's own global-pooling bias is ablated rather than assumed.

Polygames (Cazenave et al., 2020) is the strongest representative of the grid paradigm done right: fully convolutional bodies with global pooling make the network independent of board size, and the framework produced the first zero-learning program to beat strong human players at 19×19 Hex. That invariance holds inside the fixed-topology grid paradigm: it scales boards whose adjacency is known in advance. Hive is not among its games, and a boardless, stacking game offers no fixed lattice to scale.

Jones (2021) is the closest methodological licence for conclusions at small scale: AlphaZero-style agents trained on Hex from 3×3 to 9×9 with ≈500 GPU-hours in total showed smooth compute-performance frontiers (≈500 Elo per order of magnitude of training compute within a board size), evidence that small self-play experiments carry extrapolatable signal. The same work supplies a warning the protocol encodes: about 10× more training compute replaced about 15× more search at constant strength, so evaluation visit counts must be pinned or a representation comparison becomes a search-budget comparison. Jones used fully connected networks only; representation was never an axis.

3.3 Representing states and variable action sets

GraphSAGE (Hamilton et al., 2017) grounds the inductive message-passing family the graph arm belongs to: per-layer weights shared across all nodes embed any graph, including graphs never seen in training, and every Hive position is an unseen graph. Vanilla GraphSAGE has no edge semantics and keeps depth small (two layers gave most of its gains); the graph arm adds direction-typed relations (the six hex directions, one weight matrix each) and aggregates over full neighbourhoods, since a Hive cell has at most six neighbours. Depth against budget is a real axis: Keller et al. (2023) needed fifteen message-passing layers for Hex's long-range dependencies.

Pointer networks (Vinyals et al., 2015) justify scoring a variable candidate set: the output distribution is a softmax over compatibility scores of exactly the candidates present in the input, so no capacity is spent on a fixed, mostly illegal output lattice. The shared action decoder is the board-game form of that mechanism: a distribution over the legal (piece, destination) pairs of the position, which both arms emit over the same action space, built, masked, normalised and trained the same way. Because one cannot point at an element absent from the input, the graph contains empty candidate destinations as first-class nodes.

MDP-homomorphic networks (van der Pol et al., 2020) supply the theory for why representation could matter at small compute: equivariance to the joint state-action symmetry shrinks the hypothesis space and sped up learning on CartPole, a grid world and Pong at equal environment steps. The construction needs a known, small, exact group; base Hive has no board edge, so its symmetry group is large (translations combined with the twelve-element hexagonal group). This study builds no equivariant layers. The graph encoding is coordinate-free by construction, but message passing with directional edge types is not rotation-invariant, so no invariance was assumed of the graph network and none is claimed.

3.4 Direct grid-versus-graph comparisons

Three works compare a grid encoder with a graph encoder on a board game. None is on Hive, and no two share game, learning algorithm, budget and statistics.

Keller et al. (2023) ran a parameter-matched CNN-versus-GNN comparison on Hex: a fifteen-layer SAGEConv network (≈487K parameters) against a residual CNN and a U-Net (≈481K parameters), each trained for ≈110 A100-hours at 11×11. The GNN made 1 error on a long-range-dependency suite spanning 8×8 to 25×25 boards where the CNNs made 31 and 36, transferred zero-shot to unseen board sizes and overfit less, while the CNN remained sharper at local patterns in direct play. Three features limit its reach here. The controlled comparison ran under RainbowDQN, and the CNN arm never trained under tree search: the AlphaZero-style run (800 simulations per move, three A100s for ≈6 days) exists for the GNN only. The graph is a Hex-specific Shannon-game reduction in which played cells are contracted away, so every node is an empty cell and actions biject to nodes; the authors state that comparably efficient formulations are not known for other games. And the work is a preprint.

Rigaux and Kashima (2024) report the opposite sign for chess, at a peer-reviewed venue: an edge-featured graph attention network (GATEAU) whose policy is read out over move edges increases playing strength an order of magnitude faster than a same-framework convolutional AlphaZero (learning-speed experiment: eight RTX A5000 GPUs for ≈13 d 16 h), and a model trained on 5×5 Gardner chess transfers to 8×8 (Elo 807±46 with no 8×8 exposure, 1876±47 after 100 fine-tuning iterations). Three features confound a representation claim, each stated by the authors or visible in the paper: every model was trained exactly once, so the reported intervals (delta-method intervals on a Jeffreys-prior Elo estimate) cover match-sampling uncertainty only; “comparable parameters” spans 1.0M for the graph model against 2.2M for the CNN; and the action parameterisation changes together with the representation (an edge readout against a fixed grid head). Chess also gives each edge rich hand-designed semantics (direction, promotion, piece capabilities) that an adjacency graph of Hive does not have.

Ben-Assayag and El-Yaniv (2021) replaced AlphaZero's CNN with a three-layer graph isomorphism network over the square lattice, plus a global dummy node, to train on small Othello, Gomoku and Go boards and play larger ones on one TITAN X: ≈3 days of small-board training beat an AlphaZero trained for up to 30 days on the target board (54% on 16×16 Othello, 84% on 20×20, 100% on 17×17 Gomoku), each figure averaged over five independent runs with standard errors. This is a transfer claim under deliberately asymmetric budgets, not an equal-budget representation comparison, and its actions live on nodes. It nevertheless has the best replication hygiene of the three, and its dummy node is a reminder that shallow message passing needs an explicit global path.

Read together, the three comparisons do not fix a sign for Hive. They make a graph advantage plausible (long-range structure, size transfer, less overfitting) while showing that a CNN can win on local patterns, that the published positive result rests on one run per model with a confounded decoder, and that what the edges encode may matter more than graph structure itself. Hive's play is dominated by local adjacency tactics around the queen, where the Hex evidence favours the grid.

3.5 Evidence standards for few-run comparisons

Agarwal et al. (2021) supply the statistical frame for this regime. Re-analysing published deep-RL results, they show that conclusions from point estimates frequently reverse under interval analysis, that comparisons with 3–5 runs are often not supported by their own data, and that uncertainty is badly underestimated below about ten runs; they prescribe interval estimates everywhere, aggregation per run before any comparison, stratified bootstrap resampling of runs, and a probability-of-improvement statistic in place of bare means. With a single game, stratification collapses to a bootstrap over seeds, so the number of independent training runs is what matters. This study follows the prescriptions with the seed as the resampling unit: one score per seed and opponent, intervals on the graph–grid difference computed over seeds with pairing preserved, and a decision rule written before any run. By contrast, AZ-Hive repeated each 4 h configuration 5 times, Ben-Assayag and El-Yaniv averaged five runs, Keller et al. and KataGo report one main run, and Rigaux and Kashima trained each model once.

3.6 Summary of the closest works

Table 2 places the eight closest works beside the present design.

Table 2: Closest prior work compared with this study (environment, representation, search, budget, evidence protocol, and the difference from the present study).

Work	Environment	Representation	Search	Budget	Evidence protocol	Difference from this study
AZ-Hive (de Goede et al., 2022)	Hive, base game	5 dense hex-lattice array encodings × 2 action encodings, all CNN; 26×26 frame	AlphaZero-style MCTS	4 h per configuration, 5 repeats; one 24 h engine	Loss curves; win rate vs random; BayesElo round robin	No graph arm; conclusions on early learning speed; engines below plain MCTS

Work	Environment	Representation	Search	Budget	Evidence protocol	Difference from this study
Kampert et al. (2021)	Hive, base game	Hand-crafted heuristic features	Minimax (alpha-beta, transposition table), MCTS variants; 0.01–1 s per move	One compute node	Win rate vs random; turns-to-win; Elo round robin	No learning, no representation question; agents below human level
Keller et al. (2023)	Hex 8×8–25×25 (Shannon-game graph)	15-layer SAGEConv GNN (≈487K) vs ResNet and U-Net (≈481K)	RainbowDQN for the comparison; AlphaZero-style (800 simulations) for the GNN only	≈110 A100-hours per model	Long-range test suite; size transfer; one run per model	CNN arm never under self-play search; Hex-specific graph; per-node actions; preprint
Rigaux & Kashima (2024)	Chess (8×8) and 5×5 Gardner	Edge-featured graph attention (GATEAU) with edge policy readout, vs CNN planes	AlphaZero-style MCTS	8×A5000, ≈13 d 16 h	Relative Elo with delta-method intervals; one run per model	Single run, no seed variance; 1.0M vs 2.2M parameters; decoder changes with representation
Ben-Assayag & El-Yaniv (2021)	Othello, Gomoku, Go; small boards → large	3-layer GIN over the square lattice + global dummy node; node policy	AlphaZero-style MCTS, 100 simulations	1×TITAN X; ≈3 d vs 30 d, asymmetric by design	Win rate vs random, greedy and AlphaZero; 5 runs with standard errors	Transfer claim, not an equal-budget comparison; node-only actions; code unreleased
Polygames (Cazenave et al., 2020)	Hex, Havannah, Othello and others; not Hive	Fully convolutional + global pooling (grid)	AlphaZero-style MCTS	Not comparable per run	Competition results; wins vs strong humans at 19×19 Hex	Grid paradigm only; fixed-topology boards
KataGo (Wu, 2020)	Go	CNN planes + global features and pooling	PUCT-MCTS with playout-cap randomization	Fewer than 30 V100 for 19 days (≈1.4 GPU-years)	Bayesian Elo vs an external ladder; per-technique ablations; one main run	Cost-reduction study, not a representation comparison; Go-specific features
Jones (2021)	Hex 3×3–9×9	Fully connected residual networks	AlphaZero-style MCTS	≈500 GPU-hours total, swept	Elo anchored to perfect play; compute frontiers	Representation never varied; no perfect-play anchor exists for Hive
This study	Hive, base game	Grid: 77 planes on a 32×32 frame, residual CNN (1.44M); graph: cell graph with six direction-	Same PUCT-MCTS for both arms; 128/32 simulations with playout-cap randomiza-	One machine; 10 generations × 500 games per run; same-examples and same-wall-	5 independent seeds per arm; frozen three-opponent population; seed-level bootstrap in-	none

Work	Environment	Representation	Search	Budget	Evidence protocol	Difference from this study
		typed relations, message passing (1.47M); shared action decoder	tion; 400 at evaluation	clock readings	tervals; pre-registered rejection rule	

3.7 Positioning

What is reproduced. The AlphaZero-style loop as published (policy-value network, PUCT search with root noise, visit-distribution policy targets, joint policy-value loss; Silver et al., 2018), with KataGo's playout-cap randomization in self-play (Wu, 2020). The grid arm reproduces the AZ-Hive family of encodings in spirit: the hex lattice embedded in a fixed frame and read by a residual CNN (de Goede et al., 2022), here a 32×32 frame unwrapped by breadth-first traversal, centred on the occupied bounding box and carrying 77 planes. Evaluation follows Kampert et al. (2021) and AZ-Hive in measuring against fixed, non-learned opponents. The rules engine and classical search that serve both arms predate the study (built in July 2026 as an engine project and validated against the two reference engines listed in the webography); the study-specific pipeline, both encoders, the graph network, the baselines and the protocol were built from September 2026.

What is modified. The representation axis gains the graph arm that none of the Hive works has: a cell graph over the position's candidate set (every occupied cell plus its ring of empty neighbours, so that every scorable destination is a node), with 56 node features carrying the full stack composition, six direction-typed relations realised as relation-specific weight matrices, a global vector and a global-pooling bias. The two arms are capacity-matched to within +1.5% (1.44M against 1.47M parameters), far tighter than the 1.0M-against-2.2M comparison of Rigaux and Kashima (2024). And the evidence protocol replaces single-run Elo curves with the few-run discipline of Agarwal et al. (2021): five independent seeds per arm, seed-level bootstrap intervals on the graph–grid difference, and a rejection rule written into a protocol frozen on 10 September 2026 against an opponent population frozen on 9 September 2026, both before any training run.

What stays identical across arms. Everything except the state encoder and network body: the engine, the tree search and its budgets (128 or 32 simulations per decision with playout-cap randomization in self-play, 400 at evaluation), the action space over (piece, destination) pairs with its legality mask, targets and loss, the training loop, the data pipeline, the generation schedule, the evaluation harness, the pre-drawn paired openings, the three opponents (legal-random, heuristic, and search at 6,400 simulations without a network), and the machine. Budget is read twice, as the same number of training examples and as the same wall-clock on the same hardware, because a representation that is better per example but slower per example can lose the second reading; both are reported side by side.

What is claimed. No prior work runs a controlled, budget-matched grid-versus-graph comparison for Hive with both arms under one self-play pipeline. The scoped contribution is therefore the first controlled, multi-seed grid-versus-graph comparison for Hive that is budget-matched under both the same-examples and the same-wall-clock reading,

with both arms trained under one AlphaZero-style self-play pipeline, in a frameless, stacking game whose actions are (piece, destination) pairs and whose graph must carry empty candidate destinations. It is deliberately not "the first grid-versus-graph comparison in a board game", which Keller et al. (2023) and Rigaux and Kashima (2024) preclude. The perimeter comprises one variant (base Hive), one grid network and one relational message-passing network at one matched capacity, one machine, frozen opponents, openings and protocol, two budget readings, five seeds per arm. Within it the question is answered in the results chapters; outside it nothing is claimed. The contrast with the positive chess result and the long-range-versus-local asymmetry in Hex is taken up in the discussion chapter.

Part II. Building the system

4 How and when the system and the study were built

The software behind this study has three layers of provenance with different evidential weight. The rules engine, the classical search and the first neural pipeline were built in July 2026 as an engine project aimed at playing strength, not at a controlled comparison. A first AlphaZero-style self-play loop ran on that engine from July to August 2026; it is a demonstration and nothing more. Everything that carries evidence here (corpora, opponent population, protocol, both encoders and networks, campaigns and analyses) was built and run from 9 September 2026 onward (Table 3, Figure 2).

4.1 The engine project (July 2026)

The engine project set out to build a Hive program for all eight game types of the Universal Hive Protocol (the base game and every combination of the Mosquito, Ladybug and Pillbug expansions), strong enough to play automated matches against the two public reference engines, MzingaEngine and nokamute. Its design was staged: a classical alpha-beta engine with a handcrafted evaluation first, as sparring partner and test oracle; then an AlphaZero-style combination of Monte-Carlo tree search and a neural network trained by self-play. The core was written in Rust, training in Python and PyTorch, and in-engine inference used ONNX with the CoreML execution provider at fixed shapes. The plan was sized for a single Apple M4 Pro (14 cores, 24 GB).

The repository's first commits and the project's status note both carry the date 12 July 2026. The note records what that first build day reached: rules kernel, protocol input/output, alpha-beta engine and match arena complete; perft counts matching the published reference tables to depth 7 for all eight game types; differential fuzzing against MzingaEngine and nokamute passed over 27k and 66k positions; conformance 21/21; the neural pipeline (frame, policy encoding, records, data generation, a 1.44M-parameter network, ONNX export) complete; the inference go/no-go passed at 2,246 evaluations per second at batch 128 on CoreML; the network-guided search started. The note also records strength figures (100% against MzingaEngine, 79.2% against nokamute at 1 s per move, +232 Elo); these are the engine project's own numbers, and the study makes no use of them.

4.2 The prior self-play loop (July–August 2026)

A supervised bootstrap from alpha-beta games ran overnight on 12–13 July 2026, and the first generation of the self-play loop started on 13 July 2026. Each generation played 2000 games at 600 full and 150 cheap simulations per decision on 2 threads, taking roughly 150,000–157,000 s; a candidate was gated over 60 games at 2 s per move against the incumbent and promoted above 50%. Nineteen generations completed. Promotions were intermittent (generations 6, 7, 11, 14 and 19 among those logged, gate scores from about 45% to 63%), training metrics fluctuated rather than climbed (policy top-1 roughly

43–47%, value accuracy roughly 58–63%), and generation 19 was promoted at 56.7%. Generation 20 aborted on 14 August 2026 when the disk filled; the loop has not run since.

The loop is classified as a demonstration, not evidence, for reasons fixed in writing on 9 September 2026 before any study work began: single seed, single architecture, no controlled compute budget, no opponent population fixed in advance. It shows that an end-to-end self-play pipeline was built and ran; it shows nothing about representation quality or learning efficiency. The study inherits the engine and its validation evidence, not the loop's results. The author stopped the loop permanently on 9 September 2026, excluded its generation-19 checkpoint from the frozen opponent population the same day, and deleted the original workspace on 10 September 2026, leaving the research copy as the only copy of the prior work.

4.3 The research programme (from 9 September 2026)

9 September 2026: inventory, validation, baselines. The programme opened with an inventory separating working demonstrations from reproducible comparisons, and two framing choices: base game only, and a game reaching the move cap is never scored as a draw. The engine's validation suites were re-run on the study machine, an Apple M1 Pro (10 cores, 16 GB): perft to depth 6 for all eight game types in 8.32 s, conformance 21/21, differential fuzzing at 27,829 positions against each reference engine, plane crosscheck at 240 positions. A 30-case corpus of critical positions, annotated by hand from the publisher's rules and committed before any engine run, gave 29/30 on the first run, with the single disagreement resolved against the corpus, and 30/30 on the second. Seeded random-game invariant checks applied and undid 10.9M transitions without a violation, and a throughput profile measured the costs the protocol would later carry. The three baseline opponents were built, characterised over 100 paired games per pairing, and frozen by the author that day with the search opponent at 6,400 simulations and the prior checkpoint excluded; the author postponed the protocol freeze until a pilot had re-measured the proposed budgets with a study-scale network. The CoreML inference defect found during profiling was fixed the same evening.

9–10 September 2026: pipeline, pilot, freezes, encoders, launch. The study-specific pipeline was built on 9 September (a record format with a distinct truncation outcome, legal-masked training, seven automated pre-training checks); its pilot self-play began at 20:44 that evening, and the pilot's single generation of 300 games, training and evaluation completed on 10 September. On the pilot's measurements the author froze the protocol at version 1.0 on 10 September 2026, with its hash and commit recorded. The same day the grid encoder was re-used with a hardened bounds check, the graph encoder and network were written and property-tested, both encoders were pinned by cross-language golden tests (240 and 160/160 positions), 250 four-ply openings were generated blind and frozen by the author, and the author approved the campaign at its full size, launched at 12:42.

10–19 September 2026: main campaign and three-seed analysis. Six runs (two arms × three seeds) executed in sequence and finished on 17 September 2026 at 22:31. On 16 September, with five of six runs complete, the equal-time cutoff was computed from the grid arm's wall-clocks alone, as pre-registered (18.77 h), before any cross-arm number existed, and the evaluation volume was kept at 100 games per pairing after a precision check. Cutoff-checkpoint evaluations ran on 18 September (about 11 h); that day the rule

that every selected failure position be reproduced exposed a record-loss defect in the match runner, fixed after an audit showed no campaign file affected. The three-seed analysis of 19 September 2026 rejected the hypothesis under the pre-registered rule.

19 September – 9 October 2026: ablations, extension, final analysis. Ablation configurations were prepared on 19 September, with one substitution decided that day; the author approved the ablation campaign on 20 September 2026, both ablations at full three-seed parity with the main comparison. The edge-typing ablation's divergence (non-finite outputs in generation 0 for 3/3 seeds) was diagnosed on 23 September; the pooling ablation's runs completed on 24, 26 and 27 September and were analysed on 2 October. On 26 September 2026 the author approved, with pre-commitments stated before any new run, a symmetric extension to five seeds per arm and the two-component supplement to the diverged ablation. Seeds 4 and 5 ran from 27 September to 2 October, with cutoff evaluations on 6 October; the supplement ran from 2 to 6 October. The final five-seed analysis of 9 October 2026 confirmed the rejection; the report was assembled in English and French the same day, then expanded at the author's request with no number, claim or frozen artifact changed.

9–27 October 2026: release and the second study. On 9 October 2026 the author approved the release: the repository was made public under its tag with the report in both languages, the short article and the records archive. The same day the complementary study was queued and its driver support written: two options of the campaign driver, warm start and replay window, off by default, with a regression test showing that the default command lines reproduce the first study's recorded manifests on four runs and forty generations. A pilot approved on 12 October 2026 ran two generations of 200 games per arm under the new regime and the check battery on its windowed shards, which generalised one check; on 13 October 2026 the author froze protocol version 2.0 (hyperparameter option A, window of three generations, cutoff rule) and approved the campaign, launched at 15:02. Ten runs executed in sequence without incident and finished on 26 October 2026 at 23:00; the second study's equal-time cutoff was computed from the five grid clocks on 25 October, before any score had been read; the cutoff evaluations ran overnight and the analysis of 27 October 2026 found no graph advantage under either reading. This revision of the report followed the same day.

4.4 Timeline

Table 3: Chronology of the engine project and the study, from the repository's commit history, the status note of 12 July 2026, the loop logs, the dated journal entries and the decision records carrying the author's approvals; the last column names the artifact documenting each milestone.

Date	Milestone	Evidence produced
12 July 2026	Engine project: first commits; kernel, protocol, alpha-beta, arena, neural interface, tree search	Status note: perft to depth 7 (8 types), fuzz 27k/66k positions, conformance 21/21, inference 2,246 evals/s at batch 128
12–13 July 2026	Supervised bootstrap from alpha-beta games	Bootstrap logs (demonstration only)
13 July – 14 August 2026	Prior self-play loop: 19 generations; promotions at 6, 7, 11, 14, 19; generation 20 aborted on a full disk	Loop logs; generation-19 checkpoint (preserved, excluded)

Date	Milestone	Evidence produced
9 September 2026	Programme opens: inventory; validation suites re-run; corpus 30/30; 10.9M-transition invariant checks; throughput profile; CoreML fix; baselines characterised and frozen	Inventory; validation entries; population freeze record with hashes
9–10 September 2026	Pipeline; pilot (300 games); protocol frozen v1.0; encoders golden-tested (240, 160/160); 250 openings frozen; campaign launched 12:42	Pilot entry; protocol hash; encoder entries; openings hash; campaign log
10–17 September 2026	Main campaign: 6 runs in sequence, finished 17 September 22:31	Per-run wall-clock logs, checkpoints, evaluation records
16 September 2026	Equal-time cutoff from grid wall-clocks (18.77 h) before any cross-arm comparison; 100 games per pairing kept	Monitoring entry with the derivation
18–19 September 2026	Cutoff evaluations; figures; record-loss defect fixed; three-seed analysis: hypothesis rejected	Result tables, arm contrasts, figures; methodology log entry
20 September 2026	Ablation campaign approved and launched (two ablations × 3 seeds)	One-component configuration diffs
23 September 2026	Edge-typing ablation diagnosed: non-finite from generation 0 in 3/3 seeds; guards added	Divergence entry
26 September 2026	Five-seed extension and two-component supplement approved with pre-commitments	Decision records quoting the pre-commitments
27 September – 6 October 2026	Seeds 4–5 (to 2 October), cutoff evaluations (6 October); supplement (2–6 October); pooling ablation analysed (2 October)	Run logs; ablation entries
9 October 2026	Final five-seed analysis (rejection stands); supplement analysed; report assembled in English and French, then expanded	Final result tables; report
9 October 2026	Repository made public at tag v2.0-report with the report, the short article and the records archive; second study queued; driver options for warm start and replay window written and pinned to the first study by a regression test	Release record; driver test entry
12–13 October 2026	Pilot of the second study (two generations × 200 games per arm; check battery on windowed shards); protocol v2 frozen; campaign launched 15:02	Pilot entry; protocol v2 hash; campaign log
13–26 October 2026	Second campaign: ten warm-started runs in sequence, finished 26 October 23:00; equal-time cutoff of the second study (17.63 h) computed from the five grid clocks on 25 October before any score was read	Per-run clock logs, manifests, evaluation records; cutoff entry
26–27 October 2026	Cutoff evaluations (21 sets); second-study analysis: no graph advantage detected; report revised	Second-study result tables and figures; analysis entry

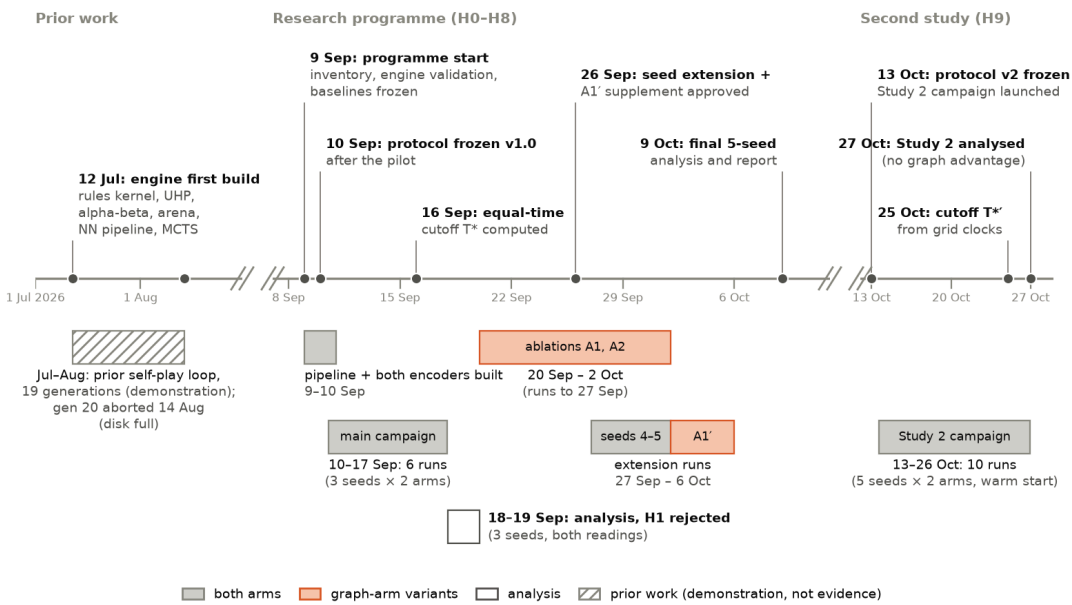


Figure 2: Project chronology, July–October 2026: engine project (12 July), prior self-play loop classified as a demonstration (13 July – 14 August), and research programme (9 September – 9 October) with freezes (population 9 September; protocol and openings 10 September), main campaign (10–17 September), equal-time cutoff (16 September), three-seed analysis (19 September), ablations (20 September – 6 October), seed extension (27 September – 6 October) and final five-seed analysis (9 October); second study (13–27 October): protocol v2 frozen and campaign launched (13 October), ten warm-started runs (13–26 October), equal-time cutoff from the grid clocks (25 October), analysis (27 October).

5 The Hive engine: design, implementation and validation

Every arm, opponent and evaluation in this study shares the engine. A self-play study trains on positions the engine itself generates and scores, so a rules defect would teach both arms the same wrong game and silently invalidate the comparison rather than merely adding noise. The engine was built in July 2026 as an independent project (Section 4) and validated along four independent lines on 9 September 2026, before any baseline or training work. This chapter covers its goals, architecture, board representation, move generation, validation, throughput, and the two fixes recorded before training.

5.1 Design goals

The engine project fixed five goals. The first was rules fidelity: all eight game types of the Universal Hive Protocol, with exact agreement to the reference engine's published perft tables and per-ply agreement of legal-move sets with two independent reference engines as acceptance criteria. The second was protocol interoperability over standard input and output, so that MzingaEngine and nokamute can be driven as subprocesses. The third was staged strength: a classical alpha-beta engine with a handcrafted evaluation first, as sparring partner and test oracle, then AlphaZero-style tree search guided by a self-play-trained network (Silver et al., 2018). The fourth was single-machine feasibility, meaning an allocation-free Rust kernel, fixed-shape ONNX exports for the CoreML execution provider, and an inference go/no-go before any self-play. The fifth was verification built in from the start: unit tests per rule, perft fast in the standard suite and deep nightly, cross-engine differential fuzzing, make/unmake hash property tests, and a tactical regression suite.

Two further properties mattered more for the study than strength: the rules kernel performs no input or output and holds no global state, so one kernel referees arena games, generates self-play and serves the protocol; and every stochastic component is seeded, so any recorded game replays deterministically.

5.2 Architecture

The engine is a Rust workspace of small crates with one-way dependencies: the rules kernel at the bottom, protocol and evaluation above it, the two search backends above those, and the arena, self-play generators and shipped binary on top (Table 4). Three choices deserve a word. The classical search has no null-move pruning, because Hive's forced-pass states make tempo-based pruning unsound in exactly the positions that matter, the queen races; its quiescence covers only Hive's forcing moves, those landing on or beside the enemy queen. The network-guided search is generic over an evaluator with one output contract, so the handcrafted evaluation with uniform priors (the study's no-network opponent), the grid network on CoreML and the graph network on the CPU provider all run through the identical tree search. The self-play generator implements

the KataGo economies of Wu (2020): playout-cap randomization, Dirichlet root noise, early-move temperature, and resignation with an audit fraction. Root noise stays off in every other backend.

Table 4: Components of the engine, their roles, and the properties the study relies on. One rules kernel generates, referees and serves every game in this report; the three frozen opponents and both study arms are backends of the same binary.

Component	Role	Properties the study relies on
Rules kernel	State, move generation, make/unmake, hashing, notation, perft, canonicalisation	No input/output, no global state; all eight game types; referees every game
Protocol server and client	Universal Hive Protocol over standard streams; subprocess driver for other engines	Every response block ends with `ok`; any backend plugs in behind one trait
Handcrafted evaluation	Static evaluation, negamax convention	Queen liberties dominant, piece activity, tempo; weights hash-pinned for the heuristic opponent
Classical search	Principal-variation search, iterative deepening	Lock-free shared transposition table (XOR-validated), lazy SMP, killer/history ordering, late-move reductions, aspiration windows; no null move; queen-targeting quiescence
Neural interface	32×32 frame of 77 planes; 28,673-way (piece, destination) policy index; training records (112-byte one-hot, 176-byte with visit distribution, study version with model stamp, legal-index list, truncation outcome)	Byte-identical to the Python decoder (golden-tested)
Network-guided search	Batched PUCT tree search over an evaluator	Replay-based descent, virtual loss; three evaluators, one output contract; noise off unless self-play enables it
Match arena	Paired colour-swapped games from seeded random openings or a fixed file (pair i plays line i)	Every reply validated by the kernel; 300-ply cap a separate outcome, excluded from the score and reported as a rate
Self-play generators	Alpha-beta bootstrap data; tree-search self-play	Playout-cap randomization, root noise, temperature, resignation with audit; seeded
Shipped binary	Protocol front-end over all backends	Alpha-beta by default; tree search with heuristic, grid or graph network; legal-random; seed and simulation flags

5.3 Board representation and hashing

Hive has no board: pieces define the playing surface, and a hive drifts across the plane as it grows. The engine nevertheless uses a fixed 64×64 grid of cells in axial coordinates, wrapping into a torus, on a counting argument: a hive contains at most 28 pieces, so it spans at most 28 cells along any axis, and on a 64×64 torus every local query (adjacency, slides, jumps) is indistinguishable from the same query on an infinite plane. Each cell stores its top piece, stacks live in a side table, and the maximum stack height is 7 (a ground piece under 4 beetles and 2 mosquitoes).

Because the hive never leaves the torus, coordinates never need renormalising during a game, which keeps the position hash fully incremental: make and unmake XOR the moved pieces' keys in and out, and no re-centring ever forces a recomputation or inval-

idates transposition-table entries. Translation symmetry is therefore irrelevant inside a search tree; canonicalisation under translation and the 12 hex symmetries is isolated at the boundary with the network frame and the opening book. The Zobrist hash is computed by a mixing function rather than lookup tables (a full table would be about 6 MB; mixing a packed (piece, cell, level) key costs about 2 ns with zero memory) and folds in the side to move, the last-moved piece and an early-game phase component. Repetition detection drops the phase component, so the same arrangement reached at different plies compares equal, but keeps side to move and last-moved piece, the stun rights, much as chess repetition keys keep en-passant rights; a third occurrence is a draw unless the game is already decided by surround.

5.4 Move generation and rule edge cases

Move generation follows the reference engine's conventions, because those are what its published perft tables count: the queen may not be placed on either player's first turn (the tournament opening rule), for identical in-hand bugs only the lowest ordinal is placeable, a player with no placement and no movement receives the single move `pass`, and a finished game generates no moves.

Sliding and gate legality is evaluated on a *lifted* view of the board, the occupancy with the mover removed from its origin cell. On that view the **freedom-to-move** rule for a ground-level slide step reads: the destination must be empty, and of the two cells adjacent to both origin and destination exactly one must be occupied; with zero the piece would detach in transit, and with two the gate is too narrow to pass. The **height gate** for climbing moves (beetle and ladybug steps, both legs of a pillbug throw) blocks a step if and only if both common neighbours are strictly higher than both the level the mover starts above and the destination stack; the above-ground beetle gate of the tournament FAQ falls out of this rule without a special case. The **One-Hive** rule is an articulation-point computation: a ground-level piece may be lifted if and only if its cell is not a cut vertex of the occupied-cell graph, found by a lowlink depth-first search once per move generation; tops of stacks are exempt, since the cell keeps its node, and a pinned pillbug may still throw, since it does not itself move.

The **stun rule** of the Pillbug expansion, under which a piece moved on the previous turn may neither move nor be moved, needs a single field: the piece physically moved or placed on the previous ply. A piece is stunned if and only if it is that piece and belongs to the side to move; a pillbug may not throw that piece; the stun lapses after one opponent turn; placement also sets the field. The **mosquito** copies each adjacent bug type and is a pure beetle on top of the hive; the **ladybug** moves exactly two steps on top and one down. Because mosquito multi-copy, ladybug multi-path and walk-versus-throw can derive the same move more than once, the generated list is sorted and de-duplicated; if it is then empty, `pass` is generated.

5.4.1 Why a (piece, destination) pair identifies a move

The policy spaces of both arms and the de-duplication above rest on the claim that (piece, destination cell) determines the successor state. A placement takes the piece from hand to an empty cell, so the pair fixes the result. A movement starts from the piece's current location, which the state determines uniquely, and ends on top of the destination stack, whose height the state also determines; so the pair fixes the board.

The only way two distinct actions share a pair is a pillbug throw of a piece to a cell it could also have walked to. Both produce the same board, so the successors could differ only in the record of which piece moved last, and the engine stores only *which* piece moved, not how. That suffices: at the start of a player's turn, if the last-moved piece is their own, the opponent can only have thrown it, so it is frozen this turn; if it is the opponent's, the player's pillbug may not throw it either way, and nothing else depends on the distinction. Hence the walk–throw collision yields identical states, and a policy head indexed by (piece, destination) plus pass loses nothing.

5.5 Validation campaign (September 2026)

All validation evidence was re-established on 9 September 2026 on the study machine, an Apple M1 Pro (10 cores, 16 GB), with the reference engines freshly fetched (MzingaEngine v0.16.0, the protocol's reference implementation, and nokamute 1.0.3 built from source), before any baseline or training work. Four lines of evidence bound four different failure modes; a fifth check pins the data path to the training code.

5.5.1 Perft against published tables

Perft counts legal-move paths to a given depth under the reference engine's conventions, so its published tables serve as ground truth; it is exhaustive within its depth, and any discrepancy in move generation (a missing move, an extra move, a wrong stacking rule) shifts a count. The standard suite checks depth ≤5 for all eight game types on every build, a dedicated run checks depth 6, and the nightly run checks depth 7. On 9 September 2026 the standard suite passed and the depth-6 run completed in 8.32 s with every count matching; depth 7 was not repeated that day, so the study claims depth ≤6, with depth 7 last matched in July 2026 (Table 5, Table 6).

Table 5: Perft node counts for the base game, depths 1–6, as published for the reference engine and reproduced exactly by the rules kernel on 9 September 2026 (no queen on a player's first turn; identical in-hand bugs counted once; forced pass counts as one move; a finished game generates none).

Depth	Nodes
1	4
2	96
3	1,440
4	21,600
5	516,240
6	12,219,480

Table 6: Perft node counts at depths 4–6 for all eight game types (M = Mosquito, L = Ladybug, P = Pillbug), as published for the reference engine and matched by the rules kernel; the Mosquito and Ladybug variants share counts through depth 5 and separate only at depth 6, hence the depth-6 run.

Game type	Depth 4	Depth 5	Depth 6
Base	21,600	516,240	12,219,480
Base+M	45,414	1,252,800	34,233,432
Base+L	45,414	1,252,800	34,233,672
Base+P	45,414	1,255,932	34,395,984
Base+ML	86,400	2,725,920	85,201,200

Game type	Depth 4	Depth 5	Depth 6
Base+MP	86,400	2,730,888	85,492,248
Base+LP	86,400	2,730,240	85,457,136
Base+MLP	151,686	5,427,108	192,353,904

5.5.2 Reference-engine agreement

The protocol conformance harness shipped with nokamute passed 21/21. More stringently, differential fuzzing plays seeded random games while asserting, after every ply, *set equality* of the legal-move sets returned by the engine and by a reference. At seed 20260909 with 25 games per game type, both references agreed on every one of 27,829 positions (the identical count is expected: same seed, same games); the nightly run repeats this at 200 games per type against nokamute and 100 against MzingaEngine. Agreement with two independent code bases bounds a misreading shared with the engine, not one shared by the whole community of implementations, which the next line addresses.

5.5.3 The hand-annotated critical corpus

Thirty critical positions were annotated by hand from the publisher's rules: the base-game rulesheet cited by page, the Pillbug sheet, and the World Hive Tournaments rules FAQ for above-ground gates and stun clarifications. The tournament opening rule was declared as the study's convention. Every expectation was written from the rules text, never from or against the engine, and the corpus was committed before the first run. Each case is a protocol move sequence from the empty board, hence reachable and independently checkable, with an expectation of one of seven kinds: the exact move set of a focal piece (possibly empty), a single move legal or illegal, every move a placement of a named piece, no movement moves, forced pass, or a terminal state. The runner drives the engine over the protocol, resolves move strings to cells with a geometry-only parser that knows no game rules, and compares (piece, destination) sets. Table 7 lists the coverage; the full corpus with every justification is in Appendix A.

Table 7: Coverage of the 30-case hand-annotated critical corpus by rule area, with case identifiers as used in the corpus appendix. Expectations were written from the publisher's rules and committed before any engine run; three Pillbug cases guard the shared kernel's stun logic and lie outside the study's base-game perimeter.

Rule area	Cases
Opening and placement rules	C001–C005
Sliding, freedom to move, gates	C006–C012
Beetle and stacking, beetle gate above ground level, stack colour	C013–C018, C028
One-Hive, rings, grasshopper cut point	C019–C020, C027
Terminal states: win, draw by simultaneous surround	C021–C022
Forced pass	C023
Pillbug ability, stun, just-moved guards (kernel guards, Base+P)	C024–C026
Spider step count; queen move enumerations	C008, C029; C006, C011, C030

The first run returned 29 passes and one setup error. The disputed case was the simultaneous-surround draw: its setup moved the black queen one step with both flanking cells empty, so the hive would have been "left unlinked while the piece is in transit", which is precisely the illegal-move example the rulesheet gives for the One-Hive rule; the engine's

rejection was the rule-mandated behaviour. The disagreement was resolved *against the corpus*: the setup was rerouted through an intermediate cell so every step keeps an occupied flanking cell, the expected outcome (a draw) was left unchanged, and the correction was documented in the case file. The second run returned 30/30. The record of the corpus being wrong is kept deliberately: committed-before-run expectations cut both ways. Two limits are declared: no external review by a Hive-literate reader yet, and 30 cases sit at the bottom of the 30–50 range the research plan proposed; the strongest cases are the enumerations (13 destinations in one sliding case, 6 in a beetle case).

5.5.4 The tactical suite

A second hand-annotated set verifies the *search* rather than the rules. It holds five positions with a known good outcome: a win in one by a walking piece and by a jumping piece, each for both colours, and a position where the mover must avoid completing its own queen's surround. Since several pieces can often reach the winning cell, expectations are destination-based: the searcher's move must, or must not, land on a named cell. The network-free tree search solved 5/5 at 400, 1,600 and 6,400 simulations. Value signs under player alternation are covered by automated tests (the evaluation negates when the side to move flips; mate-in-one scores and root values are positive for the mover, both colours). One setup slip, a piece-ordering error in one case, was caught at the first run because the cases had been committed beforehand; the engine was right, so the setup was corrected and the incident recorded. The same external-review limit applies.

5.5.5 Random invariant checks

Seeded random games across all eight game types check four invariants after every transition: every generated move is accepted by the apply path and undo restores the exact game string; the hive remains connected; the protocol game string round-trips to the same position, result and sorted legal-move set; and pass is played exactly when the move list is empty. A violation would halt with game type, seed, game, ply and game string for archiving as a regression case; none arose (Table 8). The default run is in the standard suite on every build, the deep run in the nightly script.

Table 8: Seeded random-game invariant checks of 9 September 2026 over all eight game types (apply/undo exactness, hive connectivity, game-string round trip, pass exactly when no move exists). Deterministic given the seeds; durations on the study machine.

Run	Games	Plies walked	Generated moves applied and undone	Violations	Duration
Default (every build)	4 per type × 8 types, ≤150 plies	4,029	253,936	0	2.7 s
Deep (nightly)	3 rounds × 20 per type × 8 types, ≤400 plies	161,546	10,665,686	0	214 s

5.5.6 Cross-language data-path check and scope

The grid arm's plane encoder in Rust and the decoder in the Python training code are required to be byte-identical; a golden-file crosscheck over 240 positions confirmed exact agreement on 9 September 2026 and runs nightly. The graph encoder built on 10

September received the same treatment, with 160 of 160 positions across all eight game types agreeing exactly on the first run.

These lines bound different failure modes (exhaustiveness at depth, agreement with references, fidelity to the rules text, search correctness, invariant stability), but none proves full-game perfection, and code coverage is deliberately not claimed as correctness evidence. The stop rule that an incorrect engine suspends all training-side work was not triggered; baseline and pipeline work proceeded the same day.

5.6 Throughput profile

Before any simulation budget or move cap was fixed, the engine's costs were measured on the study machine with short runs on the existing binaries, the prior loop's generation-19 checkpoint serving only as a realistic inference workload (Table 9).

Table 9: Throughput profile measured on 9 September 2026 on the study machine (Apple M1 Pro, 10 cores, 16 GB), release builds, profiling seed 20260909, game-length seeds 1001–1030 (lengths include 6 random opening plies); network inference measured with the prior loop's generation-19 checkpoint (77×32×32 input) as workload.

Measurement	**Value**
Perft(5), base game	516,240 nodes in 1.36 ms (≈380 MN/s)
Perft(6), base game	12.2 M nodes in 26.5 ms (≈461 MN/s)
Alpha-beta, opening position, depth 10	7.20 Mnps
Alpha-beta, midgame position, depth 8	4.67 Mnps
Alpha-beta decision at depth 4	≈40 ms, single thread
Full games at depth 4 (6 threads, 100 games)	22 s wall, 4.6 games/s, 4,397 recorded positions
Game length at depth 4 (30 seeded games)	mean 55.0, median 39, 90th percentile 80, maximum 202 plies; none reached the 300-ply cap
Network inference, Python ONNX package, CPU provider, batch 1	23.5 ms per evaluation (43 evaluations/s)
Network inference, Python ONNX package, CoreML provider, batch 1	2.62 ms per evaluation (382 evaluations/s)
Network inference, Rust bindings, CoreML provider	crashed (fixed below)
Protocol round trip from Python	21.7 µs per `validmoves` (46,051 requests/s); 21.6 µs per play–undo pair

Three conclusions followed, each a measured proposal later confirmed by the pilot and frozen in the protocol. Legal-move generation is nowhere near a bottleneck; network evaluation dominates every decision, and the CoreML path decides feasibility: at 2.62 ms per evaluation a 128-simulation decision costs about 0.34 s and a 64-simulation decision about 0.17 s on one thread, whereas the CPU path is about 9× slower. The observed game lengths supported a 300-ply cap, above the observed maximum of 202 yet bounded, with truncation as its own outcome. Lastly, a protocol round trip at 21.7 µs is three to four orders of magnitude below any per-decision cost; since self-play generation lives entirely in Rust and exchanges data with Python through binary shard files, no in-process binding was built.

5.7 Two engine-side fixes recorded before training

Profiling surfaced two defects, each recorded as a finding first and fixed afterwards in the study's own code rather than silently patched.

CoreML execution provider. The Rust inference path panicked on the study machine with the message "Unable to compute the prediction using a neural network model", while the Python ONNX package ran the same model on CoreML at 2.62 ms per evaluation (55 of 64 nodes on CoreML, 5 partitions). The message names the cause: the Rust bindings (version 2.0.0-rc.12) default to CoreML's legacy *NeuralNetwork* format, whereas the Python package uses *MLProgram*. Requesting MLProgram, a one-option change, restored CoreML inference, verified on 9 September 2026 with the self-play generator: at 600 full and 150 cheap simulations, 2 games on 2 threads took 44 s (≈44 thread-seconds per game, against ≈320 on the CPU fallback and ≈150 on the prior loop's working path); at the study's eventual 128/32 budget, 8 games on 4 threads took 24 s, about 3 s of wall-clock and 12 thread-seconds per game, which put a 2000-game generation at roughly 1.7 h. The full test suite, the tactical suite (5/5) and the critical corpus (30/30) were green after the change. These probes are feasibility anchors; the pilot re-measured with the study's own networks.

Truncation is not a draw. Both prior-work generators capped games at 300 plies and mapped the cap to a draw, and the tree search's internal rollouts capped at 120 plies. Scoring a truncated game as a draw would let the cap manufacture or erase an advantage undetected, so the study's convention, fixed on 9 September 2026 before any measurement code was written, makes truncation a fourth outcome category everywhere: excluded from the primary score, reported as a separate rate, scored 0.5 in a sensitivity check. The prior-work code was left as a recorded finding; the study's record format carries a distinct truncation value in the outcome byte, the arena excludes truncated games from the score and reports the rate, and training excludes truncated records from the value loss. All of this landed with the study's record format on 9–10 September 2026, before the pilot.

With the engine validated, profiled and corrected on these two points, the study-specific pipeline, the opponents and the two encoders were built on top of it; the following chapters describe them.

6 The self-play learning pipeline

Both arms of the comparison are trained and evaluated by one pipeline, built in September 2026 on top of the engine of Section 5. It is deliberately ordinary, an AlphaZero-style loop in the sense of Silver et al. (2018) with the compute-saving devices of Wu (2020), since its only requirement is to be identical for the two arms. The search that generates games, the record that stores them, the loss, the export path, the evaluation arena and the automated checks are all shared; only the state encoder and the network body (Section 7) differ between the grid arm and the graph arm. Figure 3 gives the overview; the sections below describe each stage, then the pilot of 10 September 2026 that validated the loop end to end, and the one harness defect it caught.

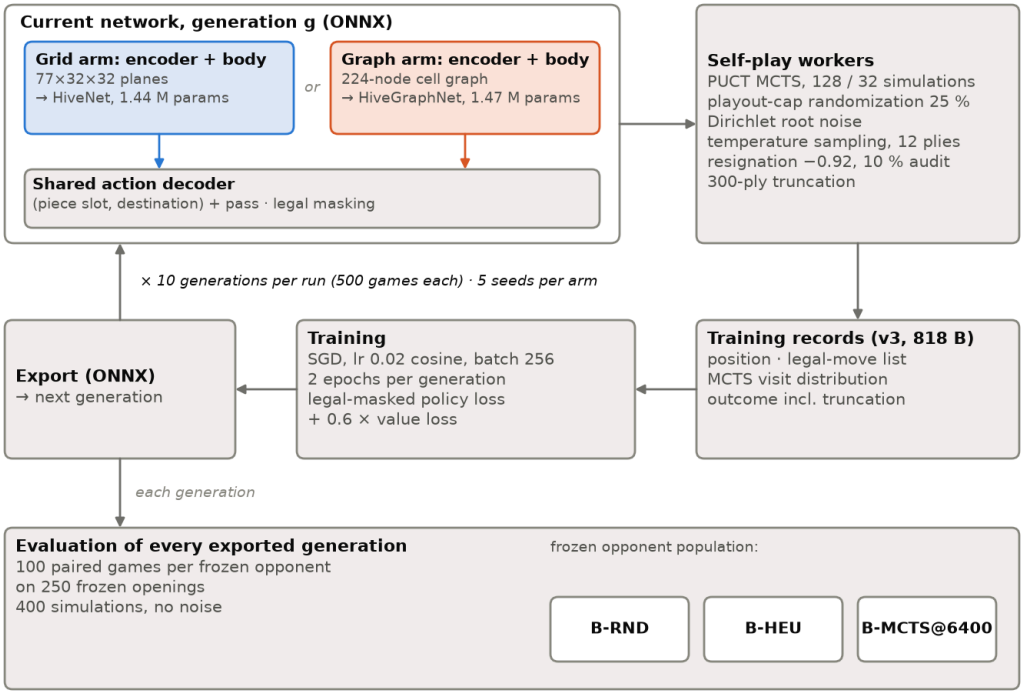


Figure 3: The generation loop shared by both arms. Self-play with Monte-Carlo tree search produces training records; the records train a network; the network is exported to ONNX and drives the next generation's self-play; at fixed generations the exported network is evaluated against the frozen opponent population (B-RND, B-HEU, B-MCTS) on frozen openings. The grid arm and the graph arm differ only in the state encoder and the network body inside the boxes marked "network".

6.1 The generation loop

A training run is identified by an arm and a seed. The generation-0 network is a seeded random initialisation, exported to ONNX before any game is played; the base seed of run s is $100,000 \times s$, and generation g uses base seed $+ g$ for both self-play and training, so no two runs share a random stream. In the configuration under which all study runs executed, a run comprises 10 generations of 500 self-play games each, with four worker threads, on one machine.

Each generation proceeds as in the pseudo-code below. The current network plays the generation's games against itself; the positions searched at the full simulation budget become records, stamped with the generation number and a hash of the generating network. A network is trained on those records, exported, and becomes the current network; a checkpoint is kept at every generation so that the same-wall-clock reading (Section 9) can later select the last checkpoint completed before the equal-time cutoff. Generations are numbered 1 to 10 throughout this report; the run logs and the pseudo-code below count them from 0. After the fifth and the eighth generation, and after the tenth and last, the exported network is evaluated against the frozen opponent population. The driver passes neither a warm-start nor a resume checkpoint to the trainer: the network of generation g is a fresh seeded initialisation trained for two epochs on the records of generation g alone, which were produced by the network of generation $g-1$ (by the random initialisation when $g = 0$). There is no replay window across generations.

```
run(arm, seed s):
    θ ← seeded random initialisation (base seed 100,000 × s), exported to ONNX
    for g in 0 … 9:
        D_g ← SELF-PLAY(θ, 500 games; seed base + g)
                 128 / 32 simulations, 25 % of decisions at 128 (recorded)
                 Dirichlet root noise ε = 0.25; visit-proportional sampling for
12 plies
                 resign below −0.92 (10 % of games never resign); truncate at 300
plies
        θ ← TRAIN(fresh seeded initialisation, D_g; seed base + g)
                 2 epochs, batch 256, SGD lr 0.02 (cosine), momentum 0.9, wd 1e-4
                 loss = masked policy cross-entropy + 0.6 × value cross-entropy
                 (truncated records excluded from the value term)
        EXPORT(θ) → ONNX, static shapes; checkpoint kept; wall-clock of g logged
        if g ∈ {4, 7} or g = 9:
            EVALUATE(θ) vs B-RND, B-HEU, B-MCTS on the frozen openings
                 400 simulations, no noise, argmax; 20 games/opponent (g ∈ {4,
7}), 100 (g = 9)
```

The seven-check battery of Section 6.7 is a separate command run against real shards. The pilot driver ran it after every generation's self-play; the campaign driver that executed the frozen matrix did not re-run it per generation, relying on the battery having passed on real shards and on the engine-side tests that run with every build.

6.2 Self-play workers and search settings

Self-play runs entirely in the Rust engine; the Python side never sits in a per-move loop and receives its data as binary shard files. The search is PUCT Monte-Carlo tree search with exploration constant $c = 1.4$, batched leaf evaluation and exact back-up of terminal values. Four devices, applied identically to both arms, shape the games into training data.

Playout-cap randomization (Wu, 2020). Each decision is searched at the full budget of 128 simulations with probability 0.25 and at the cheap budget of 32 simulations otherwise; only the full-budget decisions are recorded. The device trades policy-target quality on a quarter of the moves for many more games per hour, which is what the value head needs. The 128/32 pair was not taken from the research plan: it was measured on the study machine after the CoreML execution provider was restored on 9 September 2026 and confirmed with a study-scale network in the pilot.

Exploration. Dirichlet noise with $\varepsilon = 0.25$ is mixed into the root prior, and for the first 12 plies the move is sampled in proportion to the root visit counts; from ply 12 onward the most-visited move is played. The evaluation path carries $\varepsilon = 0$ as its code default and no sampling, a separation enforced by test (check 6 below).

Resignation with audit. A side resigns when the root value from its perspective falls below -0.92; in 10 % of games, drawn at the start of the game, resignation is disabled so that the threshold can be audited against played-out outcomes.

Truncation. A game that reaches 300 plies without a result is truncated. Truncation is a distinct outcome (the fourth value of the outcome byte) and is never recorded as a draw; the cap sits beyond the longest search-guided game observed during engine profiling (202 plies). Generation-0 play from a random initialisation truncates often (56.7 % of games in the pilot), which the separate category absorbs by design and the protocol's cap-sensitivity procedure exists to probe.

6.3 The training record

Each recorded position is a fixed-length record of 818 bytes (format version 3), written by the generator and read identically by both arms' data loaders. It stores the position in absolute piece coordinates: 28 pieces × (x, y, level), with a sentinel for pieces in hand. Alongside the position it stores the side to move, the last-moved piece (for the stun rule), the ply, the game-type bits, both sides' queen liberties and reserve counts, and the one-hive-pinned bitmask. Any consumer reconstructs the state, its frame and its candidate set exactly from these bytes, so the grid encoder and the graph encoder derive from the same source.

Three fields were added for this study. A *model stamp* (generation number and network hash) in bytes 100–107 answers, for every shard, which network generated it; the check battery asserts that every record's stamp matches the run manifest. The *legal-move index list* (bytes 178–817, capped at 320 entries; the largest branching measured in real play is 213) enables legal-masked training in both arms without re-running move generation. The *outcome byte* takes four values from the perspective of the side to move: 0 loss, 1 draw, 2 win, 3 truncated. The policy target is the search's root visit distribution over the shared action space, stored as its top-15 entries with the total visit count.

Records of the prior demonstration loop (versions 1 and 2) carry no stamp and no legal list, and their cap games were labelled as draws; they are not training inputs for the study. Every run keeps three separate trees, for self-play records (the training inputs), checkpoints and evaluation outputs; an audit asserts that no evaluation file is ever referenced by a training configuration (check 7).

6.4 The training procedure

Training is identical for both arms down to the optimiser state: stochastic gradient descent with learning rate 0.02 cosine-annealed to one hundredth of its initial value over the generation, momentum 0.9, weight decay 10^{-4}, batch size 256, two epochs per generation, and a value-loss weight of 0.6. One record in fifty is held out as a validation split, on which policy top-1 (masked as in training) and value accuracy (over non-truncated records) are reported after each epoch. No hyperparameter search was performed for either arm.

The loss is the one written out in Equation 2 of Section 2: the cross-entropy between the recorded visit distribution and the network policy over the legal set, plus 0.6 times the three-way outcome cross-entropy, the value term averaged over the batch's non-truncated records only. The policy distribution $p_\theta(\cdot \mid s)$ is normalised over exactly the legal set: the grid arm fills the illegal entries of its flat logit tensor with -10^9 before the log-softmax, while the graph arm computes logits only for legal (slot, destination) rows, so the two arms produce the same family of distributions over the same legal sets. The value head is a three-way win/draw/loss classifier; a truncated record contributes to the policy term but is excluded from the value term, never trained as a draw.

Checkpoints carry the model, optimiser and scheduler state, the step and epoch counters, the random-number state and the run configuration; the per-epoch shuffle is re-seeded from the run seed and the epoch index, so a run resumed from an epoch boundary replays the identical batch order (check 5). One change was made to the loop after the main campaign: on 23 September 2026, after an ablation run had diverged to a non-finite loss and self-played on it silently for ten generations, a guard was added that halts training on a non-finite loss; it leaves every finite computation unchanged. Training throughput was benchmarked on 10 September 2026 at 274 positions/s for the grid network and 138 positions/s for the graph network (batch 128, forward and backward, on the machine's GPU backend); the campaign runs loaded their data in-process rather than through worker processes, so their training phases ran at or below these figures. Generation rather than training dominates a generation's wall-clock in either arm.

6.5 Export and inference

After training, the checkpoint is exported to ONNX with static input shapes, one graph per batch size (the campaign exported the batch-1 graph that match play and self-play use). Static shapes are required by the CoreML execution provider; the graph arm obtains them by padding every position to a fixed node capacity and a fixed move capacity with masks. The exported graph then runs inside the Rust search through the same inference path for both arms.

The execution provider is where the two arms genuinely differ in cost, and the difference is reported rather than equalised away (Table 10, measured 10 September 2026). The grid arm's convolutional network runs fastest on CoreML; the graph arm's gather-heavy network is split by CoreML into 15 sub-graphs with 147 of 287 nodes supported, and runs fastest on the CPU provider. Each arm therefore used its best available provider: grid on CoreML at 2.62 ms per evaluation, graph on CPU at 3.67 ms, a ratio of ≈1.4× against the graph arm. The same-wall-clock reading of the protocol charges each arm this true cost on this machine; the same-examples reading is unaffected.

Table 10: Inference latency of the two capacity-matched networks (grid 1.44 M parameters, graph 1.47 M) at batch size 1, in milliseconds per network evaluation, measured on the study machine (Apple M1 Pro, 10 cores, 16 GB) on 10 September 2026 across three execution paths; one measurement series per cell, no interval reported. Bold marks the provider each arm used in self-play and evaluation.

Execution path	Grid arm	Graph arm
PyTorch, CPU	11.03	10.51
ONNX Runtime, CPU	23.5	**3.67**
ONNX Runtime, CoreML	**2.62**	9.84

Execution path	Grid arm	Graph arm
Provider used in the study	2.62 (CoreML)	3.67 (CPU)

The CoreML path itself had to be repaired first: on 9 September 2026 the Rust inference layer crashed under CoreML because it requested the provider's legacy model format; requesting the modern format restored it, and a probe at 128/32 simulations with the prior loop's network as workload ran at ≈3 s per game of wall-clock on four threads. This probe is the origin of the "≈3 s/game with a trained network" estimate quoted in the protocol.

6.6 The evaluation arena

Evaluation is independent of training in data, settings and seeds. The arena plays *paired* games: pair i of every match plays opening line i of a frozen file of 250 unique legal four-ply base-game openings, once with each colour, so that every arm, seed and opponent faces an identical opening-and-colour schedule (verified by a test that hashes the schedules). The openings were generated blind by a seeded random walk over the engine's legal moves and frozen on 10 September 2026, before any comparison run, with their content hash recorded. Final evaluations use 100 games per opponent (openings 0–49), intermediate ones 20.

The network under evaluation searches 400 simulations per decision with no Dirichlet noise and no temperature: the most-visited move is played, deterministically. These settings were pinned on 9 September 2026 in a versioned configuration and by a unit test asserting that the search's defaults carry $\varepsilon = 0$; the match-play interface exposes no way to enable noise, so the evaluation path cannot explore by accident. The visit count is fixed for a reason: Jones (2021) measures a train-time/test-time compute trade-off under which a floating evaluation budget would let measured strength drift. The opponents are invoked exactly as frozen (Section 8), with their own fixed seeds; the network side uses seed 9000 plus the generation index (9500 for the evaluations at the equal-time cutoff). Games are capped at 300 plies; truncations are excluded from the score, reported as a separate rate, and scored 0.5 in a sensitivity line. An evaluation game costs ≈23–32 s at this budget.

Scores are win = 1, draw = 0.5, loss = 0 over non-truncated games, aggregated to one score per (seed, opponent) cell before any statistics are computed; the seed rather than the game is the resampling unit (Section 9).

6.7 Seven automated pre-training checks

Before any training output was trusted, seven properties of the data path and the training loop were turned into automated assertions and run as one command against real self-play shards (Table 11). They go beyond unit tests of isolated functions: checks 1–3 run on recorded positions through a real forward pass, check 4 is a short experiment with a stated criterion, and checks 5–7 exercise the resume, evaluation and layout contracts end to end.

Table 11: The seven automated pre-training checks, run as one command against real self-play shards before training output was trusted. Each row states the property asserted and the

mechanism that asserts it; check 4 is an experiment with a numeric criterion, the others are assertions that pass or fail.

Check	What it asserts	How it is tested
1 Legal-set normalisation	The masked policy puts exactly zero probability on illegal actions and sums to one over the legal set	Real forward pass on recorded positions; illegal mass asserted equal to 0
2 Move-index identity	Encode → index → decode is the identity on all legal moves across game types; every stored target and played index is legal	Rust round-trip test; data-side scan of every record's target entries against its legal list
3 Outcome perspective	The outcome byte is one of four values, correct for the side to move in both colours; truncated records are excluded from the value loss	Rust sign batteries (evaluation, alpha-beta, search root); data-side comparison of the value loss with its truncation-filtered reference
4 Tiny-batch overfit	A fixed small batch reaches near-perfect fit	Policy argmax 15/15, value 15/15, residual KL 0.09 to the soft targets
5 Save and resume	An interrupted run resumed from a checkpoint reproduces bitwise-identical model and optimiser state and keeps the planned learning-rate schedule	Two training legs on CPU; every model tensor and optimiser tensor compared for exact equality
6 No exploration at evaluation	The evaluation path carries $\varepsilon = 0$ and no temperature	Unit test on the search defaults; settings pinned in a versioned configuration
7 Evaluation never feeds training	No evaluation game is a training input	Audit of every training configuration's shard list against the evaluation tree

Check 4 deserves a note on its criterion. Visit distributions are soft targets with irreducible entropy, so "loss tends to zero" is the wrong test; the criterion is the Kullback–Leibler divergence to the target-entropy floor. The check's first run was reported as a failure for two reasons unrelated to the network: the loss had been compared against zero rather than the floor, and the validation holdout had been included. The criterion was corrected and the correction recorded. On the pilot shard the corrected check gave KL 0.093, policy argmax 15/15, value 15/15.

6.8 The pilot of 10 September 2026

The loop was first run end to end at a deliberately small budget: one generation from a seeded random initialisation of the grid network (1.44 M parameters), 300 self-play games at 128/32 simulations with playout-cap randomization, three training epochs (the campaign later used two), batch 256, the masked policy loss, and an independent evaluation against the frozen population under the pinned settings with 30 games per opponent. Self-play started on the evening of 9 September 2026; the evaluation completed on 10 September.

Generation. The 300 games yielded 17,237 recorded positions; 170 of 300 games (56.7 %) were truncated at the 300-ply cap and none ended by resignation, since a random-initialisation value head never crosses the threshold. Generation took ≈60 min on four threads, ≈12 s per game: untrained play runs long, whereas the trained-network probe of the previous day measured ≈3 s per game at the same budget, so per-generation cost falls as play sharpens.

Checks. All seven passed on the real shard: illegal policy mass exactly 0 and legal mass 1; every stored and played index legal; a valid outcome domain with 12,806 truncated

records excluded from the value loss; the overfit result above; save and resume bitwise-identical at step 2110; evaluation noise structurally off; evaluation and training trees disjoint.

Training. On 16,893 training and 344 validation records, the losses decreased (policy ≈3.4 nats at the end against ≈4.1 for a uniform distribution over ≈60 legal moves); validation policy top-1 reached 4–5 % against a uniform-legal chance level of ≈1–2 %; validation value accuracy was 42–46 % over the 94 non-truncated validation records; throughput ≈772 positions/s. The value signal is thin at generation 0 because 74 % of records are truncation-masked.

Evaluation. Table 12 gives the result. The generation-0 network beat legal-random in every decided game (28 wins, 2 truncations) and scored 3.3 % against both the heuristic and the 6,400-simulation search.

Table 12: Independent evaluation of the pilot's generation-0 grid network (one training run, 300 self-play games at 128/32 simulations, three epochs) against the three frozen opponents, at 400 simulations per decision without noise, 30 paired colour-swapped games per opponent. Score = wins + ½ draws over non-truncated games, in %; truncations at the 300-ply cap are counted separately. Pilot volume: no interval is reported and this is not a study result.

Opponent	W/D/L	Score	Truncated
B-RND (legal-random)	28/0/0	100.0 %	2/30
B-HEU (heuristic)	0/2/28	3.3 %	0/30
B-MCTS (search, 6,400 simulations)	0/2/28	3.3 %	0/30

What the pilot diagnosed. The profile (dominance over legal-random, near-total loss to the two mid-band opponents) was interpreted in the mandated order: rules, value signs, search, data. The first three had been validated independently (Section 5, Section 8), so the gap after one generation reflects data and iteration rather than a defect; the indicated lever is more generations rather than more capacity, and no capacity change was made. The pilot also supplied the two budget confirmations the protocol freeze was waiting for: 128/32 simulations are feasible at study scale (worst case ≈12 s per game at generation 0, improving toward ≈3 s), and the 300-ply cap is workable precisely because truncation is a separately reported outcome. The protocol was frozen with these values the same day. The pilot's limits are those of its size: one seed, 30 games per opponent, generation 0 only, grid arm only. The iteration of the loop was exercised in structure but was not actually run.

6.9 A harness defect caught during the pilot

The first evaluation pass of the pilot did not produce Table 12. It returned 0/2/28 against all three opponents, including legal-random, which a network that had learned anything should not lose to. Following the same investigation order, the game records were inspected: the opponents' replies were identical across the three pairings. The interactive shell loop used for that first pass had passed each opponent's command-line flags as a single argument, so every opponent fell back to the engine's default search backend and all three matches had in fact been played against the heuristic. The pilot driver proper, which builds argument lists explicitly, does not have this defect; rerun with explicit arguments, the evaluation produced the coherent table above.

The incident was kept as a worked example rather than discarded, because identical results across supposedly different conditions should be treated as a harness alarm before being read as a finding. The rule proved its worth two weeks later, when three "independent" ablation seeds returned game-count-identical evaluations and the alarm led directly to the silent divergence described in Section 13. A second, minor slip of the same pilot, in which the check battery initially invoked a system Python interpreter with a broken deep-learning installation for check 5, was fixed by pinning the project's own environment.

7 The two state representations and their networks

The study's independent variable is the state representation together with the network body that reads it, and nothing else. This chapter specifies both arms as they were run: encodings, networks, the shared action decoder, capacity matching, the cost asymmetries that were measured rather than removed, the tests that pin each encoder, and the ablation variants. Figure 1 shows one position under both encodings, Figure 4 the two networks, and Appendix B the layer tables, the decoder arithmetic and the record layout.

The two sides do not have the same history. The grid frame, its planes and the convolutional network were built in July 2026 within the engine project and carried the earlier 19-generation self-play demonstration; on 10 September 2026 they were reused unchanged as the baseline representation, with one hardening of the frame's overflow check. Everything on the graph side (encoding, network, tensor builders in both implementation languages, and their tests) was built on 10 September 2026 for this study, against a decoder contract fixed the day before.

7.1 What varies between the arms, and what provably does not

Both arms share the rules engine, the search, the frozen opponent population and pinned evaluation settings, the record format, the training targets and outcome conventions (truncation a fourth outcome, excluded from the value loss), the action space with its legal-set normalisation, and the training loop (two epochs per generation, batch 256, learning rate 0.02, value-loss weight 0.6, stochastic gradient descent with momentum 0.9 under a cosine schedule). Neither arm was tuned beyond capacity matching. Each shared element is enforced rather than asserted: one decoder contract with identical masking, one record format, a cross-language golden test per encoder, and one Rust Monte-Carlo tree search evaluating both arms through static-shape ONNX exports. What differs is exactly enumerable: the representation, the network body, and their measured hardware interactions.

7.1.1 The shared action decoder

A Hive move is uniquely identified by (piece, destination cell): a walk and a throw landing the same piece on the same cell produce identical successor states. The decoder addresses the piece by a side-to-move-relative slot: the mover's 14 pieces in roster order, then the opponent's 14 in slots 14–27 (they exist because the Pillbug expansion moves enemy pieces; in the base game they are never legal). It addresses the destination by a cell of the candidate set, every occupied cell plus the ring of empty cells adjacent to the hive, which contains every legal destination by construction. One extra action, pass, is legal exactly when no move exists. The grid arm materialises this space as a flat vector of 28,673 logits indexed by slot × 1024 + y × 32 + x over its 32 × 32 frame, the last index being pass. The graph arm materialises no such vector: for each legal (slot, destination) pair it computes one logit from the embeddings of the destination node, the piece's source and the slot, and scores a pass row the same way. The action a logit refers to is the identical

pair in both arms; this is the pointer-network mechanism (Vinyals et al., 2015) of scoring each element of a variable candidate set and normalising over exactly that set.

Four properties are shared verbatim. Masking: logits exist only for the legal set the engine generates, the softmax runs over exactly that set, and probability mass on illegal actions is identically zero; the first of the seven automated pre-training checks asserts this through a real forward pass for whichever arm is under test. Order independence: scores attach to (slot, destination) pairs, never to positions in the legal list. Tiebreak: argmax ties break toward the lowest flat index, the grid indexing defining the tiebreak for both arms. Targets: the root visit distribution of the search over the same space, stored as its top 15 (index, visit weight) entries with the total visit count; the value target is a scalar in [−1, 1] from the side to move (win +1, loss −1, draw 0), and truncation contributes no value target in either arm. The contract was fixed on 9 September 2026, before either encoder was built for the study. The rationale is that the confounder is controlled by identity of action space, mask and targets rather than by both arms producing the same tensor; imposing the flat tensor on the graph network would have carried the frame, a grid artefact, into the graph arm.

7.2 The grid encoding

7.2.1 Frame, anchoring and overflow

The engine's board is a 64 × 64 wrapping byte grid (a torus) with absolute axial coordinates. For encoding, a position is unwrapped from the torus by breadth-first search from an arbitrary occupied cell (wrapping cannot split the hive, which is connected by rule) and translated so that the centre of the occupied bounding box lands at (16, 16) of a fixed 32 × 32 frame. Hex adjacency on axial coordinates is a 7-cell subset of the 3 × 3 neighbourhood, so ordinary 3 × 3 convolutions cover it, the two non-neighbour corners of each kernel becoming learnable dead weights. Anchoring is by bounding-box centre only: no rotation or reflection canonicalisation, and no symmetry augmentation (see the end of this chapter).

A 28-piece hive spans at most 28 cells per axis after unwrapping, so the occupied box plus the full ring of candidate destinations fits the frame with margin, by construction. The argument is also enforced at run time: the frame constructor carries an always-on assertion, present in release builds, so that any cell mapping outside the frame stops the program loudly and no piece or destination can vanish or alias silently. Until 10 September 2026 this was a debug-only assertion compiled out of release builds, a silent-corruption hazard that the overflow review closed. Extremal tests place all 28 pieces in a straight line along each axis and prove that every occupied and ring cell maps without aliasing; a long random-game drift test covers ordinary play.

7.2.2 The 77 feature planes

Table 13: The 77 input planes of the grid encoding. Each plane is a 32 × 32 float32 map with values in [0, 1]; a "constant" plane broadcasts one scalar over the whole frame. Owner is side-to-move-relative, matching the decoder's slots and the value head's perspective.

Planes	Content
0–63	Piece planes: owner (mover = 0, opponent = offset 32) + bug type (8 types: Q, S, B, G, A, M, L, P) × 4 + min(stack level, 3); owner, type and height are encoded jointly, one plane per combination
64	One-hive-pinned top pieces (articulation cells of the hive)
65	Cell of the last-moved piece (stun-relevant state)
66	Legal placement cells for the side to move
67	Legal placement cells for the opponent
68	Side to move is white (constant plane)
69, 70	Queen liberties (mover, opponent) / 6 (constant planes; 0 if the queen is unplaced)
71	Ply / 100 (constant plane)
72–74	Game-type bits M, L, P (constant planes)
75, 76	Reserve counts (mover, opponent) / 14 (constant planes)

The joint piece planes record, per cell, who owns the piece at each stack level, what it is and how high it sits, levels 3 and above merged. The placement planes follow the standard adjacency rule (an empty cell adjacent to at least one of the player's top pieces and none of the opponent's) without the opening-turn exceptions; they are hints only, since legality itself is supplied to both arms by the engine's mask. The graph encoder uses the same simplified rule.

7.2.3 The grid network

HiveNet is a residual convolutional network in the style of KataGo (Wu, 2020), sized at 96 channels and 8 residual blocks. A 3 × 3 convolutional stem takes the 77 planes to 96 channels (batch normalisation, ReLU); each residual block applies two 3 × 3 convolutions with batch normalisation, a residual addition and a ReLU; blocks 2 and 5 (counting from 0) add a global-pooling bias before the residual addition: the channel-wise mean and maximum over the frame are concatenated, passed through a linear layer and added back per channel. This injects a global signal twice, queen safety being a global property. The policy head is a 1 × 1 convolution to 28 piece-slot planes, flattened to 28,672 spatial logits, plus a pass logit from the mean-pooled features: 28,673 outputs. The value head maps the mean-pooled features through a 64-unit hidden layer to three logits (win, draw, loss from the side to move). The network has 1.44 M parameters, counted by the training code.

7.3 The graph encoding

7.3.1 Nodes, pieces and destinations

The graph encoding is coordinate-free: no absolute coordinate appears in it. Its nodes are the cells of the candidate set (every occupied cell and every empty cell adjacent to the hive), which is exactly the decoder's destination universe, so every scorable destination is a first-class node. The choice follows from the rules: Hive's destinations and sliding constraints are properties of empty space, and a graph over occupied cells alone would have nothing to score for most moves. Keller et al. (2023) reached the analogous conclusion for Hex, whose formulation keeps only empty cells as nodes, and a pointer-style decoder can only point at elements that exist (Vinyals et al., 2015). An explicit occupancy feature distinguishes empty candidates from occupied nodes. Pieces are not separate

nodes: since the decoder addresses a move as (slot, destination cell), a piece's identity enters the policy through its slot embedding and its location through the node it stands on; cell nodes carry the full stack composition level by level, so no piece information is lost and the graph stays half the size it would have with piece nodes. This is a documented design choice.

7.3.2 Node features, typed relations and global features

Table 14: The 56 node features of the graph encoding, one vector per candidate-set cell. Stacking is represented level by level up to height 5 (heights above 5 cannot occur in the base game; the height scalar still records them). Owner is side-to-move-relative.

Features	Content
0–49	Five stack levels (0–4), ten features each: present bit, owner-is-mover bit, bug-type one-hot over the 8 types
50	Stack height / 5
51	Empty-candidate bit (1 for an empty ring cell)
52	One-hive-pinned bit (the top piece is an articulation point of the hive)
53	Last-moved bit (stun-relevant)
54	Legal-placement bit for the side to move
55	Legal-placement bit for the opponent

Edges are the directed adjacencies between candidate-set cells, typed by the six hex directions (east, north-east, north-west, west, south-west, south-east), stored as a neighbour-index tensor (per node, the index of its neighbour in each direction, with a sentinel for none) and realised in the network as six relation-specific weight matrices. The reverse of direction d is (d + 3) mod 6, a symmetry the property tests check. Adjacency to cells outside the candidate set is excluded: those cells are empty and not adjacent to the hive, so they cannot influence legality or value.

Table 15: The 23 global features of the graph encoding. The vector is concatenated to every node's input and again to the pooled representation in the value head.

Features	Content
0	Side to move is white
1	Ply / 100
2, 3	Queen liberties (mover, opponent) / 6; 0 if the queen is unplaced
4–11	Mover's reserve count per bug type / 3 (8 types)
12–19	Opponent's reserve count per bug type / 3 (8 types)
20–22	Game-type bits M, L, P

Reserves enter per bug type because placement legality and material planning depend on which bugs remain in hand; the grid arm carries the two totals as constant planes and can recover per-type counts from its piece planes, so neither arm receives information the other cannot reconstruct.

7.3.3 Fixed capacities and loud overflow

The tensors have fixed shapes, 224 nodes and 321 move rows (320 legal moves, the record's cap, plus one pass row), and, like the frame, an always-on overflow check: a position exceeding either capacity fails loudly rather than silently. Measured maxima over 300 real self-play records were 67 nodes and 124 legal moves, well under capacity. Fixed shapes keep the ONNX export static, so the graph arm runs under the same Rust

search and inference path as the grid arm; dynamic shapes would have forced a different inference route and a per-arm asymmetry where the design must be identical.

7.3.4 Coverage of the engine's state, and what the network is not given

Table 16: Coverage map from the components of the engine's game state to the elements of the graph encoding. The last two rows state what is deliberately absent.

State component (influences legality or outcome)	Graph element
Piece positions, owners, types	node stack-level features
Stacking (beetle climbs, buried pieces)	per-level features and height
Empty candidate destinations	empty-candidate nodes
Adjacency geometry and directions	typed edges (6 directions)
One-hive pins	pinned bit (engine-computed articulation)
Stun state (last-moved piece)	last-moved bit
Side to move	mover-relative features and global bit
Queen placement deadline	ply global and reserve features
Reserves	per-type global counts
Game type	global bits
Placement legality regions	placement bits (engine-derived rule)
Move legality itself	excluded: supplied per position by the engine through the decoder's legal mask, identically to the grid arm; the network never computes legality
Absolute board coordinates	excluded: coordinate-free by design, so anchoring questions do not arise

No invariance and no rule is granted for free. The encoding contains no absolute coordinates, but the learned function is not thereby translation- or rotation-invariant: message passing with direction-typed relations is not rotation-invariant, receptive fields are limited by depth (one hop per layer), and no rule of Hive is known to the network; legality arrives from the engine through the mask, for both arms alike. The protocol treats any invariance as a question to be measured rather than presumed, and this chapter asserts none.

7.3.5 The graph network

HiveGraphNet begins with a linear layer from each node's 56 features, concatenated with the 23 globals, to 152 channels, masked to real nodes; a zero row at index 224 stands in for absent neighbours. Eight relational message-passing layers follow, each computing

$$h_i' = \mathrm{ReLU}\left(h_i + W_{\mathrm{self}}\, h_i + b + \sum_{d=1}^{6} W_d\, h_{n_i(d)} \right) \tag{4}$$

where $n_i(d)$ is the neighbour of node i in direction d (the zero row when absent), W_{self} carries the bias b, and the six W_d are the direction-typed matrices without bias; the output is masked to real nodes. Layers 2 and 5 (counting from 0), that is every third layer, add a global-pooling bias inside the non-linearity: the masked mean and masked maximum over the real nodes are concatenated, passed through a linear layer and added to every node. Message passing alone is limited by depth, whereas queen safety is a global property.

The value head concatenates the masked mean, the masked maximum and the global vector and maps them through a 64-unit hidden layer to the same three win/draw/loss logits as the grid arm. The policy head is the decoder's per-candidate scorer: for each legal (slot, destination) row it concatenates the destination node's embedding, a source embedding and a 32-dimensional slot embedding and maps them through a 128-unit hidden layer to one logit; the source embedding is the piece's standing node for a movement and a learned reserve vector for a placement. The pass row is scored by the same scorer from the pass-slot embedding, the reserve vector and the zero row; it is a learned constant and immaterial, because pass is legal only when no move exists and then wins the softmax alone. Illegal rows are masked before the softmax, which runs over the legal rows in the loss and in the inference evaluator exactly as the grid arm's masked softmax. The network has 1.47 M parameters, +1.5 % relative to the grid network. It is an inductive message-passing encoder in the sense of Hamilton et al. (2017), extended with relation-specific weights per direction over exact neighbourhoods of degree at most six, which is exactly what the protocol prescribes.

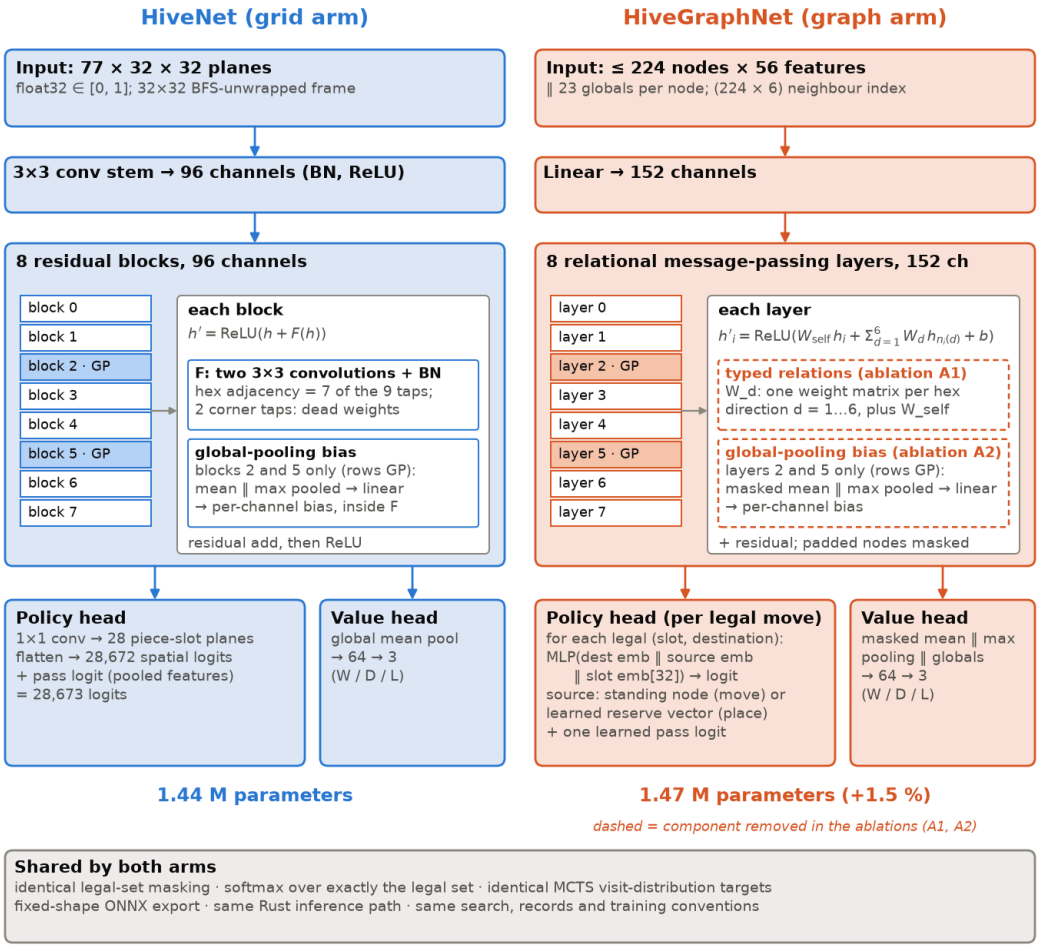


Figure 4: Block diagrams of the two networks. Left, HiveNet (grid arm): 3 × 3 convolutional stem, 8 residual blocks of 96 channels with a global-pooling bias in blocks 2 and 5, a flat 28,673-way policy head and a 3-way value head, 1.44 M parameters in total. Right, HiveGraphNet (graph arm): input linear layer to 152 channels, 8 relational message-passing layers with six direction-typed matrices and a global-pooling bias every third layer, a per-candidate policy scorer over destination, source and slot embeddings, and a masked mean‖max value head, 1.47 M parameters in total. Both networks feed the shared action decoder; only the encoder and the body differ.

7.4 Capacity matching and measured cost asymmetries

Capacity was matched by sizing the graph network's width and depth (152 channels, 8 layers) to the grid network's parameter count (96 channels, 8 blocks): 1.47 M against 1.44 M, +1.5 %, reported. What could not be matched is the cost of running each network on the study machine, where the two representations interact with the hardware in opposite ways.

Table 17: Inference cost per position evaluation at batch size 1 for the two networks, measured on 10 September 2026 on the study machine (Apple M1 Pro, 10 cores, 16 GB, macOS 15.3.1). Milliseconds per evaluation; one measurement configuration, no interval.

Inference path	Grid (HiveNet)	Graph (HiveGraphNet)
PyTorch, CPU	11.03 ms	10.51 ms
ONNX inference, CPU provider	23.5 ms	3.67 ms
ONNX inference, CoreML provider	2.62 ms	9.84 ms
Best available provider	2.62 ms (CoreML)	3.67 ms (CPU)

The convolutional network runs fastest on the CoreML accelerator; the graph network runs fastest on the CPU, because under CoreML only 147 of its 287 operators are supported, the gather-heavy operations fall back across 15 partitions, and the accelerator path ends up slower than the CPU path. At each arm's best provider the per-evaluation cost ratio is ≈1.4× against the graph arm. Self-play and evaluation therefore ran each arm on its best provider through the same search.

Table 18: Training throughput at batch size 128 under matched conditions for the two networks, in positions per second, same machine and date as the inference table. The study trains on the MPS path.

Training path	Grid (HiveNet)	Graph (HiveGraphNet)
PyTorch, CPU, forward only	138 pos/s	478 pos/s
PyTorch, MPS, forward + backward	274 pos/s	138 pos/s

Training shows the reverse pattern: on the CPU the graph network is ~3.5× faster per position, on the MPS path used for training ~2× slower, gather and scatter dominating there. Training is a minor share of a generation's wall-clock either way (in the pilot, ≈3.5 min of training against ≈60 min of self-play for a generation-0 generation), so the inference asymmetry drives the campaign-level cost difference. The figures are single-machine and single-configuration, exclude the graph dataloader's Python build cost, and the MPS numbers are a plain forward-and-backward pass without the optimizer step. These asymmetries are reported rather than equalised. They are genuine interactions between a representation and the hardware, and equalising them, whether by throttling the grid arm or by forcing the graph arm onto a slower provider, would manufacture a parity that no user of either representation would experience. The protocol instead reads the comparison twice: under the same-examples reading the asymmetries are irrelevant, both arms playing the same number of games with the same generator settings; under the same-wall-clock reading each arm is charged its true cost on this machine. The campaign-level consequence is reported with the cost results.

7.5 Pinning the encoders: golden and property tests

Each encoder exists twice, in Rust inside the engine and the search and in Python inside the training loop, and agreement between the two is proven rather than assumed. For the grid arm the Rust encoder and the Python decoder are byte-identical by contract, enforced by a nightly golden crosscheck over 240 positions; after the assertion hardening the crosscheck passed and the interface crate's suite stood at 7/7 including the new extremal tests; any later change must keep it green or be recorded as a finding.

For the graph arm, training builds the tensors in Python from the engine-generated (and themselves crosschecked) records. A property battery over 300 real records passed in full: no information loss (every piece, stack level, reserve count, stun state and turn datum in the record appears in the tensors); capacities respected, with an overflow probe raising loudly; edge symmetry; legal-move tensor validity with every stored target inside the legal list; and zero illegal probability mass through a real forward pass. One builder fix was needed: the empty board at ply 0 has an empty candidate set, and the builder now mirrors the frame's canonical first cell, (16, 16). The Rust builder used at inference mirrors the Python builder (same cell ordering, feature layout, direction order, simplified placement rule and loud overflow), and a golden crosscheck over 160 pseudo-random positions across all 8 game types found 160/160 exactly equal on its first run; it runs nightly beside the plane crosscheck. The graph evaluator inside the search builds tensors and move rows in the search's move order, runs the static-shape export on the CPU provider, applies the softmax over the legal rows and converts the three-way output to P(win) − P(loss), which is the grid evaluator's output contract.

A wiring validation (two epochs on the generation-0 pilot data, one seed) ran at 194 positions per second on MPS, dataloader included, with a validation policy top-1 of 4.7 % and a value accuracy of 40.4 %, the same profile as the grid arm on the identical data (4–5 %, ~43 %); a smoke match of 6 games at 200 simulations against the legal-random opponent gave 2 wins, 0 losses and 4 truncations with no illegal reply. These figures establish only that the wiring is sound; they are not a comparison result.

7.6 The two ablation variants

Table 19: The graph-arm ablation variants. Each differs from the full graph arm in exactly one network component; the encoder, the decoder, the training settings, the budgets, the evaluation settings, the opponents and the openings are identical. Parameters in millions, measured by the training code's parameter counter.

Variant	Single component changed	Parameters
Full graph arm (reference)	none	1.47 M
Untyped edges ("naive adjacency")	the six direction-typed matrices replaced by one shared matrix	0.54 M
No global pooling	the global-pooling bias removed from every layer	1.37 M

The parameter differences are inherent to the removed components and are reported rather than equalised; widening the untyped network to compensate would change a second component. The naive-adjacency variant is the ablation the research plan called for; it tests the premise, written into the protocol before any run, that a naive adjacency graph may not suffice. The no-global-pooling variant was substituted on 19 September 2026 for the planned augmentation-removal ablation, which had nothing to remove

once augmentation was excluded from the full method; it was chosen because the main comparison's mechanism signal (the graph arm's relative strength sat in its value head while its policy stayed locally weak) made the pooled signal the sharpest remaining single-component question. A supplementary two-component variant, untyped edges plus a global gradient-norm clip of 1.0, was approved on 26 September 2026 after the untyped variant proved untrainable at parity settings, diverging to non-finite values in generation 0 in 3 of 3 seeds; its numbers are labelled as a two-component difference wherever they appear and nothing it shows is attributed to edge typing alone. Outcomes are reported with the ablation results.

7.7 Symmetry augmentation: excluded by decision

Neither arm trains with symmetry augmentation, by a decision of 10 September 2026 taken before any comparison run. Exclusion makes the identical-data rule trivially true: implementing the hex symmetries consistently across two representations (rotations and reflections of the frame on one side, permutations of the direction types on the other) is subtle, and an asymmetry there would contaminate the main comparison. It also keeps the secondary symmetry question separable as an additive ablation, and it matches the characterised baseline: the earlier pipeline had documented a 12-fold augmentation as an intention but never implemented it. The frozen protocol never specified augmentation; both arms therefore see each position in whatever orientation the game produced.

7.8 Limits of what this chapter establishes

This chapter establishes the identity of everything except the representation; it says nothing about the merit of either representation. The cost figures come from one machine in one configuration; another accelerator could reverse the inference asymmetry. The capacity match, to +1.5 %, is a match of parameter counts rather than of compute. The graph arm is one point in a large design space (cell nodes, six direction types, a pooled bias every third layer, a per-candidate scorer), and the study compares one grid network with one graph network at one capacity and budget; nothing here says another graph design would behave the same. Finally, the hex symmetries are neither canonicalised nor augmented in either arm, so any symmetry-related difference between the arms is a property of the learned functions rather than of the encodings.

8 The frozen opponent population

8.1 Why a fixed population rather than a rating against moving targets

The primary metric of the study is a mean score against a fixed population of opponents (win = 1, draw = 0.5, loss = 0 over non-truncated games, with truncations reported as their own rate) rather than an Elo rating; the reason is comparability. A rating estimated from games among the agents being trained moves whenever the agents move: a later checkpoint changes the scale against which an earlier one was measured, and two arms trained separately share no scale at all. Go has an external ladder of public networks and Hex a perfect-play anchor, which Wu (2020) and Jones (2021) respectively use to pin their ratings; Hive has neither. A population fixed in advance and shared by both arms, all seeds and both budget readings is the substitute: it makes scores comparable across everything the study varies, and it closes the channel by which an opponent might be adjusted after results are seen. The population was therefore frozen on 9 September 2026, before any training run, and has not been added to, removed from, retuned or re-versioned since. Where tooling prints an Elo-style number in this report, it is descriptive and relative to this population only.

8.2 The three opponents

All three opponents run on the validated engine, the same rules kernel that generates both arms' training games, over the Universal Hive Protocol; they are fully seeded and reproducible.

B-RND, legal-random. A uniform draw over the legal moves of the position. The generator advances per query and is mixed with the position hash, so the agent is deterministic given its seed and the game history. It is the floor of the population and anchors the score scale; any learned or heuristic agent must dominate it.

B-HEU, documented heuristic. A greedy one-ply argmax of the engine's handcrafted evaluation, single-threaded, extended by the searcher's quiescence over moves that target the enemy queen. The evaluation is the weighted sum of the features in Table 20, in centipawn-like units with positive values favouring the side to move; its negation under player alternation is pinned by an automated test. The weights are the engine's defaults as built in July 2026, inherited unchanged and fixed for the study by a cryptographic hash of the weight file and by a test that fails if any value changes: no tuning occurred while the population was characterised and none can occur afterwards without breaking the suite. Tuning after the results have been seen is therefore ruled out by construction.

Table 20: Features and weights of the heuristic opponent. The evaluation is the weighted sum of these terms for the side to move minus the same for the opponent, in centipawn-like units (positive = good for the side to move). Weights are the engine's July 2026 defaults, hash-pinned and test-enforced; the pillbug term belongs to the shared kernel and never fires in the base game used by the study.

Feature	Weight(s)	Rationale
Queen liberties (empty cells around own queen, 0 to 6)	−2000, −700, −350, −150, −50, 0, +20	Dominant term: distance to surround is the game's objective; steeply convex as liberties vanish
Enemy piece adjacent to own queen	−90 each	Enemy neighbours are permanent surround material
Friendly piece adjacent to own queen	−20 each	Own pieces crowd escape cells and can be pinned there
Enemy piece on top of own queen	−180	A covered queen cannot flee and the cell counts against it
Piece free to move (per bug: Q, S, B, G, A, M, L, P)	15, 35, 55, 40, 80, 55, 50, 40	Mobility approximates material in Hive; the ant is the most valuable mover
Piece pinned or immobile (same order)	0, 4, 8, 6, 10, 6, 6, 4	A pinned piece is nearly dead material; small residual for latent value
Piece in hand	+6 each	Placement flexibility and tempo
Own pillbug adjacent to own queen	+40	Rescue-throw availability; inert in the base game

B-MCTS, search without a network. PUCT Monte-Carlo tree search on the same engine with uniform priors and the same handcrafted evaluation, squashed through a hyperbolic tangent, as leaf value; no neural network and no exploration noise in match play. Its budget is 6,400 simulations per decision, measured at ≈27 ms per decision on one thread of the study machine; the budget was set from this measurement rather than from the placeholder budgets of the research plan.

8.3 Tactical verification before use

The search opponent was verified on five tactical positions annotated by hand from the publisher's rules and committed before any run, under the same discipline as the engine-validation corpus of Section 5. The positions are a mate in one by a perimeter walk and by a grasshopper jump, each from both colours, and a position in which the only safe move avoids surrounding one's own queen. B-MCTS solved 5 of 5 at 400, 1,600 and 6,400 simulations. Value signs under player alternation are pinned by automated tests at three levels: the evaluation negates exactly when the side to move flips; alpha-beta scores a mate in one above the mate threshold for the mover, whichever colour moves; and the search's root value is strongly positive for the winning mover, whichever colour moves. A piece-ordering slip in the set-up of the tactical cases was itself caught by this procedure (the engine was right and the set-up wrong) and was recorded rather than silently corrected.

8.4 Characterisation

The three agents were played round-robin on 9 September 2026: 100 paired, colour-swapped games per pairing from common seeded random four-ply openings, with referee-validated moves and a 300-ply cap, truncations excluded from the score and reported separately. The whole characterisation took about ten minutes on the study machine; Table 21 gives the results.

Table 21: Round-robin characterisation of the three opponents on 9 September 2026, 100 paired colour-swapped games per pairing from common seeded four-ply openings, 300-ply cap; B-HEU at depth 1, B-MCTS at 6,400 simulations per decision. Score = wins + ½ draws over non-truncated

games, in %; truncations are counted separately (none occurred). The Elo column is descriptive only: a logistic transform of the score with a normal-approximation 95 % interval that treats games as independent, which the paired design makes slightly conservative; a perfect score has no finite Elo, so the 100–0 result is shown as the approximate value the tooling prints.

Pairing (A vs B)	W/D/L for A	Score of A	Truncated	Elo difference (descriptive)
B-HEU vs B-RND	100/0/0	100.0 %	0/100	≈+2400
B-MCTS vs B-RND	99/1/0	99.5 %	0/100	+920 [+730, +1200]
B-MCTS vs B-HEU	23/29/48	37.5 %	0/100	−89 [−150, −32]

The heuristic won all 100 games against legal-random; the search won 99 and drew 1; against the heuristic the search scored 23 wins, 29 draws and 48 losses. The last pairing is a score of 37.5 %, −89 Elo with interval [−150, −32], which excludes parity; none of the 300 games reached the 300-ply cap, consistent with the game-length profile measured during engine validation. Within this population, legal-random is a clean floor, and the heuristic and the search form a mid band roughly 90 Elo apart with different styles, sharp greedy tactics on one side and sampled search on the other. As for the limits, 100 games per pairing is pilot volume, giving intervals of about ±5 percentage points at scores near 40–60 %; the arena's interval is an unpaired approximation; everything was measured on one machine, and the search's cost figures are machine-specific by design.

One ordering was not expected and is reported as found: the 6,400-simulation search sits *below* the one-ply heuristic. The diagnosis recorded at the time is that uniform priors spread 6,400 simulations thinly over Hive's branching factor of roughly 60 moves, yielding an effectively shallow search, while the heuristic's depth-1 argmax with queen-targeting quiescence is tactically sharp; the tanh-squashed leaf value also saturates on queen-danger positions, flattening the signal the search receives. A diagnostic probe supports this reading: at 25,600 simulations (four times the budget, ≈110 ms per decision) the search scored 9/9/6 = 56.2 % against the heuristic over 24 games, descriptive Elo [−66, +163], so the search does scale past the heuristic with budget. The finding is consistent with the Hive literature, where learned engines lose to plain search (de Goede et al., 2022) and strong heuristic minimax is hard to beat (Kampert et al., 2021). No opponent was retuned after this observation: the population went to the freeze exactly as characterised, with the choice between 6,400 simulations (characterised, 27 ms per decision) and 25,600 (near parity, 110 ms per decision, requiring re-characterisation) left to the human.

8.5 The freeze of 9 September 2026

The population was presented for freezing with its configurations, the weight file's hash, the characterisation above, the budget sub-choice for the search opponent, and one open question: whether to include the checkpoint promoted by the prior self-play loop of August 2026 as a fourth, "frozen-network" member. The author approved the freeze on 9 September 2026 with the search opponent at 6,400 simulations and the prior checkpoint excluded; the decision log records the approval and the hashes.

The frozen population is exactly the three agents above, B-MCTS at 6,400 simulations, identified by the hash of each configuration file and of the weight file together with the engine code commit at the freeze. From that moment, any touch of any opponent constitutes a new study. Later engine commits touched the record format, the graph arm

and the arena's input/output, but no searcher, evaluation weight or search default; the weight-pinning test was green at every commit. The third-party reference engines used for differential testing of the rules (Section 5) are not members of the population, and no study result derives from them.

The exclusion of the prior checkpoint was deliberate, proposed before the human decided it. That checkpoint is the nineteenth generation of a loop that ran in August 2026 with 2000 games per generation at 600 full and 150 cheap simulations, gated by 60 games against the incumbent and promoted at 56.7 %. It rests on a single seed and a single architecture, without a controlled compute budget or an opponent population fixed in advance. Three reasons were recorded. Its provenance adds no controlled information and invites misreading: any score against it would look like a comparison with "the old AI". Running it at the time depended on an inference path that was being repaired that very day, which would have coupled a frozen artifact to an unresolved fix. And the three agents already span the floor and a mid band of two distinct styles. The checkpoint is preserved and may appear as descriptive context; it is never a member of the population and no number in this report is measured against it.

Two limits carry forward. The characterisation volume is 100 games per pairing with an unpaired-approximation interval, which is adequate to place the opponents relative to one another but not to support study conclusions; those rest on the seed-level analysis of Section 9. And the five tactical cases, like the engine-validation corpus, have not been reviewed by an external Hive-literate reader.

Part III. Methodology

9 Experimental methodology: a pre-registered, frozen protocol

This chapter states what the comparison fixed in advance, how, when, and above all why. The governing protocol was frozen as version 1.0 on 10 September 2026, before any comparison run and after a pilot had confirmed every measured value in it; it declares any later change a new study, and none was made. Constants, hyperparameters and seeds are tabulated in Appendix C; raw per-seed tables and the cutoff checkpoint map in Appendix D. A second protocol, version 2.0, was frozen on 13 October 2026 for the complementary study reported in Section 15; it changes the training regime and nothing else, and is described there.

9.1 Why pre-register, and why freeze, at small compute

A comparison on one machine with a handful of seeds leaves the experimenter great freedom after the fact: which checkpoint counts as "the" result; which opponents, at what strength; which seeds are reported; whether a game that hits the move cap is a draw, a loss, or nothing; which metric becomes the headline. Each choice, made after the numbers are visible, can manufacture or erase an effect of exactly the size such a study can detect; the garden of forking paths needs no conscious dishonesty. Two further risks are silent retuning of a baseline in view of the test result, and asymmetric attention to the arm one expects to win.

Pre-registration removes these degrees of freedom by one rule: every commitment that could bias an observation is made before the observation exists. Here that meant artifacts frozen under a content hash, each dated and approved by the author, and a protocol that names them and fixes the metric, the budget readings, the resampling unit and what outcome would count against the hypothesis. The protocol states that the hypothesis "is not presumed true" and that "a negative or null result is a publishable outcome of this study; the work does not need to confirm H1 to count". The negative answer eventually obtained was therefore a planned deliverable rather than a failure to be explained away.

Freezing followed a fixed order (Table 23): the opponent population (Section 8) and the evaluation search settings on 9 September 2026; the protocol on 10 September 2026, once the pilot had re-measured the budgets and the move cap with the study's own networks, so that the frozen text carries no placeholders; the 250 shared openings and the comparison matrix with its cutoff rule the same day. The first comparison run started on 10 September 2026 at 12:42. Freezing after the pilot was the author's choice: it allowed the budgets to be confirmed on the actual arms and avoided a second freeze, which would have weakened what a freeze means.

The research plan's stop rules also bounded the design: an incorrect engine suspends training, so engine validation (Section 5) precedes all training; a budget overrun cuts configurations, never honesty; two weeks without an interpretable result shrink the scope; once the final evaluation set has been consulted the method is no longer adjusted against it. Hence the ablations were declared secondary and cuttable from the outset, and neither arm gained a second architecture when the graph arm underperformed.

9.2 Research questions and the hypothesis

The frozen protocol poses one primary question and one hypothesis, reproduced as frozen.

RQ-H1. At comparable training budget, does a graph architecture learn a better policy than a grid architecture for base-game Hive, within an AlphaZero-style self-play pipeline?

Hypothesis H1. A simple message-passing graph network, receiving the hive as a graph, reaches a higher mean score against a fixed opponent population than a grid CNN receiving the 32×32 BFS-unwrapped frame, at equal training budget.

The protocol itself gives reasons for not presuming H1. Stacking and above all the *empty* candidate destinations must be represented correctly, and a naive adjacency graph over occupied cells may not suffice, because destinations and sliding constraints are properties of empty space. Prior evidence was also split, with graph networks favouring long-range structure and convolutions local patterns in Hex (Keller et al., 2023).

The rejection rule, written before any comparison run, is quoted verbatim; its section references point to the protocol's own sections on budget readings and on seeds and uncertainty:

> *What would reject H1 [...]: against the frozen opponent population, under **both** budget readings of §5, the graph arm shows no seed-consistent advantage (no consistent per-seed ordering in its favour), **and** the interval on the score difference — computed with the resampling unit stated in §6 — excludes a meaningful graph advantage. That outcome is reported as-is: "at this budget, on this variant, the graph representation did not help."*

The rule is conjunctive: per-seed ordering *and* interval must both fail to favour the graph arm, so one lucky seed cannot rescue the hypothesis and one unlucky seed cannot reject it. It applies under both budget readings, and it fixes the resampling unit in advance. One limit should be stated plainly: the frozen text says "meaningful" without a numeric threshold. The verdict therefore reports the largest upper bound of all graph–grid intervals, so that a reader can apply any threshold they consider meaningful; the study claims none it did not pre-register.

Two secondary questions follow. RQ-H2 asks whether the comparison changes when the budget is read per example rather than per hour; it is pre-registered in the matrix as the two budget readings. RQ-H3 asks what the graph arm's distinctive components contribute; its ablation specifications were written on 19 September 2026, after the main result was known (Section 9.8). The protocol's own time-permitting question, robustness to translation and symmetry, was declared the first thing to drop under schedule pressure, and it was dropped.

9.3 Design: two arms, five seeds, ten generations

Only the state encoding and the network body differ between arms (Section 7): a residual CNN over the 32×32 BFS-unwrapped, bounding-box-centred frame (1.44 M parameters) against a relational message-passing network over the cell graph (1.47 M, +1.5%). Both share the rules engine, the search, the shared action decoder, the training loop, the budgets and the evaluation harness. Neither received any hyperparameter search.

Each run comprises 10 generations of 500 self-play games. Self-play searches 128 simulations per decision on a random quarter of decisions and 32 on the rest, following the playout-cap randomization of Wu (2020), and only full-budget decisions yield targets. It samples from the visit distribution for 12 plies, resigns below −0.92 with a 10% no-resign audit, and truncates at 300 plies. Each generation trains 2 epochs at batch 256 and learning rate 0.02 with the legal-masked policy loss. A checkpoint is kept and the wall-clock logged at every generation, which makes the second budget reading possible.

These values were measured rather than taken from the plan's starting points of 64 and 128 simulations. Profiling on 9 September 2026 put one network evaluation at 2.62 ms on the accelerated provider, and the pilot of 10 September 2026 confirmed ≈12 s per game at generation 0, falling toward ≈3 s as play sharpens. The 300-ply cap lies beyond the longest search-guided game observed (202 plies). The pilot showed random-initialisation self-play truncating 56.7% of games at that cap, which is tolerable only because truncation is its own reported outcome.

Seeds came in two stages, both disclosed. The matrix pre-registered three seeds per arm (1–3; base seed 100,000 × seed) and noted a five-seed option "if budget allows". The three-seed campaign, approved on 10 September 2026, ran sequentially on one machine, four worker threads per run, under an estimate of ≈4.6 days and a 20 GB free-disk guard. After both readings of the three-seed matrix had been analysed (19 September 2026), the author approved on 26 September 2026 seeds 4 and 5 for both arms under three pre-commitments written before any new run: (a) all five seeds per arm enter the final analysis regardless of the new seeds' direction; (b) the equal-time cutoff stays at its already-computed value; (c) the two-stage collection is disclosed. Seeds added symmetrically under an unchanged rule are a decision about statistical power rather than a tuning channel; the three-seed verdict stays on record.

9.4 The two budget readings and the equal-time cutoff

A network that learns more per example but costs more per example can lose at equal hours, so the protocol mandates two readings of one campaign. The asymmetry is real: the best available provider gives 2.62 ms per evaluation for the grid arm (accelerated) but 3.67 ms for the graph arm (on the CPU, since its gather-heavy operations fall back from the accelerator), and the graph runs took 2.0× the grid training wall-clock (36.7 vs 18.0 h over five seeds; 35.7 vs 18.2 h over the original three pairs). The interaction is reported and charged rather than equalised away.

Same-examples reading. Both arms train for 10 generations × 500 games with identical generator settings, hence on the same number of games at the same simulation budget; each run is read at its final checkpoint (generation index 9 in the run logs, which count from 0).

Same-wall-clock reading. The cutoff (T* in tables and figures) was defined by a rule in the pre-registered matrix: the median full-run wall-clock of the three grid runs, every run of both arms then read at its last checkpoint completed at or before it. The rule gives the grid arm its full budget by construction, charges the graph arm its true cost, and is score-free. It was executed on 16 September 2026, with five of six runs complete and the last graph run still training, from grid clocks alone and before any cross-arm number existed: median(18.77, 16.73, 19.07) = 18.77 h. Per-generation clocks count self-play, training and export; evaluation time is excluded for every run alike. The exact median, which is the

first grid run's own total, is the operative value. When tables were regenerated for five seeds, the rounded constant briefly excluded that run's last checkpoint at its inclusive boundary; restoring the exact value reproduced the record of 16 September exactly.

Selection is mechanical and its outcome is part of the result (Table 67): generation indices 9, 9, 8, 9, 9 for the grid runs and 3, 4, 4, 2, 5 for the graph runs. At equal wall-clock the graph arm had completed 3–6 of its ten generations, the grid arm nine or ten. Cutoff checkpoints were evaluated after the campaigns (18 September 2026; 6 October 2026 for the extension) at the same 100-game volume; where the cutoff checkpoint is the final one, the final evaluation is reused.

9.5 Evaluation: frozen population, frozen openings, pinned search

Opponent population. The population comprises three opponents, frozen by the author on 9 September 2026 before any training run: B-RND, uniform over the engine's legal moves, seeded; B-HEU, the documented hand-written evaluation played greedily at depth 1, weights pinned by hash and by a test; B-MCTS, search without a network at 6,400 simulations per decision. The prior demonstration loop's checkpoint was excluded: its single-seed, uncontrolled provenance would have invited a reading as "the old AI". The search opponent's 37.5% against the heuristic was diagnosed and deliberately not retuned, since the freeze forbids adjusting an opponent in view of results; no opponent was touched afterwards.

Openings and pairing. Games start from 250 unique legal four-ply openings generated blind by a seeded random walk over the engine's legal moves (generator seed 20260910), frozen on 10 September 2026 under their content hash. Pair i of every match plays line i once with each colour, so every arm, seed, checkpoint and opponent faces an identical opening-and-colour schedule (a harness test showed byte-identical schedules for different agents and match seeds). Final and cutoff evaluations play 100 paired games per opponent (openings 0–49, both colours); intermediate evaluations at generations 4 and 7 play 20 per opponent and serve only the trajectory figures.

Why 100 games. The final volume was recalibrated after the first finals: on 16 September 2026 the seed-to-seed spread against the heuristic (0.080–0.190, SD ≈ 0.056) was about twice the per-pairing game noise at 100 games (SE ≈ 0.03), so more games could not materially narrow the seed-level interval. The volume stayed at 100; the only lever is more seeds, which the extension later provided.

Pinned search. Every evaluation of a trained network runs 400 simulations per decision, with no Dirichlet root noise (the code default, asserted by a test) and a deterministic best move; these settings were pinned on 9 September 2026 by a versioned configuration and never changed. Fixed visit counts matter because training and test compute trade off (Jones, 2021); a floating evaluation budget would let measured strength drift with the search rather than with the network. The evaluated network is seeded 9000 + generation for in-run evaluations and 9500 for cutoff sets, B-RND 9101 and B-MCTS 9201 (B-HEU is deterministic). Evaluation games never feed back into training; an audit check verifies the separation.

9.6 Statistics and the truncation policy

Metric and unit. The primary metric is the mean score against the population (win 1, draw 0.5, loss 0) over the non-truncated games of a (seed, opponent) cell; game-level data are first aggregated to one score per cell, and the seed is the unit of analysis. Games played by one trained network are not independent samples of the *method*: they share that network's weights, and how well a representation learns varies across independent runs far more than across games of one run. Pooling thousands of games from one model yields intervals precise about the wrong thing (Agarwal et al., 2021); no table in this report does so.

Intervals. Seed-level means and their 95% intervals are percentile bootstraps with 10,000 resamples over seeds. For the graph–grid contrast the two seed sets are resampled independently: training seeds are not paired across arms, whereas the pairing that does exist (identical openings and colours) is held constant across arms and absorbed within each cell. Every table shows every seed; none reports a best seed. Elo, where printed, is descriptive and relative to this population only.

Truncation. A game reaching the 300-ply cap is never a draw: truncations are a fourth outcome category everywhere, and each table carries the truncation rate beside the score. The primary score excludes truncated games; a sensitivity column scores them 0.5; two bounding treatments score every truncated game of the arm under test as a loss and as a win; and the direction of the contrast is reported under all four treatments. The "win" treatment bounds what any larger cap could do, which is why bounds replace a rerun at a larger cap. The policy proved necessary: against the random opponent the graph arm truncated 20–57% of its games where the grid arm truncated 0–1%, and, scored as draws, that pathology would have disappeared into half-points.

9.7 Exclusion criteria and the record

A run may be excluded only for a process failure (a crash, corrupted data, or an engine error such as a rules defect exploited by an agent); a bad score is never an exclusion criterion. Across the main campaign, the extension and the ablations, no run failed, was restarted, or was excluded. Two incidents touched the measurement machinery. On 18 September 2026 the match runner was found to drop its records when every game of a match truncated; an audit showed no campaign file affected, and the defect was fixed before the cutoff evaluations ran. On 23 September 2026 the first ablation's three runs, complete without process failure, were found to have diverged to non-finite outputs in generation 0; the divergence was detected because three independently seeded runs returned evaluations identical to the game count. The divergence is reported as the result; the runs are not excluded, but their evaluation tables are not strength measurements. A loud halt on non-finite loss was added to the training loops the same day; healthy runs are unaffected.

9.8 Ablation design

The ablation rules were fixed before the ablation runs: at most two ablations, each removing exactly one component from the full graph arm to answer one question, at full parity with the reference (seeds 1–3, the same budgets, frozen opponents, openings and pinned evaluation); a capacity change inherent to the removed component is reported,

never equalised by changing a second component; and where more than one component differs, no effect is attributed to the graph. A1 replaces the six direction-typed edge matrices by one shared matrix (naive adjacency; 0.54 M parameters, the typed matrices being the component), testing the protocol's own premise. A2 removes the global-pooling bias from every layer (1.37 M). A2 is a substitution: the plan's first ablation removes symmetry augmentation, but the full method was fixed without augmentation on 10 September 2026, because consistent hex-symmetry transforms across two representations risked contaminating the main comparison, so there was nothing to remove. The substitute question was chosen on 19 September 2026 in view of the main result's mechanism signal (the graph arm's relative strength sat in its value head). This is a post-result choice of *question*, disclosed here, under an unchanged analysis. The author approved both ablations at three seeds on 20 September 2026.

A1 proved untrainable at parity, so a supplement A1′ was added and labelled explicitly as two-component: untyped edges plus a global gradient-norm clip of 1.0, the only optimizer change in the study. Approved on 26 September 2026, it answers only the question "what does a *trained* naive-adjacency network score?"; no A1′ number is attributed to edge typing alone.

9.9 Resources actually consumed

All runs executed on one Apple M1 Pro (10 cores, 16 GB, macOS 15.3.1), sequentially, four worker threads per run; no paid compute or data was used. Table 22 lists the four campaigns; training wall-clock per run sums the per-generation clocks (self-play, training, export), evaluation games excluded. Over the ten main-comparison runs these totals ranged from 16.7 to 19.1 h (grid) and 26.6 to 49.9 h (graph), 273.5 h in all; the three-seed campaign occupied the machine ≈7.4 days, ≈163 h including its cutoff evaluations; an evaluation game at 400 simulations took ≈23–32 s. The first ablation's runs are short (7.61–8.01 h) because a non-finite policy plays short degenerate games.

Table 22: Compute consumed by the four campaigns on the single study machine (four worker threads per run, runs sequential, no paid compute). Wall-clock per run is the training wall-clock under one accounting for every campaign (per-generation self-play, training and export seconds summed, evaluation games excluded); the first-ablation runs are short because their training had diverged.

Campaign	Runs	Dates (2026)	Wall-clock per run
Main comparison, seeds 1–3	6	10 Sep 12:42 → 17 Sep 22:31; cutoff evaluations 18 Sep (~11 h)	grid 18.77 / 16.73 / 19.07 h; graph 43.07 / 32.71 / 31.28 h
Ablations A1, A2	6	20 Sep 14:08 → 27 Sep 17:13	A1 7.75 / 8.01 / 7.61 h (diverged); A2 39.90 / 47.44 / 32.05 h
Extension, seeds 4–5	4	27 Sep → 2 Oct; cutoff evaluations 6 Oct (≈6 h)	grid 18.23 / 17.19 h; graph 49.86 / 26.57 h
Supplement A1′	3	2 Oct → 6 Oct	23.89 / 30.35 / 27.68 h

9.10 What was fixed before the first comparison run, and what came after

Table 23 places every commitment relative to the first comparison run; hashes and values are in Appendix C, the provenance index in Appendix E.

Table 23: Chronology of the study's commitments relative to the first comparison run (10 September 2026, 12:42). Rows one to five preceded any comparison number; the later rows were added afterwards under the safeguard stated.

Element	Fixed on (2026)	Timing	Safeguard
Opponent population; prior checkpoint excluded	9 Sep	before training	hashes; never touched
Evaluation search settings	9 Sep	before training	configuration and test
Protocol v1.0, including the rejection rule	10 Sep	after the pilot, before the first comparison	content hash; change = new study
250 shared openings	10 Sep	before the first comparison	generated blind; content hash
Comparison matrix, including the cutoff rule	10 Sep	before the first comparison	rule is score-free
Equal-time cutoff value, 18.77 h	16 Sep	mid-campaign, before any cross-arm number	grid clocks only; exact median
Final evaluation volume, 100 games per opponent	16 Sep	after the first finals	pre-configured value kept
Ablation specifications A1, A2	19 Sep	after the main result	one-component rule; choice disclosed
Seeds 4–5	26 Sep	after the three-seed analysis	three written pre-commitments
Supplement A1′	26 Sep	after A1′s divergence	labelled two-component

9.11 Question → experiment → result matrix

Table 24: The study's three questions, the experiment that addresses each, and the form and location of its result. RQ-H1 is the pre-registered primary question; RQ-H2 and RQ-H3 are secondary.

Question	Experiment	Result and where
RQ-H1 (representation): does the graph arm learn a better policy than the grid arm at comparable budget?	Grid vs graph, 5 seeds per arm, frozen population and openings, read at the same examples and at the equal-time cutoff	Score and uncertainty per seed and opponent; graph–grid contrast with seed-level bootstrap intervals; verdict under the frozen rule (Section 11; raw tables in Appendix D)
RQ-H2 (efficiency): does the answer change per example versus per hour?	The same campaign read at the final checkpoints, then at the cutoff checkpoints	Score-versus-training-time curves and the score/cost table (Section 12)
RQ-H3 (components): what do the graph arm's distinctive components contribute?	One component removed at a time at full parity, 3 seeds each (A1 edge typing; A2 global pooling), plus the labelled two-component supplement A1′	Ablation table: per-seed scores, truncation rates, seed-level intervals against the full graph arm (Section 13)

10 Working methodology: gated human–AI research

This chapter describes how the study was conducted, as a collaboration between one human researcher and an AI assistant under written operating rules that reserve every irreversible or scientific decision to the human, and why that arrangement was chosen. It is the expanded form of the AI-assistance statement in the front matter; the identifiers behind every statement made here are listed in Appendix E.

10.1 Why a gated method

The study was carried out by a single researcher with limited time and a single laptop (Apple M1 Pro, 10 cores, 16 GB) as its only compute. An AI coding assistant multiplies throughput under those constraints but introduces a specific hazard: generated text and code fail *plausibly*, in that they look right at exactly the places nobody checks, and an assistant has no persistent memory worth trusting between working sessions. A study produced this way can accumulate fluent but untraceable claims. The response was to make research quality a property of **process artifacts** rather than of memory or trust: a journal entry for every measurement, frozen documents identified by cryptographic hash, a register pairing every claim with its evidence and its limit, and an append-only decision log in which a reversed decision is never edited but superseded by an entry that links back. Working sessions are stateless; each rebuilds its understanding from these files, so a crashed session loses nothing.

Three further choices follow. The assistant is routed by *goals*: the plan defines end states, each phase is an executable document with ordered tasks, acceptance checks and exit criteria, and the assistant optimises for "the phase's evidence exists" rather than "the requested edit was made". The irreversible categories of action are enumerated as **gates** and reserved for the human, keeping the assistant's autonomy where mistakes are cheap and reversible. And commitments precede the observations that could bias them (annotations before engine output, behavioural pins before code changes, measurements before budgets), while the assistant's substantial outputs, the operating documents included, pass through verification passes prompted to *refute* them. Report sections are written while the work happens, because a report assembled afterwards turns memory into narrative.

10.2 Division of labour

The human is the principal investigator. He owns the research questions, every scientific commitment (protocol, population and opening freezes), every expenditure of compute, everything that leaves the machine, and the final word on every claim. The scientific responsibility for this report is his alone and cannot be delegated.

The AI assistant executes the plan. The assistant is Claude (Anthropic), operating as Claude Code sessions. Given the plan and the state files, it plans, implements, tests, measures, journals and drafts toward the plan's end state; sessions start from an intent

("continue", or a phase name) rather than from a task list. It implemented the encoders, the graph network, the training pipeline, the baselines and the evaluation harness inside the perimeter each phase document fixes; pinned existing behaviour before changing it; profiled before proposing budgets and piloted before campaigns; journaled every experiment with hypothesis, commit, configuration, seeds, data version, hardware, duration, cost, metrics, failures and interpretation; and drafted the report as it went. Its own decisions are logged with the same discipline, and several are explicit *proposals* that became binding only through the author's approval at a gate; the proposal to exclude the prior demonstration checkpoint from the opponent population is one example.

What the assistant never decides is listed in the operating rules: it never crosses a gate (no freezing, spending, publishing, contacting or destroying without a recorded human decision); never writes an oracle after seeing model output, because that contamination is irreversible; never produces an untraceable number; never touches a frozen artifact, since any post-freeze change is by definition a new study; and never makes a claim without a row in the claims register.

10.3 The six gates

The boundary is drawn mechanically rather than left to judgement. **G-FREEZE**: freezing a protocol, split or test set, and any later touch of a frozen artifact. **G-SPEND**: paid calls, purchases, budget caps, starting or restarting long compute jobs. **G-PUBLIC**: anything leaving the machine (pushes, licence choices, publication). **G-RIGHTS**: reuse of material whose ownership is not established. **G-ADMIN**: institutional contact. **G-DESTRUCTIVE**: deleting data, models or results, overwriting raw results, killing running jobs. At a gate the assistant records the pending request, parks that phase and routes to other work; a phase document saying "do X" is never authorisation to cross. Table 25 lists every gate decision the author took and summarises the basis of each; the decision log holds the author's approvals in their original wording.

Table 25: Gate decisions taken by the author during the study, in the order recorded in the decision log. G-ADMIN was not crossed; G-RIGHTS was resolved without ambiguity when the reference engines' licences were established; G-PUBLIC was crossed once, on 9 October 2026, for the first study's release.

Date	Gate	Decision	Basis and conditions recorded
2026-09-09	G-PUBLIC	Licence chosen (MIT, copyright 2026); remotes created, kept **private** until the research is finished	Decided on the author's written instruction; the remotes to become public only through a further decision at this gate, once the research is finished
2026-09-09	G-SPEND / G-DESTRUCTIVE	Prior self-play loop permanently stopped; never restarted	Decided on the author's written instruction; the loop's checkpoint preserved in the research copy; no restart permitted without a new decision
2026-09-09	G-FREEZE (review)	Protocol freeze deferred until the pilots had re-measured the proposed budgets with the study's own networks	Decided on a draft carrying measured values labelled as proposals; the freeze scheduled after the pilots so that it would consume confirmed values rather than proposals
2026-09-09	G-FREEZE	Opponent population frozen: legal-random, heuristic, search at 6,400 simulations;	Approved after the characterisation round-robin; the search opponent fixed at 6,400 simulations; the prior

Date	Gate	Decision	Basis and conditions recorded
		prior checkpoint excluded; hashes and engine commit recorded	checkpoint excluded; configuration and weight hashes and the engine commit recorded
2026-09-10	G-FREEZE	Protocol frozen as version 1.0, hash and commit recorded, no placeholders	Approved on the pilot's measured values; protocol hash and commit recorded; no placeholder left
2026-09-10	G-DESTRUCTIVE	Pre-study workspace archive deleted by the author directly; the research copy became the sole copy of the prior work	Carried out by the author himself; the research copy declared the sole copy of the prior work, its pre-study files placed under the destructive-action gate
2026-09-10	G-FREEZE	250 shared 4-ply openings frozen, generated blind from a documented seed	Approved on a file generated blind with nothing tunable; content and file hashes and the generator seed recorded
2026-09-10	G-SPEND	Main campaign: 2 arms × 3 seeds, 10 generations × 500 games, ≈4.6 days estimated	Approved at the full sizing among the options presented from measured per-game costs; runs sequential and resumable; raw results never overwritten
2026-09-20	G-SPEND	Ablation campaign: A1 and A2, 3 seeds each at full parity, ≈9 days estimated	Approved at full three-seed parity with the main comparison, among the sizings presented from measured graph-arm run costs
2026-09-26	G-SPEND	Supplementary two-component A1′ run at a corrected ≈4-day sizing	Approved at a corrected sizing of about four days, after the initial estimate had been revised; queued after the extension; its results confined to a labelled two-component supplement
2026-09-26	G-SPEND	Main comparison extended to 5 seeds per arm, ≈4.8 days, under three pre-commitments stated before any new run	Approved with three written pre-commitments (all-seeds analysis, cutoff unchanged, two-stage collection disclosed); seeds added symmetrically to both arms under identical settings
2026-10-09	G-PUBLIC	Repository made public at tag v2.0-report with the report in both languages, the short article and the records archive; preprint submission and a public post by the author approved	Decided on the author's written instruction after the final-control record and a pre-release scrub; the programme's other repositories stay private; the third-party reference engines had been found MIT-licensed, closing the rights question without ambiguity
2026-10-12	G-SPEND	Pilot of the second study: two generations × 200 games per arm under warm start and a three-generation window, one seed outside both studies' seed sets	Approved to measure the regime's cost and exercise the full path before the freeze; its scores enter no analysis
2026-10-13	G-FREEZE	Protocol v2 frozen: warm start, replay window of three generations, hyperparameter option A (no tuning), seeds 11–15, equal-time cutoff rule; hash and commit recorded	Approved on the pilot's measured budget; option B (an identical small tuning budget per arm) considered and rejected; any later change a new study
2026-10-13	G-SPEND	Second campaign: 2 arms × 5 seeds × 10 generations × 500 games, ≈295 h estimated, cutoff evaluations included	Approved after the freeze and launched the same day under the first study's execution rules (se-

Date	Gate	Decision	Basis and conditions recorded
			quential, resumable, disk guard, no other load on the machine)

Two entries deserve more comment than the table gives. The deferral of 9 September shows a gate working against haste: offered a protocol with measured values, the author waited for pilot confirmation so that the freeze would consume confirmed numbers rather than proposals. The 5-seed extension shows a post-hoc power decision kept from becoming a tuning channel: before any new run it was recorded that (a) the final analysis would use all five seeds per arm regardless of the new seeds' direction, (b) the equal-time cutoff would stay at the value computed on 16 September, never recomputed after seeing results, and (c) the report would disclose that seeds 4–5 were collected after the 3-seed analysis. The results chapters honour all three. The scope of this report was itself an authorial decision, recorded on 9 October 2026 with the constraint that no frozen artifact, number or claim could change.

10.4 Scientific principles enforced in every phase

Beyond the gates, the operating rules bind every phase regardless of who executes it.

1. **Oracles before model output.** Test positions and benchmark cases are annotated before any engine or model sees them; afterwards an unbiased expectation can no longer be written.
2. **Characterise before changing.** Existing behaviour is pinned with tests before modification; a failing pin is recorded as a finding rather than fixed silently.
3. **Everything is journaled**: identifier, date, hypothesis, commit, configuration, seed, data version, hardware, duration, cost, metrics, artifact paths, failures, interpretation.
4. **No untraceable numbers.** Every figure links to a journal entry or raw result; numbers inherited from the plan are labelled planning proposals until measured.
5. **Negative results are kept** and reported alongside the best runs.
6. **Fair baselines.** Compared methods receive the same model, budget and attempt count; a strawman baseline invalidates a study.
7. **Truncation is not a draw.** A game stopped at the move cap is a separate outcome, reported separately and sensitivity-tested.
8. **Multi-seed honesty.** No conclusion rests on one seed; per-seed results and uncertainty are reported, with the seed as the unit of resampling.
9. **Prior work is read-only.** Pre-study material is never edited in place; imports carry a provenance note; unclear rights go to G-RIGHTS.
10. **A claims register.** Every claim has a row (claim, evidence, section, limit) or it does not appear.
11. **Write as you go.** A phase is not closed until its report section exists: protocol before experiments, method during implementation, results only from frozen raw tables.

10.5 What the discipline caught

The operating rules require an append-only log of every incident in which the method caught something, missed something, changed an outcome through a gate, or cost real

overhead; entries are never forced. Table 26 assembles those entries with incidents recorded in the experiment journals.

Table 26: Incidents recorded by the methodology log and the experiment journals during the study: what happened, which rule or check caught it, and what followed.

Date	Incident	How it was caught	Consequence
2026-09-09	First draft of the phase documents held 33 defects, including a validation↔freeze deadlock and two unexecutable tasks	refutation-prompted verification pass	28 fixes, re-verified clean before any research ran
2026-09-09	A hand-annotated critical position disagreed with the engine	expectations committed before the run	resolved *against* the corpus (One-Hive transit case); record kept
2026-09-09	Prior self-play generators mapped the 300-ply cap to a draw	truncation principle applied retroactively at review	fixed before the pipeline phase; truncation a separate outcome everywhere
2026-09-09	Python–Rust boundary about to become a preference debate	measurement first: 21.7 µs per round-trip against ≥40 ms per decision	no in-process binding built
2026-09-09	Setup slip (piece ordering) in the tactical test set	five cases hand-annotated and committed before being run	engine right, setup wrong; journaled
2026-09-09	Search opponent scored only 37.5% against the heuristic; a 4× budget probe reached 56.2%	diagnosed at characterisation	deliberately **not** retuned: the population freeze forbids post-hoc tuning
2026-09-09	Two concurrent working sessions allocated the same decision identifier (**missed** at the time)	later cross-check	entry renumbered; single-writer rule for shared state files
2026-09-09	Overhead: the operating documents consumed a full working session (≈1M agent tokens, 17 subagents) before any research work	none	front-loaded fixed cost, accepted
2026-09-10	First pilot evaluation returned identical 0/2/28 against all three opponents, legal-random included	identical-results alarm; diagnosis order rules → signs → search → data	a shell loop had passed each opponent's flags as one argument: every match was against the heuristic; rerun
2026-09-18	Reproducing an all-truncated pair for a failure figure returned no records	every selected position must be reproduced and verified against its record	latent record-loss path fixed; audit: no campaign evaluation affected
2026-09-23	All three A1 ablation seeds returned game-count-identical evaluations	the same identical-results alarm	training had diverged to NaN at generation 0 and self-played on NaN for nine more generations; non-finite guards added; divergence kept as the finding
2026-10-09	Regenerated equal-time checkpoint map for seeds 1–3 differed from the journaled original	regenerated outputs checked against the journal before use	a rounded cutoff constant had broken an inclusive boundary; fixed, original reproduced exactly
2026-10-09	Generated arm-contrast caption still read "3 per arm" under 5-seed numbers	page-by-page render pass	generator derives the seed count and rewrites

Date	Incident	How it was caught	Consequence
			the file whole; every number byte-identical
2026-10-09	Stale seed counts in two chapters; stale "pending" limits in the claims register	placeholder sweep before assembly	fixed in both languages together
2026-10-09	Cost-table generator divided five runs' wall-clock by 3 (printing 30.0 h / 61.2 h for 18.0 h / 36.7 h) and carried a training-throughput row (≈770 / ≈195 pos/s) measured under pilot conditions but presented as campaign throughput	every number re-derived from its raw source with an explicit definition while writing this report	generator fixed (journaled benchmark 274 / 138 pos/s at batch 128, conditions stated); no score, interval or claim affected; per-paragraph source citations and a token-level traceability checker now gate the report
2026-10-09	The capacity difference "+2.1%" quoted in every document was the ratio of the rounded parameter counts (1.47/1.44 M); the exact counts give +1.5%	layer-by-layer tabulation of the architectures recounted the parameters from the code	corrected everywhere except the append-only journals; exact counts now a generated results artifact; no verdict depends on it
2026-10-12	The stamp check of the pretraining battery failed on the second study's windowed shard set: records of two generations against a single manifest	the existing pin, run on the new regime before the protocol was frozen	the check generalised to the manifests of every generation in the window rather than relaxed; it still passes on first-study shards and still fails on a wrong manifest
2026-10-27	The rejection rule's "meaningful" threshold, left non-numeric in both protocols, became operative: the second study's largest upper bound (+0.090) is no longer small enough for the verdict to be independent of a threshold	applying the rule as frozen	verdict stated with its bounds and the thresholds under which it holds; a numeric threshold to be pre-registered in any successor

None of these catches required insight; each came from a mechanical check. One alarm (independent conditions cannot agree to the game count) found a harness defect in the pilot and a silent numerical divergence in an ablation; one rule (commit the expectation, then run) found an annotation error and a setup error on the same day; the reproduce-and-verify step behind a figure found a record-loss path before the equal-time evaluations (where early graph checkpoints genuinely can truncate every game) would have tripped it. Two entries are *gate effects*: the population freeze visibly prevented a results-flattering retuning of the search opponent, and measure-before-commit dissolved an architecture argument with one number. At this scale, harness error was a larger threat than statistical noise, and only mechanical verification found it; the claims register carries this as a process observation from one study, whose stated limit is that no counterfactual exists for what an ungated workflow would have caught.

The cost side is recorded with the same honesty: the method's fixed cost was front-loaded, and the "missed" entry shows the first failure mode of multi-session operation: shared state files need write discipline as well as read discipline. The method did not prevent errors in drafts; it surfaced them before this version through translation passes, numeric identity checks between the two language versions, page renders and

adversarial re-reads. A check comparing two languages cannot, however, see a staleness shared by both. The last incident sharpened the standard: a number present in a file is not thereby provenanced; the file must itself derive it from raw data under a stated definition, which is why every paragraph of this report carries a source citation that is checked mechanically.

10.6 What the assistant did not do, and the author's responsibility

The assistant chose no hypothesis, froze nothing, spent nothing, published nothing, deleted nothing, and decided no claim; it wrote no test expectation after seeing engine output and did not recompute the equal-time cutoff after results were known. The limits of the arrangement are recorded too: the assistant can misread the plan, and the phase documents are its interpretation (the decision log records where reality corrected them); adversarial self-verification remains self-verification at the level of the programme, so the external human review of a subset of annotations and of the report, foreseen in the plan, is not replaced by it; and the assistant's training data may overlap the public Hive literature, so no claim of the form "unseen by the model" is made anywhere in this report.

AI-assistance declaration. Implementation, experiment execution, journaling and drafting for this study were carried out with substantial assistance from an AI coding agent (Claude Code, Anthropic), operating under the gated methodology described in this chapter. Every scientific decision, every frozen commitment and every expenditure was made by the author at a gate, as recorded in Table 25; every claim was made or verified by the author, who has personally checked the conclusions he signs and takes full responsibility for the results. The exact wording of this declaration will follow the rules of any venue at diffusion time.

Part IV. Results

11 Results: the main comparison (RQ-H1)

This chapter reports the pre-registered comparison exactly as the frozen protocol defined it: two arms, five independent seeds each, evaluated against the frozen three-opponent population on the 250 frozen openings, under two budget readings. Every table shows every seed. Scores exclude truncated games, which are reported in their own column; intervals are 95% percentile bootstrap intervals over seeds (10,000 resamples). The statistical procedures and the raw tables with all auxiliary columns are in Appendix D; the provenance of every number is in Appendix E.

11.1 The verdict in one paragraph

Under both budget readings, the graph arm shows no seed-consistent advantage against any opponent, and every interval on the graph–grid score difference excludes a meaningful graph advantage: the largest upper bound across the six contrasts is +0.028. The pre-registered rejection rule therefore fires, and **H1 is rejected**. At this budget, on base-game Hive, the graph representation did not help: it scored lower against two of the three frozen opponents under both readings, and no better against the third. The remainder of the chapter shows the evidence behind this paragraph, seed by seed.

11.2 Reading 1: equal number of training examples

In the first reading both arms are compared at their generation-10 checkpoints: each run has then generated and trained on ten generations of 500 self-play games under identical simulation budgets, so the two arms have consumed the same number of training examples.

Table 27: Same-examples reading. Final-checkpoint (generation 10) score of every run against each frozen opponent; 100 paired colour-swapped games per opponent per run on the 250 frozen openings at 400 simulations per move without exploration noise. Score = mean over decided games (win 1, draw 0.5, loss 0). Truncation = share of the 100 games stopped at the 300-ply cap; no game against B-HEU or B-MCTS was truncated by either arm.

Arm	Seed	B-RND score	B-RND truncation	B-HEU score	B-MCTS score
grid	1	0.995	0%	0.150	0.125
grid	2	0.975	1%	0.080	0.125
grid	3	0.990	1%	0.190	0.075
grid	4	0.949	11%	0.105	0.085
grid	5	0.995	0%	0.120	0.210
graph	1	0.733	57%	0.025	0.115
graph	2	0.981	20%	0.050	0.125
graph	3	0.722	55%	0.090	0.100
graph	4	0.688	44%	0.125	0.120
graph	5	0.935	23%	0.035	0.115

Against the legal-random opponent every grid seed scores at least 0.949 while graph seeds range from 0.688 to 0.981; against the heuristic, grid seeds range from 0.080 to 0.190 and graph seeds from 0.025 to 0.125; against the search opponent the two arms

overlap (0.075–0.210 against 0.100–0.125). The graph arm truncates 20–57% of its games against the random opponent; the grid arm 0–11%.

Table 28: Same-examples reading. Seed-level mean score per arm and opponent with the 95% bootstrap interval over the five seeds.

Arm	vs B-RND	vs B-HEU	vs B-MCTS
grid	0.981 [0.964, 0.994]	0.129 [0.098, 0.165]	0.124 [0.087, 0.168]
graph	0.812 [0.710, 0.920]	0.065 [0.034, 0.100]	0.115 [0.107, 0.121]

11.3 Reading 2: equal wall-clock on the same machine

In the second reading each run is taken at the last checkpoint it had completed within the equal-time cutoff of 18.77 hours, the median total wall-clock of the three original grid runs, computed on 16 September 2026 before any cross-arm comparison. Because the graph arm's generations are roughly twice as expensive on this machine, the cutoff selects generation 9 or 10 for the grid runs (the generation-10 checkpoint is reached within the cutoff by four of the five) and generations 3 to 6 for the graph runs: grid seeds 1–5 are read at generations 10, 10, 9, 10 and 10; graph seeds 1–5 at generations 4, 5, 5, 3 and 6 (counting the initial network as generation 1; in the run logs these checkpoints are labelled gen009/gen009/gen008/gen009/gen009 and gen003/gen004/gen004/gen002/gen005).

Table 29: Same-wall-clock reading. Score of every run at its last checkpoint completed within the 18.77-hour cutoff, against each frozen opponent; same evaluation as Table 27 (100 paired games per opponent, 250 frozen openings, 400 simulations, no noise). Truncation = share of games at the 300-ply cap; none against B-HEU or B-MCTS.

Arm	Seed	B-RND score	B-RND truncation	B-HEU score	B-MCTS score
grid	1	0.995	0%	0.150	0.125
grid	2	0.975	1%	0.080	0.125
grid	3	0.939	1%	0.145	0.080
grid	4	0.949	11%	0.105	0.085
grid	5	0.995	0%	0.120	0.210
graph	1	0.798	58%	0.045	0.075
graph	2	0.926	39%	0.055	0.110
graph	3	0.713	53%	0.060	0.105
graph	4	0.631	39%	0.100	0.150
graph	5	0.980	24%	0.045	0.095

Table 30: Same-wall-clock reading. Seed-level mean score per arm and opponent with the 95% bootstrap interval over the five seeds.

Arm	vs B-RND	vs B-HEU	vs B-MCTS
grid	0.971 [0.950, 0.991]	0.120 [0.098, 0.142]	0.125 [0.090, 0.168]
graph	0.810 [0.697, 0.922]	0.061 [0.047, 0.081]	0.107 [0.087, 0.130]

The picture of the first reading persists at equal time: the grid arm's means are higher against the random and the heuristic opponents and overlapping against the search opponent; the graph arm's truncation against the random opponent is 24–58%.

11.4 The arm contrast

Table 31: The pre-registered contrast, the difference of seed-level mean scores (graph arm minus grid arm), with the 95% bootstrap interval over seeds (five independent seeds per arm, resampled independently, 10,000 resamples). A negative value favours the grid arm.

Opponent	Same-examples: graph − grid	Same-wall-clock: graph − grid
B-RND (legal-random)	−0.169 [−0.272, −0.062]	−0.161 [−0.278, −0.048]
B-HEU (heuristic)	−0.064 [−0.111, −0.017]	−0.059 [−0.087, −0.029]
B-MCTS (search, 6,400 simulations)	−0.009 [−0.055, +0.028]	−0.018 [−0.066, +0.025]

Four of the six contrasts exclude zero, all in the grid arm's favour; the two contrasts against the search opponent straddle zero.

Against the legal-random opponent the graph–grid difference is −0.169 [−0.272, −0.062] at equal examples and −0.161 [−0.278, −0.048] at equal time; against the heuristic, −0.064 [−0.111, −0.017] and −0.059 [−0.087, −0.029]; against the search opponent, −0.009 [−0.055, +0.028] and −0.018 [−0.066, +0.025]. The largest upper bound of any interval is +0.028, which is the largest graph advantage the data leave room for under the most favourable reading and opponent.

The frozen rejection rule required, under both readings, the absence of a seed-consistent graph advantage together with intervals excluding a meaningful graph advantage. Both conditions hold: no opponent shows a consistent per-seed ordering in the graph arm's favour (Table 27, Table 29), and no interval admits an advantage above +0.028. H1 is rejected. The result is consistent across seeds and holds under the two budget equalisations; it does not rest on a single interval.

Five seeds per arm bound the statistics: against the search opponent, where both arms score around 0.1, the data cannot distinguish the arms. The rejection concerns this simple relational message-passing network at this capacity and this budget; it says nothing about graph representations at larger budgets or with other architectures.

11.5 How the five-seed analysis relates to the three-seed analysis

The protocol pre-registered three seeds per arm with an option of five. The three-seed analysis of 19 September 2026 already rejected H1 under both readings (contrasts against the random opponent −0.175 [−0.268, −0.007] and −0.158 [−0.254, −0.050]; against the heuristic −0.085 [−0.143, −0.025] and −0.072 [−0.100, −0.028]; against the search opponent +0.005 [−0.020, +0.035] and −0.013 [−0.040, +0.015]). Seeds 4 and 5 were then added to both arms under written pre-commitments made on 26 September 2026: the final analysis would use all five seeds under the same rule, the cutoff would stay unchanged, and the two-stage collection would be disclosed. The extension tightened four of the six intervals, absorbed the single graph-better seed pair observed against the heuristic (graph seed 4 at 0.125 against grid seed 4 at 0.105) and the best search-opponent score of any run (grid seed 5 at 0.210), and did not change the verdict. The five-seed tables above are the study's final numbers.

11.6 Can the move cap rescue the hypothesis?

Truncation was defined as a separate outcome before the first run, and the protocol required the primary result to be recomputed under alternative treatments of truncated games. The strongest treatment in the graph arm's favour scores every truncated game as a win for the arm under test, which is an upper bound on what any larger cap could deliver. Under that bound, computed from the per-game records of the three original seeds, the graph–grid difference against the random opponent is still −0.072 at equal examples and −0.058 at equal time; the direction of the contrast is unchanged under every treatment (excluded, scored 0.5, scored as loss, scored as win). The cap value therefore cannot reverse the verdict; it can only change its size.

11.7 Where the deficit lives

Putting the three opponents side by side shows where the deficit lies: it is largest against the weakest opponent and absent against the strongest. Against legal-random, where strength is mostly the ability to finish a surround that nothing resists, the graph arm wins material and then fails to convert; the truncation column is the trace of that failure. Against the heuristic, which attacks the queen directly, the graph arm's defence is weaker than the grid arm's. Against the search opponent, both arms are in a low-score regime where ten generations of small-budget self-play have not produced an attack either arm can sustain. Section 14 examines these mechanisms on individual games; Section 16 weighs the explanations.

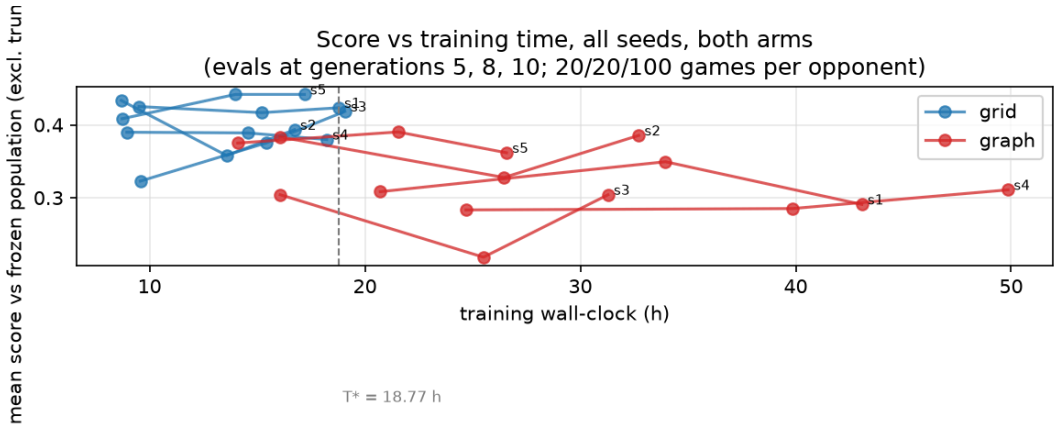


Figure 5: Mean score against the frozen population (average of the three opponents, truncations excluded) as a function of cumulative training wall-clock, for all ten main runs; evaluations at generations 5, 8 and 10 (20, 20 and 100 games per opponent). Blue: grid arm; red: graph arm; one line per seed. The dashed vertical line marks the equal-time cutoff of 18.77 hours.

Figure 5 shows the two readings at once: at any given wall-clock the grid arm sits above the graph arm, and at the cutoff the graph runs have completed less than half of their generations. Section 12 quantifies the cost side.

12 Results: cost and efficiency (RQ-H2)

The second research question asks how the two representations compare when the budget is read as a number of training examples and when it is read as time on the same hardware. The protocol required both readings and required every run to report its wall-clock, its hardware, the number of training states consumed and the simulations per decision. This chapter reports those costs and puts the two readings side by side.

12.1 What each arm costs on the study machine

Table 32: Cost and score summary per arm (H-T2). Training wall-clock covers self-play generation and training for the ten generations of a run, evaluation games excluded, averaged over the five seeds; the equivalent self-play cost divides it by the 5,000 games of a run. Inference and training-throughput figures are benchmark measurements of 10 September 2026 on the study machine (Apple M1 Pro, 10 cores, 16 GB). Population score = mean over the three frozen opponents of the seed-level mean score (truncations excluded).

Metric	Grid arm	Graph arm
Parameters	1.44 M	1.47 M (+1.5%)
Best-provider inference, batch 1	2.62 ms (neural accelerator)	3.67 ms (CPU)
Training wall-clock per run, mean of 5 seeds (10 generations × 500 games)	18.0 h	36.7 h (2.0×)
Equivalent self-play cost	13.0 s/game	26.4 s/game
Training throughput benchmark (batch 128, forward + backward)	274 pos/s	138 pos/s
Population score, same-examples reading	0.411	0.331
Population score, same-wall-clock reading (cutoff 18.77 h)	0.405	0.326

The two networks are matched in size. The graph arm's single evaluation is 1.4× slower at its best provider, its training throughput is half that of the grid arm, and a full run costs it twice the wall-clock. Per run, the five grid seeds took 18.77, 16.73, 19.07, 18.23 and 17.19 hours of training wall-clock; the five graph seeds took 43.07, 32.71, 31.28, 49.86 and 26.57 hours.

The measured wall-clock ratio is 2.0× (36.7 h against 18.0 h on average). At the sizing stage the ratio had been estimated at 1.7×; the measured ratio over the first five completed runs was 2.1×, and the estimate was replaced by the measured figure in all budget accounting before any comparison was computed.

Self-play generation dominates the wall-clock of both arms; in the pilot, generation 0 took about 60 minutes of self-play against about 3.5 minutes of training. The graph arm's extra cost is therefore paid mostly in the search, through its slower per-position inference and its CPU execution. The asymmetry is a property of the deployment hardware: the convolutional network maps cleanly onto the machine's neural accelerator (2.62 ms per evaluation), whereas the gather-heavy graph network falls back to the CPU for a large share of its operations and runs fastest there (3.67 ms on the CPU against 9.84 ms on the accelerator, with 147 of 287 operator nodes supported). The protocol chose to report and charge this asymmetry rather than to equalise it away. The same-wall-clock reading

thus reflects what each representation costs on this machine, while the same-examples reading is unaffected by it.

On a different accelerator the ratio could change in either direction; on the CPU alone the graph network is the faster of the two at inference (3.67 ms against 23.5 ms). The cost figures in this chapter are therefore specific to this hardware by construction; the parameter counts and the throughput figures are the portable part.

12.2 The two readings side by side

Under the same-examples reading, both arms are compared after ten generations; the grid arm's population score is 0.411 and the graph arm's 0.331. Under the same-wall-clock reading, each run is taken at its last checkpoint within 18.77 hours (generation 9 or 10 for the grid runs, generation 3 to 6 for the graph runs), and the population scores are 0.405 and 0.326. The two readings therefore give the same ordering and nearly the same gap: 0.081 at equal examples and 0.079 at equal time, in the grid arm's favour.

The pre-registered reason for reading the budget twice was the possibility of a representation that learns more per example but less per hour, in which case the two readings would disagree and both would have to be reported. Here they agree: the graph arm learns less per example and also less per hour. The equal-time reading does not create the deficit; it enlarges one that the per-example reading already shows, since within the cutoff the graph arm completes fewer than half of its generations.

The population score averages three opponents of very different difficulty, so it is a summary for this comparison only and hides the opponent structure of the result; the per-opponent tables of Section 11 are the primary evidence. Scores at the intermediate generations (5 and 8) come from 20-game evaluations sized for monitoring and do not enter any interval.

12.3 Score as a function of time

Figure 5 in Section 11 plots the population score of every run against its cumulative training wall-clock. Two features of the curves answer RQ-H2 directly. First, at every wall-clock at which both arms have an evaluation, every grid run lies above every graph run; the arms' bands do not cross. Second, the grid runs' curves flatten after their first evaluations (against the random opponent they are near the ceiling by generation 5), while the graph runs' curves keep moving non-monotonically throughout their longer runs; at the cutoff the graph runs have had 3 to 6 generations and are still in the regime where seed-to-seed variation dominates. Within this budget, the extra time gave the graph arm more generations without producing a crossing.

12.4 Resources consumed by the study

The main comparison consumed ten training runs totalling about 274 hours of training wall-clock (five grid runs, 90.0 h; five graph runs, 183.5 h), plus the evaluation games (100 games per opponent per checkpoint at roughly 23–32 seconds per game at 400 simulations, for the final and the cutoff checkpoints of every run) and the monitoring evaluations at generations 5 and 8. The three ablation cells added nine graph-arm runs (training wall-clock under the same accounting: A1 7.61–8.01 h each because the divergent

networks played short games; A2 32.05–47.44 h; A1′ 23.89–30.35 h). All computation ran sequentially on one laptop kept awake, with no cloud or paid resources; the campaigns span 10 September to 6 October 2026.

13 Results: component ablations (RQ-H3)

This chapter answers the third research question: which of the graph arm's two distinctive components, its six direction-typed relations and its global-pooling bias, carry its behaviour. Each ablation removes one component and nothing else, at the full method's settings (same budgets, same seeds, same frozen opponents and openings, same evaluation), with three seeds per cell; a third run, explicitly labelled as a two-component change, supplements the first ablation. Throughout, nothing is attributed beyond the single component that changed.

13.1 Design of the three cells

Table 33: The ablation cells. Parameter counts are measured; the parameter differences are inherent to the removed components and are reported rather than compensated.

Cell	What changes relative to the full graph arm	Parameters	Seeds	Status
A1: naive adjacency	the six direction-typed message matrices are replaced by a single shared matrix	0.54 M	3	one-component, at parity
A2: no global pooling	the global-pooling bias is removed from every layer	1.37 M	3	one-component, at parity
A1′: naive adjacency, stabilised	as A1, plus gradient-norm clipping at 1.0	0.54 M	3	**two-component**, never attributed to typing alone
Reference: full graph arm	none	1.47 M	5	main comparison

13.2 A1: removing the typed relations removes trainability

With one shared message matrix in place of six typed ones, training diverged to non-finite values in generation 0 for all three seeds; every later generation self-played and trained on non-finite network outputs. The symptom that triggered the diagnosis was statistical rather than numerical. The three seeds' final evaluations were identical down to the game count (for instance 0 wins, 0 draws and 100 losses against the heuristic for each seed), which independent seeded trainings cannot produce. The checkpoints have distinct hashes, but every forward pass returns non-finite policy and value from the first checkpoint onward.

The cell's evaluation tables are therefore not strength measurements, since a non-finite policy plays a deterministic, degenerate game independent of its weights; they are excluded as scores, and the H-T3 cell reads "training diverged (3/3 seeds)". The runs took 7.75, 8.01 and 7.61 hours of training wall-clock against a graph-arm mean of 36.70 hours, because non-finite priors degrade the search into short games of about 42 plies with no truncations and no resignations; the configured budget in games and simulations was fully consumed.

At parity settings, the naive-adjacency variant cannot be trained at all, so the typed relations contribute, at minimum, the optimization stability of the whole arm. Nothing

about their contribution to playing strength can be measured at parity, because the variant never trains. The explanation offered here, a reasoned argument rather than a measured decomposition, is that a single shared matrix receives the summed gradient of six neighbour terms, roughly six times the per-matrix gradient scale of the typed variant, at an identical learning rate and momentum. The protocol had stated, before any run, that a naive adjacency graph "may not suffice"; under these conditions it does not even optimize.

No learning-rate sweep was run, because changing the learning rate would have made A1 a two-component difference. The silent passage of non-finite values for ten generations was a tooling gap: the training loops had no non-finite guard, and their loss print cadence (every 100 steps) never fired on epochs of about 20 steps. The gap was closed on 23 September 2026 by halting both training loops loudly on a non-finite loss; the behaviour of healthy runs is unchanged, and the ablation result stands.

13.3 A2: removing global pooling changes nothing measurable

Table 34: Ablation A2: final-checkpoint scores per seed (same-examples reading; truncation rate in parentheses where non-zero; 100 games per opponent per seed on the 250 frozen openings) and the seed-bootstrap interval on the difference against the full graph arm's three original seeds (10,000 resamples, independent seed sets).

Opponent	A2 (no pooling), seeds 1/2/3	Full graph arm, seeds 1/2/3	Difference A2 − full [95% interval]
B-RND	0.768 (59%) / 0.714 (65%) / 0.952 (17%)	0.733 (57%) / 0.981 (20%) / 0.722 (55%)	−0.001 [−0.170, +0.165]
B-HEU	0.050 / 0.070 / 0.025	0.025 / 0.050 / 0.090	−0.007 [−0.043, +0.030]
B-MCTS	0.070 / 0.125 / 0.105	0.115 / 0.125 / 0.100	−0.013 [−0.043, +0.013]

All three A2 runs trained healthily (finite outputs, distinct per-seed results); their training wall-clocks of 39.90, 47.44 and 32.05 hours lie within the full graph arm's band (26.57–49.86 h).

Removing the global-pooling bias changed the score by −0.001 [−0.170, +0.165] against the random opponent, −0.007 [−0.043, +0.030] against the heuristic and −0.013 [−0.043, +0.013] against the search opponent. Every interval straddles zero, and the random-opponent cell is dominated by seed variance in both variants (truncation rates between 17% and 65% across seeds).

The hypothesis that pooling carried the graph arm's value learning is not supported. Eight layers of directional message passing alone reproduce the full arm's behaviour, including its failure mode: the conversion pathology against the random opponent is present in both variants. At this scale the global-pooling bias is dispensable.

The cell has three seeds; the 0.10 M parameter difference is inherent to the removed component; and "no measurable effect" is bounded by intervals of width ±0.17 against the random opponent, where a small effect could hide.

13.4 A1′: the stabilised naive-adjacency variant (two-component supplement)

Table 35: Supplement A1′, untyped relations plus gradient clipping at 1.0 (two components differ from the full method). Final-checkpoint scores per seed, three-seed means, the full graph arm's five-seed means, and the seed-bootstrap interval on the difference over the two unequal seed sets.

Opponent	A1′ seeds 1/2/3 (truncation)	A1′ mean (3 seeds)	Full graph mean (5 seeds)	Difference [95% interval]
B-RND	0.566 (47%) / 0.671 (59%) / 0.995 (0%)	0.744	0.810	−0.066 [−0.254, +0.130]
B-HEU	0.035 / 0.135 / 0.110	0.093	0.061	+0.032 [−0.011, +0.078]
B-MCTS	0.090 / 0.195 / 0.060	0.115	0.107	+0.008 [−0.040, +0.063]

With a single added stabiliser, all three trainings stayed finite and the non-finite guard never fired, which confirms the remedy implied by the divergence diagnosis. Training wall-clocks of 23.89, 30.35 and 27.68 hours lie at the fast end of the graph arm's band.

The stabilised variant scores within the full graph arm's band against all three opponents: −0.066 [−0.254, +0.130], +0.032 [−0.011, +0.078] and +0.008 [−0.040, +0.063]; every interval straddles zero.

Taken together with A1, the most that can be said is that at this scale the measurable contribution of direction-typed relations lies in optimization stability; no strength contribution beyond it is detectable.

Every number in this cell is confounded by the gradient clip by construction and is never attributed to edge typing alone. The cell has three seeds and wide intervals, and seed 3′s 0% truncation against 47–59% for seeds 1–2 shows that the conversion pathology remains seed-volatile in this variant too.

13.5 Summary table H-T3

Table 36: Component attribution for the graph arm (H-T3). Differences are variant minus full graph arm, seed-level means, three seeds per ablation cell.

Ablation	Component removed	Outcome
A1 naive adjacency	direction-typed relation matrices → one shared matrix	training diverged (non-finite, generation 0, 3/3 seeds); untrainable at parity; typed relations ⇒ at minimum optimization stability
A2 no global pooling	global-pooling bias in all layers	null effect: −0.001 / −0.007 / −0.013 against the three opponents, all intervals straddling zero; failure modes unchanged; pooling dispensable
A1′ supplement (two-component)	untyped relations + gradient clipping 1.0	trains; scores within the full arm's band (−0.066 / +0.032 / +0.008); never attributed to typing alone

Within the tested perimeter, the answer to RQ-H3 is that of the graph arm's two distinctive components, one is necessary for trainability itself and the other is dispensable. Whether the typed relations also contribute playing strength beyond stability cannot be decided from these cells.

14 Results: failure positions and training behaviour

Aggregate scores say that the graph arm learned less; they do not say how it fails. This chapter looks at three individual games selected by mechanical criteria, at the truncation channel that carries the mechanism, and at what the training metrics of all ten main runs show.

14.1 Three failure positions, selected mechanically

The research plan asks for three commented failure positions with explicit selection criteria. The criteria are mechanical and were applied over the raw per-game records: F1 is the longest truncated game of the graph arm against the legal-random opponent; F2 is the shortest decided loss of the grid arm against the heuristic opponent; F3 is the longest drawn game of the graph arm against the search opponent. Each selected game was reproduced deterministically from its recorded opening and per-game seeds and verified move by move against its recorded result row before being rendered; all three reproductions matched. This verification step exposed a latent defect in the match runner (a path that skipped writing records when every game of a match was truncated); the defect was fixed before the equal-time evaluations, and an audit found that it had affected no campaign data (Section 10).

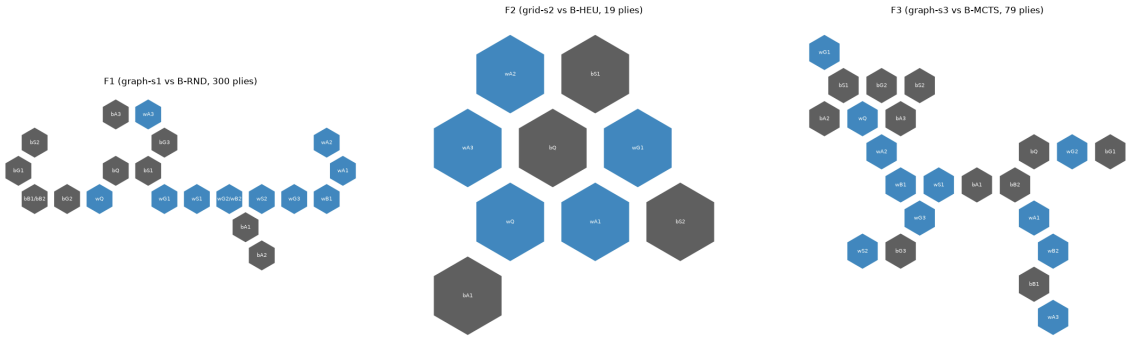


Figure 6: Three commented failure positions, reproduced and verified against their recorded result rows. F1 (left): graph arm, seed 1, against legal-random, opening 2, white; the game reaches the 300-ply cap with the graph arm materially ahead but unable to complete the surround. F2 (centre): grid arm, seed 2, against the heuristic, opening 2, black; a decided loss in 19 plies to the heuristic's queen-targeting tactics. F3 (right): graph arm, seed 3, against the search opponent, opening 19, white; a 79-ply draw in which the graph arm never generates a winning threat.

F1: a won position the graph arm cannot close. The graph arm wins material early, then shuffles pieces around a partially surrounded enemy queen until the cap. The failure is tactical rather than evaluative: the arm is ahead and stays ahead, but it does not find, or does not prefer, the forcing sequence that completes the surround. This is the pattern behind the arm's 20–57% truncation rate against the random opponent, and it is invisible in any evaluation that scores capped games as draws.

F2: the price of early-regime policies against sharp tactics. The grid arm's shortest loss against the heuristic lasts 19 plies. The heuristic's dominant queen-liberty term drives it straight at the queen before the network's policy has consolidated a defence. The game illustrates why both arms score low against the two strong opponents after ten generations, since the regime is still early, and why the comparison's resolution is highest against the random opponent.

F3: avoiding defeat without creating threats. Against the search opponent, the graph arm's longest draw (79 plies) shows a network that defends adequately, in that it avoids losing, but never builds a winning attack. Against this opponent the two arms are statistically indistinguishable (Section 11); F3 is a reminder that "indistinguishable" here means both arms draw or lose, not that either plays well.

14.2 The truncation channel, run by run

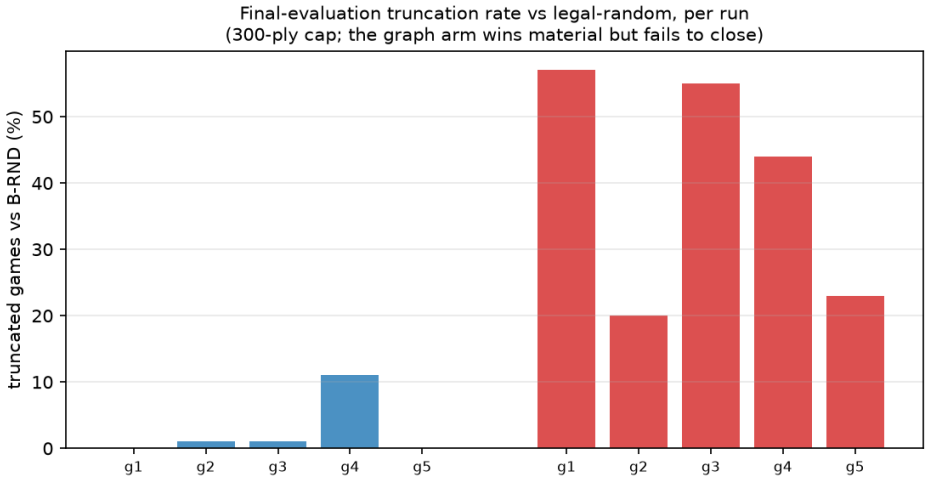


Figure 7: Final-evaluation truncation rate against the legal-random opponent for each of the ten main runs (final checkpoints; 100 games per run at 400 simulations on the 250 frozen openings; 300-ply cap). Blue: grid arm seeds 1–5; red: graph arm seeds 1–5. Truncation is reported separately from draws throughout the study.

The graph arm truncated 57%, 20%, 55%, 44% and 23% of its games against the random opponent at the final checkpoint for seeds 1 to 5; the grid arm truncated 0%, 1%, 1%, 11% and 0%. Against the heuristic and the search opponent, no game of either arm reached the cap.

The truncation gap is the largest behavioural difference between the arms, larger than any score gap, and its direction is consistent across seeds: every graph seed truncates more than every grid seed. Grid seed 4 (11%) exceeds no graph seed, but it shows that the pathology is not strictly exclusive to one arm. Because truncation was defined as its own outcome before the first run, the pathology is visible instead of being absorbed into draws; and because the cap cannot rescue the hypothesis (Section 11), the channel is diagnostic rather than decisive.

The graph arm's random-opponent score is consequently measured on fewer decided games (between 43 and 80 per seed), which widens its interval; the sensitivity analysis bounds this effect without removing it.

14.3 What the training metrics show

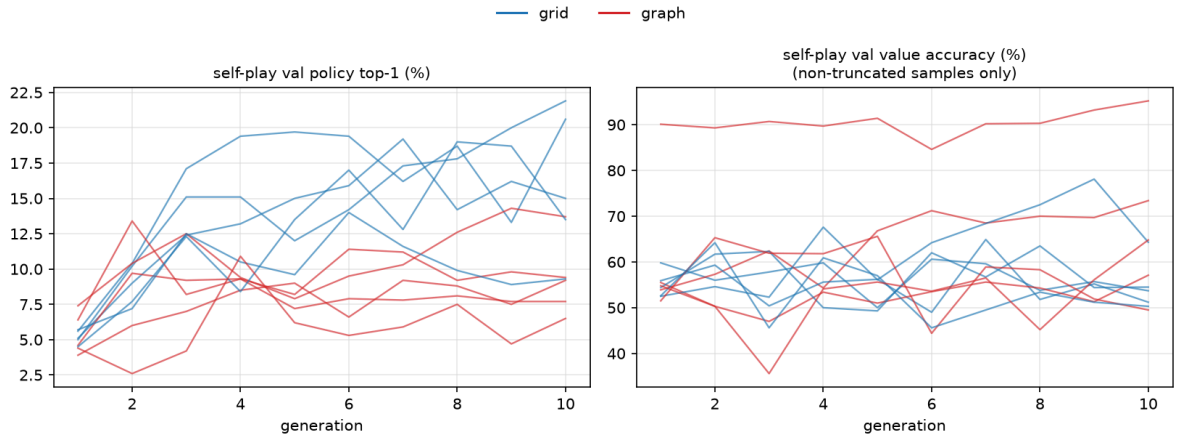


Figure 8: Training metrics by generation for the ten main-campaign runs (epoch-1 line of each generation): left, policy top-1 agreement with the MCTS visit distribution on the self-play validation split; right, value accuracy on non-truncated samples. Blue: grid arm; red: graph arm; one line per seed. Ablation runs are excluded.

Both arms fit their self-play data throughout the ten generations: policy top-1 rises with generation in every run, and value accuracy on non-truncated samples rises for both arms. The graph arm's value accuracy is comparable to the grid arm's, while its policy top-1 trails.

The gap between the arms is therefore not a plain optimization failure of the full graph arm (unlike the ablated naive-adjacency variant, which did not optimize at all). The graph arm learns a usable evaluation of positions and a weaker policy. The failure positions give the same picture, adequate judgement of who is better and inadequate generation of forcing moves, and this motivates the first follow-up in Section 17.

Training metrics are computed on each run's own self-play validation split, whose distribution shifts with the generation and differs between arms; they are evidence about fitting, not about strength, and no number from this figure enters any interval or any claim of the main comparison.

14.4 Per-opponent trajectories

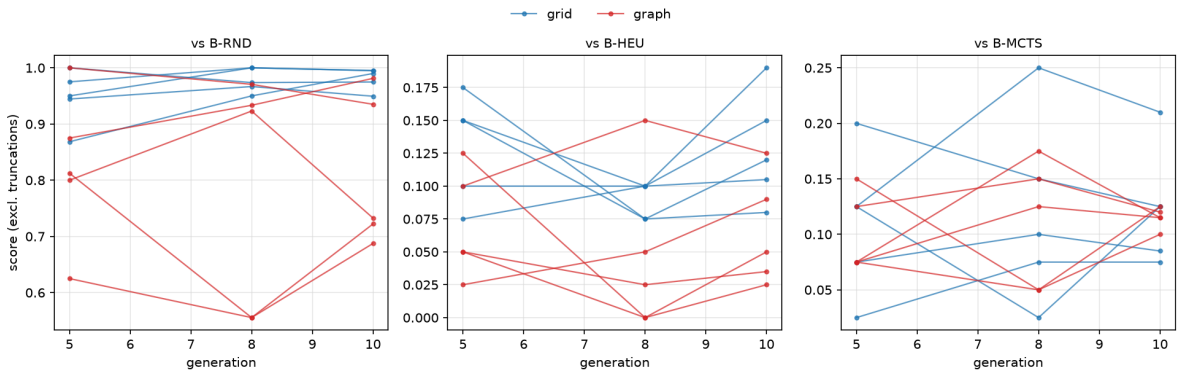


Figure 9: Score against each frozen opponent at the evaluated generations (5, 8 and 10; 20, 20 and 100 games per opponent respectively) for every main run. Blue: grid arm; red: graph arm; one line per seed. Scores exclude truncated games.

The per-opponent trajectories decompose the aggregate curves of Section 12: against the random opponent, grid seeds approach the ceiling early and stay there, while graph seeds spread widely and move non-monotonically; against the heuristic and the search opponent, both arms remain in a low band whose generation-to-generation movement is of the order of the game-count noise at 20 games (the intermediate evaluations were sized for monitoring and not for inference; only the 100-game final evaluations enter the tables).

15 A second study: the comparison under a warm-started loop with a replay window

The first study answered its question inside a pipeline that trained every generation's network from a fresh initialisation on that generation's games alone (Section 6, Section 16). This chapter reports a second, separately pre-registered study that applies the two standard remedies, weights carried across generations and training on a window of recent generations, identically to both arms, and asks whether the ordering of the two representations survives. Its protocol was frozen on 13 October 2026, before any run; its campaign started the same day. Everything the first study froze (opponents, openings, evaluation settings, networks, decoder) is reused untouched, so that the two studies differ in the training regime and in nothing else.

15.1 Why a second study

The per-generation re-initialisation of the first study is neutral for the comparison, since both encoders saw exactly the same records under the same conventions, but it bounds the regime reached: each network learned from the positions of 500 games, and the absolute strength after ten generations stayed near the ceiling against the random opponent and around 0.1 against the two strong opponents. A reviewer's first objection to the generality of a negative verdict is therefore that the graph representation might need more data per network, or a longer optimisation trajectory, before its structural prior pays. The large self-play systems carry weights from generation to generation and train on a replay window of recent games; a second study under that regime is the direct test of the objection. The question it asks is the first study's question under the new regime (RQ-H1′), with the same hypothesis (H1′): the graph arm reaches a higher mean score against the frozen population than the grid arm at equal budget. A secondary, descriptive question asks whether the graph–grid contrast moves toward zero or changes sign relative to the first study; the two studies are compared side by side and never pooled.

15.2 What changed, and what did not

Exactly two things change, in both arms. First, the network of generation g is initialised from the final checkpoint of generation $g-1$ (model weights only; the optimizer and the cosine learning-rate schedule restart at every generation, as before), generation 0 starting from the seeded random initialisation. Second, generation g trains on the records of generations $\max(0, g-2)$ to g, a replay window of three generations: 500 games at generation 0, 1,000 at generation 1 and 1,500 from generation 2 onward, still for two epochs, so that the number of gradient steps grows with the window.

The hyperparameter policy was decided by the author at the freeze and recorded with it. Both arms keep the first study's training settings unchanged (learning rate 0.02, batch 256, weight decay 10^{-4}, value-loss weight 0.6, two epochs per generation, legal-masked policy loss; 96 channels and 8 blocks for the grid network, 152 hidden units and 8 layers for the graph network). The alternative considered was an identical, pre-registered

budget of three learning rates (0.02, 0.01 and 0.005) for both arms on the first seed only, the best rate per arm then used for the remaining seeds. It was rejected because it would have tripled the cost of one seed per arm (about four more days of compute), selected on a single seed, and the pilot described below showed no sign of optimisation instability under the warm-started regime. The objection that both arms run under untuned defaults therefore remains open for both studies and is stated as a limit.

Everything else is the first study's: the engine and its search settings (128 and 32 simulations with playout-cap randomization, exploration noise, 12 temperature plies, resignation below −0.92 with the 10% audit, the 300-ply cap), the record format, the loss and masking, both encoders and networks at 1,443,168 and 1,465,452 parameters, the shared decoder, the three frozen opponents, the 250 frozen openings, the pinned evaluation (400 simulations, no noise, 100 paired colour-swapped games per opponent), the statistics (the seed as the unit, 10,000-resample percentile bootstrap, independent seed sets for the contrast, truncation as an outcome of its own) and the rejection rule, which is the first study's rule word for word in its form: no seed-consistent graph advantage under both budget readings, and seed-level intervals on the graph–grid difference excluding a meaningful graph advantage, with the largest upper bound reported so that any threshold can be applied by the reader. Intermediate evaluations are descriptive and enter no verdict. The new runs use seeds 11 to 15, so that no random stream is shared with the first study's seeds 1 to 5.

The design is again two arms, five seeds, ten generations of 500 games, read twice. The same-examples reading compares final checkpoints. The same-wall-clock reading compares each run's last checkpoint completed at or before an equal-time cutoff defined, as in the first study, by a score-free rule written before any run: the median of the total training wall-clock of the five grid runs of this study, each total being the sum of the ten per-generation clocks (self-play, training and export; evaluations excluded), computed once from the clock files when the five grid runs exist and before any cross-arm number is assembled, and never recomputed. The cutoff of the second study is its own value, measured under the new regime; it is not the first study's 18.77 h.

The pre-commitments repeat the first study's: all five seeds per arm enter the final analysis whatever their direction; a seed is excluded only for a crash, a corruption or an engine error, each recorded; no opponent, opening, evaluation setting, hyperparameter or architecture is touched; the chapter is written from the frozen tables. The stop rules are unchanged, with one addition made explicit: a running campaign is never killed or restarted without the author.

15.3 Implementation and verification before the freeze

The two changes were implemented as two options of the campaign driver, identical for both arms and switched off by default, so that the driver with default options still produces the first study's commands. That property was pinned by a regression test before the protocol was frozen: for four runs of the first study (two per arm) and their ten generations each, the training command produced by the driver with default options was compared with the recorded training configuration of every generation (shard list, seed, epoch count, absence of an initialisation checkpoint), and the self-play command with the recorded manifest (seed, number of games, simulation budgets, generation index); all forty generations matched. The same test checks that, with the two options on, only

the shard list and the initialisation checkpoint change, that generation 0 never warm-starts, and that generation g initialises from generation $g-1$. The second study's runs are written in their own directories, and each run records its regime in a configuration file, so that the first study's records cannot be touched or confused with the new ones.

A pilot approved by the author on 12 October 2026 then ran the complete path on both arms, one seed outside both studies' seed sets, two generations of 200 games, warm start and three-generation window, with a reduced final evaluation of 20 games per opponent that served as a check of the path and entered no analysis. Table 37 gives the measured phases. On both arms the second generation trained from the first generation's weights on the shards of both generations, as the configuration files and the training logs record; training time grew in proportion to the number of records (a factor 2.15 on the grid arm and 2.02 on the graph arm when the second generation entered the window) and remained a small share of a generation, because self-play dominates: in the first study training took 3% of a grid run (194 to 250 s per generation) and 5% of a graph run (516 to 833 s). With the window reaching three generations from generation 2, training time over a run is about 2.7 times the first study's, which adds about one hour to a grid run and three hours to a graph run. The projected cost of the campaign is therefore the first study's plus that increment: about 18 to 20 h per grid run, 30 to 53 h per graph run, about 295 h or 12 to 13 days of sequential laptop time for the ten runs, plus about one day of evaluations.

Table 37: Pilot of the second study (12 October 2026): wall-clock of the self-play and training phases of each generation, one seed per arm, 200 games per generation, warm start and three-generation replay window, on the same laptop as the campaigns. Generation 1 trains on the records of generations 0 and 1; the training record count excludes the validation split. Export took 2 s per generation on both arms. The pilot measured costs and exercised the path; its 20-game evaluation carries no statistical meaning and is not reported.

Arm	Generation	Self-play (s)	Training (s)	Training records (shards)
grid	0	2,196	78	10,188 (4)
grid	1	2,584	168	22,399 (8)
graph	0	5,821	303	10,655 (4)
graph	1	5,800	613	21,386 (8)

The seven automated checks of the pipeline (Section 6) were then run on each pilot run's windowed shard set. One check failed as designed. The stamp check required every record of a training set to carry the generation-and-network stamp of a single manifest, which encodes the first study's one-generation-per-training assumption; on the windowed set it reported records of two generations against one manifest. The check was generalised rather than relaxed: it now takes the manifests of every generation in the window and requires the set of record stamps to equal the set of listed stamps, so that every record comes from a listed generation and every listed generation is present; it still passes on a first-study shard set with one manifest and still fails with a wrong manifest. With that generalisation all checks passed on both arms (policy normalisation under the legal mask on a real forward pass, legality of all stored targets, outcome domain with 17,054 of 22,856 grid records and 11,298 of 21,822 graph records truncated and excluded from the value loss, record stamps, bitwise-identical save and resume, and separation of training inputs from the evaluation files).

15.4 Execution

The protocol was frozen on 13 October 2026 under its content hash, with the hyperparameter policy, the window of three generations and the cutoff rule decided by the author, and the campaign was approved and launched the same day at 15:02 CET, after a check that the machine was idle, that 161 GB of disk were free against a 20 GB guard, and that the frozen protocol's hash was unchanged. The ten runs execute sequentially on the same laptop as the first study, interleaved grid then graph for each seed from 11 to 15, so that an interruption would leave both arms with equal seed counts; each run is resumable at generation granularity from its recorded manifests, and intermediate evaluations of 20 games per opponent at generations 5 and 8 serve the trajectory figures only. No other heavy process runs on the machine during the campaign, since the per-generation clocks are the measurement of the same-wall-clock reading. Table 38 records each run as it completes.

Table 38: Execution record of the second study's campaign: start and end of each run (CET) and its total training wall-clock, the sum of the ten per-generation clocks (self-play, training and export; evaluation time excluded). Ten generations of 500 games per run, warm start and three-generation replay window, sequential on one laptop.

Run	Start	End	Training clock (h)
grid, seed 11	13 Oct 2026 15:02	14 Oct 2026 09:33	17.46
graph, seed 11	14 Oct 2026 09:33	16 Oct 2026 02:39	39.28
grid, seed 12	16 Oct 2026 02:39	16 Oct 2026 21:46	17.95
graph, seed 12	16 Oct 2026 21:46	18 Oct 2026 13:40	37.33
grid, seed 13	18 Oct 2026 13:40	19 Oct 2026 08:19	17.63
graph, seed 13	19 Oct 2026 08:19	21 Oct 2026 17:55	52.85
grid, seed 14	21 Oct 2026 17:55	22 Oct 2026 12:18	17.46
graph, seed 14	22 Oct 2026 12:18	24 Oct 2026 10:44	42.20
grid, seed 15	24 Oct 2026 10:44	25 Oct 2026 06:27	18.52
graph, seed 15	25 Oct 2026 06:27	26 Oct 2026 23:00	36.27

15.5 Results

Every table in this section shows every seed; scores exclude truncated games, reported in their own column; intervals are 95% percentile bootstrap intervals over seeds (10,000 resamples), the two arms resampled independently for the contrasts. The raw tables with the sensitivity columns are in Appendix D.8; the provenance of every number in Appendix E.

15.5.1 The verdict in one paragraph

Under both budget readings the graph arm shows no seed-consistent advantage against any opponent, and every interval on the graph–grid score difference includes zero: the six point contrasts lie between −0.052 and +0.013, and the largest upper bounds are +0.090 at equal examples, against the search opponent, and +0.055 at equal wall-clock. The first condition of the frozen rule therefore holds under both readings. Its second condition speaks, as in the first study, of a "meaningful" advantage without a number, so the bounds are stated rather than a threshold chosen after the fact: the rule rejects H1′ for any threshold above +0.090; for thresholds between +0.055 and +0.090 the same-wall-clock reading rejects while the same-examples reading leaves a graph advantage against

the search opponent unexcluded; the first study's realised bound was +0.028. **H1′ is not supported**: no graph advantage was detected under either reading, and the data cannot exclude a small one, below about 0.09, against the search opponent at equal examples. The rest of the section shows the evidence seed by seed.

15.5.2 Reading 1: equal number of training examples

Both arms are compared at their generation-10 checkpoints, each run having generated and trained on ten generations of 500 games under identical simulation budgets; under the warm-started regime each checkpoint has also inherited the weights of the nine before it.

Table 39: Second study, same-examples reading. Final-checkpoint (generation 10) score of every run against each frozen opponent, five independent warm-started training runs per arm (seeds 11 to 15, ten generations of 500 games, replay window of three generations); 100 paired colour-swapped games per opponent per run on the 250 frozen openings at 400 simulations per move without exploration noise. Score = mean over decided games (win 1, draw 0.5, loss 0). Truncation = share of the 100 games stopped at the 300-ply cap; no game against B-HEU or B-MCTS was truncated by either arm.

Arm	Seed	B-RND score	B-RND truncation	B-HEU score	B-MCTS score
grid	11	0.984	5%	0.040	0.100
grid	12	0.984	4%	0.055	0.075
grid	13	0.995	1%	0.105	0.175
grid	14	0.990	2%	0.120	0.260
grid	15	0.989	7%	0.065	0.045
graph	11	1.000	0%	0.125	0.245
graph	12	0.995	4%	0.030	0.070
graph	13	0.940	42%	0.080	0.145
graph	14	0.928	24%	0.030	0.100
graph	15	0.992	41%	0.105	0.110

Against the legal-random opponent grid seeds score 0.984 to 0.995 and graph seeds 0.928 to 1.000; against the heuristic, grid seeds 0.040 to 0.120 and graph seeds 0.030 to 0.125; against the search opponent, grid seeds 0.045 to 0.260 and graph seeds 0.070 to 0.245. The graph arm truncates 0 to 42% of its games against the random opponent, three seeds at 24 to 42% and two at 0 to 4%; the grid arm 1 to 7%. The per-seed ranges of the two arms overlap against every opponent, and the spread between seeds of one arm against the search opponent (0.045 to 0.260 for the grid arm) is larger than any difference between the arms. Five seeds per arm cannot resolve differences inside such a spread.

Table 40: Second study, same-examples reading. Seed-level mean score per arm and opponent with the 95% bootstrap interval over the five seeds (10,000 resamples); same runs, opponents and evaluation volume as Table 39.

Arm	vs B-RND	vs B-HEU	vs B-MCTS
grid	0.989 [0.985, 0.992]	0.077 [0.051, 0.104]	0.131 [0.068, 0.200]
graph	0.971 [0.943, 0.996]	0.074 [0.040, 0.108]	0.134 [0.090, 0.196]

15.5.3 Reading 2: equal wall-clock on the same machine

Each run is taken at the last checkpoint it had completed within the second study's own equal-time cutoff, 17.63 hours, the median of the total training wall-clock of the five grid

runs (17.46, 17.95, 17.63, 17.46 and 18.52 hours), computed on 25 October 2026 from the clock files alone, before any score of the study had been read, and never recomputed. The cutoff selects generation 10, 9, 10, 10 and 9 for grid seeds 11 to 15, and generations 4, 5, 3, 3 and 4 for graph seeds 11 to 15 (counting the initial network as generation 1; in the run logs these checkpoints are labelled gen009/gen008/gen009/gen009/gen008 and gen003/gen004/gen002/gen002/gen003). Where the cutoff checkpoint is the final one the final evaluation is reused; the other seven sets were played after the campaign at the same volume and with the pinned cutoff seeds.

Table 41: Second study, same-wall-clock reading. Score of every run at its last checkpoint completed within the 17.63-hour cutoff (grid seeds at generations 10, 9, 10, 10, 9; graph seeds at generations 4, 5, 3, 3, 4), against each frozen opponent; same evaluation as Table 39 (100 paired games per opponent, 250 frozen openings, 400 simulations, no noise). Truncation = share of games at the 300-ply cap; none against B-HEU or B-MCTS.

Arm	Seed	B-RND score	B-RND truncation	B-HEU score	B-MCTS score
grid	11	0.984	5%	0.040	0.100
grid	12	0.984	5%	0.060	0.110
grid	13	0.995	1%	0.105	0.175
grid	14	0.990	2%	0.120	0.260
grid	15	0.973	7%	0.065	0.075
graph	11	0.995	1%	0.110	0.160
graph	12	0.984	4%	0.115	0.095
graph	13	0.762	58%	0.140	0.140
graph	14	0.965	28%	0.065	0.120
graph	15	0.958	40%	0.025	0.080

Table 42: Second study, same-wall-clock reading. Seed-level mean score per arm and opponent with the 95% bootstrap interval over the five seeds (10,000 resamples); same runs and evaluation volume as Table 41.

Arm	vs B-RND	vs B-HEU	vs B-MCTS
grid	0.985 [0.979, 0.992]	0.078 [0.053, 0.105]	0.144 [0.092, 0.206]
graph	0.933 [0.846, 0.986]	0.091 [0.056, 0.124]	0.119 [0.094, 0.144]

At equal time the graph arm is read at its third to fifth checkpoints and the grid arm at its ninth or tenth. The graph arm's mean against the random opponent falls to 0.933 [0.846, 0.986] against 0.985 [0.979, 0.992], a gap carried by seed 13 (0.762 with 58% truncation) and seeds 14 and 15 (28% and 40% truncation); against the heuristic its mean is 0.091 [0.056, 0.124] against 0.078 [0.053, 0.105], and against the search opponent 0.119 [0.094, 0.144] against 0.144 [0.092, 0.206], every interval overlapping its counterpart. Three to five warm-started generations of the graph arm therefore reach, against the two strong opponents, the level that nine or ten grid generations reach, and the random-opponent gap is the truncation channel again. The reading charges the graph arm its measured cost on this machine, 2.34 times the grid arm's per run, which is what it should do; what it cannot do with five seeds is separate means whose intervals overlap by more than half their width.

15.5.4 The arm contrast

Table 43: Second study, the pre-registered contrast: difference of seed-level mean scores (graph arm minus grid arm) with the 95% bootstrap interval over seeds (five independent warm-started

seeds per arm, resampled independently, 10,000 resamples), per frozen opponent and budget reading. A negative value favours the grid arm.

Opponent	Same-examples: graph − grid	Same-wall-clock: graph − grid
B-RND (legal-random)	−0.018 [−0.045, +0.008]	−0.052 [−0.140, +0.002]
B-HEU (heuristic)	−0.003 [−0.046, +0.039]	+0.013 [−0.032, +0.055]
B-MCTS (search, 6,400 simulations)	+0.003 [−0.082, +0.090]	−0.025 [−0.091, +0.034]

None of the six contrasts excludes zero. Against the legal-random opponent the graph–grid difference is −0.018 [−0.045, +0.008] at equal examples and −0.052 [−0.140, +0.002] at equal time; against the heuristic, −0.003 [−0.046, +0.039] and +0.013 [−0.032, +0.055]; against the search opponent, +0.003 [−0.082, +0.090] and −0.025 [−0.091, +0.034]. The point contrasts are within 0.05 of zero in five cells of six and the sixth touches zero at its upper bound. The intervals are wider than the first study's in four of the six cells, the widest spanning 0.17 against the search opponent, where the two arms' seeds spread from 0.045 to 0.260.

The frozen rule's first condition asks for the absence of a seed-consistent ordering in the graph arm's favour. Graph seeds outscore grid seeds in 3, 2 and 2 of the 5 same-index pairs against the three opponents at equal examples (13, 11 and 13 of the 25 cross pairs), and in 2, 3 and 2 of 5 (7, 15 and 11 of 25) at equal time: no opponent orders the seeds in the graph arm's favour under either reading. The second condition asks for intervals excluding a meaningful graph advantage, with no number attached: the largest upper bound is +0.090 at equal examples and +0.055 at equal time. The verdict is the one stated at the head of the section, in those terms; the study claims no threshold it did not pre-register, and the reader can apply any.

15.5.5 Truncation and the sensitivity reading

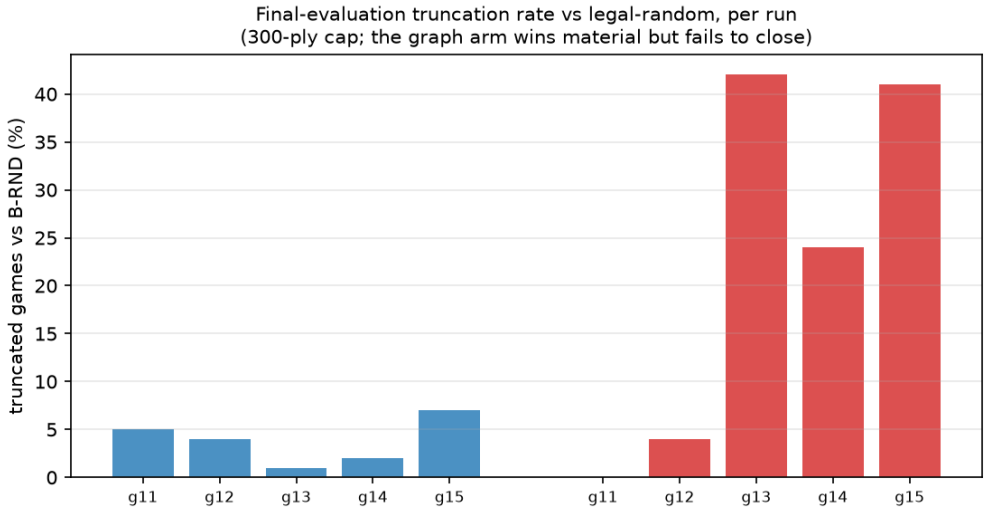


Figure 10: Second study: share of the 100 final-evaluation games against the legal-random opponent (B-RND) stopped at the 300-ply cap, per run; blue, the five grid seeds (11 to 15, left to right); red, the five graph seeds. Final checkpoints (generation 10) of ten warm-started runs, 100 paired colour-swapped games each on the 250 frozen openings at 400 simulations.

Truncation again occurs only against the random opponent, in none of the 20 cells against the heuristic and the search opponent. At the final checkpoints the graph arm

truncates 0%, 4%, 42%, 24% and 41% of its games against the random opponent for seeds 11 to 15, the grid arm 5%, 4%, 1%, 2% and 7% (Figure 10); at the cutoff checkpoints, 1%, 4%, 58%, 28% and 40% against 5%, 5%, 1%, 2% and 7%. The conversion pathology of the first study, present in every graph seed there (20 to 57%), persists in three graph seeds of five and is absent in two, whose rates sit inside the grid arm's band. The score that excludes truncations is close between the arms, but under the sensitivity treatment, every truncated game counted 0.5, the graph–grid difference against the random opponent is −0.101 [−0.187, −0.012] at equal examples and −0.131 [−0.256, −0.015] at equal time, and both intervals exclude zero; the other four contrasts are unchanged under every treatment, having no truncations. Under the warm-started regime the graph arm's deficit against the weakest opponent is therefore no longer a deficit in decided games; it is a deficit in deciding them, concentrated in three seeds.

15.5.6 Trajectories and cost

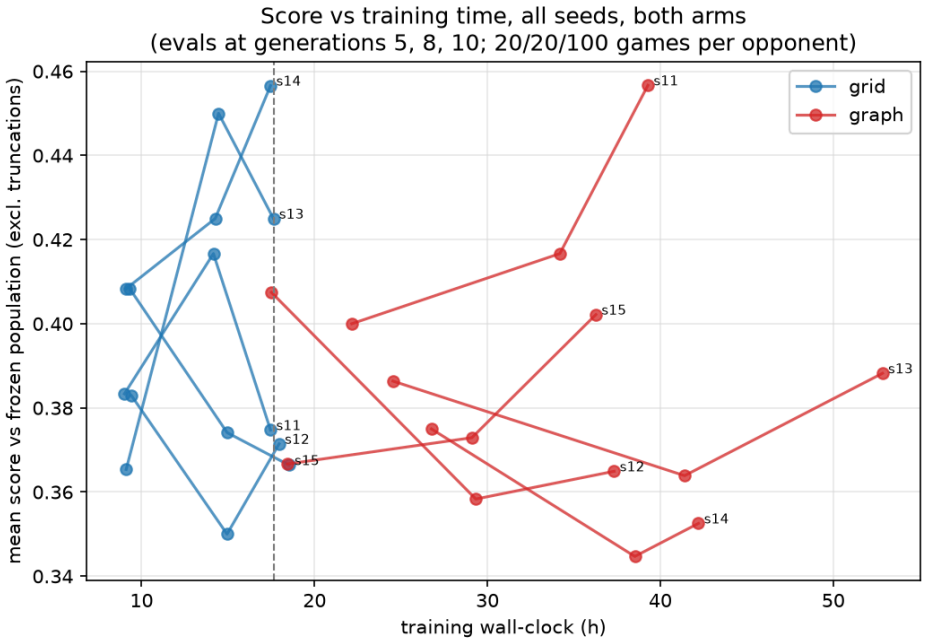


Figure 11: Second study: mean score against the frozen population (average of the three opponents, truncations excluded) as a function of cumulative training wall-clock, for all ten warm-started runs; evaluations at generations 5, 8 and 10 (20, 20 and 100 games per opponent). Blue: grid arm; red: graph arm; one line per seed, labelled at its end. The dashed vertical line marks the second study's equal-time cutoff of 17.63 hours.

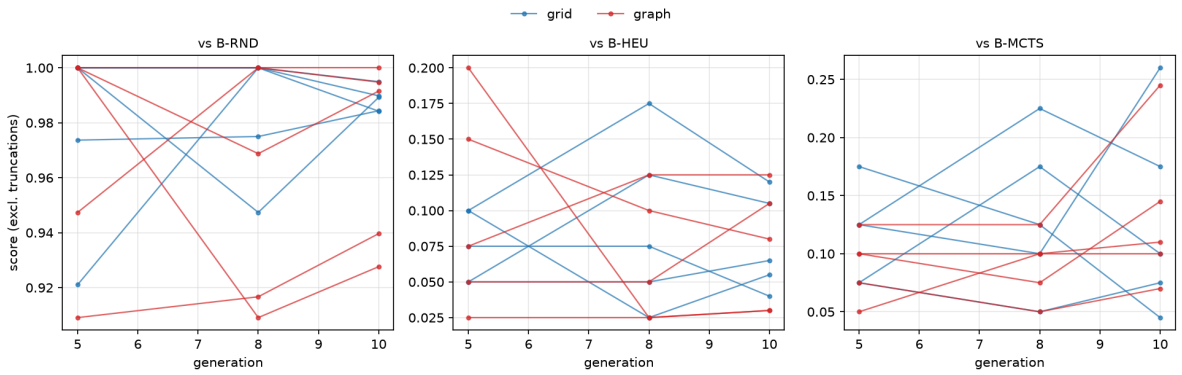


Figure 12: Second study: score against each frozen opponent at generations 5, 8 and 10 (20, 20 and 100 paired games per opponent), one line per run, five warm-started seeds per arm; blue, grid arm; red, graph arm; truncations excluded. Intermediate points carry 20 games and serve the trajectory only.

Figure 11 places the ten runs on a common time axis: the grid runs finish between 17.46 and 18.52 hours of training wall-clock, the graph runs between 36.27 and 52.85 hours, and at the cutoff the two clouds overlap in score. Figure 12 shows the per-opponent trajectories; the intermediate points rest on 20 games and are read for shape only. The training-clock profiles of the graph runs took two shapes, a rise to a peak around generation 4 followed by a fall to grid-like durations by generation 10 in seeds 11 and 14, and a flat profile in seeds 12, 13 and 15; the per-generation clock of seed 13 rose monotonically from 14,456 s to 21,112 s and made it the slowest run of the campaign at 52.85 hours.

Table 44: Second study, cost and score summary per arm. Training wall-clock covers self-play generation, training and export for the ten generations of a run, evaluation games excluded, on the study machine (Apple M1 Pro, four worker threads, sequential runs); self-play cost = that wall-clock divided by 5,000 games. Population score = mean over the three frozen opponents of the seed-level mean score, truncations excluded. Parameters, inference and throughput benchmarks are unchanged from the first study.

Metric	Grid arm	Graph arm
Training wall-clock per run, seeds 11 to 15	17.46 / 17.95 / 17.63 / 17.46 / 18.52 h	39.28 / 37.33 / 52.85 / 42.20 / 36.27 h
Training wall-clock per run, mean of 5 seeds	17.80 h	41.59 h (2.34×)
Mean self-play cost	12.8 s/game	29.9 s/game
Population score, same-examples reading	0.399	0.393
Population score, same-wall-clock reading (cutoff 17.63 h)	0.402	0.381

The ten runs consumed 296.9 hours of training wall-clock (grid 89.0 h, graph 207.9 h), against a projection of about 295 hours from the pilot, on one laptop with no paid compute. The graph arm cost 2.34 times the grid arm per run, against 2.04 in the first study: the replay window adds training time in proportion to the record count on both arms, and the graph arm's training is the more expensive. The population scores differ by 0.006 at equal examples and 0.021 at equal time in the grid arm's favour, against 0.081 and 0.079 in the first study.

15.6 The two studies side by side

The comparison between the studies is descriptive. The two studies use different seed sets, two regime changes were applied at once, and no interval is computed across studies; the two sets of intervals are placed beside each other and read as such.

Table 45: The graph–grid contrast of the two studies, per frozen opponent and budget reading: difference of seed-level mean scores with the 95% bootstrap interval over seeds (five independent seeds per arm in each study, resampled independently, 10,000 resamples). First study: seeds 1 to 5, each generation's network trained from a fresh initialisation on that generation's games (cutoff 18.77 h). Second study: seeds 11 to 15, warm start and three-generation replay window (cutoff 17.63 h). The studies are not pooled.

Opponent	Reading	First study: graph – grid	Second study: graph – grid
B-RND	same-examples	−0.169 [−0.272, −0.062]	−0.018 [−0.045, +0.008]

Opponent	Reading	First study: graph − grid	Second study: graph − grid
B-HEU	same-examples	−0.064 [−0.111, −0.017]	−0.003 [−0.046, +0.039]
B-MCTS	same-examples	−0.009 [−0.055, +0.028]	+0.003 [−0.082, +0.090]
B-RND	same-wall-clock	−0.161 [−0.278, −0.048]	−0.052 [−0.140, +0.002]
B-HEU	same-wall-clock	−0.059 [−0.087, −0.029]	+0.013 [−0.032, +0.055]
B-MCTS	same-wall-clock	−0.018 [−0.066, +0.025]	−0.025 [−0.091, +0.034]

The contrast moved toward zero on every opponent under both readings. Against the random opponent it went from −0.169 to −0.018 at equal examples and from −0.161 to −0.052 at equal time; against the heuristic from −0.064 to −0.003 and from −0.059 to +0.013; against the search opponent from −0.009 to +0.003 and from −0.018 to −0.025. Four contrasts that excluded zero in the first study include it in the second, and the sign changed in two cells, both inside intervals that include zero. The graph arm's deficit of the first study is no longer visible at equal examples and is reduced at equal wall-clock, where it survives as an interval touching zero against the random opponent. What the second study does not show is any advantage of the graph arm: its two positive point contrasts (+0.003 and +0.013) are the smallest numbers in their rows.

The arms' absolute levels moved in different directions. The graph arm's mean against the random opponent rose from 0.812 to 0.971 at equal examples, which is where its first-study deficit lived; its means against the heuristic and the search opponent moved from 0.065 to 0.074 and from 0.115 to 0.134. The grid arm's mean against the heuristic fell from 0.129 [0.098, 0.165] to 0.077 [0.051, 0.104], two intervals that do not overlap, while its means against the random and the search opponent stayed within 0.01 (0.981 to 0.988; 0.124 to 0.131). Read descriptively, the warm-started regime with a three-generation window did not raise the strength reached against the strong opponents within ten generations at these settings; it narrowed the gap between the arms, and for the grid arm against the heuristic it coincided with a lower level. Whether the lower level is an effect of the regime (a network carried across generations trains on a window that includes its own earlier games) or of the seed set cannot be decided from two studies of five seeds, and the study does not decide it.

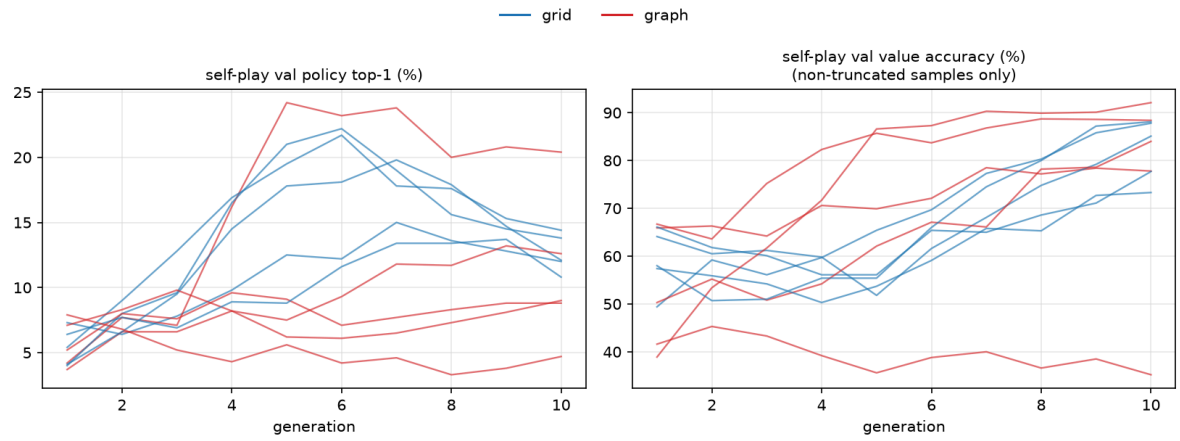


Figure 13: Second study: validation policy top-1 accuracy (left) and value accuracy on non-truncated samples (right) after the first epoch of each generation, as printed by the trainer on the held-out split of the generation's windowed training set; one line per run, five warm-started seeds per arm; blue, grid arm; red, graph arm. Training diagnostics, not strength measurements.

The training diagnostics (Figure 13) separate one run from the other nine. Nine runs end the tenth generation with a value accuracy between 73.3% and 92.1% on their held-out split; graph seed 15 never exceeds 45% and ends at 35.2%, with a policy top-1 falling from 7.9% to 4.7%, yet its final checkpoint scores 0.992 against the random opponent (41% truncation), 0.105 against the heuristic and 0.110 against the search opponent, inside the other seeds' bands. In the first study every graph run's value accuracy rose into the grid arm's band. The run is kept, as the pre-commitments require; it shows that under this regime a network can play at the level of its peers while fitting its own windowed data poorly, and the study has no measurement that explains why.

Three limits bound this section. The between-study movement is not attributable to the warm start or to the window separately, since both were applied at once by design. The hyperparameters are the first study's untuned defaults in both arms, a choice made at the freeze and stated as a limit of both studies; a tuned graph arm, or a tuned grid arm, might sit elsewhere. And ten generations remain an early regime: the second study moved the pipeline toward the standard practice of self-play systems and the absolute strength against the strong opponents did not move with it, which says that the budget, not only the regime, bounds what either representation shows here.

Within these limits the second study answers the objection it was designed to test. The ordering of the first study does not survive the warm-started regime as a deficit: the graph representation, carried across generations and trained on a window of recent games, plays the frozen population as well as the grid representation at equal examples, within five-seed intervals, and within 0.05 of it at equal time, at 2.34 times the cost. It does not play better. The original hypothesis, that the graph representation learns a better policy at equal budget, is unsupported in both regimes; what changed between them is the size of the gap, from a clear grid advantage to none that five seeds can detect, with a conversion pathology that the regime removed in two graph seeds and left in three.

Part V. Discussion and conclusion

16 Discussion and threats to validity

The hypothesis was rejected exactly as the frozen protocol defined rejection. This chapter asks what the result means, which explanations the data support and which they merely suggest, how the result relates to the prior work that motivated it, and what could be wrong with it. The threats are organised as the research plan prescribes: internal, measurement, statistical and external.

16.1 Reading the result

Three observations shape how the result should be read.

First, the deficit is opponent-shaped. It is largest against the legal-random opponent, clear against the heuristic and absent against the search opponent (Table 31). A graph arm that was simply weaker everywhere would show a uniform deficit; a deficit concentrated where games are decided by finishing a surround that nothing resists points at *local tactical conversion* rather than positional judgement.

Second, the truncation channel carries the mechanism. The graph arm wins material against the random opponent and then fails to close: 20–57% of those games end at the 300-ply cap, against 0–11% for the grid arm (Figure 7, Figure 6 F1). This pattern is invisible in any study that scores capped games as draws, and it is why the protocol insisted, before the first run, that truncation be an outcome of its own.

Third, the ablations sharpen the reading. At parity settings, removing the direction-typed relations did more than weaken the graph arm; it destroyed trainability outright (non-finite divergence in generation 0 for all three seeds), so the typed relations carry at minimum the optimization stability of the whole arm. Removing the global-pooling bias changed the scores by ≈0.00 ± 0.17, −0.01 ± 0.04 and −0.01 ± 0.03 against the three opponents, with failure modes unchanged. The arm's distinctive machinery therefore lives in its directional relations rather than in its pooling. Once the naive variant is stabilised by gradient clipping, however, it plays within the full arm's band, so the measurable contribution of the typed relations at this scale is concentrated in optimization rather than in representation (Section 13).

16.2 Which explanations are demonstrated, and which are plausible

The research plan distinguishes an explanation demonstrated by ablation from one that is merely plausible. Four candidate explanations of the gap deserve this treatment.

Geometry (plausible, not demonstrated). Hive's tactics are dominated by short-range surround geometry, the regime in which Keller et al. (2023) found convolutional networks stronger than graph networks in Hex. A 3×3 convolution over the unfolded frame sees a cell's whole neighbourhood, and stacked residual blocks see the neighbourhood of the neighbourhood, at no learning cost; the message-passing network must compose the same patterns from six typed relations one hop at a time. This is consistent with every

observation above, but no ablation in this study isolates receptive-field composition, so it remains a hypothesis.

Capacity (controlled). The two networks differ by +1.5% in parameter count (1.44 M against 1.47 M), in the graph arm's favour. Capacity does not explain a graph deficit.

Decoder (controlled). Both arms score the identical set of legal (piece, destination) pairs with identical masking, normalisation, targets and tie-breaking; the decoder cannot favour one arm. What does differ is the *parameterisation* behind the shared interface: a flat 28,673-way tensor for the grid arm, a per-candidate scoring function for the graph arm. That difference is part of what "representation" means in this study rather than a confound of it.

Speed (demonstrated for the second reading only). The graph arm's generations cost roughly twice as much wall-clock on this machine, so at equal time it completes fewer than half of its generations (Section 12). This demonstrably widens the gap in the same-wall-clock reading; it plays no role in the same-examples reading, where the deficit already exists.

Optimization (partly demonstrated). The ablations show that the full graph arm's optimization is fragile in one specific way (it depends on typed relations for stability), but the training curves of the full arm (Figure 8) show it fitting its data in every seed. Nothing indicates that the full arm failed to optimize; the gap is a gap between two trained networks. Whether a different learning rate or normalisation would have favoured the graph arm was not tested, because neither arm received any tuning. The zero-tuning policy is symmetric, but it is not neutral if one architecture is more sensitive to its defaults, a point taken up under external validity.

16.3 Relation to prior work

The result does not contradict the positive graph findings of Rigaux and Kashima (2024) in chess or the long-range advantage Keller et al. (2023) measured in Hex. It bounds where their optimism transfers. The chess result rests on a single training run per model with intervals covering Elo estimation only, a loose capacity match and arm-specific decoders; the Hex result was obtained under a value-based learner, with a game-specific graph construction, and found the convolutional arm sharper at local patterns, which is the regime that decides Hive games at small budgets. The present rejection is seed-consistent across five runs per arm under two pre-registered budget readings, with capacity matched and the decoder shared. Read together, the three studies suggest that the sign of the grid-versus-graph comparison depends on the game's tactical range and on the evidence standard, rather than on the graph paradigm as such. The one prior AlphaZero study of Hive (de Goede et al., 2022) had already shown that encoding choice changes learning within the grid family; this study extends the finding across families and finds the grid side ahead.

16.4 Internal validity: bugs and comparability

The engine is validated independently of learning: perft equality with published tables to depth 6 for all eight game types, 21/21 protocol conformance, 27,829-position differential agreement of legal-move sets with two reference engines, a 30-case rules-derived corpus annotated before any run, and 10.9 million random transitions without an invariant violation (Section 5). Both encoders are pinned byte-exactly against cross-language golden

tests run nightly (240 and 160 positions). The arms share the decoder, the records, the losses, the budgets, the search and the frozen evaluation; capacity differs by +1.5% and is reported.

Residual risks remain. The graph architecture is one point in design space; a different graph network might behave differently, and nothing is claimed beyond this one. Tooling defects were found *during* the study: a match-runner path that dropped records when every game of a match was truncated, a training loop that let non-finite values pass silently for ten generations, and a results generator whose per-run wall-clock average still divided by the original three runs after the extension to five. Each was caught by the verification discipline (Section 10), fixed, and audited: the first touched no campaign data; the second affected only the ablation it exposed and became that ablation's result; the third affected two cost-table rows and no score, interval or claim. They illustrate the study's most practical lesson: at this scale the dominant failure mode is harness error rather than chance, and only mechanical verification catches it.

16.5 Measurement validity: opponents and truncation

Scores are relative to a population of three fixed opponents spanning floor-to-mid strength; no universal strength claim is made. All trained arms still lose heavily to the two strong baselines after ten generations, so the comparison lives in a low-score regime in which differences against the search opponent cannot be resolved. Truncation is handled as its own outcome with sensitivity bounds: the cap cannot reverse the verdict, but the graph arm's high truncation rates mean its random-opponent score is measured on fewer decided games (43–80 per seed), which widens its interval. The evaluation openings are frozen and shared, so an opening-set effect would affect both arms alike; it could still shape absolute scores.

16.6 Statistical validity: seeds and dependencies

Five seeds per arm bound the statistics. The rejection rests on more than a single interval: on seed-consistency across two opponents and two readings simultaneously, on four of six intervals excluding zero, and on a bounds analysis of the cap. The pre-committed extension from three to five seeds tightened four of the six intervals and absorbed the most graph-favourable seeds observed without changing the verdict; the contrast against the search opponent remains indistinguishable from zero under both readings and is reported as such. The arm contrast bootstraps seeds rather than games, so no game-level pseudo-replication enters any interval, and the two arms' seed sets are resampled independently, which is conservative for a paired design in which both arms play the same openings.

16.7 External validity: variant, hardware and budget

One variant (base Hive without expansions), one machine, one small budget (ten generations of 500 games; between 16.7 and 19.1 hours of training wall-clock per grid run and between 26.6 and 49.9 hours per graph run), early-regime self-play throughout. The inference-provider asymmetry is a genuine property of the deployment hardware: the convolutional network runs fastest on the machine's neural accelerator, the gather-

heavy graph network fastest on the CPU. It is reported and charged, and on different accelerators the wall-clock reading could shift while the same-examples reading would not. The zero-tuning policy is symmetric but may not be neutral: graph networks are commonly more sensitive to optimizer defaults than residual convolutional networks, and the first ablation showed that this family is fragile under these defaults; a tuned graph arm might narrow the gap, at the cost of breaking the symmetry of the comparison. Nothing here generalises to other games, larger budgets or richer graph architectures. The study answers its pre-registered question inside its pre-registered perimeter, and the negative answer is the finding. Encoding choice matters, as the earlier Hive study found, and for Hive at small budget it favours the grid.

16.8 A property of the pipeline that bounds the absolute strength reached

One property of the shared pipeline deserves to be stated as plainly as possible, because it shapes how the "ten generations" of this study should be read. The campaign driver trains the network of each generation from a fresh seeded initialisation, for two epochs, on the records of that generation alone; it passes neither a warm-start checkpoint nor a replay window to the trainer (Section 6). The loop still improves across generations, because the network of generation g is trained on games played by the search guided by the network of generation $g-1$, and that search is stronger than the raw network; but no weights are carried forward and no record is reused. Each network therefore sees the positions of 500 games, which is a small training set by the standards of self-play learning, and the absolute strength reached after ten generations (near the ceiling against the random opponent, around 0.1 against the two strong opponents) must be read against that fact. For the *comparison* the property is neutral: it is identical in both arms, both encoders see exactly the same records, and the frozen protocol specified identical training conventions rather than a particular warm-start policy. For the *generalisation* of the result it is a limit in addition to those already listed: a pipeline that accumulated data and carried weights would reach a different regime, in which the ordering of the arms is an open question. That successor study was run after this report had first been written, with both changes applied identically to both arms under a second frozen protocol, and is reported in Section 15: under a warm-started loop with a three-generation replay window the deficit is no longer visible at equal examples and reduced at equal time, no graph advantage appears, and the strength reached against the two strong opponents does not rise.

16.9 What we would do differently

Four changes would strengthen a successor study without changing its logic. The first, the warm start and replay window just described, was carried out as the second study; it leaves the tuning question below open, and its wider intervals showed that the rejection rule needs a numeric threshold for what counts as a meaningful advantage (Section 15.5). The intermediate evaluations (20 games per opponent at generations 5 and 8) were sized for monitoring and are too noisy to carry inference; a successor should evaluate every generation at the final volume if the curves are to be analysed. The equal-time reading should be accompanied by an equal-*energy* reading on heterogeneous accelerators, since

wall-clock alone hides the provider asymmetry. And the zero-tuning symmetry should be complemented by a small, identical, pre-registered tuning budget for both arms, so that fragility under defaults is measured rather than inherited.

17 Conclusion and perspectives

17.1 The answer to the research question

The tested perimeter is base-game Hive, one capacity-matched simple relational message-passing network against one grid convolutional network, ten generations of small-budget self-play, five independent seeds per arm, a frozen three-opponent population and 250 frozen openings. Within that perimeter, **the answer to the main research question is no**: the graph representation did not learn a better policy than the grid representation, neither when both arms consumed the same number of training examples nor when both received the same wall-clock on the same machine, and the pre-registered rejection rule fired exactly as it had been frozen. The graph arm's deficit is largest where Hive is most tactical (against the legal-random opponent, −0.169 and −0.161 under the two readings) and clear against the heuristic opponent (−0.064 and −0.059); against the search opponent the two arms are indistinguishable (−0.009 and −0.018, intervals straddling zero). The graph arm pays roughly twice the wall-clock per run. Its characteristic failure is to win material without converting the win; this shows as 20–57% of its games against the random opponent ending at the move cap, and it is observable only because truncation was never folded into draws.

The second study, pre-registered separately and run under a warm-started loop with a three-generation replay window on five new seeds per arm, tested the main objection to that answer. It found no graph advantage under either reading: every graph−grid interval includes zero, with the largest upper bounds at +0.090 and +0.055; the first study's deficit is no longer visible at equal examples and reduced at equal time; the conversion pathology persists in three graph seeds of five; and the graph arm costs 2.34 times the grid arm per run. The answer to the research question therefore stands in the form the two studies together allow: at this budget, on this variant, the graph representation did not learn a better policy than the grid representation in either training regime, and under the standard regime it learned an equivalent one, within five-seed intervals, at more than twice the cost.

The result bounds, rather than contradicts, the positive graph findings reported for Hex under value-based learning and for chess under self-play. Where short-range surround tactics decide games and budgets are small, the frame artifacts of a grid encoding turn out to be cheaper than framelessness. The component analysis adds a mechanistic reading: of the graph arm's two distinctive components, the direction-typed relations are necessary for optimization itself (removing them at identical settings made training diverge in every seed), while the global-pooling bias is dispensable at this scale.

17.2 Contributions, as demonstrated

Four contributions were announced in Section 1; each is demonstrated in the chapters named there. (1) A controlled, budget-matched, multi-seed grid-versus-graph comparison for Hive, with both arms under one self-play pipeline and a pre-registered negative answer (Section 11, Section 12). (2) A reproducible comparison harness for frameless,

stacking games, released with the raw records: a rules-validated engine, two encoders pinned byte-exactly across two languages, a shared variable-action decoder, truncation-aware evaluation and frozen-artifact discipline (Section 5 to Section 9). (3) A component attribution for the graph arm with an honestly labelled two-component supplement (Section 13). (4) A second pre-registered comparison under the standard warm-started regime, with its protocol, cutoff and seeds fixed before any run, whose null result bounds the first study's negative one (Section 15).

17.3 Two follow-ups motivated by the data

The first follow-up targets the **conversion pathology**, which the data isolate as a policy defect rather than a value defect: the graph arm's value accuracy in training is comparable to the grid arm's while its policy fails at forcing sequences (Section 14). A natural experiment is a search-time remedy applied identically to both arms under the same frozen evaluation, as a one-component change to the shared pipeline: a deeper evaluation budget in positions the value head already judges won, or auxiliary training targets for forcing moves. The second follow-up concerns the **stability finding**: the ablation and its supplement showed that direction-typed relations matter mostly for optimization at this scale, but the supplement's gradient clipping confounds the attribution. A controlled study of normalisation and gradient-scale choices for relation-shared graph layers would decouple trainability from representational content and would say whether naive adjacency, properly stabilised, is sufficient for Hive. Either follow-up now has the warm-started regime as its baseline, and either should pre-register a numeric threshold for a meaningful advantage, which the second study's intervals showed to be necessary.

17.4 A method that stands regardless of the sign

Pre-registration with frozen artifacts, two budget readings, truncation as an outcome, seed-level inference and reporting written as the work progressed turned a negative answer into a usable scientific object on a single laptop. The same discipline, described in Section 10 together with the incidents it caught, is the part of this work most directly transferable to other small-compute studies.

Bibliography

Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., & Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems* (NeurIPS 2021) (peer-reviewed; Outstanding Paper Award). arXiv:2108.13264 (v4, 5 January 2022). https://doi.org/10.48550/arXiv.2108.13264

Ben-Assayag, S., & El-Yaniv, R. (2021). Train on small, play the large: Scaling up board games with AlphaZero and GNN. arXiv preprint arXiv:2107.08387v1 (18 July 2021) (no peer-reviewed version found as of 26 September 2026). https://doi.org/10.48550/arXiv.2107.08387

Cazenave, T., Chen, Y.-C., Chen, G.-W., Chen, S.-Y., Chiu, X.-D., Dehos, J., Elsa, M., Gong, Q., Hu, H., Khalidov, V., Li, C.-L., Lin, H.-I., Lin, Y.-J., Martinet, X., Mella, V., Rapin, J., Roziere, B., Synnaeve, G., Teytaud, F., Teytaud, O., Ye, S.-C., Ye, Y.-J., Yen, S.-J., & Zagoruyko, S. (2020). Polygames: Improved zero learning. *ICGA Journal*, 42(4), 244–256. https://doi.org/10.3233/ICG-200157. Preprint: arXiv:2001.09832 (January 2020).

de Goede, D., Kampert, D., & Varbanescu, A. L. (2022). The cost of reinforcement learning for game engines: The AZ-Hive case-study. In *Proceedings of the 13th ACM/SPEC International Conference on Performance Engineering* (ICPE 2022, Beijing), pp. 145–152. https://doi.org/10.1145/3489525.3511685

Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems* (NIPS 2017) (peer-reviewed). arXiv:1706.02216 (v4, 10 September 2018). https://doi.org/10.48550/arXiv.1706.02216

Jones, A. L. (2021). Scaling scaling laws with board games. arXiv preprint arXiv:2104.03113v2 (15 April 2021) (not peer-reviewed). https://doi.org/10.48550/arXiv.2104.03113

Kampert, D., Varbanescu, A.-L., Müller-Brockhausen, M., & Plaat, A. (2021). Mimicking the human approach in the game of Hive. In *2021 IEEE Symposium Series on Computational Intelligence* (SSCI 2021, Orlando). IEEE Xplore document 9659999. https://ieeexplore.ieee.org/document/9659999/. Preprint circulated as "Better AI for Hive: Mimicking human game-play strategies".

Keller, Y., Blüml, J., Sudhakaran, G., & Kersting, K. (2023). From images to connections: Can DQN with GNNs learn the strategic game of Hex? arXiv preprint arXiv:2311.13414 (22 November 2023) (not peer-reviewed; OpenReview submission dYaeDrazj5). https://arxiv.org/abs/2311.13414

Rigaux, T., & Kashima, H. (2024). Enhancing chess reinforcement learning with graph representation. In *Advances in Neural Information Processing Systems 37* (NeurIPS 2024, main conference track). https://doi.org/10.52202/079017-0006. Preprint: arXiv:2410.23753v1 (31 October 2024), https://doi.org/10.48550/arXiv.2410.23753

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144 (peer-reviewed). https://doi.org/10.1126/science.aar6404. Preprint (2017): Mastering chess and shogi by self-play with a general reinforcement learning algorithm, arXiv:1712.01815v1 (5 December 2017), https://doi.org/10.48550/arXiv.1712.01815

van der Pol, E., Worrall, D. E., van Hoof, H., Oliehoek, F. A., & Welling, M. (2020). MDP homomorphic networks: Group symmetries in reinforcement learning. In *Advances in Neural Information Processing Systems* (NeurIPS 2020) (peer-reviewed). arXiv:2006.16908 (v2, 20 January 2021). https://arxiv.org/abs/2006.16908

Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. In *Advances in Neural Information Processing Systems 28* (NIPS 2015) (peer-reviewed). arXiv:1506.03134 (v2, 2 January 2017). https://arxiv.org/abs/1506.03134

Wu, D. J. (2020). Accelerating self-play learning in Go. arXiv preprint arXiv:1902.10565v5 (9 November 2020); presented at the AAAI-20 Workshop on Reinforcement Learning in Games (not a full peer-reviewed proceedings paper). https://doi.org/10.48550/arXiv.1902.10565

Webography

edre (GitHub user). *nokamute*: Hive engine in Rust, with a Universal Hive Protocol conformance tester and a built-in match runner [source-code repository]. GitHub. https://github.com/edre/nokamute. Version 1.0.3 used as a reference engine. Consulted July 2026 (engine design) and 9 September 2026 (validation campaign).

jonthysell (GitHub user). *Mzinga*: reference Hive engine and project wiki, including the Universal Hive Protocol specification and the perft tables [source-code repository and wiki]. GitHub. https://github.com/jonthysell/Mzinga perft tables: https://github.com/jonthysell/Mzinga/wiki/Perft. Release MzingaEngine v0.16.0 used as a reference engine. Consulted July 2026 (engine design) and 9 September 2026 (validation campaign).

Pfeifer, J. (GitHub user janpfeifer). (2018–2026). *hiveGo — Go implementation of Hive game* [source-code repository]. GitHub. Created 24 August 2018; main branch, commit d6ff95418d2b (20 August 2026). https://github.com/janpfeifer/hiveGo. Consulted 26 September 2026.

World Hive Tournaments. *Rules of Hive: Rules FAQ* [web page]. https://www.worldhivetournaments.com/rules-of-hive/. Consulted 9 September 2026.

Yianni, J. (2010). *Hive rules* [publisher's rules sheet]. Gen42 Games. https://www.gen42.com/wp-content/uploads/Hive-rules.pdf. Consulted 9 September 2026.

Yianni, J. *The Pillbug: Additional Hive Pieces* [publisher's rules sheet]. Gen42 Games. https://www.gen42.com/wp-content/uploads/Pillbug_Rules.pdf. Consulted 9 September 2026.

Appendices

A Annotated position corpora

Every expectation below was written by hand from the publisher's rules before any engine run, so that the corpus could serve as an unbiased oracle; the engine was then run against it. The single disagreement found was resolved against the corpus (a One-Hive transit error in one setup sequence), and the engine was vindicated. These renderings are generated directly from the executable test cases, so the tests and this appendix cannot drift apart. Setup sequences are given in the move notation of the Universal Hive Protocol.

A.1 Critical rules corpus (30 cases)

Rules-correctness cases: placement, sliding/freedom to move, gates, stacking, One-Hive, terminal states, forced pass, and kernel-guard Pillbug stun cases.

A.1.1 C001: Queen Bee may not be placed on the first turn (tournament rule)

- **Rule:** Tournament opening rule (*Tournament variant of the official rules (README source 4); Gen42 2010 rulesheet p. 3 alone would allow it*)
- **Setup:** ``
- **Expectation:** move_illegal `{'move': 'wQ'}`
- **Hand-written justification:** The 2010 base rulesheet says the Queen 'can be placed at any time from your first to your fourth turn' (p. 3), but the tournament variant (adopted by UHP and both reference engines, and the convention this study fixes) forbids placing the Queen Bee on either player's first turn. White's first move 'wQ' must therefore be rejected.

A.1.2 C002: Queen must be placed on the fourth turn if not placed before

- **Rule:** Placing your Queen Bee (*Gen42 Hive rulesheet p. 3*)
- **Setup:** `wS1;bS1 wS1-;wG1 -wS1;bG1 bS1-;wA1 -wG1;bA1 bG1-`
- **Expectation:** all_moves_place `{'piece': 'wQ'}`
- **Hand-written justification:** 'You must place your Queen Bee on your fourth turn if you have not placed it before.' (p. 3). It is White's fourth turn and wQ is still in hand, so every legal move must be a placement of wQ. (Movement moves are additionally excluded by the Moving rule, p. 3: no moving before the queen is placed.) Setup legality: each white placement touches only white pieces, each black placement only black (Placing, p. 2).

A.1.3 C003: No piece may move before that player's queen is placed

- **Rule:** Moving (*Gen42 Hive rulesheet p. 3*)
- **Setup:** `wS1;bS1 wS1-`
- **Expectation:** all_moves_are_placements
- **Hand-written justification:** 'Once your Queen Bee has been placed (but not before), you can decide whether to use each turn after that to place another tile or to move one

of the pieces that have already been placed.' (p. 3). White's queen is unplaced on turn 2, so wS1 must have no movement moves; only placements are offered.

A.1.4 C004: After the first pieces, placements may not touch the opponent's colour

- **Rule:** Placing (*Gen42 Hive rulesheet p. 2*)
- **Setup:** `wG1;bS1 wG1/`
- **Expectation:** moves_for_piece `{'piece': 'wB1', 'moves': ['wB1 wG1\\', 'wB1 /wG1', 'wB1 -wG1']}`
- **Hand-written justification:** '...with the exception of the first piece placed by each player, pieces may not be placed next to a piece of the opponent's colour.' (p. 2). wG1 sits at the origin with bS1 to its north-east. Of wG1's five empty neighbours (E, SE, SW, W, NW), the E and NW cells each also touch bS1 (they are the two cells adjacent to both wG1 and its NE neighbour), so a new white piece may go only SE, SW or W of wG1. Expected wB1 placements: exactly those three cells. Hand geometry: axial E=(1,0), NE=(1,-1); neighbours of NE-cell (1,-1) include (1,0)=E-of-origin and (0,-1)=NW-of-origin.

A.1.5 C005: Second player's first piece joins the first piece (may touch enemy)

- **Rule:** Playing the Game / Placing (*Gen42 Hive rulesheet pp. 2-3*)
- **Setup:** `wS1`
- **Expectation:** moves_for_piece `{'piece': 'bG1', 'moves': ['bG1 wS1-', 'bG1 wS1/', 'bG1 wS1\\', 'bG1 -wS1', 'bG1 /wS1', 'bG1 \\wS1']}`
- **Hand-written justification:** 'Play begins with one player placing a piece from their hand in the centre of the table and the next player joining one of their own pieces to it edge to edge.' (p. 2). This is the first-piece exception to the own-colour placing rule. Black's first piece must join wS1 edge to edge, so bG1 may be placed on any of the six cells adjacent to wS1, and nowhere else.

A.1.6 C006: Queen at the hive tip: exactly the two slides that keep contact

- **Rule:** Queen Bee / Freedom to Move / One Hive (contact) (*Gen42 Hive rulesheet pp. 4, 9, 10*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-`
- **Expectation:** moves_for_piece `{'piece': 'wQ', 'moves': ['wQ \\wS1', 'wQ /wS1']}`
- **Hand-written justification:** The Queen 'can move only one space per turn' (p. 4) in a sliding movement (p. 10), and 'all pieces must always touch at least one other piece' (p. 3 NB). wQ sits at the west tip of a straight line of four. Of its five empty neighbours, only the two cells that are also adjacent to its neighbour wS1 (the cells NW and SW of wS1) keep contact with the hive after the slide; the three cells further west touch nothing once the queen leaves. Neither destination is gated (each slide's two flanking cells are one occupied, one empty). Expected: exactly those two moves.

A.1.7 C007: Ant enclosed in a pocket: only exit is a gate, so it cannot move

- **Rule:** Freedom to Move (*Gen42 Hive rulesheet p. 10*)
- **Setup:** `wA1;bS1 wA1-;wQ \wA1;bQ bS1-;wG1 -wA1;bB1 bQ/;wS1 /wA1;bB1 bS1/;wB1 \wQ;bB1 wA1/`
- **Expectation:** moves_for_piece `{'piece': 'wA1', 'moves': []}`

- **Hand-written justification:** 'If a piece is surrounded to the point that it can no longer physically slide out of its position, it may not be moved.' (p. 10). Five of the ant's six neighbours are occupied. The only empty neighbour (SE of the ant) is flanked by bS1 (E of the ant) and wS1 (SW of the ant) - the two cells adjacent to both the ant and that space - so the ant cannot physically slide into it. Removing the ant would NOT split the hive (the ring bB1-wQ-wG1-wS1 plus bS1 stays connected), so the block is purely freedom-to-move, not One Hive. The Ant, normally the most mobile piece, has zero legal moves.

A.1.8 C008: Spider moves exactly three spaces along the hive edge - two destinations

- **Rule:** Spider (*Gen42 Hive rulesheet p. 7*)
- **Setup:** `wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wS1 \wQ;bG1 bQ-`
- **Expectation:** moves_for_piece `{'piece': 'wS1', 'moves': ['wS1 bS1/', 'wS1 wQ\\']}`
- **Hand-written justification:** 'The Spider moves three spaces per turn - no more, no less. It must move in a direct path and cannot backtrack on itself. It may only move around pieces that it is in direct contact with on each step.' (p. 7). The hive minus the spider is a straight five-piece line whose boundary is a single 14-cell ring with no gates; each ring cell touches the line, and cells off the ring touch nothing (excluded by the contact requirement). From its ring position the spider therefore has exactly two three-step walks - three cells clockwise and three cells anticlockwise: the cell NE of bS1, and the cell SE of wQ. One- and two-step stops are excluded ('no less'), backtracking is excluded.

A.1.9 C009: Grasshopper: jumps only along occupied rows, no one-space slides

- **Rule:** Grasshopper (*Gen42 Hive rulesheet p. 6*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 \wS1;bG1 bQ-`
- **Expectation:** moves_for_piece `{'piece': 'wG1', 'moves': ['wG1 wS1\\', 'wG1 /wQ']}`
- **Hand-written justification:** 'It jumps from its space over any number of pieces (but at least one) to the next unoccupied space along a straight row of joined pieces.' (p. 6). The grasshopper touches occupied cells in exactly two of its six directions: SE (over wS1, landing in the next space, SE of wS1) and SW (over wQ, landing SW of wQ). In the other four directions the adjacent cell is empty, and a jump 'over at least one' piece is impossible - in particular the four adjacent empty cells are NOT destinations: the grasshopper 'does not move around the outside of the Hive like the other creatures'. Expected: exactly the two landing cells.

A.1.10 C010: Grasshopper jumps a full five-piece row to the first empty space

- **Rule:** Grasshopper (*Gen42 Hive rulesheet p. 6*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-`
- **Expectation:** moves_for_piece `{'piece': 'wG1', 'moves': ['wG1 bG1-']}`
- **Hand-written justification:** '...over any number of pieces (but at least one) to the next unoccupied space along a straight row of joined pieces.' (p. 6). Due east the grasshopper faces the unbroken row wQ, wS1, bS1, bQ, bG1; the first unoccupied space beyond it is the cell E of bG1 - the single destination. It must land there, not earlier (every nearer cell in the row is occupied). In all five other directions the adjacent cell is

empty, so no jump exists. The grasshopper is a leaf of the hive, so One Hive does not restrict it.

A.1.11 C011: Queen's only open neighbour is behind a gate: zero moves

- **Rule:** Freedom to Move (*Gen42 Hive rulesheet p. 10*)
- **Setup:** `wS1;bS1 -wS1;wB1 wS1/;bQ -bS1;wQ wB1-;bG1 \bQ;wS2 wQ/;bA1 /bQ;wG1 -wS2;bS2 / bS1;wA1 wS2\;bB1 \bG1;wG2 wQ\;bG2 \bB1`
- **Expectation:** moves_for_piece `{'piece': 'wQ', 'moves': []}`
- **Hand-written justification:** 'Similarly, no piece may move into a space that it cannot physically slide into.' (p. 10). Five of the queen's six neighbours are white pieces; the sixth (the cell W of wG2, equally SE of wB1) is empty, but the two cells adjacent to both the queen and that space are wG2 and wB1 - both occupied - so the queen cannot physically slide in. Removing the queen leaves the white horseshoe wS1-wB1-wG1-wS2-wA1-wG2 connected (and the black chain hangs off wS1 via bS1), so One Hive would allow the move; the block is purely Freedom to Move. Expected: the queen has no legal move.

A.1.12 C012: Ant reaches every cell of the hive perimeter (13 destinations)

- **Rule:** Soldier Ant / Freedom to Move (*Gen42 Hive rulesheet pp. 8, 10*)
- **Setup:** `wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wA1 \wQ;bG1 bQ-`
- **Expectation:** moves_for_piece `{'piece': 'wA1', 'moves': ['wA1 -wQ', 'wA1 \\wG1', 'wA1 \\bS1', 'wA1 \\bQ', 'wA1 \\bG1', 'wA1 bG1/', 'wA1 bG1-', 'wA1 bG1\\', 'wA1 bQ\ \', 'wA1 bS1\\', 'wA1 wG1\\', 'wA1 wQ\\', 'wA1 /wQ']}`
- **Hand-written justification:** 'The Soldier Ant can move from its position to any other position around the Hive provided the restrictions are adhered to.' (p. 8). The hive minus the ant is a straight five-piece line; its boundary is a single 14-cell ring with no gates (every slide step is flanked by one line cell and one empty cell), and every ring cell touches the line. The ant starts on the ring at the cell NW of wQ, so it can stop on any of the other 13 ring cells: the west cap (W of wQ), the five north-shoulder cells (NW of each line piece plus NE of bG1), the east cap (E of bG1), and the six south-shoulder cells (SE of each line piece plus SW of wQ). Cells off the ring touch no piece and are excluded (p. 3 NB: pieces must always touch at least one other piece).

A.1.13 C013: Beetle on the ground: two slides and two climbs

- **Rule:** Beetle (*Gen42 Hive rulesheet pp. 4-5, 10*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-`
- **Expectation:** moves_for_piece `{'piece': 'wB1', 'moves': ['wB1 wS1', 'wB1 wQ', 'wB1 \\bS1', 'wB1 \\wQ']}`
- **Hand-written justification:** 'The Beetle, like the Queen Bee, moves only one space per turn. Unlike any other creature though, it can also move on top of the Hive.' (p. 4). From (NW of wS1) the beetle may climb onto either adjacent piece - wS1 or wQ - or slide along the ground to the two empty cells that keep contact with the hive: NW of bS1 (touching wS1 and bS1) and NW of wQ (touching wQ). The two remaining empty neighbours touch no piece after the beetle lifts, so they are excluded (p. 3 NB). No gate blocks any of the four moves (each is flanked by at most one occupied cell, and for the climbs the flanking stacks are not taller than the destination). Exactly four moves - matching the rulesheet's own beetle example count.

A.1.14 C014: Beetle on top of the hive: all six neighbouring cells

- **Rule:** Beetle (*Gen42 Hive rulesheet p. 5; beetle-gate ruling, World Hive Tournaments Rules FAQ*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-;wB1 wS1;bA1 bG1-`
- **Expectation:** moves_for_piece `{'piece': 'wB1', 'moves': ['wB1 bS1', 'wB1 wQ', 'wB1 \\wS1', 'wB1 \\bS1', 'wB1 wS1\\', 'wB1 wQ\\']}`
- **Hand-written justification:** 'From its position on top of the Hive, the Beetle can move from tile to tile across the top of the Hive. It can also drop into spaces that are surrounded and therefore not accessible to most other creatures.' (p. 5). Sitting on wS1, the beetle may move to every one of the six neighbouring cells: across onto bS1 or wQ (both height-1 stacks), or drop to any of the four empty cells around wS1 - each of which still touches wS1 itself, so contact holds. No pair of flanking stacks is taller than both origin (height 1 under the beetle) and destination, so no beetle gate applies (FAQ). One Hive cannot be violated: wS1 stays where it is. Exactly six destinations.

A.1.15 C015: Beetle gate: drop between two height-2 stacks is blocked

- **Rule:** Freedom to Move above ground level (beetle gate) (*World Hive Tournaments Rules FAQ; Gen42 Hive rulesheet p. 10*)
- **Setup:** `wS1;bG1 wS1/;wQ /wS1;bQ bG1/;wG1 wS1\;bB1 bQ/;wB1 -wS1;bB1 bQ;wB2 /wQ;bB1 bG1;wB1 wS1;bQ bB1-;wB2 wQ;bQ bB1/;wB2 wG1;bA1 bQ/`
- **Expectation:** move_illegal `{'move': 'wB1 bB1\\'}`
- **Hand-written justification:** 'When a piece climbs up or down the hive, or moves staying on top of the hive, it must be able to slide according to the freedom to move rule which applies to higher levels than the ground. If two stacks form a gate above the ground level (we call it beetle gate), pieces won't be able to slide through.' (WHT Rules FAQ). wB1 sits on wS1 (its own level: on top of a height-1 piece); the target cell SE of the bB1 stack is empty (height 0). The two cells adjacent to both origin and target carry the stacks bG1+bB1 and wG1+wB2, both height 2 - strictly taller than both the origin without the beetle (1) and the destination (0) - so the beetle cannot slide down between them. The drop must be rejected. (One Hive would allow it: wS1 stays in place; contact holds via the flanking stacks.)

A.1.16 C016: A piece with a beetle on top of it cannot move

- **Rule:** Beetle (stack immobility) (*Gen42 Hive rulesheet p. 5*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-;wB1 wS1;bA1 bG1-`
- **Expectation:** move_illegal `{'move': 'wS1 bS1\\'}`
- **Hand-written justification:** 'A piece with a beetle on top of it is unable to move' (p. 5). wS1 lies under wB1, so any attempt to move wS1 - here a spider move towards the cell SE of bS1 - must be rejected, regardless of whether the path would otherwise be legal for a spider.

A.1.17 C017: Stack takes the beetle's colour: white may place beside a covered black queen

- **Rule:** Beetle (stack colour) / Placing (*Gen42 Hive rulesheet pp. 2, 5*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bA1 bS1\;wB1 \bS1;bG1 bA1\;wB1 \bQ;bG2 bG1\;wB1 bQ;bB1 bG2\`

- **Expectation:** move_legal `{'move': 'wG1 wB1-'}`
- **Hand-written justification:** '...for the purposes of the placing rules on p. 2, the stack takes on the colour of the Beetle.' (p. 5). White's beetle sits on the black queen at the east end of the hive. The cell E of that stack touches no other piece, so a white placement there is adjacent only to a stack whose colour is - by the rule - white. Placing wG1 there must be accepted. (Without the stack-colour rule the cell would be adjacent to a black piece and the placement would be illegal, p. 2.)

A.1.18 C018: Stack takes the beetle's colour: black may NOT place beside its own covered queen

- **Rule:** Beetle (stack colour) / Placing (*Gen42 Hive rulesheet pp. 2, 5*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bA1 bS1\;wB1 \bS1;bG1 bA1\;wB1 \bQ;bG2 bG1\;wB1 bQ;bB1 bG2\;wG1 -wQ`
- **Expectation:** move_illegal `{'move': 'bB2 wB1-'}`
- **Hand-written justification:** Mirror of C017: the stack bQ+wB1 counts as WHITE ('the stack takes on the colour of the Beetle', p. 5). The cell E of the stack touches only that stack, so for black it is adjacent to a white piece and 'pieces may not be placed next to a piece of the opponent's colour' (p. 2). Black's attempt to place bB2 there must be rejected - even though the buried piece is black's own queen.

A.1.19 C019: One Hive: the only connection between two parts may not move

- **Rule:** One Hive rule (*Gen42 Hive rulesheet pp. 3, 9*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-`
- **Expectation:** moves_for_piece `{'piece': 'wS1', 'moves': []}`
- **Hand-written justification:** 'All pieces must always touch at least one other piece. If a piece is the only connection between two parts of the Hive, it may not be moved.' (p. 3 NB); 'The pieces in play must be linked at all times. At no time can you leave a piece stranded (not joined to the Hive) or separate the Hive in two.' (p. 9). wS1 is the interior link between wQ on one side and bS1-bQ on the other: lifting it splits the hive, so the spider has no legal move at all - every destination, however valid as spider movement, is excluded by One Hive.

A.1.20 C020: Ring: a piece on a closed loop may move (not a cut point); the ring's eye is gated

- **Rule:** One Hive rule / Freedom to Move (*Gen42 Hive rulesheet pp. 9, 10*)
- **Setup:** `wS1;bS1 -wS1;wG1 wS1/;bQ -bS1;wQ wS1\;bG1 -bQ;wG2 wG1-;bG2 -bG1;wA1 wQ-;bA1 -bG2;wS2 wG2\;bB1 -bA1`
- **Expectation:** moves_for_piece `{'piece': 'wQ', 'moves': ['wQ /wS1', 'wQ /wA1']}`
- **Hand-written justification:** The six white pieces form a closed ring, so removing wQ leaves the other five connected around the loop (and the black tail hangs off wS1): One Hive permits the queen to move. Sliding one space (p. 4), the queen has three empty neighbours: the ring's eye and two outside cells. The eye is flanked by wS1 and wA1 - both occupied - so the queen 'may not move into a space that it cannot physically slide into' (p. 10). The two outside cells (SW of wS1, which touches wS1 and bS1; and SW of wA1, which touches wA1) are unobstructed slides keeping contact. Expected: exactly those two destinations.

A.1.21 C021: Game ends when a queen is fully surrounded - even by its own colour

- **Rule:** The Object of Hive / The End of the Game (*Gen42 Hive rulesheet pp. 1, 11*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-;wA1 -wG1;bG2 bQ/;wA2 -wA1;bB1 \bQ;wA3 -wA2;bA1 bS1\;wS2 -wA3;bA2 bQ\`
- **Expectation:** game_over `{'state': 'WhiteWins'}`
- **Hand-written justification:** 'The pieces surrounding the Queen Bee can be made up of a mixture of both your pieces and your opponent's.' (p. 1). 'The game ends as soon as one Queen Bee is completely surrounded by pieces of any colour. The person whose Queen Bee is surrounded loses the game.' (p. 11). Black's final placement (bA2, SE of its own queen) fills the sixth and last cell around bQ. The surrounding pieces are all black - irrelevant per p. 1 - and it is Black's own move that completes the surround: Black loses, the GameString state must read WhiteWins immediately after that move.

A.1.22 C022: One move surrounds both queens simultaneously: draw

- **Rule:** The End of the Game (*Gen42 Hive rulesheet p. 11*)
- **Setup:** `wS1;bS1 wS1/;wQ wS1\;bB1 bS1-;wA1 -wQ;bQ bB1\;wS2 /wQ;bQ /bB1;wG1 -wS2;bQ wQ-;wB1 -wA1;bA1 bQ\;wG1 wS2-;bG1 \bS1;wB1 \wA1;bA2 bQ-;wB1 \wS1;bA3 bB1\;wG2 -wB1;bG1 bS1\`
- **Expectation:** game_over `{'state': 'Draw'}`
- **Hand-written justification:** 'The person whose Queen Bee is surrounded loses the game, unless the last piece to surround their Queen Bee also completes the surrounding of the other Queen Bee. In that case the game is drawn.' (p. 11). Before Black's last move, each queen has exactly one empty neighbour - the same cell (1,0), adjacent to both queens (the queens sit side by side, wQ NE-shoulder wS1, bQ beside it). Black's grasshopper at (1,-2) jumps SE over bS1 into that cell, filling the sixth neighbour of both queens with a single piece: the game must end as a Draw, not a win for either side.

A.1.23 C023: A player who can neither place nor move must pass

- **Rule:** Unable to move or place (*Gen42 Hive rulesheet pp. 2, 5, 10, 11*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bQ bS1/;wA1 /wQ;bQ bS1-;wB1 \bS1;bQ bS1/;wB1 bS1;bQ wB1-;wA1 /bQ;bQ wB1/;wA2 /wQ;bQ wB1-;wA2 bQ\;bQ wB1/;wA3 /wQ;bQ wB1-;wA3 bQ-;bQ wB1/;wG1 /wB1;bQ wB1-;wG1 wB1/`
- **Expectation:** must_pass
- **Hand-written justification:** 'If a player can neither place a new piece or move an existing piece, the turn passes to their opponent who then takes their turn again.' (p. 11). After White's final move Black has: bS1 under wB1 - 'a piece with a beetle on top of it is unable to move' (p. 5), and the stack counts as white for placing (p. 5); and bQ, whose five neighbours are the stack, wG1, wA1, wA2 and wA3, with its single empty neighbour reachable only between wG1 and wA3 - a gate the queen 'cannot physically slide into' (p. 10). No placement is possible either: the only cell adjacent to a black-topped piece is that same gated cell, which also touches white pieces (p. 2). Black is not lost - bQ has an empty neighbour, so it is not surrounded - but must pass.

A.1.24 C024: Pillbug special ability: moving an adjacent friendly piece (kernel guard)

- **Rule:** Pillbug special ability (*Gen42 Pillbug rulesheet (English section)*)
- **Setup:** `wP;bS1 wP-;wQ -wP;bQ bS1-`

- **Expectation:** move_legal `{'move': 'wQ wP\\'}`
- **Hand-written justification:** 'The special ability allows the Pillbug to move an adjacent piece (friend or enemy) two spaces; up onto itself and then down into another empty space adjacent to itself.' (Pillbug rulesheet). wQ is adjacent to wP; the target cell SE of wP is empty and adjacent to wP. None of the four exceptions applies: wQ was not just moved by the other player (Black's last move was placing bQ), wQ is not in a stack, removing wQ does not split the hive (it is a leaf), and no stacked pieces form a gap on the up-and-over path. NOTE: this is a variant-tagged kernel-guard case (protocol §2) - the study variant is base game; Pillbug cases only protect the shared rules kernel.

A.1.25 C025: A piece just moved by the enemy pillbug is stunned for one turn (kernel guard)

- **Rule:** Pillbug special ability (immobility of the moved piece) (*Gen42 Pillbug rulesheet (English section); World Hive Tournaments Rules FAQ*)
- **Setup:** `wS1;bP wS1-;wQ -wS1;bQ bP-;wA1 \wQ;bG1 bQ-;wA1 \bP;bG1 -wQ;wG1 -wA1;wA1 bP\`
- **Expectation:** move_illegal `{'move': 'wA1 bP/'}`
- **Hand-written justification:** 'Furthermore, any piece moved by the Pillbug may not be moved at all (directly or via Pillbug action) on the next player's turn.' (Pillbug rulesheet); FAQ: 'any piece that just moved, in the turn of the other player immediately after is unable to: move, be moved or use the pillbug's ability.' Black's pillbug just threw wA1 up over itself and down to the cell SE of bP (a legal use: wA1 last moved two plies earlier, so the just-moved exception did not block the throw; removing it kept the hive whole since wG1 also touches wS1 and wQ). On White's very next turn the thrown ant is stunned: the attempted ant move to NE of bP must be rejected. Variant-tagged kernel-guard case (protocol §2).

A.1.26 C026: Pillbug may not move the piece the opponent just moved (kernel guard)

- **Rule:** Pillbug special ability (exceptions) (*Gen42 Pillbug rulesheet (English section)*)
- **Setup:** `wS1;bP wS1-;wQ -wS1;bQ bP-;wA1 \wQ;bG1 bQ-;wA1 \bP`
- **Expectation:** move_illegal `{'move': 'wA1 bP\\'}`
- **Hand-written justification:** 'The Pillbug may not move the piece which was just moved by the other player.' (Pillbug rulesheet, first exception). White's ant moved to the cell NW of bP on the immediately preceding ply; Black's attempt to use the pillbug's ability on that same ant - throwing it to SE of bP - must be rejected. (The identical throw becomes legal two plies later, which is case C025′s setup.) Variant-tagged kernel-guard case (protocol §2).

A.1.27 C027: One Hive binds even the grasshopper: a cut-point cannot jump

- **Rule:** One Hive rule / Grasshopper (*Gen42 Hive rulesheet pp. 3, 6, 9*)
- **Setup:** `wG1;bS1 wG1-;wQ -wG1;bQ bS1-`
- **Expectation:** moves_for_piece `{'piece': 'wG1', 'moves': []}`
- **Hand-written justification:** The grasshopper is exempt from the sliding restriction (p. 10: it 'can jump into or out of a space'), but not from One Hive: 'If a piece is the only connection between two parts of the Hive, it may not be moved.' (p. 3 NB). wG1 sits

between wQ and the black pair; lifting it for any jump splits the hive in two, so despite having jump lines in both E and W directions the grasshopper has no legal move.

A.1.28 C028: A beetle cannot be PLACED directly on top of the hive

- **Rule:** Beetle (placement NB) (*Gen42 Hive rulesheet p. 5*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-`
- **Expectation:** move_illegal `{'move': 'wB1 wS1'}`
- **Hand-written justification:** 'When it is first placed, the Beetle is placed in the same way as all the other pieces. It cannot be placed directly on top of the Hive, even though it can be moved there later.' (p. 5 NB). wB1 is still in hand; the attempt to introduce it on top of wS1 must be rejected. (C013/C014 verify that the same beetle may climb there by a move once placed.)

A.1.29 C029: Spider may not stop after one step ('no more, no less')

- **Rule:** Spider (*Gen42 Hive rulesheet p. 7*)
- **Setup:** `wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wS1 \wQ;bG1 bQ-`
- **Expectation:** move_illegal `{'move': 'wS1 \\wG1'}`
- **Hand-written justification:** 'The Spider moves three spaces per turn - no more, no less.' (p. 7). The cell NW of wG1 is exactly one sliding step from the spider's position, and no legal three-step non-backtracking path ends there (the two three-step walks end NE of bS1 and SE of wQ - case C008); a path through that cell passes it at step one and may not stop. The one-step move must be rejected.

A.1.30 C030: Queen between two pieces: two slides along the shoulder

- **Rule:** Queen Bee / One Hive / Freedom to Move (*Gen42 Hive rulesheet pp. 4, 9, 10*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-`
- **Expectation:** moves_for_piece `{'piece': 'wQ', 'moves': ['wQ -wB1', 'wQ /wS1']}`
- **Hand-written justification:** The queen touches wS1 (E) and wB1 (NE). Removing her keeps the hive whole (wB1 still touches wS1), so One Hive allows a move. One-space slides (p. 4): of her four empty neighbours, only the cell W of wB1 (keeping contact with wB1) and the cell SW of wS1 (keeping contact with wS1) still touch the hive after she lifts; the two far-western cells touch nothing and are excluded (p. 3 NB). Neither slide is gated (each flanked by exactly one occupied cell). Expected: exactly those two destinations.

A.2 Tactical verification set (5 cases)

Search-correctness cases: mate-in-1 by walk and by jump from both colours, and self-surround avoidance; solved 5/5 by the MCTS baseline at 400, 1600 and 6400 simulations.

A.2.1 T001: White mates in 1: occupy the black queen's last liberty (SE of bQ)

- **Rule:** The End of the Game (*Gen42 Hive rulesheet pp. 1, 11*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-;wA1 -wG1;bG2 bQ/;wA2 -wA1;bB1 \bQ;wA3 -wA2;bA1 bS1\`
- **Expectation:** bestmove_to_cell `{'target': 'bQ\\'}`

- **Hand-written justification:** The black queen has exactly one empty neighbour, the cell SE of bQ. Any white piece landing there completes the surround and wins immediately ('the game ends as soon as one Queen Bee is completely surrounded by pieces of any colour', p. 11; mixture of colours allowed, p. 1). The cell is reachable: a white ant can walk the south perimeter in one move (entry past bA1 is not gated), so a winning move exists. No other single move ends the game. The searcher must play onto that cell.

A.2.2 T002: Black mates in 1: occupy the white queen's last liberty (SE of wQ)

- **Rule:** The End of the Game (*Gen42 Hive rulesheet pp. 1, 11*)
- **Setup:** `wS1;bS1 -wS1;wQ wS1-;bQ -bS1;wG1 wQ-;bG1 -bQ;wG2 wQ/;bG2 -bG1;wB1 \wQ;bA1 -bG2;wA1 wS1\;bA2 -bA1;wA2 wG1-`
- **Expectation:** bestmove_to_cell `{'target': 'wQ\\'}`
- **Hand-written justification:** Mirror of T001 with the colours exchanged and black to move. This pair is the player-alternation check at the move level: the winning pattern must be found from both sides. The white queen's only empty neighbour is the cell SE of wQ; a black ant reaches it along the south perimeter (route via SE of bS1's column: the step into the cell is flanked by the occupied wA1 cell, so contact holds and no gate blocks). Landing there completes the surround: Black wins (p. 11).

A.2.3 T003: White mates in 1 by grasshopper jump over four pieces (E of bQ)

- **Rule:** Grasshopper / The End of the Game (*Gen42 Hive rulesheet pp. 6, 11*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ/;wA1 \wQ;bB1 \bQ;wA2 \wA1;bA1 bS1\;wA3 \wA2;bA2 bQ\`
- **Expectation:** bestmove_to_cell `{'target': 'bQ-'}`
- **Hand-written justification:** The black queen's only empty neighbour is the cell E of bQ. Due east from wG1 at the west cap runs the unbroken occupied row wQ, wS1, bS1, bQ; the next unoccupied space along that row is exactly the winning cell, so the grasshopper jumps over four pieces and completes the surround (p. 6: 'over any number of pieces ... to the next unoccupied space along a straight row of joined pieces'; p. 11: surround ends the game). White ants can also walk in around the perimeter; the expectation is the destination cell, whichever piece the searcher sends.

A.2.4 T004: Black mates in 1 by grasshopper jump over four pieces (W of wQ)

- **Rule:** Grasshopper / The End of the Game (*Gen42 Hive rulesheet pp. 6, 11*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 \wQ;bG1 bQ-;wG2 \wS1;bA1 bQ\;wA1 wQ\;bA2 bA1\;wA2 /wQ;bA3 bA2\;wB1 \wG1`
- **Expectation:** bestmove_to_cell `{'target': '-wQ'}`
- **Hand-written justification:** Alternation mirror of T003 with black jumping. The white queen's neighbours: E wS1, NW wG1 (placed NW of wQ), NE wG2 (placed NW of wS1 = NE of wQ), SE wA1, SW wA2. Five are occupied; only W of wQ is empty. Due east of that gap runs the unbroken row wQ, wS1, bS1, bQ with bG1 at the east cap (3,0): from bG1 the next unoccupied space westward along the row is exactly the gap, so the grasshopper jumps over four pieces and completes the surround (pp. 6, 11). Black's southern tail (bA1..bA3 SE of bG1) keeps black's earlier placements legal and away from white.

A.2.5 T005: Do not fill your own queen's last liberty

- **Rule:** The End of the Game (*Gen42 Hive rulesheet p. 11*)
- **Setup:** `wS1;bS1 -wS1;wQ wS1-;bQ -bS1;wG1 wQ-;bG1 -bQ;wG2 wQ/;bG2 -bG1;wB1 \wQ;bA1 -bG2;wA1 wS1\;bA2 -bA1`
- **Expectation:** bestmove_avoid_cell `{'target': 'wQ\\'}`
- **Hand-written justification:** The cell SE of wQ is the white queen's last liberty. White placing or moving any piece there completes the surround of White's own queen: 'the person whose Queen Bee is surrounded loses the game' (p. 11) regardless of who supplied the sixth piece. The placement is perfectly legal (the cell touches only white pieces), so only search judgment prevents it. Any move except one landing on that cell passes; the searcher must not play into it. (This does not assert White survives long-term, since black threatens the same cell; it asserts only that the immediate self-kill is avoided.)

B Architectures, decoder and data formats in detail

This appendix complements Section 7 with layer-by-layer tables of the two networks and their ablation variants, the exact arithmetic of the shared action decoder, the tensor interface of the graph arm, the byte layout of the self-play record, and the search settings shared by both arms. Parameter totals are the values measured by the training code's parameter counter and were re-verified for this appendix; per-layer counts were not separately recorded, so the tables give shapes and widths only.

B.1 HiveNet, the grid network

The input is a tensor of 77 planes × 32 × 32 with float32 values in [0, 1] (Table 13). The body and heads are listed in Table 46. Hex adjacency on axial coordinates is a 7-cell subset of the 3 × 3 neighbourhood, so the 3 × 3 kernels cover it with two learnable dead corners per kernel.

Table 46: HiveNet, the grid arm's network, layer by layer. Blocks are numbered from 0 as in the implementation; the two global-pooling blocks are the ones at one third and two thirds of the depth. Widths are channel counts; the total is the measured parameter count.

Stage	Operation	Width	Notes
Input	77 planes × 32 × 32	77	float32 in [0, 1]
Stem	Conv 3 × 3 (no bias), BatchNorm, ReLU	77 → 96	padding 1
Block 0	Conv 3 × 3, BatchNorm, ReLU; Conv 3 × 3, BatchNorm; residual add; ReLU	96 → 96	convolutions without bias
Block 1	as block 0	96 → 96	
Block 2	as block 0, plus a global-pooling bias after the second BatchNorm	96 → 96	mean ‖ max over the frame (2 × 96) → linear → 96, added per channel
Block 3	as block 0	96 → 96	
Block 4	as block 0	96 → 96	
Block 5	as block 2	96 → 96	global-pooling bias
Block 6	as block 0	96 → 96	
Block 7	as block 0	96 → 96	
Pooling	mean over the 32 × 32 frame	96	feeds the pass logit and the value head
Policy, spatial	Conv 1 × 1 (with bias)	96 → 28	flattened to 28 × 1024 = 28,672 logits, index slot × 1024 + y × 32 + x
Policy, pass	Linear on the pooled features	96 → 1	appended as index 28,672
Value	Linear, ReLU, Linear on the pooled features	96 → 64 → 3	win / draw / loss logits from the side to move
Total			1.44 M parameters

B.2 HiveGraphNet, the graph network

The graph arm's network consumes fixed-shape tensors (Table 47) built from one record by the Python builder for training and by its Rust mirror for inference. Node and global feature layouts are given in Table 14 and Table 15; the body and heads are listed in Table 48, and the ablation variants in Table 49.

Table 47: Tensor interface of the graph arm, per position. Capacities: 224 nodes (occupied cells plus the empty ring) and 321 move rows (320 legal moves, the record's cap, plus one pass row); exceeding either capacity raises an error rather than truncating. Node order is deterministic: occupied cells first, then ring cells, each sorted by (y, x) in frame coordinates.

Tensor	Shape	Type	Content
nodes	224 × 56	float32	node features, zero-padded beyond the real nodes
neighbours	224 × 6	int64	index of the neighbour in each hex direction; 224 = none
node mask	224	bool	1 for a real node
globals	23	float32	global features
moves	321 × 3	int64	(slot, destination node, source node) per legal move; source = 224 for placements and pass; the pass row uses slot 28
move mask	321	bool	1 for a legal row
target	321	float32	visit distribution over the legal rows (training only)
outcome	scalar	int64	0 loss / 1 draw / 2 win / 3 truncated, from the side to move (training only)

Table 48: HiveGraphNet, the graph arm's network, layer by layer. Layers are numbered from 0 as in the implementation; the pooling layers are every third layer. Widths are channel counts; the total is the measured parameter count.

Stage	Operation	Width	Notes
Input	node features ‖ broadcast globals	56 + 23	per node
Input linear	Linear	56 + 23 → 152	masked to real nodes; a zero row at index 224 stands in for absent neighbours
Layer 0	relational layer: W_{self} (with bias) + six W_d (no bias); residual; ReLU; mask	152 → 152	Equation 4
Layer 1	as layer 0	152 → 152	
Layer 2	as layer 0, plus a global-pooling bias inside the non-linearity	152 → 152	masked mean ‖ masked max (2 × 152) → linear → 152, added to every node
Layer 3	as layer 0	152 → 152	
Layer 4	as layer 0	152 → 152	
Layer 5	as layer 2	152 → 152	global-pooling bias
Layer 6	as layer 0	152 → 152	
Layer 7	as layer 0	152 → 152	
Readout	masked mean ‖ masked max over the real nodes	2 × 152	
Value	Linear, ReLU, Linear on readout ‖ globals	2 × 152 + 23 → 64 → 3	win / draw / loss logits from the side to move
Slot embedding	embedding table	28 piece slots + pass → 32	

Stage	Operation	Width	Notes
Reserve vector	learned vector	152	source embedding for placements and for the pass row
Policy	Linear, ReLU, Linear per move row on destination ‖ source ‖ slot	2 × 152 + 32 → 128 → 1	illegal rows filled with a large negative constant before the softmax
Total			1.47 M parameters

Table 49: The graph-network variants used in the ablations, with their measured parameter counts in millions. The encoder, the decoder, the training settings, the budgets and the evaluation settings are identical across variants; only the named component changes.

Variant	Difference from the full graph arm	Parameters
Full graph arm	none	1.47 M
Untyped edges	one shared matrix for the six directions, in every layer	0.54 M
No global pooling	no pooling bias in any layer	1.37 M
Untyped edges + gradient clipping (supplement)	one shared matrix for the six directions, plus a global gradient-norm clip of 1.0 during training (a two-component difference)	0.54 M

B.3 The shared action decoder in detail

Slots. A piece is addressed by a side-to-move-relative slot (Table 50): the mover's pieces in roster order, then the opponent's. In the base game, which has 11 pieces per side, only the mover's first 11 slots can ever be legal; the expansion slots and all opponent slots exist for kernel uniformity (the Pillbug expansion moves enemy pieces) and remain inert. Bug types are coded 0–7 in the order Q, S, B, G, A, M, L, P; this code indexes the piece planes of the grid encoding (bug type × 4) and the one-hot block of the graph encoding.

Table 50: Piece slots of the shared action decoder. Slot 28 is the pass slot in the graph arm's move rows; the grid arm's pass is flat index 28,672.

Slot	Mover's piece	Slot	Opponent's piece
0	Queen	14	Queen
1–2	Spiders	15–16	Spiders
3–4	Beetles	17–18	Beetles
5–7	Grasshoppers	19–21	Grasshoppers
8–10	Ants	22–24	Ants
11	Mosquito	25	Mosquito
12	Ladybug	26	Ladybug
13	Pillbug	27	Pillbug

Flat index (grid arm). With (x, y) the destination's frame coordinates, the policy index is $\text{slot} \times 1024 + y \times 32 + x$; pass is index 28,672 and the vector has 28,673 entries. The same index is what records store, for the played move and for every entry of the visit distribution and of the legal list.

Per-candidate rows (graph arm). Each legal index from the record is decoded into a move row: slot = index div 1024; remainder = index mod 1024; x = remainder mod 32, y = remainder div 32; the destination node is the index of cell (x, y) in the candidate-set ordering; the source node is the standing node of the piece if it is on the board, and the reserve sentinel 224 otherwise; the pass index becomes the row (28, 224, 224). The

identity encode → index → decode on all legal moves across game types is the second of the seven automated pre-training checks.

Shared properties. Masking: the softmax runs over exactly the legal set (plus pass when legal), implemented as a fill of illegal entries with a large negative constant in both training loops and as a softmax over the legal rows in both inference evaluators; the first automated check asserts zero illegal mass through a real forward pass. Order independence: scores attach to (slot, destination) pairs, never to list positions. Tiebreak: argmax ties break toward the lowest flat index in both arms. Targets: the top 15 (index, visit weight) entries of the root visit distribution, normalised over their support; when every stored weight is zero the target falls back to the played move. Value: a win/draw/loss head trained with truncated records excluded; the search consumes P(win) – P(loss) from the side to move, with terminal values +1, –1 and 0.

B.4 Record format

Self-play positions are written as fixed-size little-endian records in shards that begin with a 16-byte header (8 magic bytes identifying the format version, 8 reserved). The format evolved in three versions: 112 bytes with a one-hot played-move target; 176 bytes adding the top-15 visit distribution; and the study's 818-byte version 3 adding the generating-model stamp and the legal-move index list, which is what enables legal-masked training in both arms. Only version-3 records are study inputs. Table 51 gives the layout.

Table 51: Byte layout of the version-3 self-play record (818 bytes, little-endian). "Mover" is the side to move at the recorded position.

Bytes	Field	Encoding
0–83	28 pieces × (x, y, level)	one byte each, frame coordinates, absolute piece order; x = 255 means in hand
84	side to move	0 white, 1 black
85	last-moved piece	piece id; 255 = none (stun state)
86	ply	clamped at 255
87	game-type bits	1 = M, 2 = L, 4 = P
88, 89	queen liberties (mover, opponent)	255 = queen not placed
90, 91	reserve counts (mover, opponent)	
92–95	one-hive-pinned bitmask	32-bit, one bit per piece id
96–97	played-move policy index	16-bit flat index
98	outcome from the mover's perspective	0 loss / 1 draw / 2 win / 3 truncated (never a draw)
99	record version	3
100–107	generating-model stamp	generation (32-bit) then network hash (32-bit); 0 = unstamped
up to 112	reserved	zero
112–175	root visit distribution	15 × (policy index 16-bit, visit weight 16-bit), then total stored visits (32-bit)
176–177	legal-move count	16-bit; 0xFFFF = list unavailable (overflow)
178–817	legal policy-index list	16-bit × 320 (cap; measured maximum branching 213); unused slots zero

Both arms read the same record. The grid arm decodes the 77 planes from it; the graph arm builds the cell graph from it; both take their policy target from bytes 112–175, their legal mask from bytes 176–817, and their outcome from byte 98. The Rust writer and the

Python readers of the planes and of the graph tensors are pinned byte-identical by the nightly cross-language golden tests over 240 and 160 positions respectively.

B.5 Search settings

Table 52: Search settings shared by both arms. Self-play settings come from the frozen protocol; evaluation settings were pinned by configuration on 9 September 2026, before any training run, and any later change would count as a new study.

Setting	Self-play (training data)	Independent evaluation
Search	PUCT Monte-Carlo tree search, c = 1.4, batched leaf evaluation, terminal values backed up exactly	same
Simulations per decision	128 on 25 % of decisions (recorded), 32 on the other 75 % (unrecorded); playout-cap randomization	400
Dirichlet root noise	ε = 0.25	0 (the code default, asserted by a test; the engine binary exposes no noise flag)
Move selection	temperature sampling for the first 12 plies, then argmax	deterministic argmax
Resignation	below −0.92, with 10 % of games never resigning (audit)	not part of the pinned settings
Ply cap	300; a capped game is recorded as truncated, never as a draw	300; truncation reported separately
Openings	none	4 opening plies, paired colour-swapped, shared opening identifiers across all runs and arms
Inference provider	each arm's best: CoreML for the grid arm, CPU for the graph arm	same

The exploration machinery is structurally confined to self-play: the evaluation path's defaults carry ε = 0 and no temperature, a test asserts those defaults, and the engine binary exposes no flag to enable noise, so the evaluation route cannot opt into it. Self-play opts into noise explicitly. The two paths therefore differ by construction rather than by convention.

C Frozen constants, hyperparameters, seeds and configurations

Every value in this appendix is the one the campaigns actually ran; the configuration and manifest files kept with each run are authoritative, and nothing here is a recommendation. Dates are those of the author's approval or of the mechanical computation that fixed a value; Appendix E indexes the corresponding records, and Section 9 explains why each value was fixed when it was.

C.1 Frozen constants

Table 53: Frozen experimental constants with the date each was fixed and the approval under which it was fixed. No constant was changed after its date.

Constant	Value	Fixed on (2026)	Approved by
Game variant	base game (queen, spider, beetle, grasshopper, ant), tournament opening rule; expansions outside the perimeter	9 Sep (scope); frozen in the protocol 10 Sep	the author
Move cap and truncation	300 plies; a truncated game is its own outcome, never a draw; rate reported separately	9 Sep (convention and measured value); frozen 10 Sep	the author
Self-play search budget	128 full / 32 cheap simulations per decision; full fraction 0.25	10 Sep (protocol)	the author
Self-play exploration	temperature sampling for 12 plies; Dirichlet root noise $\varepsilon = 0.25$; resignation at −0.92 with a 10% no-resign audit	10 Sep (pre-registered matrix)	the author
Evaluation search	400 simulations per decision; no root noise; deterministic argmax	9 Sep	the author
Evaluation volume	100 paired games per opponent at final and cutoff checkpoints; 20 per opponent at generations 4 and 7	10 Sep; confirmed unchanged 16 Sep	the author
Opponent population	B-RND (legal-random), B-HEU (heuristic, weights hash-pinned), B-MCTS (search, 6,400 simulations); prior checkpoint excluded	9 Sep	the author
Openings	250 unique legal 4-ply openings; generator seed 20260910; pair i plays line i with both colours	10 Sep	the author
Equal-time cutoff	18.77 h = median total wall-clock of the three original grid runs; rule in the matrix	rule 10 Sep; value 16 Sep	rule: the author; value: computed mechanically
Training seeds	1–3 per arm; 4–5 added under three written pre-commitments	10 Sep; extension 26 Sep	the author
Protocol document	version 1.0; any later change is a new study	10 Sep	the author

Frozen artifacts are identified by content hash (Table 54). The protocol file carried its hash at the freeze and carries it unchanged; the openings hash covers the 250 opening lines independently of the file header; the heuristic weights are additionally pinned by an automated test that fails if any weight changes.

Table 54: Content hashes of the frozen artifacts; these are the scientific identifiers of the protocol, the openings and the opponent population.

Artifact	SHA-256
Protocol, version 1.0	f340a6b64db0f5f0bf126ffb251c3de339450bde192fd54b 719036a8a3aefeb5
Openings, content (250 lines)	63b318d071dfc3ecfae3585636c8e6f7327ddc08e7aed86a 466f915f8005af7b
Heuristic weights (B-HEU)	d0602f1895fbed70b6f84ac2a3eb87bd68e811e53acf24d4 d7814a1495b0b97a
B-RND configuration	f2fc4a06441d3c1a7922838a6693dbb48ec54543bc34fd81 4d41c9a742514cd7
B-HEU configuration	7210a0a349c5bad5dcd2df099cc6865ee3cb5d8a2c30e106 ce58137804818999
B-MCTS configuration	3fc8f75cf2b4f21012dd61e9924408fbfc32c8ea561aa96e b44f1091ba07364e

C.2 Training hyperparameters

The training loop and every hyperparameter are identical for both arms and for all ablation variants, with the single exception noted in the last row.

Table 55: Training hyperparameters, identical for the grid arm, the graph arm and the ablation variants; the gradient clip of the A1′ supplement is the only optimizer change in the study.

Hyperparameter	Value (both arms)
Optimizer	SGD, momentum 0.9, weight decay 1e-4
Learning rate	0.02, cosine-annealed to lr/100 over each generation's steps
Batch size	256
Epochs per generation	2
Policy loss	cross-entropy against the MCTS visit distribution, softmax over exactly the legal move set
Value loss	win/draw/loss from the side to move, weight 0.6; truncated records excluded
Training data per generation	the generation's 500 self-play games; only full-budget (128-simulation) decisions are recorded
Gradient clipping	none, except in the A1′ supplement (global norm 1.0)
Hyperparameter search	none, for either arm

C.3 Seed derivation

Every random choice in the study descends from a recorded seed, so that a run, an evaluation set or a bootstrap interval regenerates identically.

Table 56: Derivation of every seed used in training, evaluation and analysis.

Quantity	Seed
Training run (arm, seed s)	base seed = 100,000 × s; seed spaces disjoint across runs
Generation g (0–9)	self-play seed = base + g; training seed = base + g
Generation-0 network	seeded random initialisation with the base seed, exported before any self-play
Network under evaluation	9000 + g for in-run evaluations (generations 4, 7 and 9); 9500 for cutoff-checkpoint sets
Opponents	B-RND 9101; B-MCTS 9201; B-HEU deterministic (no seed)

Quantity	Seed
Match runner	777,000 + *g* for in-run evaluations; 888,000 for cutoff sets; with fixed openings the opening/colour schedule is seed-independent (schedule hashes reproduced across runs with different agents and match seeds: 8cd84b6564440666 for the 20-game sets, 3bf5d3e51cea0c0d for the 100-game sets, identical in both studies)
Opening generator	20260910
Bootstrap	fixed resampling seed; 10,000 resamples

C.4 Configuration of each arm and variant

All five configurations share the frozen constants of Table 53 and the hyperparameters of Table 55; they differ only as stated in the table. Parameter counts are exact counts from the instantiated models; the +1.5% capacity difference between the arms is the exact ratio. Capacity differences in the ablation variants are inherent to the removed component and are reported rather than equalised.

Table 57: Configuration of the two arms and the three ablation variants, with exact parameter counts and the difference relative to the grid arm in parentheses. Everything not listed is identical across the five rows. The inference provider is the measured best available per arm on the study machine.

Variant	Body	Parameters	Difference from the full graph arm	Seeds	Inference provider
Grid arm	residual CNN, 96 channels × 8 blocks, over 77 planes × 32 × 32	1,443,168	not applicable (the other arm)	1–5	CoreML
Graph arm (full method)	relational message passing, 152 hidden × 8 layers, slot embedding 32, node capacity 224, six direction-typed relations, global-pooling bias	1,465,452 (+1.5%)	reference	1–5	CPU
A1 (naive adjacency)	one shared edge matrix in place of the six direction-typed matrices	541,292 (−62.5%)	edge typing removed	1–3	CPU
A2 (no global pooling)	global-pooling bias removed from every layer	1,372,732 (−4.9%)	pooling removed	1–3	CPU
A1′ (supplement)	as A1 plus global gradient-norm clip 1.0	as A1	two components: edge typing removed and clip added	1–3	CPU

C.5 Measured machine profile

All wall-clock figures in this report were measured on one Apple M1 Pro (10 cores, 16 GB, macOS 15.3.1), with 4 worker threads per run, runs executed sequentially and the machine kept awake. The inference provider used for each arm's self-play and evaluation is the measured best available on this machine; the graph network's gather-heavy operations are only partly supported by the accelerator (147 of 287 nodes, 15 partitions), which is why its CPU path wins.

Table 58: Inference cost per network evaluation on the study machine, measured on 10 September 2026; the best available path per arm is the one used in self-play and evaluation.

Path (batch 1, per decision-evaluation)	**Grid**	**Graph**
PyTorch, CPU	11.03 ms	10.51 ms
ONNX, CPU provider	23.5 ms	3.67 ms
ONNX, CoreML provider	2.62 ms	9.84 ms
Best available (used)	2.62 ms (CoreML)	3.67 ms (CPU)

Training throughput was benchmarked on 10 September 2026 at batch 128, forward and backward, on the machine's GPU backend: 274 positions/s for the grid network and 138 positions/s for the graph network (forward-only on the CPU: 138 and 478 positions/s; the graph network is faster per position on the CPU and slower on the GPU). The campaign runs loaded their data in-process and trained at or below these figures; generation, not training, dominates a generation's wall-clock in either arm. Per-run training wall-clock (self-play, training and export over 10 generations × 500 games, evaluation games excluded) was 16.7–19.1 h for grid runs and 26.6–49.9 h for graph runs, with means of 18.0 h and 36.7 h (per-seed values in Table 68). An evaluation game at 400 simulations took ≈23–32 s; the search opponent at 6,400 simulations decides in ≈27 ms single-threaded; a subprocess round-trip to the engine costs 21.7 µs. Disk use was estimated at ≈3–5 GB per three-seed campaign against a 20 GB free-space guard.

C.6 The second study: frozen constants, regime and seeds

The second study (Section 15) reuses every constant of Table 53 except where Table 59 states otherwise, and every hyperparameter of Table 55 except the training data per generation. Its protocol was frozen as version 2.0 on 13 October 2026 under the content hash given in Table 60; the two arms are the configurations of the first two rows of Table 57, run with seeds 11 to 15, and no ablation variant was run.

Table 59: Frozen constants of the second study, with the date each was fixed and the approval under which it was fixed; everything not listed is the first study's.

Constant	**Value**	**Fixed on (2026)**	**Approved by**
Training regime	warm start: the network of generation g is initialised from the final checkpoint of generation $g-1$ (model weights only; optimizer and cosine schedule restart every generation); generation 0 starts from the seeded random initialisation	13 Oct (protocol v2)	the author
Replay window	3 generations: generation g trains on the records of generations $\max(0, g-2)$ to g, that is 500, 1,000, then 1,500 games from generation 2 onward; 2 epochs per generation	13 Oct	the author
Hyperparameter policy	option A: the first study's settings in both arms, no tuning; option B (three learning rates on one seed per arm) considered and rejected	13 Oct	the author
Training seeds	11–15 per arm; base seed 100,000 × seed; no random stream shared with seeds 1–5	13 Oct	the author
Equal-time cutoff of the second study	17.63 h = median of the total training wall-clock of the five grid runs of the second study (17.46, 17.95, 17.63, 17.46, 18.52 h; sum	rule 13 Oct; value 25 Oct	rule: the author; value: computed mechanically

Constant	Value	Fixed on (2026)	Approved by
	of the ten per-generation clocks, evaluations excluded), computed once from the clock files on 25 October 2026 before any cross-arm number, never recomputed		
Evaluation schedule	generations 5 and 8 at 20 paired games per opponent (descriptive only); final checkpoint at 100; cutoff checkpoints at 100 with network seed 9500 and match seed 888,000	13 Oct (as in the first study)	the author
Reused frozen assets	opponent population, 250 openings, pinned evaluation settings, both networks and the shared decoder, untouched	13 Oct (reuse declared)	the author
Execution order	ten runs sequential on the study machine, interleaved grid then graph for seeds 11 to 15; no other heavy process during the campaign	13 Oct	the author
Protocol document	version 2.0; any later change is a new study	13 Oct	the author

Table 60: Content hash of the second study's frozen protocol; the hashes of the reused artifacts are those of Table 54.

Artifact	SHA-256
Protocol, version 2.0	e296a12e8d1e3dee36c3b210ecec ced62314e64657aede1c b91b1ea452b461eb

The seed derivation of Table 56 applies unchanged with s = 11 to 15: the base seed is 100,000 × s, generation g uses base + g for self-play and training, the generation-0 network is initialised from the base seed, and the evaluation seeds (9000 + g in-run, 9500 for cutoff sets; match runner 777,000 + g in-run, 888,000 for cutoff sets) are the first study's. The campaign driver records the regime of every run (warm start on, window 3) in a configuration file kept with the run, and every generation's training configuration lists the initialisation checkpoint and the shards actually read.

D Statistical procedures and raw result tables

This appendix states the estimators of the study exactly as they were computed and reproduces, without rounding or recomputation, every per-run number behind the main comparison and the ablations. Values are copied from the regenerated result files and from the dated analysis records; where a record does not state a quantity, the cell says so.

D.1 Score, truncation rate and sensitivity score

An evaluation match between one checkpoint and one opponent is 100 games on the frozen opening lines, each line played once with each colour. A game ends in a win, a draw, a loss, or a truncation at the 300-ply cap. From the game rows of one (seed, opponent) cell three statistics are derived: the **score**, the mean of win = 1, draw = 0.5, loss = 0 over the *non-truncated* games; the **truncation rate**, the number of truncated games divided by the 100 games played; and the **sensitivity score**, the mean over *all* 100 games with every truncated game counted 0.5, reported as a column and never used as the primary metric. Games are aggregated to one score per cell first; the seed is the unit of every later step, and no game enters any interval as an independent observation.

D.2 The seed-level bootstrap

Every interval in the report is a percentile bootstrap whose resampling unit is the seed, that is, one independent training run. For a cell series $s_1, \ldots, s_n$ (one score per seed, with n = 5 in the final analysis and n = 3 in the analysis of 19 September 2026), the procedure draws n indices uniformly with replacement, averages the corresponding scores, repeats this B = 10,000 times, sorts the resample means, and reports the elements at zero-based positions $\lfloor 0.025\,B \rfloor$ and $\lfloor 0.975\,B \rfloor$ as the 95% interval. No bias correction or acceleration is applied; the point estimate printed beside each interval is the plain mean of the n per-seed scores. The pseudo-random generator is Python's `random.Random` seeded with the constant 0 for every call, so every interval is reproducible to the last digit from the same per-seed inputs.

The arm contrast is the difference of seed-level means, graph minus grid. In each of the B = 10,000 resamples the graph seed set and the grid seed set are resampled *independently*, each with its own size, and the difference of the two resample means is recorded; the sorted differences are cut at the same two positions. Seeds are independent across arms by design (seed k of one arm shares nothing with seed k of the other beyond the frozen openings and opponents), so no pairing across arms is imposed. The same estimator serves seed sets of unequal size: the A1′ supplement (three seeds) against the full graph arm (five seeds) resamples three and five scores respectively.

```
procedure PERCENTILE-CI(x[1..n]; B = 10,000; seed = 0)
    rng <- Random(seed)
    for b in 1..B:
        m[b] <- mean of n draws x[rng.randrange(n)]
```

```
    sort m ascending
    return m[floor(0.025 * B)], m[floor(0.975 * B)]       # zero-based positions

procedure DIFF-CI(a[1..p], c[1..q]; B = 10,000; seed = 0)   # a = graph, c = grid
    rng <- Random(seed)
    for b in 1..B:
        d[b] <- (mean of p draws a[rng.randrange(p)]) - (mean of q draws
c[rng.randrange(q)])
    sort d ascending
    return d[floor(0.025 * B)], d[floor(0.975 * B)]
```

D.3 Raw per-seed tables: same-examples reading

The same-examples reading evaluates each run's checkpoint after the tenth and last generation (checkpoint identifiers are zero-based, so this is gen009), every run having consumed the same self-play budget of 10 generations × 500 games. Table 61 gives the scores and the truncation rate against legal-random; Table 63, after the second reading, gives the sensitivity scores against legal-random under both readings; legal-random is the only opponent against which any game was truncated.

Table 61: Per-run scores, same-examples reading: the tenth-generation checkpoint of each of five independent training runs per arm (10 generations × 500 self-play games each), 100 paired colour-swapped games per opponent on the frozen openings at 400 simulations per decision, against the frozen population (B-RND legal-random, B-HEU heuristic, B-MCTS search at 6,400 simulations). Score = mean of win 1 / draw 0.5 / loss 0 over non-truncated games (a fraction); trunc. = share of the 100 games stopped at the 300-ply cap, 0 % against B-HEU and B-MCTS in every cell and omitted. Raw values, no interval.

Arm	Seed	B-RND score	B-RND trunc.	B-HEU score	B-MCTS score
grid	1	0.995	0 %	0.150	0.125
grid	2	0.975	1 %	0.080	0.125
grid	3	0.990	1 %	0.190	0.075
grid	4	0.949	11 %	0.105	0.085
grid	5	0.995	0 %	0.120	0.210
graph	1	0.733	57 %	0.025	0.115
graph	2	0.981	20 %	0.050	0.125
graph	3	0.722	55 %	0.090	0.100
graph	4	0.688	44 %	0.125	0.120
graph	5	0.935	23 %	0.035	0.115

D.4 Raw per-seed tables: same-wall-clock reading

The same-wall-clock reading evaluates each run's last checkpoint completed within the equal-time cutoff (Appendix D.6). Where that checkpoint is the final one, the final evaluation set serves both readings, so the grid rows of seeds 1, 2, 4 and 5 are identical in both readings.

Table 62: Per-run scores, same-wall-clock reading: for each of five independent training runs per arm, the last checkpoint completed within 18.77 h of cumulative training wall-clock (identifiers zero-based; gen009 is the tenth generation), 100 paired colour-swapped games per opponent on the frozen openings at 400 simulations per decision, against the frozen population (B-RND legal-random, B-HEU heuristic, B-MCTS search at 6,400 simulations). Score = mean of win 1 /

draw 0.5 / loss 0 over non-truncated games (a fraction); trunc. = share of the 100 games stopped at the 300-ply cap, 0 % against B-HEU and B-MCTS in every cell and omitted. Raw values, no interval.

Arm	Seed	Checkpoint	B-RND score	B-RND trunc.	B-HEU score	B-MCTS score
grid	1	gen009	0.995	0 %	0.150	0.125
grid	2	gen009	0.975	1 %	0.080	0.125
grid	3	gen008	0.939	1 %	0.145	0.080
grid	4	gen009	0.949	11 %	0.105	0.085
grid	5	gen009	0.995	0 %	0.120	0.210
graph	1	gen003	0.798	58 %	0.045	0.075
graph	2	gen004	0.926	39 %	0.055	0.110
graph	3	gen004	0.713	53 %	0.060	0.105
graph	4	gen002	0.631	39 %	0.100	0.150
graph	5	gen005	0.980	24 %	0.045	0.095

Table 63: Sensitivity scores against legal-random (B-RND) under both readings, for the same runs, checkpoints and evaluation volume as Table 61 and Table 62: the mean over all 100 games of the cell with every truncated game counted 0.5, at the four-decimal precision of the comma-separated result files. Against B-HEU and B-MCTS no game of any run was truncated under either reading, so there the sensitivity score equals the score of the main tables in every cell. Raw values, no interval.

Arm	Seed	Same-examples: B-RND sens.	Same-wall-clock: B-RND sens.
grid	1	0.9950	0.9950
grid	2	0.9700	0.9700
grid	3	0.9850	0.9350
grid	4	0.9000	0.9000
grid	5	0.9950	0.9950
graph	1	0.6000	0.6250
graph	2	0.8850	0.7600
graph	3	0.6000	0.6000
graph	4	0.6050	0.5800
graph	5	0.8350	0.8650

D.5 Seed-level means, intervals and arm contrasts

Table 64 gives the mean of the five per-seed scores for each arm, opponent and reading with its seed-level bootstrap interval; Table 65 gives the graph-minus-grid differences of those means. These are the final numbers of the study.

Table 64: Seed-level mean scores of the two arms against each frozen opponent under both budget readings (same-examples: tenth-generation checkpoints after 10 generations × 500 games; same-wall-clock: last checkpoint within 18.77 h), five independent training runs per arm, 100 paired colour-swapped games per opponent and run. Each entry is the plain mean of the five per-run scores (fraction of decided games, truncations excluded) with its 95% percentile bootstrap interval over seeds (10,000 resamples).

Reading	Arm	vs B-RND	vs B-HEU	vs B-MCTS
same-examples	grid	0.981 [0.964, 0.994]	0.129 [0.098, 0.165]	0.124 [0.087, 0.168]
same-examples	graph	0.812 [0.710, 0.920]	0.065 [0.034, 0.100]	0.115 [0.107, 0.121]
same-wall-clock	grid	0.971 [0.950, 0.991]	0.120 [0.098, 0.142]	0.125 [0.090, 0.168]

Reading	Arm	vs B-RND	vs B-HEU	vs B-MCTS
same-wall-clock	graph	0.810 [0.697, 0.922]	0.061 [0.047, 0.081]	0.107 [0.087, 0.130]

Table 65: Arm contrast: difference of the seed-level means of Table 64, graph arm minus grid arm, per frozen opponent and budget reading, five independent training runs per arm; unit: score difference (fraction of decided games). Brackets: 95% percentile bootstrap interval from the five graph seeds and the five grid seeds resampled independently, 10,000 resamples.

Opponent	Same-examples: graph − grid	Same-wall-clock: graph − grid
B-RND	−0.169 [−0.272, −0.062]	−0.161 [−0.278, −0.048]
B-HEU	−0.064 [−0.111, −0.017]	−0.059 [−0.087, −0.029]
B-MCTS	−0.009 [−0.055, +0.028]	−0.018 [−0.066, +0.025]

The seeds were collected in two stages: seeds 1–3 of both arms in the campaign of 10–17 September 2026, analysed on 19 September 2026; seeds 4–5 of both arms between 27 September and 2 October 2026, under a commitment made before they were run to use all five seeds in the final analysis whatever their direction. Table 66 records the three-seed analysis so that both stages are on the record; the extension tightened four of the six contrast intervals, and the largest upper bound of any contrast moved from +0.035 to +0.028.

Table 66: The original three-seed analysis of 19 September 2026 (seeds 1–3 of each arm; same opponents, budget readings and evaluation volume as Table 64): seed-level means and graph-minus-grid contrasts with 95% percentile bootstrap intervals over three seeds per arm (10,000 resamples; arms resampled independently). Superseded by the five-seed tables; kept as the record of the first stage of seed collection.

Quantity	Reading	vs B-RND	vs B-HEU	vs B-MCTS
grid mean	same-examples	0.987 [0.975, 0.995]	0.140 [0.080, 0.190]	0.108 [0.075, 0.125]
graph mean	same-examples	0.812 [0.722, 0.981]	0.055 [0.025, 0.090]	0.113 [0.100, 0.125]
grid mean	same-wall-clock	0.970 [0.939, 0.995]	0.125 [0.080, 0.150]	0.110 [0.080, 0.125]
graph mean	same-wall-clock	0.812 [0.713, 0.926]	0.053 [0.045, 0.060]	0.097 [0.075, 0.110]
graph − grid	same-examples	−0.175 [−0.268, −0.007]	−0.085 [−0.143, −0.025]	+0.005 [−0.020, +0.035]
graph − grid	same-wall-clock	−0.158 [−0.254, −0.050]	−0.072 [−0.100, −0.028]	−0.013 [−0.040, +0.015]

D.6 The equal-time cutoff and the checkpoint map

The cutoff was fixed by a rule stated before the campaign: the median of the full-run training wall-clocks of the grid arm's three original runs. Those totals were 18.77 h, 16.73 h and 19.07 h, so the cutoff is 18.77 h; it was computed on 16 September 2026, before any cross-arm number existed, and the two later grid seeds never entered the median. The final computation takes the median exactly from the per-generation clock files rather than from the rounded constant, so that the defining run's own final checkpoint sits at the cutoff inclusively; the resulting map for seeds 1–3 reproduces the one recorded on 16 September 2026 checkpoint for checkpoint. For each run the checkpoint retained is the last whose cumulative training wall-clock does not exceed the cutoff (Table 67).

Table 67: Checkpoint retained for the same-wall-clock reading in each of the ten training runs: the last checkpoint completed within the equal-time cutoff of 18.77 h of cumulative training wall-clock (identifiers zero-based; gen009 is the tenth and final generation). Cumulative hours are those stated in the analysis records; "not stated" means the record gives the checkpoint but not the hours. Where the retained checkpoint is the final one, the final evaluation set (100 paired games per opponent) serves both readings; otherwise a separate equal-time set of the same volume was evaluated.

Arm	Seed	Checkpoint at the cutoff	Cumulative wall-clock	Evaluation set
grid	1	gen009 (final)	18.77 h	final set reused
grid	2	gen009 (final)	16.73 h	final set reused
grid	3	gen008	17.22 h	equal-time set
grid	4	gen009 (final)	not stated	final set reused
grid	5	gen009 (final)	not stated	final set reused
graph	1	gen003	16.50 h	equal-time set
graph	2	gen004	16.06 h	equal-time set
graph	3	gen004	not stated	equal-time set
graph	4	gen002	not stated	equal-time set
graph	5	gen005	not stated	equal-time set

The map is the measured content of the second reading: at equal wall-clock the graph arm had completed 3–6 of its ten generations (4–5 over the original three seeds), the grid arm nine or ten. Table 68 gives the full-run training wall-clocks behind the map. Over the five seeds the means are 18.00 h (grid) and 36.70 h (graph), a ratio of 2.04×; over the original three seeds they were 18.2 h and 35.7 h, a ratio of 2.0×. The equal-time evaluation sets for seeds 1–3 were run on 18 September 2026 (about 11 h), those for the graph arm's seeds 4 and 5 on 6 October 2026 (about 6 h).

Table 68: Training wall-clock of each of the ten main-campaign runs, in hours: the per-generation self-play generation and training seconds summed over the 10 generations of the run (10 × 500 games), evaluation games excluded, from each run's clock log; one machine, four worker threads, sequential runs. The sums are 90.0 h (grid) and 183.5 h (graph), 273.5 h in all. The equal-time cutoff is the median of the three grid values of seeds 1–3.

Arm	Seed 1	Seed 2	Seed 3	Seed 4	Seed 5	Mean
grid	18.77	16.73	19.07	18.23	17.19	18.00
graph	43.07	32.71	31.28	49.86	26.57	36.70

D.7 Cap-sensitivity bounds

The protocol brackets the primary estimator (truncations excluded) with three alternative treatments of every truncated game: counted 0.5 (the sensitivity scores of Table 63), counted as a loss for the arm under test, and counted as a win for it; the last is an upper bound on what any larger cap could do for an arm that truncates. The bounds were computed from the per-game records in the analysis of 19 September 2026 (three seeds per arm): with every truncated game scored as a win for the arm under test, the graph-minus-grid contrast against legal-random is still −0.072 under the same-examples reading and −0.058 under the same-wall-clock reading, and its direction is unchanged under every treatment (excluded, 0.5, loss, win). Against B-HEU and B-MCTS no game of either arm was truncated, so all treatments coincide there. The final analysis record does not restate the bounds at five seeds. Because the graph arm truncated 20–57% of

its games against legal-random in the original seeds and 23–44% in the extension seeds (grid: 0–1%, one extension seed at 11%), its primary score against that opponent is a mean over fewer decided games (43 to 80 per seed) than the grid arm's.

D.8 The second study: sensitivity scores, cutoff map and wall-clock

The second study (Section 15) uses the estimators of this appendix unchanged: the same score, truncation rate and sensitivity score per (seed, opponent) cell, the same seed-level percentile bootstrap with B = 10,000 and the same independent resampling of the two arms for the contrast. Its per-seed score tables under both readings, its seed-level means and its contrasts are given in the chapter (Table 39 to Table 43) and are not repeated; this section adds the sensitivity scores, the cutoff map with the cumulative clocks behind it, and the per-run wall-clock.

Table 69: Second study, sensitivity scores against legal-random (B-RND) under both readings, for the runs, checkpoints and evaluation volume of Table 39 and Table 41: the mean over all 100 games of the cell with every truncated game counted 0.5, at the four-decimal precision of the comma-separated result files. Against B-HEU and B-MCTS no game of any run was truncated under either reading, so there the sensitivity score equals the score of the main tables in every cell. Raw values, no interval.

Arm	Seed	Same-examples: B-RND sens.	Same-wall-clock: B-RND sens.
grid	11	0.9600	0.9600
grid	12	0.9650	0.9600
grid	13	0.9900	0.9900
grid	14	0.9800	0.9800
grid	15	0.9550	0.9400
graph	11	1.0000	0.9900
graph	12	0.9750	0.9650
graph	13	0.7550	0.6100
graph	14	0.8250	0.8350
graph	15	0.7900	0.7750

Under the sensitivity treatment the graph-minus-grid contrast against legal-random is −0.101 [−0.187, −0.012] under the same-examples reading and −0.131 [−0.256, −0.015] under the same-wall-clock reading; the other four contrasts are those of Table 43, there being no truncation against the other two opponents. The two bounding treatments (every truncated game scored as a loss or as a win for the arm under test) were not computed for the second study.

The second study's cutoff was fixed by the rule of its protocol: the median of the full-run training wall-clocks of its five grid runs, each the sum of the ten per-generation clocks, computed exactly from the clock files on 25 October 2026, minutes after the fifth grid run had finished and while the last graph run was in its first generation, before any score of the study had been read. The five totals were 17.4567, 17.9540, 17.6263, 17.4592 and 18.5176 h; the median, 17.6263 h (63,454.8 s), is the total of grid seed 13, whose own final checkpoint therefore sits at the cutoff inclusively. For each run the checkpoint retained is the last whose cumulative training wall-clock does not exceed the cutoff (Table 70).

Table 70: Second study, checkpoint retained for the same-wall-clock reading in each of the ten warm-started runs: the last checkpoint completed within the equal-time cutoff of 17.63 h of cumulative training wall-clock (identifiers zero-based; gen009 is the tenth and final generation), with the cumulative hours at that checkpoint from the clock files. Where the retained checkpoint is the final one, the final evaluation set (100 paired games per opponent) serves both readings; otherwise a separate equal-time set of the same volume was evaluated on 26–27 October 2026 with the pinned cutoff seeds (network 9500, match runner 888,000).

Arm	Seed	Checkpoint at the cutoff	Cumulative wall-clock	Evaluation set
grid	11	gen009 (final)	17.46 h	final set reused
grid	12	gen008	16.54 h	equal-time set
grid	13	gen009 (final)	17.63 h	final set reused (defining run)
grid	14	gen009 (final)	17.46 h	final set reused
grid	15	gen008	16.77 h	equal-time set
graph	11	gen003	17.25 h	equal-time set
graph	12	gen004	17.51 h	equal-time set
graph	13	gen002	13.95 h	equal-time set
graph	14	gen002	15.38 h	equal-time set
graph	15	gen003	14.93 h	equal-time set

At equal wall-clock the graph arm had completed 3–5 of its ten generations, the grid arm nine or ten (first study: 3–6 and nine or ten). Table 71 gives the full-run training wall-clocks behind the map; their means are 17.80 h (grid) and 41.59 h (graph), a ratio of 2.34× (first study: 2.04×).

Table 71: Second study, training wall-clock of each of the ten warm-started runs, in hours: the per-generation self-play generation, training and export seconds summed over the 10 generations of the run (10 × 500 games, replay window of three generations), evaluation games excluded, from each run's clock log; one machine, four worker threads, sequential runs. The sums are 89.0 h (grid) and 207.9 h (graph), 296.9 h in all. The equal-time cutoff is the median of the five grid values.

Arm	Seed 11	Seed 12	Seed 13	Seed 14	Seed 15	Mean
grid	17.46	17.95	17.63	17.46	18.52	17.80
graph	39.28	37.33	52.85	42.20	36.27	41.59

The trajectory figures of the second study are drawn as the first study's, directly from each run's evaluation records at the checkpoints of generations 5, 8 and 10 and its clock log; the training-metric figure is parsed from the second campaign's log, and the per-run values it shows at the first and tenth generation are tabulated in the analysis record.

D.9 Ablation raw tables

Three variants of the graph arm were trained with three seeds each at the full budget (10 generations × 500 games, identical self-play, training and evaluation settings) and evaluated at their tenth-generation checkpoint under the same-examples reading. Table 72 summarises the variants; Table 73 and Table 74 give the per-seed scores of the two that trained; Table 75 gives the contrasts against the full graph arm.

Table 72: The three ablation variants of the graph arm, each trained with three independent seeds at the full budget of the main comparison (10 generations × 500 self-play games; same frozen opponents, openings and evaluation settings). Training wall-clock in hours per seed

under the same accounting as Table 68 (per-generation self-play and training seconds summed, evaluation games excluded); the reference graph runs of seeds 1–3 took 43.07, 32.71 and 31.28 h. A1′s short runs are a symptom of its divergence (a NaN policy plays short degenerate games) rather than a saving.

Variant	Component changed	Seeds	Training wall-clock	Outcome
A1 (`graph-untyped`)	six direction-typed edge matrices replaced by one shared matrix	1, 2, 3	7.75 / 8.01 / 7.61 h	training diverged to NaN in generation 0, 3 of 3 seeds
A2 (`graph-nogpool`)	global-pooling bias removed from every layer (1.37M parameters vs 1.47M)	1, 2, 3	39.90 / 47.44 / 32.05 h	trained; no measurable effect
A1′ (`graph-untyped-clip`)	untyped edges *and* gradient-norm clipping at 1.0 (two components)	1, 2, 3	23.89 / 30.35 / 27.68 h	trained; within the full arm's band

No score table is given for A1 because none exists as a strength measurement: every forward pass of all three networks returned NaN for policy and value from the first checkpoint onwards, and the final evaluations of the three independently seeded runs were identical to the game (0 wins, 0 draws and 100 losses against B-HEU; 2 wins and 45 draws with 53% truncation against B-RND), which independent trainings cannot produce. The training logs carry the signature "policy top-1 100.0%, value acc 0.0%" from generation 1 onwards, and the degenerate games averaged about 42 plies with no truncation and no resignation. The raw evaluation files are kept but excluded from every table as scores.

Table 73: Per-seed final scores of ablation A2 (global-pooling bias removed) beside the full graph arm's seeds 1–3, same-examples reading: tenth-generation checkpoints, 100 paired colour-swapped games per opponent on the frozen openings at 400 simulations against the frozen population. Score = fraction of decided games won (draws 0.5); truncation rate of the 100 games in parentheses. Three independent training runs per variant; raw values, no interval.

Opponent	A2 seed 1	A2 seed 2	A2 seed 3	full seed 1	full seed 2	full seed 3
B-RND	0.768 (59 %)	0.714 (65 %)	0.952 (17 %)	0.733 (57 %)	0.981 (20 %)	0.722 (55 %)
B-HEU	0.050 (0 %)	0.070 (0 %)	0.025 (0 %)	0.025 (0 %)	0.050 (0 %)	0.090 (0 %)
B-MCTS	0.070 (0 %)	0.125 (0 %)	0.105 (0 %)	0.115 (0 %)	0.125 (0 %)	0.100 (0 %)

Table 74: Per-seed final scores of the A1′ supplement (untyped edges with gradient clipping at 1.0, an explicitly two-component variant), same-examples reading: tenth-generation checkpoints, 100 paired colour-swapped games per opponent on the frozen openings at 400 simulations against the frozen population. Score = fraction of decided games won (draws 0.5); the truncation rate of the 100 games is given in parentheses where the record states it (legal-random only). Three independent training runs; raw values, no interval.

Opponent	A1′ seed 1	A1′ seed 2	A1′ seed 3
B-RND	0.566 (47 %)	0.671 (59 %)	0.995 (0 %)
B-HEU	0.035	0.135	0.110
B-MCTS	0.090	0.195	0.060

Table 75: Ablation contrasts against the full graph arm, same-examples reading, frozen population, 100 paired games per opponent and run. A2 – full: difference of seed-level means over seeds 1–3 of both variants. A1′ – full: difference between the A1′ mean over its three seeds and the full graph arm's mean over its five seeds. Unit: score difference (fraction of decided games). Brackets: 95% percentile bootstrap interval, the two seed sets resampled independently with their own

sizes, 10,000 resamples. The A1′ contrast is confounded by the clipping and is never attributed to edge typing alone.

Opponent	A2 − full (3 vs 3 seeds)	A1′ mean (3 seeds)	full graph mean (5 seeds)	A1′ − full (3 vs 5 seeds)
B-RND	−0.001 [−0.170, +0.165]	0.744	0.810	−0.066 [−0.254, +0.130]
B-HEU	−0.007 [−0.043, +0.030]	0.093	0.061	+0.032 [−0.011, +0.078]
B-MCTS	−0.013 [−0.043, +0.013]	0.115	0.107	+0.008 [−0.040, +0.063]

D.10 Per-generation evaluation data

No per-generation score table exists in the result files or the analysis records. The score-versus-time and per-opponent trajectory figures are drawn directly from each run's evaluation records: every main-campaign run was evaluated, with the pinned settings and volume, at the checkpoints of generations 5, 8 and 10 (identifiers gen004, gen007 and gen009) and, where different from the final one, at its equal-time checkpoint; the population score at a point is the mean over the three opponents of the cell scores defined above, and the time axis is the cumulative sum of the run's per-generation training durations. The curves are reproducible from the released per-game records and clock files; their numbers are not reproduced here.

E Provenance of every result

This appendix is the one place in the report where internal identifiers are the point. It gives, for every table and figure, the generated artifact it was taken from, the raw records that artifact was computed from, and the journal entry that recorded the measurement; it then indexes the decision log, reproduces the claims–evidence register in full, and lists where the raw data live together with the hashes of every frozen artifact.

E.1 The evidence chain

Every number in this report is reachable along one chain. **Raw per-game records** are written by the evaluation arena as one CSV per (checkpoint, opponent) pairing under the run directory; the file's header lines name both engine command lines verbatim (network path, simulation count, seeds), and every game carries its outcome with truncation as its own category. **Generated tables** are produced by `scripts/make_results.py`, which reads those CSVs and the per-run wall-clock logs, aggregates per (seed, opponent), bootstraps over seeds (10,000 resamples, seed as the unit, no game pooled as i.i.d.), and writes `results/comparison/*.{md,csv}`; it never retypes a number. **Figures** are produced by `scripts/make_figures.py` from the same CSVs, wall-clock logs and campaign logs. **The report** is assembled from these files; a source checker verifies that every numeric token in every chapter occurs in the evidence base, and a second checker verifies numeric identity between the English and French versions. The minimal reproduction script rebuilds the engine in a fresh clone, replays a recorded game to an exact match of its shipped row, and regenerates the tables byte-identically (verified 2026-10-09).

Journals. Every experiment or measurement has one entry under `journal/`, named `YYYY-MM-DD-<slug>.md` and carrying an identifier of the form `<phase>-<date>-<slug>-<nn>` (for example `H6-2026-10-09-5seed-final-01`). The header fixes id, date, hypothesis, git commit, configuration, seeds, data version, hardware, duration and cost; the body has fixed sections: Methods, Raw results and uncertainty, Failures, Limits and confounders, Interpretation, Decision, Artifacts. Negative results and tooling defects are entries like any other. **Decision log.** Every non-trivial decision is an entry `D-nnn` in the workspace file `state/decisions.md`, append-only: a reversed decision is never edited, a new entry supersedes it and links back. Gate crossings record the author's approval in its original wording. **Methodology log.** `docs/methodology-log.md` (workspace) records, append-only, every incident in which the method caught or missed something.

E.2 From each result to its artifact

Table 76 covers the generated tables and figures; Table 77 covers the validation, baseline, pipeline and encoder evidence cited in the method chapters, which lives in test suites, configurations and journals rather than in generated tables.

Table 76: Provenance of every generated table and figure: the artifact it is taken from, the raw records that artifact is computed from, and the journal entry that recorded the result. Run directories are abbreviated as `eval/…` for `data/runs/cmp-<arm>-s<seed>/eval/…`.

Result in the report	**Generated artifact**	**Raw input**	**Journal entry**
Per-seed scores and truncation rates, same-examples reading (2 arms × 5 seeds × 3 opponents, 100 games each)	`results/comparison/results-same-examples.{md,csv}`	`data/runs/cmp-{grid,graph}-s{1..5}/eval/gen009-vs-{B-RND,B-HEU,B-MCTS}.csv`	`H6-2026-09-19-comparison-01` (3 seeds); `H6-2026-10-09-5seed-final-01` (final)
Per-seed scores and truncation rates, same-wall-clock reading	`results/comparison/results-same-wallclock.{md,csv}`	`eval/tstar-gen00N-vs-*.csv` of each run (the final checkpoint's files where the cutoff checkpoint is the last one) and `wallclock.json`	same two entries
Arm contrast graph – grid, both readings, bootstrap over seeds	`results/comparison/results-arm-difference.md`	the two CSV files above	same two entries
Equal-time cutoff T* = 18.77 h and the checkpoint map at T*	numbers in the journal; map re-derived by `make_results.py`	`wallclock.json` of grid s1–s3 (median of the three totals), then of every run	`H6-2026-09-16-progress-01`; re-derivation checked in `H6-2026-10-09-5seed-final-01`
Cap sensitivity bounds (every truncation scored as a win for the arm under test)	numbers in the journal	per-game records of the final evaluations	`H6-2026-09-19-comparison-01`
Cost table: parameters, best-provider inference, run wall-clock (self-play generation + training, evaluation excluded, mean over the five seeds), self-play cost (that wall-clock / 5,000 games), training-throughput benchmark (batch 128), population scores	`paper/figures/fig2-score-cost.md` (regenerated 2026-10-09 after a generator defect was found; see Section 10)	`wallclock.json` per run; `results-*.csv`; benchmark in `docs/representations/comparison-controls.md`	`H5-2026-09-10-encoders-01`; `H6-2026-09-19-comparison-01`
Score versus training wall-clock, all seeds, both arms, T* marked	`paper/figures/fig1-score-vs-time.png`	`eval/*.csv` and `wallclock.json` per run	`H6-2026-09-19-comparison-01`; `H6-2026-10-09-5seed-final-01`
Grid planes versus cell graph for one position	`paper/figures/fig3-encodings.png`	schematic: a synthetic five-piece position drawn in the script; no measured quantity	`H5-2026-09-10-encoders-01` (encoding definitions)
Three failure positions F1–F3	`paper/figures/fig4-failures.{md,png}`	`eval/gen009-vs-*.csv` and `results/`	`H6-2026-09-19-`

Result in the report	Generated artifact	Raw input	Journal entry
		`comparison/openings-v1.txt`; each game reproduced deterministically and verified against its CSV row	`comparison-01`
Per-opponent score trajectories by generation, 5 seeds	`paper/figures/fig5-per-opponent.png`	`eval/{gen004,gen007,gen009}-vs-*.csv` and `wallclock.json` per run	`H8-2026-10-09-detailed-edition-01`
Epoch-1 policy top-1 and value accuracy by generation, 10 runs	`paper/figures/fig6-training-metrics.png`	`data/runs/campaign.log`, `data/runs/extension.log` (training logs of the 10 main runs)	`H8-2026-10-09-detailed-edition-01`
Final-evaluation truncation rate versus legal-random per run	`paper/figures/fig7-truncation.png`	`eval/gen009-vs-B-RND.csv` per run	`H8-2026-10-09-detailed-edition-01`
Ablation table: A1 divergence, A2 null effect, A1′ supplement	`results/ablations/README.md`	`data/runs/cmp-graph-{untyped,nogpool,untyped-clip}-s{1,2,3}/eval/` against the full graph arm's finals	`H7-2026-09-23-a1-divergence-01`; `H7-2026-10-02-a2-nogpool-01`; `H7-2026-10-09-a1prime-01`
Pipeline, architecture and chronology schematics	`paper/figures/fig8-pipeline.png`, `fig9-architectures.png`, `fig10-timeline.png` (`scripts/make_report_figures.py`)	the system documentation (`docs/inventory.md`), the journal headers and the decision log; no measured quantity	none
Second study: per-seed scores and truncation rates, same-examples reading (2 arms × 5 seeds × 3 opponents, 100 games each)	`results/study2/results-same-examples.{md,csv}`	`data/runs/cmp2-{grid,graph}-s{11..15}/eval/gen009-vs-*.csv`	`H9-2026-10-27-analysis-01`
Second study: per-seed scores, same-wall-clock reading	`results/study2/results-same-wallclock.{md,csv}`	`eval/tstar-gen00N-vs-*.csv` of each `cmp2-*` run (final files where the cutoff checkpoint is the last one) and `wallclock.json`	`H9-2026-10-27-analysis-01`
Second study: arm contrast, both readings; Study 1 vs Study 2 table	`results/study2/results-arm-difference.md`; `results/study2/study1-vs-study2.md`	the two CSV files above and `results/comparison/results-*.csv`	`H9-2026-10-27-analysis-01`
Second study: equal-time cutoff $T^{*\prime}$ = 17.6263 h and the checkpoint map	numbers in the journal; map re-derived by `make_results.py --study 2`	`wallclock.json` of `cmp2-grid-s11..15` (median of the five totals), then of every `cmp2-*` run	`H9-2026-10-25-tstar-01`

Result in the report	Generated artifact	Raw input	Journal entry
Second study: cost table and per-run wall-clock	`paper/figures/fig2-score-cost-study2.md`; `results/study2/wallclock-per-run.md`	`wallclock.json` per `cmp2-*` run; `results/study2/results-*.csv`	`H9-2026-10-27-analysis-01`; `H9-2026-10-13-campaign-01`
Second study: score versus wall-clock, per-opponent trajectories, truncation per run, training metrics	`paper/figures/fig{1,5,7}-*-study2.png`, `fig6-training-metrics-study2.png` (`make_figures.py --study 2`)	`eval/*.csv` and `wallclock.json` per `cmp2-*` run; `data/runs/campaign2.log`	`H9-2026-10-27-analysis-01` (training metrics tabulated in its addendum)
Second study: pilot phase durations; execution record of the ten runs	numbers in the journals	`data/runs/pilot2-{grid,graph}-s10/`; `data/runs/cmp2-*/wallclock.json`, `data/runs/campaign2.log`	`H9-2026-10-12-pilot-01`; `H9-2026-10-13-campaign-01`

Table 77: Provenance of the method-chapter evidence (engine validation, baselines, pipeline, encoders, campaign design): the artifacts and the journal entry recording each measurement.

Evidence in the method chapters	Where it lives	Journal entry (date)
Perft tables (8 game types), UHP conformance 21/21, differential fuzzing against two reference engines (27,829 positions at seed 20260909), plane-encoder crosscheck (240 positions)	test suites under `crates/`, `scripts/nightly.sh`, `scripts/crosscheck_planes.py`	`H2-2026-09-09-suite-rerun-01` (2026-09-09)
30 hand-annotated critical positions, 30/30 after one setup-side correction	`tests/critical_positions/cases/*.toml`; `scripts/run_critical_corpus.py`	`H2-2026-09-09-corpus-run-01` (2026-09-09)
Zero invariant violations over 10.9M applied transitions	`crates/hive-core/tests/random_invariants.rs`	`H2-2026-09-09-random-invariants-01` (2026-09-09)
Throughput profile: 21.7 µs UHP round-trip; 2.62 ms (CoreML) versus 23.5 ms (CPU) per evaluation; game-length distribution behind the 300-ply cap	commands recorded inline in the entry	`H2-2026-09-09-throughput-profile-01` (2026-09-09)
CoreML inference restored in the Rust pipeline (MLProgram format)	`crates/hive-mcts/src/ort_eval.rs`	`H4-2026-09-09-coreml-fix-01` (2026-09-09)
Baseline definitions, characterisation round-robin, five tactical cases, weight pin	`configs/baselines/*.toml`, `docs/baselines.md`, `tests/tactical_positions/`	`H3-2026-09-09-baselines-01` (2026-09-09)
Pipeline pilot and the seven automated pre-training checks	`scripts/run_pilot.py`, `scripts/run_h4_checks.sh`, `data/runs/pilot0/`	`H4-2026-09-10-pilot-01` (2026-09-10)
Grid and graph encoders, capacity matching (1.44M vs 1.47M), cost asymmetries, property battery	`docs/representations/{grid,graph,comparison-controls}.md`, `python/hivenet/{graph_dataset,graph_model}.py`	`H5-2026-09-10-encoders-01` (2026-09-10)
Rust mirror of the graph encoder, golden crosscheck (160 positions), end-to-end graph arm through the same search	`scripts/crosscheck_graph.py`, `hive-nn` graph module	`H5-2026-09-10-graph-wiring-01` (2026-09-10)

Evidence in the method chapters	Where it lives	Journal entry (date)
Comparison matrix, opening generation, fixed-schedule harness test, campaign sizing	`configs/comparison-matrix.yaml`, `results/comparison/openings-v1.txt`, `results/comparison/opponents-manifest.md`	`H6-2026-09-10-matrix-01` (2026-09-10)
Report assembly checks and the staleness defect found by the render pass	`paper/final-control.md`	`H8-2026-10-09-detailed-edition-01` (2026-10-09)
Second study: driver options for warm start and replay window, regression test against the first study's recorded manifests (4 runs × 10 generations)	`scripts/run_comparison.py`, `scripts/test_driver_equivalence.py`	`H9-2026-10-09-driver-01` (2026-10-09)
Second study: pilot (2 generations × 200 games per arm), check battery on windowed shards, stamp check generalised to several manifests	`scripts/pilot_h9.sh`, `scripts/run_h4_checks.sh`, `python/hivenet/checks.py`, `data/runs/pilot2-h4-checks.log`	`H9-2026-10-12-pilot-01` (2026-10-12)
Second study: frozen protocol v2, campaign script, cutoff evaluation driver, analysis switch	`docs/protocol-v2.md`, `scripts/campaign2.sh`, `scripts/tstar_evals_study2.py`, `scripts/make_results.py --study 2`	`H9-2026-10-13-campaign-01`, `H9-2026-10-25-tstar-01`, `H9-2026-10-27-analysis-01`

E.3 Decision index

Table 78 lists the decision-log entries relevant to this study with their dates; entries D-002, D-013, D-018 and D-040 concern the other projects of the research programme and are omitted. Gate crossings are marked with their gate.

Table 78: Index of the decision-log entries relevant to this study, in order of recording.

Entry	Date	Decision
D-001	2026-09-09	Goal-directed operating rules adopted: routing file, one executable document per phase, gates reserved to the human
D-003	2026-09-09	The prior 19-generation self-play loop classified as a demonstration, not evidence; its checkpoint preserved
D-004	2026-09-09	Protocol/validation ordering fixed (draft, then measure, then freeze); write-as-you-go and no-real-money rules added; gates park a phase, not the session
D-005	2026-09-09	G-PUBLIC / G-SPEND: MIT licence; prior loop permanently stopped; sequential tracks
D-006	2026-09-09	Private remotes; methodology documented; full French copy of each report
D-007	2026-09-09	Study variant: base game only
D-008	2026-09-09	Truncation convention: cap ≠ draw, separate reporting, sensitivity procedure
D-009	2026-09-09	Novelty positioning: no pivot; scoped contribution claim against three closest works
D-010	2026-09-09	Python–Rust binding by subprocess/UHP (21.7 µs round-trip), no in-process binding
D-011	2026-09-09	Measured proposals: 128/32 simulations, 300-ply cap, two follow-up fixes
D-012	2026-09-09	G-FREEZE review: protocol freeze deferred until after the pilots
D-014	2026-09-09	Baseline definitions and budgets (legal-random, heuristic at depth 1, search at 6,400 simulations)

Entry	Date	Decision
D-015	2026-09-09	Shared action decoder: the (piece slot, destination) contract reused by both arms
D-016	2026-09-09	Proposal to exclude the prior checkpoint from the frozen population
D-017	2026-09-09	**G-FREEZE**: opponent population frozen, search opponent at 6,400 simulations, prior checkpoint excluded; hashes and engine commit b94e7c1
D-019	2026-09-09	Evaluation settings pinned: 400 simulations, no exploration noise, deterministic argmax
D-020	2026-09-10	**G-FREEZE**: protocol frozen as v1.0 on the pilot's measured values; sha256 f340a6b6…aefeb5, commit 44a74ff
D-021	2026-09-10	Grid representation reused as-is; frame-bounds check made an always-on assertion
D-022	2026-09-10	Graph arm: cell graph, fixed-capacity tensors (node cap 224), Python-first with a Rust mirror
D-023	2026-09-10	Symmetry augmentation excluded from the full method, both arms
D-024	2026-09-10	**G-DESTRUCTIVE** (by the author directly): pre-study workspace archive deleted; research copy is the sole copy of the prior work
D-025	2026-09-10	**G-FREEZE**: 250 shared openings frozen, generated blind; content sha256 63b318d0…5af7b
D-026	2026-09-10	**G-SPEND**: main campaign approved at full size and launched
D-027	2026-09-16	Evaluation volume kept at 100 games per pairing; seed variance dominates game noise
D-028	2026-09-19	Ablation (a) substituted by A2 (global-pooling bias); A1 naive adjacency kept
D-029	2026-09-20	**G-SPEND**: ablation campaign approved, both ablations at 3 seeds each
D-030	2026-09-26	**G-SPEND**: supplementary two-component A1′ approved at its corrected sizing
D-031	2026-09-26	**G-SPEND**: extension to 5 seeds per arm approved with three pre-commitments stated before any new run
D-032	2026-10-09	Report scope: detailed edition, no frozen artifact, number or claim changed
D-033	2026-10-09	Report form: standalone academic report explaining what, how, when and why, with no internal file references in the body
D-034	2026-10-09	Rights: the third-party reference engines (Mzinga, nokamute) are MIT-licensed; validation results publishable; G-RIGHTS closed
D-035	2026-10-09	Diffusion plan (repository release, preprint, public post), each step a separate G-PUBLIC decision; complementary study queued with a draft protocol
D-036	2026-10-09	**G-PUBLIC**: repository made public at tag v2.0-report with the report, the short article and the records archive; preprint and post approved
D-037	2026-10-12	**G-SPEND**: pilot of the second study (2 generations × 200 games per arm, warm start and three-generation window, seed outside both studies' sets)
D-038	2026-10-13	**G-FREEZE**: protocol v2 frozen (option A, window 3, seeds 11–15, cutoff rule); sha256 e296a12e…b461eb, commit 4a22a24
D-039	2026-10-13	**G-SPEND**: second campaign launched (2 arms × 5 seeds × 10 generations × 500 games, cutoff evaluations included)

E.4 Claims–evidence register

Table 79 reproduces `paper/claims.md` with its section column removed; claim wording is compressed where needed, numbers are unchanged. The rule is: no row, no claim.

Table 79: The claims–evidence register of this report, reproduced from the repository file with the section column removed. Every claim made in the body corresponds to one row; the limit column states what prevents a broader reading.

Claim	Evidence	Limit
The rules engine reproduces Mzinga's published perft tables for all 8 game types to depth 6 (depth ≤5 in the standard suite, depth 7 in nightly runs).	`H2-2026-09-09-suite-rerun-01`	depth-bounded node-count equality; d7 re-run not repeated on 2026-09-09 (d≤6 confirmed)
The engine passes the UHP conformance harness of the nokamute reference (21/21).	`H2-2026-09-09-suite-rerun-01`	conformance = protocol behaviour, not full rules proof
Per-ply legal-move sets are identical to both reference engines (MzingaEngine v0.16.0, nokamute 1.0.3) over seeded random games (27,829 positions at seed 20260909; 200/100 games/type in nightly).	`H2-2026-09-09-suite-rerun-01`	agreement with references, not with the rulesheet directly; random-walk coverage
The engine agrees with a 30-case corpus of hand-annotated critical positions whose expectations were committed before any engine run (30/30 after one setup-side correction).	`H2-2026-09-09-corpus-run-01`; `tests/critical_positions/`	corpus annotations not externally reviewed (declared limit); 30 cases at the low end of the 30–50 proposal
Seeded random-game sessions across all 8 game types show zero invariant violations over 10.9M applied transitions.	`H2-2026-09-09-random-invariants-01`	pseudo-random breadth, not adversarial depth; serialisation via UHP GameString only
The Rust and Python plane encoders agree exactly (240 positions, byte-identical planes).	`H2-2026-09-09-suite-rerun-01`	grid-arm encoder only at that date
A UHP subprocess round-trip costs ~22 µs (negligible against per-decision costs), so the Python↔Rust binding uses subprocess/UHP (no PyO3).	`H2-2026-09-09-throughput-profile-01`; D-010	one machine (M1 Pro); revisit if Python ever enters a per-move loop
At the measured inference costs (2.62 ms/eval CoreML, 23.5 ms CPU, gen-19 net as workload), 128/32 sims with playout-cap randomization and a 300-ply cap are feasible for this study's compute envelope.	`H2-2026-09-09-throughput-profile-01`; D-011	historical: proposals later pilot-confirmed and frozen (D-020); costs are net-specific
CoreML inference restored in the Rust pipeline (MLProgram fix): self-play at 128/32 sims costs ≈12 thread-s/game (≈3 s/game wall at 4 threads), vs ≈320 thread-s/game on the CPU fallback at 600/150.	`H4-2026-09-09-coreml-fix-01`	gen-19 net as workload; small probe counts; re-measured at the pilot
The heuristic baseline defeats legal-random 100–0 (100 paired games, 0 truncations);	`H3-2026-09-09-baselines-01`	pilot volume (100 games/pairing); arena CI unpaired approximation; ordering

Claim	Evidence	Limit
MCTS-without-network at 6400 sims scores 99.5% vs random and 37.5% vs the heuristic (−89 Elo [−150, −32]); a 4×-budget probe reaches 56.2%.		finding diagnosed, not retuned; population since frozen (D-017)
The MCTS baseline solves all 5 hand-annotated tactical cases at 400, 1600 and 6400 sims, and value signs are pinned under player alternation.	`H3-2026-09-09-baselines-01`; `tests/tactical_positions/`; sign tests in `crates/*/tests`	5 cases, unreviewed externally (declared limit)
All seven pre-training checks pass as automated tests on real self-play shards: zero illegal policy mass, id↔move identity, correct outcome perspective with truncation distinct, tiny-batch overfit (KL 0.09, argmax 15/15, value 15/15), bitwise save/resume, structurally-off eval noise, eval/training separation by audit.	`H4-2026-09-10-pilot-01`; `scripts/run_h4_checks.sh`	overfit criterion is KL to the soft-target entropy floor, not loss→0
One generation of self-play from random init (300 games, 128/32 sims, 1.44M-param grid net) yields a net that beats legal-random 100–0 (28 wins, 2 truncations) while scoring 3.3% vs the heuristic and 3.3% vs 6400-sim MCTS: non-degenerate learning with a data/iteration-gap diagnosis.	`H4-2026-09-10-pilot-01`; `data/runs/pilot0/eval/`	single seed, 30 games/opponent, gen 0 only; not a study result
Gen-0 self-play truncates 56.7% of games at the 300-ply cap (170/300); generation cost ≈12 s/game wall (4 threads) at gen 0, falling toward ≈3 s/game with a trained net at the same budget.	`H4-2026-09-10-pilot-01`; `data/runs/pilot0/selfplay/gen000-manifest.json`	one machine, one seed; rates specific to random-init play
The two arms are capacity-matched to +1.5% (grid 1.44M, graph 1.47M params) behind the identical decoder, and the graph encoder loses no state information vs engine-generated records (property battery P1–P5, 300 real positions, zero illegal policy mass end-to-end).	`H5-2026-09-10-encoders-01`; `docs/representations/`	historical: Rust mirror + golden cross-check landed green the same day (`H5-2026-09-10-graph-wiring-01`)
Best-available-provider inference costs differ ≈1.4× against the graph arm (grid 2.62 ms/eval CoreML vs graph 3.67 ms ORT-CPU), while CPU training throughput favours the graph arm 3.5× and MPS favours the grid arm 2×.	`H5-2026-09-10-encoders-01`; table in `comparison-controls.md`	one machine; reported, not equalised; feeds the same-wall-clock reading

Claim	Evidence	Limit
The Rust and Python graph encoders agree exactly (160 positions across all 8 game types, byte-identical tensors, nightly-pinned), and the graph arm runs end-to-end through the identical Rust MCTS.	`H5-2026-09-10-graph-wiring-01`; `scripts/crosscheck_graph.py`	smoke-scale training; strength results belong to the comparison
H1 is rejected under the pre-registered rule: under both budget readings the graph arm shows no seed-consistent advantage against the frozen population, and every graph–grid interval excludes a meaningful graph advantage (largest upper bound +0.035).	`H6-2026-09-19-comparison-01`; `results/comparison/results-arm-difference.md`	3 seeds/arm; one graph architecture at one capacity and budget; early-regime self-play (10 gens)
Arm contrast (seed-mean, bootstrap95 over seeds): vs B-RND −0.175 [−0.268, −0.007] (same-examples) and −0.158 [−0.254, −0.050] (same-wall-clock); vs B-HEU −0.085 [−0.143, −0.025] and −0.072 [−0.100, −0.028]; vs B-MCTS +0.005 [−0.020, +0.035] and −0.013 [−0.040, +0.015].	`results/comparison/results-*.csv`; `H6-2026-09-19-comparison-01`	intervals over 3 independent seeds per arm
The graph arm truncates 20–57% of its games vs legal-random at the 300-ply cap (grid 0–1%); the rejection is cap-robust: scoring all truncations as graph wins leaves graph–grid at −0.072/−0.058 vs B-RND.	`H6-2026-09-19-comparison-01` (cap-sensitivity bounds); fig4-F1	bound argument substitutes for a larger-cap re-run (stated)
Measured campaign costs: graph runs averaged 2.0× grid training wall-clock (36.7 vs 18.0 h per 10×500-game run, mean over 5 seeds, self-play + training, evaluation excluded; 3-seed figures were 35.7 vs 18.2 h); at T* = 18.77 h the graph arm completes 3–6 of 10 generations (4–5 over the original three seeds).	`data/runs/cmp-*/wallclock.json`; `results/comparison/wallclock-per-run.md`; fig2	one machine; per-arm best available provider (reported); the cost-table generator's divisor defect of 2026-10-09 affected only the previously printed means (30.0/61.2 h), never this ratio
All comparison artifacts were frozen before any comparison run (population D-017, protocol D-020, openings D-025), T* was computed from grid wall-clocks before any cross-arm number existed, and no frozen artifact was touched.	D-017/D-020/D-025/D-026; `H6-2026-09-16-progress-01`, `H6-2026-09-19-comparison-01`	none
Ablation A1 (parity): removing geometric edge typing destroys trainability: NaN divergence at generation 0 in 3/3 seeds; typed edges contribute at minimum optimization sta-	`H7-2026-09-23-a1-divergence-01`; `results/ablations/README.md`	mechanism (6× gradient scale on the shared matrix) is a grounded hypothesis, not a measured decomposition

Claim	Evidence	Limit
bility; A1 eval tables are artifacts of a NaN policy and excluded as scores.		
Ablation A2: removing the global-pooling bias has no measurable effect: nogpool–full = −0.001 [−0.170, +0.165] (B-RND), −0.007 [−0.043, +0.030] (B-HEU), −0.013 [−0.043, +0.013] (B-MCTS); failure modes unchanged.	`H7-2026-10-02-a2-nogpool-01`; `results/ablations/README.md`	3 seeds/cell; 0.10M param difference inherent to the component (reported)
Final 5-seed analysis (precommitted, D-031): H1 remains rejected. Graph–grid contrasts: B-RND −0.169 [−0.272, −0.062] / −0.161 [−0.278, −0.048]; B-HEU −0.064 [−0.111, −0.017] / −0.059 [−0.087, −0.029]; B-MCTS −0.009 [−0.055, +0.028] / −0.018 [−0.066, +0.025] (same-examples / same-wall-clock); largest upper bound +0.028. Supersedes the 3-seed tables as the study's final numbers.	`H6-2026-10-09-5seed-final-01`; `results/comparison/`	seeds 4–5 collected after the 3-seed analysis (disclosed); T* fixed at the preregistered value
A1′ supplement (two-component: untyped edges + grad-clip): trains finite and scores within the full graph arm's band (all diff CIs straddle 0); with A1, the typed relations' measurable contribution at this scale concentrates in optimization stability; it is never attributed to typing alone (clip confound).	`H7-2026-10-09-a1prime-01`; `results/ablations/README.md`	3 vs 5 seeds; wide intervals; confounded by construction
The working methodology (goal-level AI delegation under six human-only gates, file-based state, mechanical verification) caught at least five harness/process defects before they could contaminate results (pipeline deadlock, corpus setup error, eval-harness opponent defect, silent record-loss path, silent NaN divergence), each journaled at detection time.	`docs/methodology-log.md` (workspace); journals cited per incident in Section 10	process observations from one study; no counterfactual control
Second study (protocol v2, warm start + replay window 3, seeds 11–15): H1′ not supported. Graph–grid: B-RND −0.018 [−0.045, +0.008] / −0.052 [−0.140, +0.002]; B-HEU −0.003 [−0.046, +0.039] / +0.013 [−0.032, +0.055]; B-MCTS +0.003 [−0.082, +0.090] / −0.025 [−0.091, +0.034] (same-examples /	`H9-2026-10-27-analysis-01`; `results/study2/results-arm-difference.md`	5 seeds per arm; intervals wider than the first study's; a graph advantage below ≈ 0.09 against B-MCTS not excluded at equal examples; no numeric threshold in the frozen rule

Claim	Evidence	Limit
same-wall-clock at 17.63 h); no seed-consistent graph advantage; largest upper bounds +0.090 and +0.055.		
Second study vs first (descriptive, no pooling): the contrast moved toward zero on every opponent under both readings (B-RND −0.169 → −0.018 and −0.161 → −0.052; B-HEU −0.064 → −0.003 and −0.059 → +0.013; B-MCTS −0.009 → +0.003 and −0.018 → −0.025).	`results/study2/study1-vs-study2.md`; `H9-2026-10-27-analysis-01`	different seed sets; two regime changes at once; no cross-study interval
Second study: the conversion pathology persists in three graph seeds of five (24–42% truncation vs B-RND at the final checkpoints, 28–58% at the cutoffs) and is absent in two (0–4%); grid 1–7%; with truncations scored 0.5 the B-RND contrast is −0.101 [−0.187, −0.012] and −0.131 [−0.256, −0.015].	`results/study2/results-*.csv`; `H9-2026-10-27-analysis-01`	truncation against B-RND only; 300-ply cap; sensitivity reading, not the primary score
Second study cost: training clock grid 17.46–18.52 h (mean 17.80 h), graph 36.27–52.85 h (mean 41.59 h), ratio 2.34× (first study 2.04×); the window adds training time in proportion to the record count, 3–5% of a run; at the cutoff the grid arm had completed 9–10 generations, the graph arm 3–5.	`results/study2/wallclock-per-run.md`; `H9-2026-10-13-campaign-01`, `H9-2026-10-12-pilot-01`, `H9-2026-10-25-tstar-01`	one machine; wall-clock on this machine's providers
Second study: absolute strength against the strong opponents did not rise under the warm-started regime within ten generations (grid vs B-HEU 0.077 against 0.129; vs B-MCTS 0.131 against 0.124); the graph arm's score against B-RND rose from 0.812 to 0.971.	`results/study2/study1-vs-study2.md`	descriptive; different seeds; untuned defaults in both arms
The second study was preregistered: protocol v2 frozen on 13 October 2026 (sha256 e296a12e…b461eb) before any run; its cutoff computed from grid clocks on 25 October 2026 before any score was read; all five seeds per arm analysed; no restart or exclusion.	`docs/protocol-v2.md`; `H9-2026-10-25-tstar-01`, `H9-2026-10-13-campaign-01`; D-038, D-039	hyperparameters untuned by choice (option A), stated as a limit

E.5 Raw data, identifiers and frozen hashes

Location and layout. Raw records live on disk under `data/runs/` in the study repository; they are not under version control. There is one directory per training run, `cmp-<arm>-s<seed>`, with `<arm>` in {`grid`, `graph`} for the main comparison (seeds 1–5) and in {`graph-untyped`, `graph-nogpool`, `graph-untyped-clip`} for the ablations A1, A2 and A1′ (seeds 1–3), plus `pilot0/` for the pipeline pilot. Each run directory contains: `selfplay/gen00N-*.bin` (binary record shards of generation N, v3 format, model-stamped) with `gen00N-manifest.json` (network and its fingerprint, model generation, record version, game type, seed, game count, full and cheap simulation counts, full-move fraction, temperature plies, resignation settings, counts of resigned and truncated games, number of positions, shard list); `checkpoints/gen00N/` (`hivenet-e0.pt`, `hivenet-e1.pt`, `train-config.json`) and the exported `gen00N-b1.onnx` used for play; `eval/<checkpoint>-vs-<opponent>.csv` with its `.log` for the intermediate (gen004, gen007) and final (gen009) evaluations, and `eval/tstar-gen00N-vs-<opponent>.{csv,log}` for the equal-time cutoff checkpoint; and `wallclock.json`, the per-generation wall-clock in seconds from which every time figure and the cutoff are derived. Campaign-level logs are `data/runs/campaign.log` (main runs, seeds 1–3), `extension.log` (seeds 4–5), `ablations.log` and `tstar-evals.log`. The second study's runs follow the same layout under `cmp2-<arm>-s<seed>` (seeds 11–15), each with a `driver-config.json` recording the regime (warm start on, window 3); its campaign log is `campaign2.log`, its cutoff-evaluation log `tstar2.log`, and the pilot runs are `pilot2-{grid,graph}-s10`. Seed derivation: base seed = 100,000 × seed; generation g uses base seed + g; evaluation network seed 9000 + gen (finals) or 9500 (cutoff sets), opponent seeds 9101 (legal-random) and 9201 (search).

Volumes. The main campaign comprised 10 generations × 500 self-play games per run (30,000 self-play games per three-seed arm at the 3-seed stage) and 100 paired games per (checkpoint, opponent) evaluation on the frozen openings; the campaign ran from 2026-09-10 12:42 to 2026-09-17 22:31 for seeds 1–3 (≈163 h of machine time including the cutoff evaluations), with seeds 4–5 collected 2026-09-27 to 2026-10-02. Disk footprint is recorded in the evidence base only as the launch estimates (≈3–5 GB for the main campaign, ≈3–4 GB for the ablations) against 164 GB free at launch with a 20 GB guard; exact per-run sizes were not journaled. The second campaign ran from 2026-10-13 15:02 to 2026-10-26 23:00 (296.9 h of training wall-clock over the ten runs), followed by about 14 h of cutoff evaluations on 26–27 October; its ten run directories total 4.8 GB. Since the deletion recorded in D-024, the study repository is the sole copy of the pre-study material it inherited (the prior-loop checkpoint and its data, 412 MB and 75 MB), which is why those files are protected by the destructive-action gate.

Frozen artifacts and their hashes. Table 80 lists every frozen or pinned artifact with the identifier recorded at its freeze. The protocol hash is recorded in the decision log in abbreviated form; the full value was recomputed from the frozen commit while preparing this appendix and is identical to the hash of the current file, confirming that the protocol has not changed since its freeze.

Table 80: Frozen and pinned artifacts of the study with the identifiers recorded at their freeze (hashes, commits, seeds) and the decision-log or journal entry that records them.

Artifact	Frozen / pinned on	Identifier	Record
Protocol v1.0 (`docs/protocol.md`)	2026-09-10	sha256 f340a6b6…aefeb5 at commit 44a74ff (full:	D-020

Artifact	Frozen / pinned on	Identifier	Record
		f340a6b64db0f5f0bf126ffb 251c3de339450bde192fd54b 719036a8a3aefeb5)	
Opponent B-RND (`configs/baselines/random.toml`)	2026-09-09	sha256 f2fc4a06441d3c1a7922838a 6693dbb48ec54543bc34fd81 4d41c9a742514cd7	D-017
Opponent B-HEU (`configs/baselines/heuristic.toml`)	2026-09-09	sha256 7210a0a349c5bad5dcd2df09 9cc6865ee3cb5d8a2c30e106 ce58137804818999	D-017
Opponent B-MCTS (`configs/baselines/mcts-nonet.toml`)	2026-09-09	sha256 3fc8f75cf2b4f21012dd61e9 924408fbfc32c8ea561aa96e b44f1091ba07364e	D-017
Heuristic weights (`configs/baselines/heuristic-weights.toml`)	2026-09-09	sha256 d0602f1895fbed70b6f84ac2 a3eb87bd68e811e53acf24d4 d7814a1495b0b97a; pinned by test `weights_pinned_for_h3_baselines`	D-014, D-017
Engine code at the population freeze	2026-09-09	commit b94e7c1	D-017
Shared openings (`results/comparison/openings-v1.txt`, 250 lines)	2026-09-10	content sha256 63b318d071dfc3ecfae35856 36c8e6f7327ddc08e7aed86a 466f915f8005af7b; frozen-file sha256 538497390a3787299c67c3ca 138b8feacb369d55dc562881 d1aee45200cbccb2; generator seed 20260910; schedule hashes 8cd84b6564440666 (20-game sets) and 3bf5d3e51cea0c0d (100-game sets, both studies)	D-025; `H9-2026-10-27-analysis-01`
Evaluation settings (`configs/eval-settings.toml`)	2026-09-09	400 simulations, no exploration noise, deterministic argmax, 300-ply cap	D-019
Comparison matrix (`configs/comparison-matrix.yaml`)	2026-09-10	pre-registered sizes and the cutoff rule	D-026
Equal-time cutoff T*	2026-09-16	18.77 h = median of the three original grid run totals (18.77, 16.73, 19.07 h)	`H6-2026-09-16-progress-01`; D-031
Campaign and analysis code	2026-09-10 → 2026-10-09	campaign ac58773; analysis 1c65269 (3 seeds), 7db074d (5 seeds); ablations 216fded → bffeea9 (non-finite guard); A1′ queue 1fdd7ec	journals `H6-…`, `H7-…`
Protocol v2.0 (`docs/protocol-v2.md`), second study	2026-10-13	sha256 e296a12e…b461eb at commit 4a22a24 (full: e296a12e8d1e3dee36c3b210 ecec ced62314e64657aede1c b91b1ea452b461eb)	D-038
Equal-time cutoff T*′, second study	2026-10-25	17.6263 h = median of the five grid run totals (17.4567, 17.9540, 17.6263, 17.4592, 18.5176 h)	`H9-2026-10-25-`

Artifact	Frozen / pinned on	Identifier	Record
			`tstar-01`
Second-study code	2026-10-09 → 2026-10-27	driver options 008aaee; check battery 99be4ef; campaign script dc0b6c2; analysis switch b0f1598; figures 32297fe	journals `H9-…`

F Glossary

The tables below fix the English terms used in this report, their meaning within the perimeter of the study, and the French equivalent used in the French edition. The French column reproduces the phrasing established in the French chapters; an asterisk marks a composed form, built from established pieces, that the French edition uses for the first time here.

Table 81: Study-design terms.

Term	Definition	French equivalent
grid arm / graph arm	The two compared systems, identical except for the state encoding and the network body reading it.	bras grille / bras graphe
state encoding	The function turning a position into the tensors a network reads; pinned byte-for-byte between engine and training code.	encodage d'état*
shared action decoder	The single contract by which both arms emit a policy over the same actions, namely (piece slot, destination) plus pass, with identical legal-set masking and normalisation.	décodeur d'actions partagé
opponent population	The three fixed opponents every checkpoint is scored against, frozen on 9 September 2026.	population d'adversaires
B-RND, legal-random	Uniform draw over the legal moves; the floor of the population.	aléatoire légal
B-HEU, heuristic	Greedy one-ply argmax of a documented fixed-weight evaluation with queen-targeting quiescence.	heuristique documentée à poids fixes
B-MCTS, search at 6,400 simulations	PUCT tree search without a network, uniform priors, the handcrafted evaluation as leaf value.	MCTS sans réseau à 6400 simulations
frozen	Fixed by a dated approval and never touched afterwards; any later touch would be a new study.	gelé / gelée
pre-registered rejection rule	The condition written into the frozen protocol under which the hypothesis is rejected: no seed-consistent graph advantage under both readings, intervals excluding a meaningful advantage.	règle de rejet pré-enregistrée*
seed (independent training run)	One complete training run from a seeded initialisation; the unit of replication and resampling.	graine (exécution d'entraînement indépendante)
generation	One cycle of self-play, training and checkpoint export; ten per run.	génération
checkpoint	The exported network at the end of a generation; identifiers zero-based (gen000 to gen009).	point de contrôle
paired colour-swapped games	Each opening line played once with each colour, so all arms, seeds and opponents face identical schedules.	parties appariées, à couleurs échangées
pinned evaluation settings	400 simulations per decision, no exploration noise, deterministic argmax; fixed by file and test.	réglages d'évaluation épinglés
warm start	Initialising the network of generation g from the weights of generation $g-1$'s final checkpoint, with the optimizer and the schedule restarted; used in the second study only, the first study reinitialised every generation.	démarrage à chaud

Term	Definition	French equivalent
replay window	The recent generations whose records a generation trains on; 3 generations in the second study, the current generation alone in the first.	fenêtre de rejeu
equal-time cutoff of the second study	The median total training wall-clock of the five grid runs of the second study, computed once from the clock files before any cross-arm number; a value of its own, distinct from the first study's 18.77 h.	seuil à temps mural égal de la seconde étude*
second study	The separately pre-registered comparison under warm start and replay window (protocol version 2.0, frozen 13 October 2026, seeds 11 to 15); reported beside the first study, never pooled with it.	seconde étude

Table 82: Hive terms.

Term	Definition	French equivalent
One-Hive rule	No move may split the hive, including in transit; a piece whose removal would split it is pinned.	One-Hive (kept in English)
freedom to move	A sliding piece may only pass through a gap whose two flanking cells are not both occupied.	liberté de mouvement
gate	Two occupied cells flanking a step, blocking a slide; above ground level, the beetle gate.	porte (porte du scarabée)
stacking	Beetles climb onto pieces, forming stacks; only the top piece acts.	empilement
stun	The marker on a piece just thrown by a pillbug, which cannot move next ply; inert in the base game.	étourdissement
forced pass	With no legal move a side must pass; pass is legal exactly when no move exists.	passe forcée
ply	One move by one side (half-move); the cap is 300 plies.	pli / demi-coup
opening (frozen line)	A four-ply prefix played before the agents take over; 250 frozen lines shared by every match.	ouverture

Table 83: Search and self-play terms.

Term	Definition	French equivalent
self-play	Games the current network plays against itself, through the search, to produce training records.	auto-jeu
Monte-Carlo tree search (MCTS)	The search of both arms and of B-MCTS; the network supplies priors and a leaf value.	recherche arborescente Monte-Carlo (MCTS)
PUCT	The selection rule: maximise Q + c·prior·$\sqrt{N}/(1+n)$, c = 1.4.	PUCT
policy	The network's distribution over legal actions, trained towards the recorded visit distribution.	politique
value	The network's estimate in [−1, 1] of the outcome for the side to move; a three-class head, truncated records excluded.	valeur
visit distribution	Normalised root visit counts after a search; the policy target.	distribution de visites
playout-cap randomization	A fraction 0.25 of self-play decisions gets 128 simulations and is recorded; the rest get 32 and are not.	randomisation du plafond de simulations
temperature plies	The first 12 plies of a self-play game, sampled rather than taken by argmax.	plis en température
resignation	A self-play game is abandoned when the value falls below −0.92.	abandon

Term	Definition	French equivalent
resignation audit	A 10% fraction of self-play games played to the end regardless, to check that resignation hides no wins.	audit sans abandon
policy top-1	Training-fit metric: share of records whose policy argmax matches the target's argmax.	argmax de politique (top-1)*
value accuracy	Training-fit metric: share of records whose predicted outcome class is the recorded one.	exactitude de la valeur*

Table 84: Representation and network terms.

Term	Definition	French equivalent
frame (32×32)	The fixed grid into which the grid arm embeds a position, centred on the bounding box; 77 feature planes.	cadre 32×32 ; plans de caractéristiques
message passing	The graph arm's computation: each node updates from its neighbours, layer by layer.	passage de messages
direction-typed relations	The six hexagonal directions as edge types, each with its own weights.	relations d'arêtes typées par direction
global-pooling bias	A pooled summary of all nodes added back into every node; removed in ablation A2.	biais de pooling global (biais d'agrégation globale)
capacity-matched	Near-equal parameter counts: 1.44 M (grid) and 1.47 M (graph), +1.5%.	à capacité appariée
naive adjacency	Ablation A1: the six typed edge matrices replaced by one shared matrix.	adjacence naïve
gradient clipping	The norm clip at 1.0 added in the A1′ supplement, making it a two-component variant.	écrêtage de gradient

Table 85: Outcome and statistics terms.

Term	Definition	French equivalent
truncation (never "draw")	A game stopped at the 300-ply cap; a fourth outcome, never folded into draws.	troncature (partie tronquée)
move cap	The 300-ply limit at which a game is truncated.	plafond de coups (plafond de 300 demi-coups)
score	Mean of win 1 / draw 0.5 / loss 0 over the non-truncated games of a (seed, opponent) cell.	score
truncation rate	Share of a cell's 100 games that were truncated; always reported beside the score.	taux de troncature*
sensitivity score	The cell's mean over all games with truncations counted 0.5.	colonne de sensibilité « troncatures à 0.5 »
cap-treatment bounds	The contrast recomputed with truncations as losses and as wins, bracketing any other cap.	bornes de traitement du plafond
seed-level mean	The plain mean of the per-seed cell scores of one arm against one opponent.	moyenne au niveau des graines
seed-level bootstrap interval	A 95% percentile bootstrap interval from resampling seeds (never games) 10,000 times.	intervalle bootstrap sur les graines
percentile interval	The interval whose ends are the 0.025 and 0.975 quantiles of the sorted resample statistics; no bias correction.	bootstrap percentile
arm contrast	The graph-minus-grid difference of seed-level means, with its interval from independently resampled seed sets.	contraste entre bras

Table 86: Budget terms.

Term	Definition	French equivalent
budget	A run's resources in four denominations: wall-clock, hardware, training states, simulations.	budget (temps mural, matériel, états d'entraînement, simulations)
wall-clock	Elapsed real time on the study machine, logged per generation; a run's training wall-clock excludes its evaluation games.	temps mural
same-examples reading	The comparison at equal self-play budget: both arms' tenth-generation checkpoints after 10 generations × 500 games.	lecture à exemples égaux
same-wall-clock reading	The comparison at equal training time: each run's last checkpoint within the equal-time cutoff.	lecture à temps mural égal
equal-time cutoff	18.77 h, the median full-run wall-clock of the grid arm's three original runs, fixed by a pre-registered rule on 16 September 2026.	seuil à temps mural égal
simulations per decision	The search budget: 128 full / 32 cheap in self-play, 400 in evaluation, 6,400 for B-MCTS.	simulations par décision

G Reproduction guide

This is the one place in the report where commands, file names and identifiers appear verbatim, because reproducing the study requires them: what is released, the minimal scenario run in a fresh environment on 9 October 2026, the full command set, the durations to expect, and the identifiers a reproduction must match.

G.1 What is released

The release set is the repository `hive-graph-selfplay` and the records under it. The first study was released on 9 October 2026 at tag `v2.0-report` of the public repository https://github.com/mohamedbechirkefi/hive-graph-selfplay, with the report in both languages, the short article and a records archive attached to the release; the second study's records and this revision accompany the release tagged `v2.1-report`. The third-party reference engines used only for rules validation are MIT-licensed and are not redistributed.

- **Code.** The Rust engine (rules kernel, protocol server, search, arena, self-play workers), the Python training and export code under `python/hivenet/`, the scripts under `scripts/`.
- **Frozen configurations.** `configs/comparison-matrix.yaml`, `configs/eval-settings.toml`, the opponent configurations and heuristic weight file under `configs/baselines/`, the ablation specifications under `configs/ablations/`.
- **Frozen openings and population manifest.** `results/comparison/openings-v1.txt` (250 four-ply lines) and `opponents-manifest.md`, which records the hashes of Table 88.
- **Result tables.** `results/comparison/results-same-examples.{md,csv}`, `results-same-wallclock.{md,csv}`, `results-arm-difference.md`, `wallclock-per-run.md`; `results/ablations/README.md`; for the second study, the same four tables plus `study1-vs-study2.md` under `results/study2/`.
- **Per-run records**, under `data/runs/cmp-<arm>-s<seed>/` (first study, seeds 1–5) and `data/runs/cmp2-<arm>-s<seed>/` (second study, seeds 11–15, each with a `driver-config.json` recording the regime): `eval/` (one comma-separated file per checkpoint and opponent, one row per game: `opening_id, a_is_white, score_a, truncated, plies, outcome`, metadata in `#` header lines); `wallclock.json` (seconds per generation); `selfplay/` manifests tying each shard to its generating network, and the shards; `checkpoints/gen000-b1.onnx` to `gen009-b1.onnx`. Tables and figures need only `eval/` and `wallclock.json`; the replay needs one checkpoint.

The third-party engines used in rules validation (Mzinga, nokamute) are not shipped and are not needed: no study number derives from them.

G.2 The minimal fresh-environment scenario

`scripts/reproduce_minimal.sh <workdir>` performs, from a clean clone plus the shipped records, the smallest end-to-end check that touches every link of the chain: build, play, record, aggregate. It passed on 9 October 2026 and again the same day after a prose fix to the table generator, with an empty numeric difference. Its four steps:

1. **Clone and build.** `git clone` into `<workdir>/clone`, then `cargo build --release -p hive-engine -p hive-arena`; the engine binary must exist afterwards. The inference crate

downloads the ONNX Runtime binary at first build, so the first build needs network access.

2. **Ship the records.** Copy `results/comparison/`, every run's `eval/` and `wallclock.json`, and the single checkpoint `cmp-grid-s2/checkpoints/gen009-b1.onnx` into the clone, as the release layout would.
3. **Replay one recorded game deterministically.** The game is failure position F2 of the qualitative results: the grid arm's seed-2 final checkpoint against B-HEU on opening line 2, the arm playing Black, lost in 19 plies. The arena plays that opening's colour pair (`--games 2 --depth 1 --seed 1 --threads 1`), the network at 400 simulations with seed 9009 (the final-evaluation rule, 9000 + generation index), B-HEU at depth 1 on one thread; the script asserts that the Black-side row's `score_a`, `truncated` and `plies` equal the shipped row in `cmp-grid-s2/eval/gen009-vs-B-HEU.csv` and prints `replay matches shipped row: plies 19, score 0`.
4. **Regenerate and compare.** `python3 scripts/make_results.py` (standard-library Python only) rebuilds both reading tables from the shipped records; `cmp` against the shipped `results-same-examples.md` and `results-same-wallclock.md` must report them byte-identical. The script ends with `MINIMAL REPRODUCTION: PASS`.

The scenario verifies that the released code builds from a clean checkout, that a recorded game is replayed exactly by the released checkpoint against the released opponent under the pinned settings, and that the published tables are a pure function of the released records. It does not verify training; that is the full reproduction below.

G.3 Full reproduction commands

```
# toolchain: Rust (cargo) and Python 3; the project virtual environment is
python/.venv
cargo build --release                       # engine, arena, self-play and fuzz
binaries
cargo test                                  # rules kernel: unit tests, perft to
depth 5 for all 8 game types, UHP server tests
cargo test --release -p hive-core --test perft_tables -- --ignored   # perft to
depth 6

# one training run of the matrix (resumable per generation); repeat for --seed 2
… 5
python3 scripts/run_comparison.py --arm grid  --seed 1 --gens 10 --games 500
python3 scripts/run_comparison.py --arm graph --seed 1 --gens 10 --games 500
# ablations (seeds 1–3): --arm graph-untyped | graph-nogpool | graph-untyped-clip

# equal-time checkpoint evaluations for runs whose cutoff checkpoint is not the
final one
bash scripts/tstar_evals.sh                 # seeds 1–3
bash scripts/tstar_evals2.sh                # graph seeds 4–5

# regenerate every table and figure from raw per-game records
python3 scripts/make_results.py
python/.venv/bin/python scripts/make_figures.py

# second study (protocol v2): driver defaults reproduce the first study
(regression test),
# then the ten warm-started runs, the cutoff evaluations and the analysis
python3 scripts/test_driver_equivalence.py
```

```
bash scripts/campaign2.sh                  # seeds 11–15, --warm-start --window 3
--run-prefix cmp2
python3 scripts/tstar_evals_study2.py      # T*' from the five grid clocks, then
the seven cutoff sets
python3 scripts/make_results.py --study 2
python/.venv/bin/python scripts/make_figures.py --study 2

# pre-training check battery on any shard set
bash scripts/run_h4_checks.sh '<abs>/gen000-*.bin' '<abs>/gen000-manifest.json'

# fresh-environment minimal reproduction (clone, build, replay, compare)
bash scripts/reproduce_minimal.sh /tmp/repro

# French/English numeric-identity check; report build
python3 scripts/check_fr_numbers.py
python/.venv/bin/python scripts/build_report.py all
```

The training driver is resumable per generation: a re-run skips completed generations and continues with the same learning-rate schedule, and every run writes its generator settings, seeds and model stamps into its manifests. Seeds derive from the run's seed number: base seed = 100,000 × seed, generation g using base seed + g; evaluation uses network seed 9000 + generation (final sets) or 9500 (equal-time sets), opponent seeds 9101 (B-RND) and 9201 (B-MCTS).

G.4 Expected durations and disk

All durations in Table 87 were measured on the study machine (Apple M1 Pro, 10 cores, 16 GB, macOS 15.3.1) with four worker threads per run and sequential runs; the grid arm's inference runs on CoreML (2.62 ms per evaluation), the graph arm's on CPU (3.67 ms), each the fastest provider measured for that network.

Table 87: Measured durations of the study's computations on the study machine (Apple M1 Pro, four worker threads per run, sequential runs), as recorded in the run and analysis records; ranges span the runs of the step. Training wall-clock is the sum of a run's per-generation self-play and training seconds, evaluation games excluded; campaign, extension and ablation figures are elapsed times as recorded. A1′s short runs reflect its divergence (a NaN policy plays short degenerate games) rather than a lower cost. The engine build and the minimal scenario were not timed.

Step	Measured duration
One grid training run (10 generations × 500 games): training wall-clock, evaluation games excluded	16.7–19.1 h
One graph training run (same budget): training wall-clock, evaluation games excluded	26.6–49.9 h
All ten main-campaign runs, training wall-clock summed	273.5 h (grid 90.0 h, graph 183.5 h)
Original six-run campaign (seeds 1–3, both arms, sequential), elapsed	10–17 September 2026, ≈7.4 days; ≈163 h of machine time
Extension runs (seeds 4–5, both arms), elapsed per run as recorded	18.0–54.9 h, 27 September to 2 October 2026
One evaluation game at 400 simulations	≈23–32 s
Equal-time evaluation sets, seeds 1–3 (four checkpoints × three opponents)	≈11 h, 18 September 2026
Equal-time evaluation sets, graph seeds 4–5 (two checkpoints × three opponents)	≈6 h, 6 October 2026

Step	Measured duration
Ablation runs A1 / A2 / A1′ (per seed), training wall-clock	7.61–8.01 h (a divergence symptom) / 32.05–47.44 h / 23.89–30.35 h
Second study, one grid run (warm start, window 3): training wall-clock	17.46–18.52 h
Second study, one graph run (same regime): training wall-clock	36.27–52.85 h
Second study, all ten runs, training wall-clock summed	296.9 h (grid 89.0 h, graph 207.9 h)
Second campaign (seeds 11–15, both arms, sequential), elapsed	13–26 October 2026, 13 days 8 h
Second study, equal-time evaluation sets (seven checkpoints × three opponents)	≈ 14 h, 26–27 October 2026
Second study, pilot (2 generations × 200 games, evaluation at 20 games per opponent)	grid 1 h 35 min; graph 4 h 17 min
Engine build; minimal scenario	not measured

Disk: before the campaign the six-run matrix was sized at about 3–5 GB of shards, checkpoints and records, against 164 GB free with a 20 GB guard; mid-campaign the machine reported 200 GB free. The final footprint of the first study's run directories was not recorded; the second study's ten run directories total 4.8 GB.

G.5 Identifiers a reproduction must match

A reproduction is faithful when it uses the frozen artifacts identified below and, for the minimal scenario, reproduces the shipped game row and the byte-identical tables. The frozen experimental constants that every command must leave untouched are tabulated in Appendix C: base game with the tournament opening rule, 300-ply cap, 128/32 self-play simulations with full fraction 0.25, 12 temperature plies, resignation at −0.92 with a 10% audit, 400 evaluation simulations without noise, 100 games per opponent, 250 openings, the 18.77 h cutoff, seeds 1–5 per arm; for the second study, the same constants with the warm start, the three-generation window, seeds 11–15 and the 17.63 h cutoff of protocol version 2.0 (Appendix C.6).

Table 88: Identifiers of the frozen artifacts and code states of the study. Hashes are SHA-256 hexadecimal digests of file content as recorded at the freeze; the schedule hashes are the digest prefixes printed by the analysis tool for the (opening, colour) schedule, identical for every match of a given volume in both studies (the first edition of this report attributed the 20-game hash to the 100-game matches); commits are abbreviated repository commit identifiers.

Artifact	Identifier
Heuristic weight file (`configs/baselines/heuristic-weights.toml`), SHA-256	`d0602f1895fbed70b6f84ac2a3eb87bd 68e811e53acf24d4d7814a1495b0b97a`
B-RND configuration (`configs/baselines/random.toml`), SHA-256	`f2fc4a06441d3c1a7922838a6693dbb4 8ec54543bc34fd814d41c9a742514cd7`
B-HEU configuration (`configs/baselines/heuristic.toml`), SHA-256	`7210a0a349c5bad5dcd2df099cc6865e e3cb5d8a2c30e106ce58137804818999`
B-MCTS configuration (`configs/baselines/mcts-nonet.toml`), SHA-256	`3fc8f75cf2b4f21012dd61e9924408fb fc32c8ea561aa96eb44f1091ba07364e`
Frozen openings, content of the 250 lines, SHA-256	`63b318d071dfc3ecfae3585636c8e6f7 327ddc08e7aed86a466f915f8005af7b`
Frozen openings, file as frozen, SHA-256	`538497390a3787299c67c3ca138b8fea cb369d55dc562881d1aee45200cbccb2`
Opening/colour schedule of every 100-game match (final and cutoff sets, both studies)	`3bf5d3e51cea0c0d`
Opening/colour schedule of every 20-game intermediate match (both studies)	`8cd84b6564440666`

Artifact	Identifier
Frozen protocol document	`f340a6b6…aefeb5` (recorded abbreviated), commit `44a74ff`
Frozen protocol document, version 2.0 (second study)	`e296a12e8d1e3dee36c3b210ececced6 2314e64657aede1cb91b1ea452b461eb`, commit `4a22a24`
Engine code at the population freeze	commit `b94e7c1`
Campaign code (main comparison)	commit `ac58773`
Ablation code; non-finite-loss guard	commits `216fded`; `bffeea9`
Analysis code (final five-seed tables)	commit `7db074d`
Second study: driver options; campaign script; analysis switch; figure switch	commits `008aaee`; `dc0b6c2`; `b0f1598`; `32297fe`

v2.1-draft, 2026-10-27